Linnaeus University Dissertations
No 639/2026

Rakhshanda Jabeen

Representing Architectural Evidence in Software Systems

A Graph Learning Approach

LINNAEUS UNIVERSITY PRESS

Representing Architectural Evidence in Software Systems

A Graph Learning Approach

Linnaeus University Dissertations

No 639/2026

Representing Architectural Evidence in Software Systems

A Graph Learning Approach

Rakhshanda Jabeen

LINNAEUS UNIVERSITY PRESS

The doctoral dissertation seminar will be held at 9:15 am on 4 December 2025 in room D113, at Linnaeus University, Växjö. The opponent will be Docent Sebastian Herold, Karlstad University.

Representing Architectural Evidence in Software Systems: A Graph Learning Approach
Doctoral Dissertation, Department of Computer Science and Media Technology, Linnaeus University, Växjö, 2026

ISBN: 978-91-8082-518-4 (print), 978-91-8082-519-1 (pdf)
© Rakhshanda Jabeen
DOI: https://doi.org/10.15626/LUD.639.2026
Published by: Linnaeus University Press, 351 95 Växjö
Printed by: Multiply, Malmö, 2026

Abstract

Jabeen, Rakhshanda (2026). *Representing Architectural Evidence in Software Systems: A Graph Learning Approach,* Linnaeus University Dissertations No 639/2026, ISBN: 978-91-8082-518-4 (print), 978-91-8082-519-1 (pdf).

Software architecture documentation is often missing, incomplete, or outdated, making it unclear which source files implement which architectural modules. Software architecture recovery infers a modular decomposition from the implementation, while code-to-architecture mapping assigns files to known modules using a partial mapping. Both tasks can use dependencies, folder structure, and source code, but existing methods often rely on individual sources or predefined combinations. It remains unclear what each source contributes and when propagating information between connected files helps rather than mixes evidence from different modules.

The research uses open-source Java, C, and C++ systems with published reference decompositions. Recovery studies examine learning objectives, graph construction, and information sources, while mapping studies investigate representations, graph-based propagation, and iterative self-training.

For architecture recovery, learned representations achieve agreement with reference architectures comparable to or better than the evaluated baselines. In the evaluated configurations, reconstruction generally outperforms contrastive learning, while random walks and diffusion capture structural information beyond direct dependencies. However, combining structural, lexical, and folder information can improve agreement with reference architectures while producing modules with weaker dependency cohesion than those obtained from dependency-based random-walk embeddings.

For code-to-architecture mapping, neural classifiers outperform a Naive Bayes baseline. A graph attention network outperforms a graph-free classifier on most systems, although gains vary by system and configuration. Folder information produces the largest and most consistent improvements, but higher accuracy can coincide with misclassifications of

files that a simple directory rule assigns correctly. Representations that encode dependencies generally outperform lexical ones, and dependency-guided fine-tuning improves contextual representations.

However, strong graph-model performance does not by itself demonstrate a benefit from additional message passing. Controlled experiments show that propagating dependency information can reduce performance for some predictors compared with matched models without cross-file messages. Changes to graph construction, propagation methods, or learning settings can reduce these losses or lead to improvements.

Together, these findings show that the usefulness of architectural information depends not only on what is available, but also on how it is represented, combined, and used.

Keywords: Software Architecture, Software Architecture Recovery, Representation Learning, Graph Neural Networks, Graph Autoencoders

وَفَوْقَ كُلِّ ذِي عِلْمٍ عَلِيمٌ ﴿٧٦﴾

*"And above every possessor of knowledge is **one** more knowing."*

Quran, Surah Yusuf (12:76)

In memory of my mother, Safia Kausar.

Acknowledgments

First and foremost, I thank Allah for giving me the strength, patience, and health to begin this work and bring it to completion.

These five years have taught me patience and perseverance, and I thank my main supervisor, Morgan Ericsson, who has been a great mentor to me throughout this time. I have learned a great deal from him, and when I look back, it is the small details and teaching moments that stand out. He has helped me more than I can express here, and I am deeply grateful.

I also thank my co-supervisors, Jonas Nordqvist and Anna Wingkvist, for their guidance and feedback throughout this work. Welf Löwe, thank you for your careful review and constructive feedback, which helped improve this thesis.

At Electrolux Professional, I thank my managers, Dorothee Millon and Massimo Gerolami, for their support and for understanding the time and commitment required by my PhD studies. I also thank my colleagues at Linnaeus University and Electrolux Professional for their support throughout these years.

I am deeply grateful to my father, Imtiaz Ahmad, for his love, prayers, and for always believing in me. To my brothers, Yasir and Nasir, and my sister, Farkhanda, thank you for always being there for me, especially when things were difficult. Finally, I thank my friends for their encouragement and for the good times we have shared over these years.

Rakhshanda Jabeen
4th December, 2026
Växjö, Sweden

Table of Contents

Acknowledgments **xiii**

Table of Contents **xv**

List of Figures **xvii**

List of Tables **xix**

List of Publications **xxiii**

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Research Aim and Scope . . . 4
- 1.4 Thesis Contributions . . . 4
- 1.5 Thesis Outline . . . 5

2 Background **7**
- 2.1 Prior Approaches . . . 7
- 2.2 Representing Artifact Content . . . 11
- 2.3 Graph Representation Learning . . . 12
- 2.4 Evaluation and Ground Truth . . . 16

3 Problem Formulation and Research Objectives **19**
- 3.1 Problem Formulation . . . 19
- 3.2 Research Objectives . . . 20

4 Overview of the Studies **23**
- 4.1 Relationship Between Objectives and Studies . . . 23
- 4.2 Overview of the Included Studies . . . 23

5 Research Method **27**
- 5.1 Research Design . . . 27
- 5.2 Graph and Signal Representation . . . 30
- 5.3 Learning Settings . . . 32

5.4 Benchmark Systems 35
5.5 Evaluation Protocol 36
5.6 Cross-Study Evaluation Design 37

6 Research Findings and Synthesis **41**
6.1 Results of the Cross-Study Evaluations 41
6.2 Synthesis of Research Findings 46

7 Discussion **53**
7.1 Interpretation of the Findings 53
7.2 Limitations 56

8 Conclusion and Future Work **61**
8.1 Conclusion 61
8.2 Future Work 63

Bibliography **65**

Appended Publications

Graph Representation Learning for Software Architecture Recovery **71**

GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery **79**

When Does the Dependency Graph Help Code-to-Architecture Mapping? **103**

Graph Embeddings for Software Architecture Recovery **121**

What Carries Architectural Signal? A Code-to-Architecture Mapping Study **141**

List of Figures

4.1 A high-level overview of objectives mapped to the papers. 24

5.1 Design space studied in the thesis. 29
5.2 Node-set configurations used across the thesis. 31

6.1 Overview of recovery results across studies. 42
6.2 Overview of C2A mapping results across studies. 44

7.1 Correlation between architecture recovery metrics and system properties. 55

List of Tables

2.1 Taxonomy of representative prior approaches. 8

5.1 Graph construction and signal representation across thesis. 33
5.2 Encoder configurations and training settings across studies. . . . 34
5.3 Properties and versions of the benchmark systems. 35
5.4 Properties of the benchmark systems. 39

6.1 Recovered partitions compared with the ground truth in TMQ_{norm}. 44
6.2 Typed versus merged GAT across file representations. 45

List of Abbreviations

Tasks, Data and Background

AST	Abstract Syntax Tree
C2A	Code-to-architecture
MQ	Modularization quality
SAR	Software architecture recovery
SDG	Software dependency graph

Models and Graph Processing

GAE	Graph autoencoder
GAT	Graph attention network
GAT-DC	Graph attention network with graph diffusion convolution
GAT-E	Graph attention network with edge attributes
GCN	Graph convolutional network
GDC	Graph diffusion convolution
GNN	Graph neural network
HGAT	Heterogeneous graph attention network
HGAT-TD	Heterogeneous graph attention network with typed dependencies
HGT	Heterogeneous Graph Transformer
LPE	Laplacian positional encoding
M2V	MetaPath2Vec
MLP	Multilayer perceptron
N2V	Node2Vec
PPR	Personalized PageRank
VGAE	Variational graph autoencoder

Input Representations

BoW	Bag-of-words
UXC	UniXcoder
W2V	Word2Vec

Training Objectives

BCE	Binary cross-entropy
CCE	Categorical cross-entropy
DGI	Deep Graph Infomax
KL	Kullback–Leibler

Evaluation Metrics

A2A	Architecture-to-architecture
ARI	Adjusted Rand index
$\mathbf{C2C_{cvg}}$	Cluster-to-cluster coverage
MoJoFM	Move and Join effectiveness measure
$\mathbf{TMQ_{norm}}$	Normalized turbo modularization quality

List of Publications

This dissertation is based on the following publications. I contributed to all stages of the work as lead author.

I. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Representation Learning for Software Architecture Recovery". In: *ESANN*. 2026. (page 71).

II. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery". In: *Proceedings of the ACM on Software Engineering* 3.FSE (July 2026). DOI: 10.1145/3808187. (page 79).

III. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. *When Does the Dependency Graph Help Code-to-Architecture Mapping?* 2026. arXiv: 2610.19769 [cs.SE]. URL: https://arxiv.org/abs/2610.19769. *Under review.* (page 103).

IV. Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Embeddings for Software Architecture Recovery". In: *Nordic Machine Intelligence* (2026). DOI: 10.5617/nmi.13153. URL: https://journals.uio.no/NMI/article/view/13153. (page 121).

V. Rakhshanda Jabeen, Morgan Ericsson, Yuyao Duan, and Anna Wingkvist. *What Carries Architectural Signal? A Code-to-Architecture Mapping Study.* 2026. arXiv: 2610.41628 [cs.SE]. URL: https://arxiv.org/abs/2610.41628. *Under review.* (page 141).

Chapter 1

Introduction

Contents

1.1 Motivation 1
1.2 Problem Statement 2
1.3 Research Aim and Scope 4
1.4 Thesis Contributions 4
1.5 Thesis Outline 5

Software architecture provides an abstraction of a software system in terms of its principal elements, their responsibilities, and the relations among them [10]. This abstraction supports reasoning about the system without requiring developers to inspect every implementation detail. However, the connection between the intended architecture and the source code is often incomplete or outdated [1, 13]. Recovering this connection relies on evidence from the implementation, which is distributed across multiple artifacts and their relations [8, 55]. Prior work either uses such evidence in isolation or combines multiple sources to support architectural analysis [87, 12, 8]. This thesis investigates how graph-based representation learning can use static implementation artifacts to support architectural analysis.

1.1 Motivation

Software architecture is often reflected in the implemented system as a modular decomposition: a partition of files into modules that group related functionality [68]. Such decompositions matter for program understanding, maintenance, and refactoring [10]. As a system evolves, developers introduce dependencies that were not anticipated in the original design, refactor code across module boundaries, and add functionality that fits no existing module. Documentation is often not maintained in step with these changes. The resulting gap between the intended design and the implementation is described using two related terms: *architectural drift*, where the implementation grows away from the intended design without directly violating it, and *architectural erosion*, where the implementation directly violates the intended design [13].

Architectural drift and erosion can contribute to *technical debt* by making later maintenance and change more costly [45]. This concern may become more relevant as large language models (LLMs) are increasingly used to assist software development. A multivocal literature review reports that LLM-assisted development can amplify existing forms of technical debt, including design debt [16]. An empirical study of AI-assisted development also reports short-term gains in development velocity alongside persistent increases in code complexity [28]. These findings provide additional motivation to assess architectural consistency, since changes that satisfy individual tasks may still conflict with the intended organization of responsibilities and dependencies across the system.

Two related activities address the gap between the intended architecture and its implementation. Software architecture recovery (SAR) is used when no sufficiently reliable architectural decomposition is available. It infers such a decomposition by grouping source-code files into architectural modules using evidence from the implementation. Performing recovery manually requires substantial knowledge of both the system and its domain, and becomes difficult as the number of files grows [20].

Code-to-architecture (C2A) mapping is used when the architectural modules are known, but the assignments of source-code files to those modules are absent, incomplete, or outdated. It assigns files to these predefined modules [81, 9]. The resulting mapping supports techniques such as reflexion modeling, which compare implemented dependencies with intended architectural relations [57]. Maintaining the mapping manually is costly because file assignments must be reconsidered as the implementation and architectural responsibilities evolve [1].

Although the two activities differ in the architectural information available at the outset, both use implementation evidence to assign files to architectural modules. Dependencies, project organization, and source-code content offer potentially complementary evidence, motivating an investigation into how these signals can support recovery and mapping.

1.2 Problem Statement

Both tasks assign source-code files to architectural modules, and a single implementation property does not determine a file's architectural role. Instead, evidence of module membership is distributed across several signals. Static dependencies describe how files call, import, inherit from, or otherwise use one another. Files that implement related responsibilities may be directly connected or occupy nearby regions of the dependency graph, making dependency structure a potential source of architectural information [8]. Dependency types can also describe different forms of interaction and may not contribute equally to architectural similarity [78].

Project organization, represented here by the folder hierarchy, provides a second signal. Folder hierarchies record how developers organize implementation artifacts and are commonly consulted when producing reference decompo-

sitions [20]. However, folders are implementation-level containers rather than architectural modules. A module can span several folders, and a folder can contain files belonging to several architectural concerns. Folder location, therefore, carries useful but imperfect evidence.

Source-code content provides a third signal. File names, identifiers, and comments encode domain-specific knowledge about the concepts and responsibilities a file implements [87]. Files from the same module may therefore share terminology, and lexical content has been used to group files by semantic similarity in prior recovery work [22, 12]. However, vocabulary can also recur across modules, particularly in shared abstractions and utility code. Lexical similarity is therefore informative, but may be insufficient on its own. These signals may provide complementary evidence, but their combination is not straightforward. A modular decomposition is expected to have high *cohesion* within modules and low *coupling* between them [54], and each signal is only an indirect indicator of these properties. A dependency can connect files within the same module or cross a module boundary. A folder can approximate a module or reflect only organizational conventions. Similar identifiers can indicate shared responsibility or merely the use of a common domain vocabulary. The strength of each signal can therefore vary across systems and modules.

Prior approaches, reviewed in Section 2.1, include dependency-based approaches [54, 80, 51, 8], lexical approaches [12, 22], and lexical models that encode dependencies as tokens [64]. Approaches that rely on a single source can leave potentially complementary evidence unused. To incorporate such evidence, several approaches combine multiple signals through predefined similarity functions or weighting schemes [87, 67, 53].

Graph neural networks (GNNs) offer a way to learn from these signals jointly by mapping the dependency graph, along with the attributes attached to its nodes and edges, into a common vector space. This does not by itself determine what is learned. The result depends on how files are represented, how the graph is constructed, how information is shared between connected files, and what objective drives learning. When several of these change at once, a difference in performance cannot be attributed to any one signal. Two further questions remain open: whether an added signal contributes beyond what the representation already encodes, and whether sharing information between connected files helps or instead mixes evidence from different modules when their relations cross architectural boundaries [89].

The problem addressed by this research is therefore not only how to combine implementation signals. It is to examine what architectural information each signal carries, how that information should be represented, how the signals interact when combined, and under which structural conditions propagation along the graph is beneficial. Answering this requires varying input representation, graph construction, relation modeling, and message passing separately, and examining how they interact.

1.3 Research Aim and Scope

This thesis investigates how graph-based representation learning can support software architecture recovery and code-to-architecture mapping. It examines the architectural information available in static implementation signals and how their usefulness changes with the input representation, graph structure, and message-passing mechanism.

This thesis aims to determine how graph-based representation learning can be designed for recovery and mapping, what architectural information the underlying implementation signals carry, and how the contribution of file representations changes when folder organization and member-level detail are added to the graph and dependency neighbors are made available through message passing. Both tasks are studied under a common perspective. Files and other implementation artifacts are represented as nodes, and their relations as edges; artifact-level information is provided as node attributes, and a graph representation-learning encoder produces file representations used to assign files to architectural modules.

The investigation starts with file-level dependency graphs in which the signals are supplied as node attributes, examines the signals individually, and then adds two further node types, with member nodes used only for mapping. A folder node represents a directory and is connected to its parent folder and to the files it contains. A member node aggregates all program elements of a single source file, such as its methods and fields, and is connected to that file. Adding these nodes only after the signals have been examined individually makes it possible to separate the effect of the initial file representation from the effects of graph structure and of the way information is shared between connected files. The aim is not to identify one graph model or representation that is uniformly preferable, but to examine under which conditions different signals and learning mechanisms are useful.

The thesis assigns source-code files to architectural modules and evaluates the assignments against a published reference decomposition. The file is the unit of analysis throughout. Member-level information may serve as evidence, but assignments are not made at the class, method, or subsystem level. The thesis spans unsupervised recovery and supervised mapping with limited labeled data and considers static implementation artifacts: dependencies, folder organization, and source code content. Other sources of evidence, such as commit co-change history and dynamic dependencies observed at runtime, are not studied. Chapter 3 states the formal problem and the objectives.

1.4 Thesis Contributions

The thesis makes three research contributions, corresponding to the objectives in Chapter 3, and one methodological contribution. The contributions are based on

Papers I–V, introduced in Chapter 4, and supported by the findings presented in Chapters 6 and 7.

1. **Graph-based representation learning for recovery and mapping.** The thesis develops and evaluates graph-based methods for unsupervised SAR (Papers I, II, and IV) and for C2A mapping from a small labeled set (Papers III and V). It examines alternative learning objectives (Papers I and II), encoder architectures (Papers II–V), and dependency representations (Papers III and IV), and compares the methods with established approaches (Papers I–IV).

2. **Empirical evaluation of representations of implementation signals.** The thesis examines the architectural information carried by dependency structure and by lexical and contextual source-code representations, assessed through their use in recovery and mapping. It evaluates dependency-based recovery (Paper IV) and compares initial file representations for mapping without additional information from message passing (Paper V). It also investigates whether dependency information can adapt contextual code representations to better reflect architectural proximity (Paper V).

3. **Analysis of how graph structure interacts with implementation signals.** The thesis examines whether folder organization (Papers IV and V), member-level detail (Paper V), edge-level attributes (Paper I), and dependency-based message passing (Papers III and V) contribute beyond the initial file representation. The analyses cover recovery and mapping and, where possible, hold the file representation fixed to distinguish the contribution of added structure from that of the representation.

4. **Controlled cross-study evaluation.** The thesis extends the evaluation of recovery and mapping methods to common benchmarks under controlled conditions (Section 5.6). Recovery methods from Papers I, II, and IV are compared on fourteen systems using a common clustering algorithm, reference-decomposition agreement, and dependency cohesion. Mapping methods from Papers III and V are compared on ten systems using matched input representations, initial labeled sets, and training procedures. Each method is run 100 times per system, and results are reported per system with significance tests and effect sizes.

Together, the first three contributions treat model architecture, input representation, graph construction, and system structure as connected design decisions rather than independent sources of improvement.

1.5 Thesis Outline

The remainder of the thesis is organized as follows. Chapter 2 reviews prior work on software architecture recovery and C2A mapping, introduces the structural, organizational, lexical, and contextual representations used in the thesis,

and presents the relevant graph representation learning and evaluation concepts. Chapter 3 defines the research problem, objectives, and questions.

Chapter 4 relates the included studies to research objectives and summarizes their approaches, benchmark scope, main baselines, and principal findings. Chapter 5 presents the shared empirical and constructive research design, study progression, investigated design space, benchmark systems, and evaluation protocol, including the controlled cross-study evaluation.

Chapter 6 synthesizes the findings of the included studies and the cross-study evaluation by research objective. Chapter 7 discusses the findings across studies and addresses their implications and the limitations of the work. Chapter 8 summarizes the main findings and contributions and identifies directions for further research.

Chapter 2

Background

Contents

2.1 Prior Approaches 7
2.1.1 Software Architecture Recovery 9
2.1.2 Code-to-Architecture Mapping 10
2.1.3 Limitations Across Recovery and Mapping 11
2.2 Representing Artifact Content 11
2.2.1 Lexical Representations 12
2.2.2 Contextual Representations 12
2.3 Graph Representation Learning 12
2.3.1 Random-Walk Embeddings 13
2.3.2 Graph Neural Networks 13
2.3.3 Graph Properties and Learning 14
2.3.4 Learning Paradigms 15
2.4 Evaluation and Ground Truth 16

This chapter presents the background needed to frame the research problem and the methods used in this thesis. It first reviews prior approaches to architecture recovery and C2A mapping, with attention to how they represent and combine implementation-level evidence. It then introduces artifact representations, graph-based learning methods and relevant graph properties, learning paradigms, and evaluation concepts used in subsequent chapters.

2.1 Prior Approaches

Prior approaches to architecture recovery and C2A mapping draw on implementation signals such as static dependencies, directory layout, and identifiers and comments in the code. Some also use version history or other metadata, which are not studied in this thesis. The approaches differ in how they represent, combine, and learn from these signals. Table 2.1 summarizes representative approaches, their input signals, underlying methods, and main limitations.

Table 2.1: Representative approaches to SAR and C2A mapping. SD: static dependencies; F: folder hierarchy or file paths; T: identifiers, comments, or other code text. † denotes an additional signal not studied in this thesis.

Approach	SD	F	T	Paradigm	Primary limitation
Software Architecture Recovery					
Dependency-based clustering					
ACDC [80]	✓	×	×	Pattern-driven clustering	Over-fragments into many small clusters
Bunch [54]	✓	×	×	Local search (hill climbing variants); optimizes MQ	Hubs and utility classes distort modules; slow on large systems
DAGC [69]	✓	×	×	Global search (genetic algorithm); optimizes MQ	MQ objective poorly matches expert decompositions
FCA [79]	✓	×	×	Fast clustering on the neighborhood degree matrix	Greedy and local; weights all dependencies equally
LIMBO [3]	✓	×	×	Information-bottleneck hierarchical clustering	Preset cluster count; does not scale to large systems
WCA [51]	✓	×	×	Weighted hierarchical clustering	Preset cluster count; low component-level accuracy
Semantic and name-based clustering					
Anquetil et al. [4]	×	×	✓	File-name clustering using common substrings and lexical concepts	Short or cryptic names can hinder concept extraction
ARC [22]	✓	×	✓	LDA-based topic modeling	Shared terminology mixes modules; preset cluster count
ZBR [12]	×	×	✓	TF-IDF weighted lexical zones, hierarchical clustering	Shared terminology mixes modules
Hybrid multi-signal methods					
SADE [67]	✓	×	✓	Community detection on weighted graph	Predefined, manually tuned weights
SARIF [87]	✓	✓	✓	Community detection on weighted graph	Predefined, manually tuned relation-type and signal weights
NSGA-III† [55]	✓	×	✓	Multi-objective search over structure; uses change history	Objectives fix the fusion before recovery
Representation learning and LLMs					
NEGAR [8]	✓	×	×	Node2Vec-based random walks, clustered	One dependency type at a time; no node attributes
Amalfitano et al. [2]	×	×	✓	Generative LLMs with iterative prompting	Weak on class relationships and fine-grained patterns
SemArc [88]	✓	×	✓	LLM code interpretation, anchor-based clustering	Scalability; possible benchmark overlap with pretraining data

Table 2.1 continued from previous page

Approach	SD	F	T	Paradigm	Primary limitation
Code-to-Architecture Mapping					
Rule-based and retrieval-based mapping					
Tzerpos and Holt [81]	✓	×	✓	Orphan adoption by naming and structure criteria	Hard-coded patterns
HuGMe [9]	✓	×	×	Dependency-based attraction functions for semi-automatic mapping	Requires an initial partial mapping
IRAttract LSIAttract [5]	×	×	✓	Information-retrieval based attraction functions	No explicit dependency structure
InMap [77]	×	×	✓	Interactive ranking from module descriptions	No dependency structure
Learned mapping					
NBAttract [64]	✓	×	✓	Semi-supervised Naive Bayes	Dependencies enter as per-file features, not graph structure
Pretrained embeddings and LLMs					
Johansson et al. [38, 39]	×	×	✓	Pretrained embeddings with classifiers	No explicit dependency or folder structure
Rukmono et al. [76]	×	×	✓	GPT-4 mapping to a predefined architecture	Scalability as system size grows

2.1.1 Software Architecture Recovery

Traditional recovery methods treat software artifacts as a dependency graph and recover modules through several related paradigms. A first group formulates recovery as search over a modularization objective. Some methods optimize a single modularity criterion that favors high cohesion within modules and low coupling across them [54, 69], while others use multi-objective or evolutionary search to balance several criteria at once [71]. A second group clusters directly on dependency-derived similarities or information-theoretic criteria [61, 60, 3, 51]. ACDC [80] takes a different approach by building clusters from recurring subsystem patterns. A third group focuses on the clustering procedure itself. The fast clustering algorithm (FCA) [79] speeds up clustering for large systems by working on the neighborhood degree matrix; Mohammadi and Izadkhah [56] build clusters from the neighborhoods of artifacts in the dependency graph, and Pourasghar et al. [70] use relationship depth. Overall, graph-based recovery methods can work well when dependencies reflect architectural structure, but their effectiveness depends on which dependency relations the graph includes and how the method uses them.

Semantic approaches use identifiers and comments instead of, or alongside, dependencies, and can outperform dependency-only methods in some systems [19]. ARC [22] uses latent Dirichlet allocation (LDA) to infer concerns from

source text and combines them with structural information. ZBR and Risi et al. derive semantic similarities using lexical zones and latent semantic indexing (LSI), respectively [12, 74], while Kargar et al. [40] build a semantic dependency graph from lexical cues and optimize it by local search. Anquetil et al. [4] take a related but distinct approach by clustering files according to their names, using common substrings and concepts found in routine names or comments. Non-structural signals of this kind can be noisy in isolation, since reused terminology may span modules.

Hybrid approaches use multiple sources to capture complementary evidence. Some integrate structural and semantic relations before hierarchical clustering [53, 73]. Others incorporate semantic information into search-based modularization, and NSGA-III also uses development change history [37, 55]. Puchala et al. [72] combine several clustering results into a consensus solution, while Yang et al. [86] use semantic cues to guide label propagation over the dependency graph. Naim et al. [59] combine structural and contextual information through coordinated clustering, refining the combination through incremental developer feedback. SADE [67] and SARIF [87] combine signals in weighted dependency graphs and apply community detection. SARIF also incorporates directory similarity. These methods bring information together at different stages, from constructing relations and defining search objectives to guiding or combining clustering results. In most cases, the fusion is specified by a designed procedure, consensus rule, or heuristic weighting rather than learned jointly as a representation of the signals.

Representation learning has also been applied to recovery. NEGAR [8] learns node representations from dependency graphs through random walks and clusters the resulting vectors. It applies this pipeline separately to graphs at different implementation levels and does not incorporate non-structural signals.

Another approach uses large language models to infer architectural structures and patterns from source code. SemArc [88] uses LLMs to identify implementation-level and architectural semantics, drawing on canonical architectural patterns as a knowledge base. It also incorporates explicit and implicit dependencies before applying component-as-anchor clustering. Amalfitano et al. [2] evaluate general-purpose LLMs on identifying implementation-level class diagrams, architectural and design patterns, and architectural styles. Their prompts include a self-reflection step to simulate iterative refinement. ArchAgent [66] combines static analysis and adaptive code segmentation with LLM-based synthesis to recover multi-view architectures from large, cross-repository codebases.

2.1.2 Code-to-Architecture Mapping

Tzerpos and Holt [81] introduced orphan adoption, the incremental assignment of unmapped entities to a documented architecture using naming and structural criteria. Christl et al. [9] built on this with HuGMe, a semi-automatic method that extends a partial mapping through dependency-based attraction functions

and leaves uncertain assignments for the architect to review. Bittencourt et al. [5] later introduced the information-retrieval attraction functions IRAttract and LSIAttract, which use lexical similarity between entities and modules. InMap [77] takes an interactive recommendation approach, ranking candidate mappings from natural-language module descriptions. Like HuGMe, it keeps the architect involved, but through interactive recommendations rather than review of uncertain assignments. Together, these approaches rely on predefined criteria, attraction functions, or retrieval-based ranking rather than a jointly learned representation, the mapping analog of the fusion schemes above.

Learned classifiers provide another approach to mapping. Olsson et al. [64] propose NBAttract, a semi-supervised Naive Bayes mapper that combines lexical and dependency information encoded via Concrete Dependency Abstraction (CDA). Florean et al. [18] compare text classifiers and text representations for the same task. An earlier study [33] represents dependencies directly as edges in homogeneous and dependency-type-aware graphs, in contrast to NBAttract's feature-based encoding. It evaluates graph convolutional network (GCN) variants and uses iterative pseudo-labeling to expand a small initial labeled set.

Some mapping studies use pretrained embeddings as classifier inputs, while others prompt general-purpose LLMs to produce mappings. Johansson et al. [39] evaluate pretrained embeddings alongside traditional representations such as TF-IDF as input to method-level classifiers. In follow-up work, Johansson and Caporuscio relate the performance of embedding-based mapping of functions and files to data properties, most consistently class separability [38]. Rukmono et al. [76] use chain-of-thought prompting with GPT-4 to map implementation units to a predefined reference architecture. Both approaches use lexical or semantic information, while typed dependencies remain implicit in their inputs.

2.1.3 Limitations Across Recovery and Mapping

Across recovery and mapping, many of the reviewed approaches rely on a single information source, combine sources through predefined rules, or use a fixed representation of each source within a learned model. Comparing the contributions of graph structure, directory organization, and source-code content requires varying these choices under matched conditions. The studies comprising this thesis examine how these signals can be represented, combined, and propagated for architecture recovery and C2A mapping.

2.2 Representing Artifact Content and Organization

A software system carries architectural evidence in more than one form. Its files and members are connected by static dependencies, but they also have names, occupy locations in the folder hierarchy that reflect how developers group related

functionality [20], and contain identifiers and comments that encode domain-specific knowledge [87]. In a representation learning framework, dependencies can be represented as graph relations, while the content associated with individual artifacts must be encoded numerically for a learning model to operate on. This section introduces the two families of representation used in the thesis: lexical and contextual.

2.2.1 Lexical Representations

Lexical representations treat folder-path components and source-code identifiers as tokens. The simplest representation is a binary bag-of-words (BoW) vector, in which each dimension indicates whether a token is present in the artifact's folder path or source code. Because it allocates one dimension to every vocabulary token, the representation becomes high-dimensional and sparse as the vocabulary grows. Word2Vec (W2V) [52] instead learns a dense vector for each source-code identifier from a corpus of source-code tokens using a skip-gram model.

In skip-gram, each token is used to predict tokens occurring within a fixed context window. The vectors are trained to assign higher scores to observed token–context pairs than to randomly sampled pairs through negative sampling. As a result, tokens that occur in similar contexts receive similar vectors. An artifact's Word2Vec representation is obtained by averaging the vectors of its source-code identifiers, producing a compact, fixed-dimensional encoding.

2.2.2 Contextual Representations

Contextual representations encode tokens in the context of the surrounding code rather than as an unordered collection of identifiers. The same identifier can therefore receive different representations in different contexts, whereas a trained Word2Vec model assigns a fixed vector to each token. Transformer-based encoders use attention to combine information from tokens within the input sequence, allowing each token's representation to reflect how it is used in the code. Through pretraining on large corpora, these encoders learn patterns that can support different code analysis tasks. UniXcoder [27] is one such model, producing contextual token representations that can be pooled into a single vector for a code fragment.

A pretrained code encoder can be used without further training or adapted through fine-tuning. Fine-tuning adjusts its representations using a training objective and examples relevant to the target setting, which can include dependency information.

2.3 Graph-Based Representation Learning

Content encoders process artifacts individually, although their representations can capture dependency information introduced during fine-tuning. Graph-

based representation learning uses the dependencies and containment relations between artifacts explicitly to learn node representations. The approaches used in this thesis fall into two groups: random-walk embeddings, which derive vectors directly from graph neighborhoods, and graph neural networks, which propagate and transform information along graph edges.

2.3.1 Random-Walk Embeddings

Random-walk methods learn node embeddings in two steps. First, they construct a corpus by sampling random walks over the graph, where each walk is a sequence of nodes. Node2Vec (N2V) [26] samples walks over a homogeneous graph, which in this thesis is unweighted and undirected, whereas MetaPath2Vec (M2V) [14] extends this to heterogeneous graphs by constraining walks to follow a predefined sequence of node and relation types. Second, a skip-gram model (see Section 2.2.1) is trained over this corpus, treating the nodes within a fixed context window of a target node as its observed context, so that nodes with similar walk contexts receive similar embeddings. Walk length, context window, and the number of walks per node determine which nodes are treated as similar. These methods learn from walk co-occurrence alone and therefore do not jointly incorporate artifact content.

2.3.2 Graph Neural Networks

Graph neural networks (GNNs) provide a common framework for learning from artifact-level information and graph structure, producing node representations shaped by both. A GNN encoder maps each node, together with its initial representation and graph neighborhood, to a vector through message passing [24]. At each layer, a node aggregates the current representations of its neighbors, combines them with its own, and applies a learned transformation. GNN architectures differ primarily in how they transform and aggregate these messages, and this thesis uses three variants.

The graph convolutional network [43] aggregates neighboring representations through degree-normalized message passing, which helps prevent high-degree nodes from dominating solely because they have many dependencies. The graph attention network (GAT) [82] instead learns attention weights over neighbors, allowing their contributions to differ according to their representations. Both encoders can be applied either to a graph in which dependency types are collapsed or to a multi-relational graph with relation-specific parameters. When the graph contains multiple node types and typed relations, this thesis also uses the heterogeneous graph transformer (HGT) [29], which parameterizes attention by meta-relation. Thus, the contribution of a neighbor depends jointly on the source-node type, target-node type, and edge type.

2.3.3 Graph Properties and Learning

Two properties are particularly relevant to how well graph-based encoders can support downstream tasks: homophily and structural reach. A graph is homophilic with respect to a labeling when connected nodes tend to share a label. Under homophily, neighborhood aggregation reinforces label-consistent information; under heterophily, it mixes information from different labels and may obscure distinctions needed by the downstream task [89]. Formally, let $G = (V, E)$ be a graph in which each node $v \in V$ has a label y_v. Homophily $\mathcal{H}$ is the proportion of edges whose two endpoints have the same label,

$$\mathcal{H} = \frac{|\{e = (u, v) \in E : y_u = y_v\}|}{|E|}, \tag{2.1}$$

so that $\mathcal{H} = 0$ means every edge connects nodes with different labels and $\mathcal{H} = 1$ means every edge connects nodes with the same label [89]. For software systems, the nodes are files, the edges are static dependencies between them merged across dependency types, and the label is the ground-truth module. In this setting, homophily relates to cohesion and coupling by measuring the proportion of dependencies that remain within modules rather than cross module boundaries. It provides an overall measure of this balance, but does not distinguish how those dependencies are distributed across individual modules.

Since $\mathcal{H}$ is computed from the reference decomposition, it can be determined only once such a decomposition exists. The concentration of dependencies within modules varies across systems, and so does the usefulness of dependency-driven random walks and message passing.

The second property is the structural reach of the encoder, which we use to mean the set of nodes that can contribute information to a node representation. For an L-layer message-passing encoder, this is the node's L-hop neighborhood. Reach describes which nodes can contribute by construction, rather than how much they contribute in practice. Increasing depth extends this reach, but repeated aggregation can make node representations progressively less distinguishable, an effect known as over-smoothing [49]. GNN encoders are therefore commonly kept shallow.

With a shallow encoder, access to more distant architectural information depends on how the input graph connects the nodes. Changing these connections can make that information reachable within fewer message-passing steps. Random-walk embeddings can capture longer-range structural context through sampled walks, although they do not propagate artifact-level representations along the graph.

Two further techniques alter the structural information available to a shallow encoder without increasing its number of message-passing layers. Laplacian positional encoding (LPE) augments each node representation with leading eigenvectors of the graph Laplacian, giving the encoder information about the node's structural position before message passing begins [15]. Graph diffusion convolution (GDC) [23] rewires the input graph using personalized PageRank

(PPR), connecting diffusion-close nodes directly and assigning the resulting edges diffusion-derived weights. This makes multi-hop structural proximity available within a fixed number of message-passing layers.

2.3.4 Learning Paradigms

The encoder architecture determines how information propagates through the graph, but not what drives learning. That role belongs to the training objective, which depends on the supervision available. The studies in this thesis use two learning settings that reflect the recovery and mapping tasks.

Architecture recovery. No module labels are available during recovery, so the studies use self-supervised learning, in which the model learns from an auxiliary signal derived from the graph itself. These methods construct positive and negative training examples from the graph, with the examples differing between methods. Binary cross-entropy (BCE) measures how well the model distinguishes positive from negative examples. For a single example, it is defined as:

$$\mathcal{L}_{\mathrm{BCE}} = -\left[y \log \hat{y} + (1-y) \log(1-\hat{y})\right], \tag{2.2}$$

where $y \in \{0, 1\}$ is the target and $\hat{y}$ is the predicted probability.

Contrastive learning trains representations by contrasting positive and negative pairs. Deep Graph Infomax (DGI) [83] applies this principle to graphs by treating node–summary pairs in the original graph as positive and pairs constructed from a corrupted view as negative. Graph autoencoders (GAE) [44] instead learn node representations by reconstructing observed graph edges, treating observed edges as positives and sampled non-edges as negatives. Their variational form (VGAE) [44] models each node's latent representation as a Gaussian distribution rather than a single vector and adds a Kullback–Leibler (KL) divergence term that penalizes deviations from a standard normal prior. Random-walk methods learn without module labels through the skip-gram objective described in Section 2.3.1.

The learned embeddings are then partitioned with a clustering algorithm, typically agglomerative hierarchical clustering [58], for architecture recovery. This clustering step is unsupervised: it derives module groups from similarity between embeddings. The number of clusters is either selected automatically from a predefined search range or fixed in advance.

C2A mapping. Mapping starts from known module assignments for a limited number of files, so the studies train a classifier on these labeled files with supervised learning. Because the module set is known and fixed, the classifier assigns each file to one of the K predefined modules through a softmax layer, which yields a probability distribution over the modules, and the highest softmax probability of a prediction is its confidence. It is trained with categorical cross-entropy (CCE), $-\sum_{k=1}^{K} y_k \log \hat{y}_k$, which for a one-hot target is the negative log-probability assigned to the true module.

Unlabeled files can also contribute through iterative self-training [47], which Paper III applies on top of supervised training. The classifier assigns pseudo-labels to unlabeled files whose confidence exceeds a predefined threshold, retrains on the labeled and pseudo-labeled files, and repeats over successive rounds until no prediction meets the criterion or a maximum number of rounds is reached.

2.4 Evaluation and Ground Truth

The studies in this thesis evaluate their outputs against reference decompositions from published, expert-validated benchmarks, supporting comparison with prior work. Recovery produces a partition whose cluster labels have no predefined meaning, whereas mapping assigns files to known architectural modules. Recovery is therefore evaluated through partition-agreement metrics and mapping through conventional classification metrics. This section introduces these measures and discusses how reference decompositions should be understood as ground truth. The provenance of the specific reference mappings is discussed in Chapter 5.

A reference decomposition provides an established basis for comparison, but it need not be the only reasonable architectural interpretation of a system. Experts may disagree on architectural boundaries and design intent [20], so a recovered decomposition can differ from the reference without necessarily being wrong.

Complementary metrics assess different aspects of a recovered decomposition. MoJoFM, A2A, and $C2C_{cvg}$ measure alignment with a reference decomposition. The adjusted Rand index (ARI) [30] measures partition agreement while correcting for chance. TurboMQ provides a complementary, reference-independent measure of modularization quality based on the dependency graph.

MoJoFM [84] is widely used in architecture recovery. It counts the minimum number of move and join operations needed to transform the recovered partition into the reference partition, normalized by the maximum possible number of such operations. Because its join operations are comparatively cheap, MoJoFM is less sensitive to over-fragmentation: many small clusters can still score well when they broadly align with the reference. A2A [46] extends MoJoFM with costs for adding, removing, and moving entities and clusters. It can assign a high score to two architectures that are structurally quite different, overestimating their similarity [50].

$C2C_{cvg}$ [21] measures the percentage of reference modules that are sufficiently covered by a recovered module, where coverage is judged against a threshold on the overlap between the two; conventional thresholds of 0.10, 0.33, and 0.50 correspond to weak, moderate, and majority matches. This makes it sensitive to fragmentation: a reference module split across several recovered modules does not count as matched if none of them meets the chosen overlap threshold, even where individual file assignments look reasonable under a different metric. ARI ranges from -1 to 1, where 0 is the agreement expected by chance.

Unlike the metrics above, which measure alignment with the reference decomposition, TurboMQ [50] measures modularity: it rewards intra-cluster cohesion and penalizes inter-cluster coupling, computed directly on the dependency graph without reference to any external partition. TurboMQ can favor decompositions with many clusters. Normalized TurboMQ (TMQ_{norm}) divides the TurboMQ score by the number of recovered clusters to reduce this bias.

The reference-agreement metrics described above assess how closely a recovered decomposition matches an expert-produced reference, whereas TurboMQ evaluates its dependency structure. Expert-produced references can reflect responsibilities and project organization identified through documentation, source code, and knowledge of the system. These boundaries need not coincide with groups of densely connected files, since shared utilities and libraries can create dependencies across architectural modules [20]. A reference decomposition therefore need not maximize dependency modularity, and a recovered decomposition can score well on one kind of measure without scoring well on the other.

For mapping, the evaluation is a per-file classification into known modules, and the standard metrics apply directly. Precision, recall, and their harmonic mean F1 are reported per module and aggregated at the macro and micro level. The macro-average treats every module equally and therefore surfaces performance on small modules that would be hidden by a micro-average dominated by large ones. Reporting both reveals differences in performance across modules of different sizes.

Chapter 3

Problem Formulation and Research Objectives

Contents

3.1 Problem Formulation 19
3.2 Research Objectives 20
3.2.1 Representation Learning for Recovery and Mapping 21
3.2.2 Representations of Implementation Signals 21
3.2.3 Implementation Signals and Graph Structure 22

This chapter formalizes the research problem addressed in this thesis and defines the objectives for investigating it. It first presents software architecture recovery and C2A mapping as related file-to-module assignment problems within a common graph-based representation-learning framework. It then states the research objectives and associated research questions.

3.1 Problem Formulation

Architecture recovery and C2A mapping both assign source-code files to architectural modules, but differ in the architectural information available. This thesis approaches recovery by learning file representations without module labels and clustering them into modules. In C2A mapping, the target modules and a subset of file assignments are known, and the task is to predict the remaining assignments. Both tasks are formulated here using a common graph representation.

Despite this difference in supervision, both tasks draw on dependencies, project organization, and source-code content as discussed in Section 1.1. This thesis uses graph-based representation learning to combine these signals as evidence of module membership.

Accordingly, a software system is represented as a graph:

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A}, \mathcal{R}, \mathrm{X}), \tag{3.1}$$

where $\mathcal{V}$ and $\mathcal{E}$ are the node and edge sets, $\mathcal{A}$ and $\mathcal{R}$ are the node and relation types, and X denotes node attributes. The maps $\tau : \mathcal{V} \to \mathcal{A}$ and $\phi : \mathcal{E} \to \mathcal{R}$

assign types to nodes and edges. Structure-only settings use no lexical or folder attributes. File nodes form the subset $\mathcal{V}_{\texttt{file}} \subseteq \mathcal{V}$.

The graph is homogeneous when $|\mathcal{A}| = |\mathcal{R}| = 1$, and heterogeneous otherwise. Its simplest form is a software dependency graph (SDG), containing only file nodes and a single dependency relation type. Preserving dependency types increases $|\mathcal{R}|$, while adding folder and member nodes increases both $|\mathcal{A}|$ and $|\mathcal{R}|$. Folder and member nodes provide context for learning file representations; only file nodes are assigned to architectural modules.

Formally, for recovery, we learn an encoder f_θ in a self-supervised manner:

$$f_\theta : \mathcal{G} \longrightarrow \mathrm{Z} \in \mathbb{R}^{|\mathcal{V}_{\texttt{file}}| \times d}, \tag{3.2}$$

that maps each source code file $i \in \mathcal{V}_{\texttt{file}}$ to a d-dimensional embedding $\mathrm{z}_i \in \mathbb{R}^d$, where θ denotes the learnable parameters. The resulting file embeddings are clustered into a recovered architectural decomposition $\hat{\mathcal{C}}$, which partitions the source code files into m disjoint modules:

$$\hat{\mathcal{C}} = \{C_1, \ldots, C_m\}. \tag{3.3}$$

In C2A mapping, a set of target modules $\mathcal{C} = \{C_1, \ldots, C_m\}$ is known, together with a labeled subset of files $\mathcal{L} \subseteq \mathcal{V}_{\texttt{file}}$ and their module assignments $Y_{\mathcal{L}}$. The task is to learn:

$$f_\theta : (\mathcal{G},\ \mathcal{L},\ Y_{\mathcal{L}}) \longrightarrow \hat{Y}_{\mathcal{U}}, \tag{3.4}$$

for the unlabeled files $\mathcal{U} = \mathcal{V}_{\texttt{file}} \setminus \mathcal{L}$. The model is trained on the labeled files, and the supervision is limited to this subset. Training may additionally use the model's confident predictions on unlabeled files as pseudo-labels. The setting is transductive: all files, labeled and unlabeled, are part of $\mathcal{G}$ during training.

The two problems use the same implementation signals, but differ in how representations are learned and how their contribution can be assessed. In recovery, module labels are unavailable, so representations are shaped by the graph construction, the input representations, and a self-supervised objective. The final decomposition also depends on the clustering procedure and granularity. In C2A mapping, known file-to-module assignments provide task-specific supervision, allowing matched configurations to be compared using per-file classification metrics. Mapping thus provides a complementary setting for examining whether individual signals help predict an existing decomposition. However, its results depend on the number of initially labeled files, their coverage and distribution across modules, and the training procedure. A signal's usefulness or ranking in one task therefore does not necessarily carry over to the other.

3.2 Research Objectives

The research aim is operationalized through three objectives that progress from joint representation learning to examining individual signals and file representations, and then to their interaction within richer graphs. Objective 1 designs,

compares, and benchmarks graph-based methods for recovery and mapping by combining dependency, organizational, and lexical signals. This examines their joint usefulness, but not their individual contributions. Objective 2 therefore examines the architectural information these signals and representations carry, including how source-code content should be represented. Objective 3 investigates how their usefulness changes as structural context is added and whether findings from Objectives 1 and 2 remain valid.

3.2.1 Representation Learning for Architecture Recovery and C2A Mapping

Objective 1

To design, compare, and benchmark graph-based representation learning methods for architecture recovery and code-to-architecture mapping.

RQ1.1 How can implementation-level signals be combined into a joint representation for architecture recovery and mapping?

RQ1.2 How does modeling dependency types as separate relations compare with merging them into a single relation?

RQ1.3 How sensitive is the learned representation to encoder architecture, learning objective, and model depth?

This objective is addressed through comparisons across recovery and mapping studies on file-level graphs. Most comparisons jointly encode folder, lexical, and dependency signals within one encoder. The comparisons cover GCN, GAT, and HGT encoders and merged versus relation-specific dependencies. Recovery experiments compare contrastive and reconstruction objectives using the same encoder, inputs, and benchmark systems. Mapping studies also vary model depth; one study and its cross-study extension use inputs without folder signals.

3.2.2 Representations of Implementation Signals

Objective 2

To determine what architectural information dependency structure and individual file representations provide, and how source-code content should be represented.

RQ2.1 How far can dependency structure alone support architecture recovery, in agreement with the reference and in dependency cohesion?

RQ2.2 How does dependency-guided adaptation of a contextual code representation affect the architectural signal it carries?

RQ2.3 How do structural, lexical, and contextual file representations compare in mapping without message passing?

This objective examines the architectural information provided by representations of individual implementation signals. In recovery, it uses dependency structure alone on the homogeneous SDG, without lexical or folder information, and examines how much structural context is needed across systems. In mapping, it compares file representations without adding folder organization, member-level detail, or SDG-based message passing. These representations include Word2Vec, zero-shot UniXcoder, Node2Vec, and UniXcoder fine-tuned through SDG-guided contrastive learning without module labels. The latter two incorporate dependency information during representation learning. A multilayer perceptron (MLP) classifier uses each file's representation independently, so it can access dependency information only if that information is already encoded in the representation.

3.2.3 Interaction Between Implementation Signals and Graph Structure

Objective 3

To determine how the contribution of file representations changes with dependency-based message passing, folder organization, and member-level graph detail, and whether findings from Objectives 1 and 2 remain valid.

RQ3.1 How does the usefulness of message passing over dependency neighbors vary across predictors, aggregation directions, and file representations?

RQ3.2 How does adding folder organization to the graph change what the file representations contribute to recovery and mapping?

RQ3.3 How does performance differ with and without member-level nodes and edge-level interaction details?

This objective examines how additional graph structure changes the contribution of file representations. In recovery, folder nodes and containment meta-relations are added to the SDG and evaluated through message passing or random walks; edge attributes describing the member-level interactions behind each dependency, including their type and frequency, are examined separately. In mapping, an MLP and graph-based neural predictors are compared, and each predictor is evaluated with and without access to dependency neighbors while holding file representations fixed. Aggregation direction is also varied. Folder and member nodes are evaluated with Word2Vec, Node2Vec, and fine-tuned UniXcoder. Member nodes also change how lexical and contextual content enters the model, and a rule based on the directory hierarchy and initial labels provides a reference.

Chapter 4

Overview of the Studies

Contents

4.1 Relationship Between Objectives and Studies 23
4.2 Overview of the Included Studies 23
4.2.1 Paper I: Software Architecture Recovery 24
4.2.2 Paper II: Software Architecture Recovery 25
4.2.3 Paper III: Code-to-Architecture Mapping 25
4.2.4 Paper IV: Software Architecture Recovery 25
4.2.5 Paper V: Code-to-Architecture Mapping 26

This chapter introduces the five studies that constitute this thesis and explains how they address the research objectives. It first maps the studies to the objectives, then summarizes what each study built, its benchmark scope, its main baselines, and its principal individual findings.

4.1 Relationship Between Objectives and Studies

The objectives are addressed across complementary studies rather than by a single experimental design applied identically to both recovery and mapping. Figure 4.1 maps each of the five studies to the objectives it addresses, distinguishing studies that directly address an objective's research questions from studies that partially inform it through a contrasting result or an earlier comparison with a narrower scope.

4.2 Overview of the Included Studies

The studies progress from relation-aware graph representation learning for architecture recovery to dependency-based message passing for C2A mapping, then examine structure-only recovery and systematically compare and combine implementation signals for mapping.

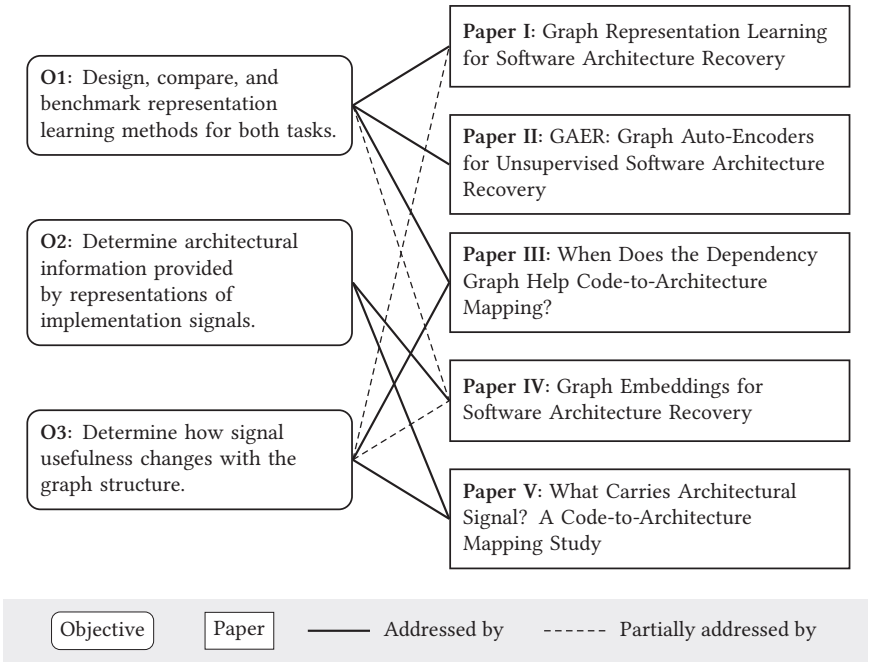


Figure 4.1: Mapping of research objectives to the five studies. Solid lines indicate direct coverage of the research questions; dashed lines indicate partial coverage through contrasting results or narrower comparisons.

4.2.1 Paper I. Graph Representation Learning for Software Architecture Recovery

The first study [35] establishes the initial feasibility of learning architectural representations from a multi-relational software dependency graph for architecture recovery. Each dependency type is modeled as a separate directed relation among source-code files, while node features combine folder-hierarchy tokens with code identifiers not involved in inter-file dependencies. Relation-specific GAT encoders produce file representations that are trained without architectural labels using a contrastive objective and subsequently clustered to recover modules. The study also examines whether member-level interaction attributes add architectural information beyond the presence of a dependency.

The study evaluates the proposed approach on four systems against SARIF [87], a hand-tuned information-fusion baseline that combines dependency, textual, and directory signals through manually designed similarity functions. The proposed approach reaches higher MoJoFM than SARIF on all four systems.

4.2.2 Paper II. GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery

The second study [32] extends the investigation of Paper I to a reconstruction-based learning approach and a broader empirical setting. GAER retains the multi-relational dependency graph but replaces contrastive learning with a graph autoencoder trained to reconstruct observed dependencies. Code identifiers are now embedded with Word2Vec rather than encoded as binary vectors, while directory tokens remain binary-encoded. The study compares GCN and GAT encoders before clustering the learned representations into architectural modules.

The study evaluates ten benchmark systems against eight established dependency-based, semantic, structural representation learning, and hybrid recovery approaches. GAER achieves higher or comparable ground-truth alignment across most systems for MoJoFM, A2A, $C2C_{cvg}$, and ARI.

4.2.3 Paper III. When Does the Dependency Graph Help Code-to-Architecture Mapping?

The third study [36] is an extended version of an earlier mapping paper [33]. It shifts from unsupervised architecture recovery to C2A mapping, in which the target modules are known, and a limited set of files is initially labeled. It reuses the graph construction and node features of Paper II, expanding the labeled set into a full mapping via iterative self-training. The study compares GCN and GAT models with an MLP that uses the same file features, classifier, and self-training procedure. It further examines the effect of preserving dependency types, message-passing depth, and aggregation direction.

Across ten benchmark systems, all learned approaches outperform the Naive Bayes mapping baseline [64]. The gain of GAT and GCN over the MLP varies by system and is largest on Chromium, the largest system.

4.2.4 Paper IV. Graph Embeddings for Software Architecture Recovery

The fourth study returns to architecture recovery but isolates the value of structural and organizational information by removing lexical file features [34]. Unlike the first three studies, it learns representations from graph structure alone. The study first evaluates recovery from a homogeneous dependency graph using Node2Vec, a GAT encoder trained within a variational graph autoencoder, and the same encoder applied to a graph rewired by PPR diffusion (GAT-DC). It then extends the graph with folder nodes and containment relations, comparing the dependency structure alone against the dependency structure combined with developer-defined folder organization, using MetaPath2Vec, a heterogeneous GAT (HGAT), and a variant that preserves dependency types rather than merging them (HGAT-TD).

The study evaluates eleven systems against four classical dependency-only recovery baselines. The dependency-only GAT-DC and Node2Vec outperform these baselines on most systems and metrics.

4.2.5 Paper V. What Carries Architectural Signal? A Code-to-Architecture Mapping Study

The fifth study examines which implementation signals contribute to C2A mapping and how their usefulness changes with graph structure and message passing [31]. It compares structural Node2Vec, lexical Word2Vec, and contextual UniXcoder file representations, with UniXcoder evaluated before and after dependency-guided contrastive fine-tuning. Each representation is first evaluated using an MLP. Node2Vec, Word2Vec, and fine-tuned UniXcoder then initialize HGT on four graph configurations: files only, files with folders, files with members, and the full folder-file-member graph, with dependencies merged into a single relation type. Each member node summarizes the program members of one file. A GAT and an MLP that receives the mean of its dependency neighbors' vectors are evaluated on the file-level dependency graph as further predictors, with dependency neighbors enabled or disabled and aggregated in one or both directions. A model that supplies folder-path tokens directly in the file vector is compared with folder nodes, and HGT is also evaluated with two layers.

The study uses the ten systems and labeled sets of Paper III. The MLP is the baseline for the graph configurations, and a directory-hierarchy rule is the reference for folder-augmented configurations, whether through folder nodes or folder-path tokens supplied in the file vector. The rule assigns an unlabeled file the most common module among the labeled files in its directory, backing off to progressively larger enclosing subtrees when the directory has no labeled file associated with it.

Chapter 5

Research Method

Contents

5.1 Research Design 27
5.1.1 Design Space 28
5.1.2 Research Trajectory 28
5.2 Graph and Signal Representation 30
5.2.1 Graph Construction 30
5.2.2 Node Feature Initialization 31
5.2.3 Edge Attributes 32
5.3 Learning Settings 32
5.3.1 Controlled-Comparison Logic 34
5.4 Benchmark Systems 35
5.5 Evaluation Protocol 36
5.5.1 Labeled Training Sets 36
5.5.2 Statistical Analysis 37
5.6 Cross-Study Evaluation Design 37
5.6.1 Cross-Study Correlation with System Properties 38

This chapter describes the research method used to investigate architecture recovery and C2A mapping. It defines the research design and the graph-based representation-learning design space, then describes the input signals, learning settings, and controlled-comparison logic used across the studies. It also presents the benchmark systems and evaluation protocol, including the sampling of labeled files and statistical analysis. Finally, it describes the additional cross-study evaluation design.

5.1 Research Design

This thesis combines method development with empirical evaluation. Each study develops and evaluates a graph-based representation learning method for architecture recovery or C2A mapping. The studies investigate what architectural information is carried by implementation signals, including dependency structure, project organization, and code content, and the system conditions under which

these signals support architectural analysis. Comparisons with established baselines assess whether the resulting methods are competitive, whereas controlled comparisons between graph configurations, input representations, and learning mechanisms examine the contribution of each.

5.1.1 Design Space

Architecture recovery and mapping are studied through a common graph-based representation-learning framework. The choices investigated across the thesis define a design space with four dimensions: graph topology, relation modeling, input representation, and learning mechanism, summarized in Figure 5.1. These dimensions are analytically distinct but not always experimentally independent: adding a node type usually requires a new relation type to connect it to the rest of the graph.

Graph topology determines which software artifacts are represented as nodes. The simplest configuration contains only file nodes. Richer variants add folder nodes, one per directory, member nodes that aggregate the program elements contained in each file, or include both folder and member nodes together. Relation modeling determines how these nodes are connected. Static dependencies may be merged into a single relation or preserved as typed relations such as *call*, *import*, and *inheritance*. Graphs containing additional artifact types also include organizational and containment meta-relations, such as folder-to-folder, folder-to-file, and file-to-member relations.

Input representation determines what information is available to the nodes before message passing. Nodes may start with learnable identity embeddings or with features derived from dependency structure, folder organization, or code content. These features range from token-based representations and structural encoding to pretrained contextual code embeddings.

The learning mechanism determines how these inputs and graph relations are used to learn file representations. The studies compare random-walk methods and graph neural networks, with learning objectives suited to recovery or mapping.

5.1.2 Research Trajectory

The studies progressively examine how dependency structure, project organization, and code content contribute to recovery and mapping. Each study addresses a question left open by earlier work and informs the choices investigated in the next. Together, they cover an expanding range of graph configurations, input representations, and learning mechanisms.

The prior approach, NEGAR [8], evaluates dependency types through separate pipelines and uses neither folder organization nor lexical information, leaving open whether typed dependencies can be modeled jointly and whether organizational and lexical signals provide complementary evidence for recovery. Papers I and II take a first step on this question with a multi-relational file-level

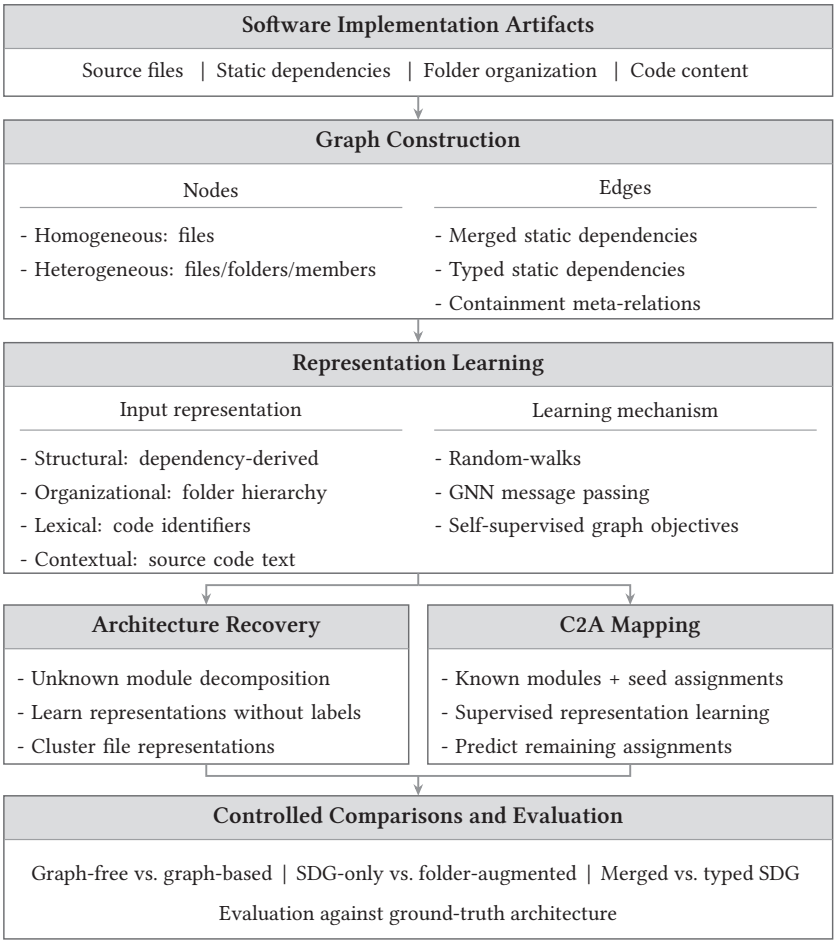


Figure 5.1: Design space studied across the thesis: graph configurations ranging from the SDG to heterogeneous folder-file-member graphs, initialized from structural, organizational, lexical, or contextual signals, and evaluated under unsupervised recovery and supervised mapping through controlled comparisons of relation modeling, graph structure, input representation, and learning mechanism.

graph. Both fuse folder-path and code-identifier tokens as node features, so the contribution of each source cannot yet be separated from the others.

Paper III keeps the graph and feature construction of Paper II, apart from the Word2Vec settings, and moves to a semi-supervised setting, formulating mapping as node classification. It introduces an MLP baseline, trained under the same features and self-training procedure, to compare GNN-based prediction against feature-only prediction.

Paper IV, a further recovery study, returns to dependency structure and project organization, without lexical features. Removing the lexical features allows the contribution of project organization to be assessed without the fused signal of the earlier studies. It compares dependency-only file graphs against folder-augmented heterogeneous graphs under matched conditions, using both random-walk and graph-attention learners, and evaluates ways to extend the effective reach of message passing.

Paper V extends the mapping design by introducing a dependency-guided contextual code representation and by expanding the graph to include folder and member nodes. It compares structural, lexical, and contextual representations with and without message passing, and across graph variants that add or remove folder and member nodes, under one matched supervised mapping setting.

5.2 Graph and Signal Representation

The design space defines the representational choices in the abstract. This section instantiates them concretely, showing which graph each study constructs and which signals it places on that graph's nodes and edges. Table 5.1 summarizes these choices across the five studies.

5.2.1 Graph Construction

Across the five studies, graph topology evolves from file-level dependency graphs to representations that additionally include folder and member nodes. Files remain the targets of recovery or mapping, whereas folder and member nodes provide contextual information through message passing. Static dependencies between code entities are extracted with the multilingual analysis tool Depends [62] and aggregated to the file level. Figure 5.2 illustrates the node-set configurations used in the thesis.

Papers I, II, and III use the file-only configuration, with static dependencies kept as separate typed relations. Paper III further tests a merged-relation variant to determine whether relation-specific message passing contributes to mapping.

Papers IV and V add context through folder nodes, and Paper V also through member nodes. Both merge dependencies into a single relation type. Paper IV compares the file-only graph with a folder-augmented configuration to isolate the contribution of project organization, with a typed-dependency variant as an ablation within the folder-augmented graph. Paper V adds summarized member nodes, each representing the program elements contained in one file, and evaluates four graph variants: file only, file with folders, file with members, and the complete file–folder–member graph. In the complete graph, dependencies keep their file- or member-level endpoints; when member nodes are removed, member endpoints are projected onto their containing files.

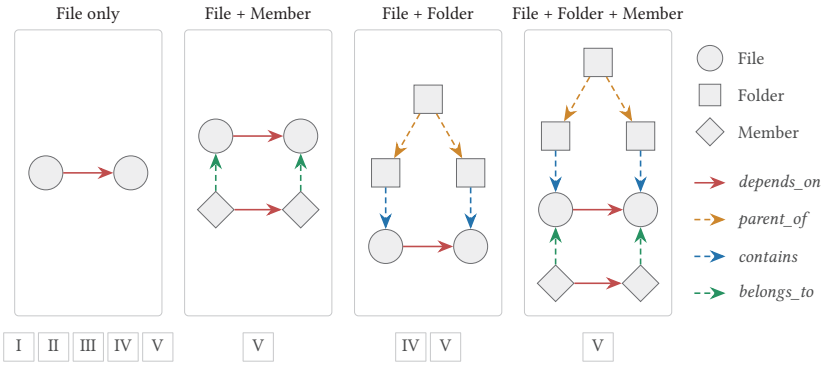


Figure 5.2: Node-set configurations used across the thesis, mapped to the studies that report results on each: file only (Papers I–V), file with folders (Papers IV and V), file with members, and file with folders and members (Paper V). Files are the prediction targets in all configurations. Solid *depends_on* edges are static software dependencies, such as *call*, *import*, and *inheritance*. Dashed edges are contextual meta-relations: folder–folder *parent_of*, folder–file *contains*, and member–file *belongs_to*.

5.2.2 Node Feature Initialization

Node-feature initialization operationalizes the input-representation dimension of the design space. Papers II and III initialize each file node by concatenating a binary BoW representation of its folder path with the ℓ_2-normalized mean of Word2Vec embeddings for its code identifiers. Paper I uses the same sources but represents both folder path and identifier tokens as ℓ_2-normalized binary BoW vectors. Organizational and lexical information therefore enter these models as a fixed input.

Paper IV uses no external lexical or organizational features. Instead, each node type receives a learnable identity embedding trained jointly with the model; file nodes additionally receive LPE, which supplies structural position in the dependency graph. Its Node2Vec and MetaPath2Vec baselines use no input feature vectors, since they learn representations directly from random-walk co-occurrence.

Paper V treats node initialization as the object of study rather than a fixed choice. File and member nodes use lexical Word2Vec embeddings over identifiers, structural Node2Vec embeddings from the dependency graph, or contextual UniXcoder embeddings from code text extracted from source files, evaluated in zero-shot (UXC_{ZS}) and dependency-guided fine-tuned (UXC_{FT}) forms. Where folder nodes are included, they are consistently initialized using binary BoW vectors derived from folder-path tokens. Paper V therefore varies the representation of file- and member-level information while holding folder-node initialization constant.

Input text is extracted using Tree-sitter [7] with the Java, C, or C++ grammar matching each file. The syntax tree guides fragment selection but is not itself passed to the encoder. A file-level view contains type declarations, fields or global variables, and constructor signatures with short body excerpts where applicable. A member-level view contains method and function declarations and definitions. Each file receives one vector per view. The file-level view is encoded as a single input, while the member-level vector is obtained by mean-pooling representations of overlapping text chunks.

In zero-shot use, UniXcoder encodes these views without further training. It is also fine-tuned separately for each system on file-level views using a contrastive objective [65] to incorporate dependency structure. Positive pairs are drawn from each file's two-hop dependency neighborhood, falling back to a direct neighbor when no two-hop candidate exists. Other files in the batch serve as negatives, except directly connected files, which are masked unless selected as the positive pair. Module labels are not used in this loss. The resulting model encodes both views, although fine-tuning uses only file-level text.

5.2.3 Edge Attributes

Edge attributes form a narrower axis of the design space and are used beyond relation type in only Papers I and IV. In the remaining studies, edges are defined only by their relation type. Paper I evaluates a variant that augments file-level dependency edges with attributes describing the member-level interactions that produce them: an ℓ_2-normalized binary encoding of interaction type and a log-normalized interaction frequency. This tests whether the origin and frequency of a dependency provide information beyond its typed presence.

Paper IV uses edge attributes differently in its diffusion variant. GDC rewires the file graph using PPR and assigns each rewired edge a scalar diffusion weight, which biases attention towards diffusion-close neighbors. Unlike the attributes of Paper I, these weights are derived from graph topology rather than from additional implementation information. They therefore extend the structural reach of the encoder without adding information beyond the dependency graph.

5.3 Learning Settings

Once the graph and node features are constructed, the studies differ in how they learn file representations and how these representations are used to obtain module assignments. This difference follows the two tasks: recovery methods learn embeddings and cluster them into modules, whereas mapping methods predict assignments to a predefined module set. Table 5.2 summarizes the encoder, learning objective, and training configuration used in each study.

For multi-relational graphs, GCN and GAT use separate layers for each dependency type and aggregate their outputs. HGT instead incorporates node and

Table 5.1: Graph construction and signal representation across the five papers (P). Numbers in parentheses give vector dimensions; ID denotes code identifiers.

P	Nodes	Edges	Node features	Edge features
I	File	typed	[BoW(*path*) ‖ BoW(ID)]	type & frequency†
II	File	typed	[BoW(*path*) ‖ W2V(ID) (100)]	–
III	File	typed / merged†	[BoW(*path*) ‖ W2V(ID) (128)]	–
IV	File / +Folder†	merged / typed†	Folder: Learnable (128) File: [Learnable (128) ‖ LPE (16)]	PPR rewired edges & diffusion weights†
V	File / +Folder / +Member	merged	Folder: BoW(*path*) W2V (128) / N2V (128) / UXC_{ZS} / UXC_{FT} (768)	–

† denotes an ablation against the other value, not the fixed design of the study.

relation types directly in its attention mechanism, processing the relation types jointly within each layer.

In the recovery studies (Papers I, II, and IV), no module labels are available during training. Paper I learns file embeddings using a GAT with a contrastive objective, while Paper II compares GCN and GAT encoders trained through graph reconstruction against a Node2Vec baseline that follows NEGAR [8]. Paper IV compares random-walk methods with GAT-based encoders trained within a variational graph autoencoder. Its graph-attention variants examine diffusion, folder context, and typed dependencies.

The learned file embeddings are then clustered into modules using agglomerative hierarchical clustering with Ward linkage, except for the Node2Vec embeddings of Papers II and IV, which use k-means, following NEGAR [8]. The number of clusters is either selected automatically or set to the number of ground-truth modules, as in the main evaluation of Paper IV. Recovery therefore infers the file assignments and, unless the number of clusters is fixed, also the number of modules.

In the mapping studies (Papers III and V), the target modules are known, and a small set of files is labeled with their ground-truth assignments. Paper III compares GCN, GAT, and MLP models under the same feature representation and training procedure, and uses iterative self-training to expand the labeled set. Paper V compares MLP, GAT, and HGT. The baseline MLP uses file features alone, GAT uses the file-only graph, and HGT is evaluated on all four graph variants. Additional comparisons examine dependency-neighbor aggregation, including a neighbor MLP that receives each file's own vector concatenated with the mean vector of its dependency neighbors. Paper V trains each model on the labeled files for 200 epochs without self-training. It uses the same loss function, optimizer, and learning rate as Paper III, with encoder configurations given in Table 5.2. Mapping therefore predicts file assignments to a predefined module set.

Table 5.2: Encoder configurations and training settings across the five studies, grouped by architecture recovery and C2A mapping. Where a study evaluates multiple encoders, each encoder is listed separately. Dim. denotes the encoder embedding dimension; LR denotes learning rate. Dataset-specific settings for the largest systems are omitted and reported in the individual papers.

	P	Encoder	Objective	Dim.	Layers	Heads	Optimizer LR	Epochs
Recovery	I	GAT	Contrastive	128	1	1	Adam [42] 10^{-3}	50
		GAT-E		128	1	1		
	II	GCN	Reconstruction	128	1	–	Adam 10^{-4} $(10^{-3})^{\dagger}$	30 (50)†
		GAT		128	1	1		
	IV	N2V	Skip-gram	128	–	–	SGD [25] 0.025	5
		M2V		64	–	–	SparseAdam 0.05	20
		GAT	Reconstruction + KL	256	2	2	Adam 10^{-3}	100
		GAT-DC		256	1 or 2‡	2		
		HGAT		256	1	2		
		HGAT-TD		256	1	2		
Mapping	III	GCN	CCE	256	2	–	Adam 10^{-3}	100 warm-up + $\leq 4 \times 30^{*}$
		GAT		256	2	4×64		
		MLP		256	1	–		
		HGT		256	1	4×64		
	V	GAT		256	1	4×64		200
		MLP		256	2	–		

SGD: stochastic gradient descent.
For the mapping studies, layer counts exclude the linear classification head.
† Very small systems (fewer than 100 entities).
‡ GAT-DC: one layer for more than 500 files and more than 10 modules, two otherwise.
∗ Paper III epochs: warm-up plus at most the stated rounds of 30 epochs each, with pseudo-labels requiring confidence > 0.95.

5.3.1 Controlled-Comparison Logic

The studies examine selected choices within the design space through controlled comparisons. Some choices are coupled, since adding folder or member nodes also introduces relations and changes the paths available for message passing. A full factorial evaluation of all combinations would be computationally prohibitive. Each comparison therefore varies one or two choices while keeping the remaining conditions fixed where possible, allowing differences in outcome to be related to the choices under investigation.

The mapping studies compare graph-based encoders with an MLP using the same input features and training procedure. These comparisons are consistent with a contribution of dependency-graph information, but they change the predictor together with the available structure. Paper IV compares a dependency-

only graph with the same graph augmented by folder nodes while keeping the learning setting fixed, testing the contribution of organizational structure in the absence of lexical features. Paper V extends this logic by varying feature initialization and, with HGT, graph structure across file-only, folder-augmented, member-augmented, and combined graph variants. It also enables and disables dependency neighbors within each predictor while holding the file representations fixed. This makes it possible to examine the effects of input features and structural additions separately and, where combinations are included, their interaction.

5.4 Benchmark Systems

The studies evaluate open-source systems from published architecture recovery and C2A mapping benchmarks. Reusing expert-validated reference decompositions supports comparison with prior work and avoids introducing new architectural judgments, which require substantial domain knowledge and manual effort [20]. Table 5.3 lists the fourteen systems used across the thesis; each study evaluates the subset appropriate to its setting.

Table 5.3: Properties and versions of the benchmark systems. $\mathcal{M}$: number of modules in the ground-truth architecture; |SD|: static dependencies between files.

	System	Version	$\mathcal{M}$	|Files|	|SD|	Domain
Java	ArchStudio4	4	57	583	5 418	Architecture modeling
	ArgoUML	0.24	14	766	9 491	UML modeling
	Commons-Imaging	v1.0a2	21	329	4 561	Image I/O library
	Hadoop	0.19.0	67	591	10 510	Distributed processing
	JabRef	v3.7	6	1 013	16 394	Reference management
	Lucene	r1075001	7	1 054	18 595	Search engine
	OODT	0.2	216	876	8 811	Data management
	SweetHome-3D	r002382	9	167	3 912	Interior design
	TeamMates	v5.11	15	778	17 818	Peer feedback system
C	Bash	4.2	14	292	1 222	Shell
	HDF	0e196f	9	153	1 239	Driver subsystem
	Libxml	2.4.22	17	82	986	XML parser
C++	Chromium	svn-171054	69	18 343	284 435	Web browser
	HDC	46ff87	11	207	1 117	Camera interface

HDF: Hardware Driver Foundation; HDC: Distributed Camera.
Versions and counts refer to the benchmark instances used in the cross-study evaluation. They may differ from those reported in individual papers.

ArgoUML, Commons-Imaging, JabRef, Lucene, SweetHome-3D, and TeamMates are drawn from the benchmark assembled by Olsson et al. [64] and distributed through the s4rdm3x repository [63]. The mappings for JabRef and

TeamMates were produced by system experts during the SAEroCon workshops in 2016 and 2017. The ArgoUML and Lucene mappings were studied by Brunet et al. [6] and Lenhard et al. [48], respectively, whereas the Commons-Imaging mapping was contributed as an s4rdm3x test case.

For ArchStudio4, Hadoop, Bash, OODT, and Chromium, the thesis uses the reference decompositions curated by Garcia et al. [20, 50]. These decompositions were developed through a certification process that recovered an initial view from documentation and source code and refined it with system experts. The HDF, HDC, and Libxml mappings were developed with industry partners and are included in the SARIF benchmark studied by Zhang et al. [87].

5.5 Evaluation Protocol

Every controlled comparison in the thesis follows the same evaluation protocol. Model initialization and training, and, in the mapping studies, selection of the labeled set, involve randomness, so a single run cannot distinguish a systematic difference between configurations from ordinary run-to-run variation. Each stochastic method–configuration pair is therefore run 100 times. Results are summarized by the median across runs rather than the mean, because the median is robust to occasional outlying runs.

5.5.1 Labeled Training Sets

In the mapping studies, each labeled set begins with a uniform random sample of 5% of a system's files, bounded between 30 and 200 files. One file is then added from each module not represented in that sample. The floor and cap therefore apply to the random sample before these module-coverage additions, and the final labeled set can exceed the bounded sample size. Papers III and V use the same labeled sets, which self-training expands only in Paper III.

The 30-file floor applies to Commons-Imaging, SweetHome-3D, Bash, HDF, and HDC, corresponding to approximately 9–20% of their files before module-coverage additions. The 200-file cap applies only to Chromium, corresponding to approximately 1% of its files. For ArgoUML, JabRef, Lucene, and TeamMates, the random sample follows the nominal 5% rule. The experiments therefore use a bounded labeling budget rather than a constant labeled proportion: smaller systems receive a larger labeled share, while Chromium receives a smaller share. Comparisons within each system use the same labeled sets, but cross-system performance differences must be interpreted in light of these different labeled proportions.

The mapping studies therefore use ten of the fourteen systems. ArchStudio4, Hadoop, and OODT are excluded because representing every module would require labeling approximately 10–25% of their files, and Libxml is excluded because the 30-file floor alone would label approximately 37% of its files.

5.5.2 Statistical Analysis

To assess the statistical significance of observed differences between methods or configurations, nonparametric tests are used because metric values across repeated runs are not necessarily normally distributed. The recovery studies compare independent distributions of 100 runs per configuration and use the Mann–Whitney U test [17]. In the mapping studies, the compared configurations use the same labeled set in each run, producing paired observations. These comparisons use the Wilcoxon signed-rank test [85]. Where a recovered partition is compared with the ground-truth architecture, the 100 runs of a method are tested against the single ground-truth value of the normalized TurboMQ using the one-sample Wilcoxon signed-rank test.

Statistical tests use a significance threshold of $\alpha = 0.05$. For the mapping comparisons, the resulting p-values are adjusted using Holm's procedure across the ten systems, separately for each metric and each pair of compared configurations.

A significant result indicates that the observed difference would be unlikely if the two methods or configurations produced equivalent results, but it does not indicate the magnitude of the difference. With 100 runs per configuration, even a small difference can reach statistical significance. Differences are therefore also quantified using Cliff's delta, with effect sizes classified by magnitude as negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), medium ($0.33 \leq |\delta| < 0.474$), or large ($|\delta| \geq 0.474$) [11, 75]. Between two configurations, delta is computed on the two sets of 100 scores as independent groups, and the significance test is paired. Against the ground-truth value, which is a constant, delta is the proportion of runs above that value minus the proportion below it. A difference that is significant but negligible is reported but not interpreted as an improvement.

Results are reported per system rather than averaged across systems, since averaging can conceal effects that differ across systems with different graph properties. The definitions and interpretation of the recovery and mapping metrics are given in Section 2.4.

5.6 Cross-Study Evaluation Design

The five studies were conducted on separately scoped benchmarks and, in part, with different input features and pipelines, so their reported results cannot be compared directly. For this thesis, four evaluations were therefore conducted in addition to the published work, followed by a correlation analysis with system properties. Their results are reported together with those of the studies in Chapter 6. Unless stated otherwise, the models, input features, training settings, and evaluation metrics are those described in Section 5.3.

The first two evaluations revisit recovery methods from Papers I, II, and IV on all systems in Table 5.3. Both use Ward-linkage agglomerative clustering of the learned file embeddings, with the number of clusters set to the number of modules in the ground-truth architecture. Each method is run 100 times per sys-

tem. Holding the clustering procedure fixed removes differences in clustering algorithms and cluster-count selection from the comparison.

The first evaluation compares the contrastive approach of Paper I with the reconstruction-based approach of Paper II using the GAT encoder. Paper I is extended to use the input features of Paper II while retaining its own training pipeline. The comparison therefore uses shared input features, although differences in training settings remain.

The second evaluation compares the attributed dependency graph of Paper II, using GCN and GAT, with the structure-only file-level and folder-augmented representations of Paper IV. It also compares the dependency cohesion of the recovered partitions with that of the ground-truth architectures.

The remaining two evaluations concern C2A mapping on the ten-system mapping benchmark. Within each comparison, the underlying file-level dependency graph, input features, 100 labeled sets per system, and training procedure are held fixed.

The third evaluation adds HGT to the comparison of Paper III to examine its performance with typed dependencies. It uses that study's file-level graph and input features, with two message-passing layers and messages aggregated from source to target, matching the depth and direction of its main configuration.

The fourth evaluation examines whether the contribution of dependency types varies with the input representation. It compares typed GAT, with one channel per relation type, against merged GAT, which collapses all types into a single relation, using the four input representations from Paper V: Word2Vec, Node2Vec, zero-shot UniXcoder, and fine-tuned UniXcoder. Both variants aggregate messages from source to target. None of these representations includes folder-path features, so this evaluation examines dependency types without organizational information in the node features.

5.6.1 Cross-Study Correlation with System Properties

This analysis assesses whether recovery performance varies systematically with system properties. Each system contributes one observation per method and configuration: its median metric value across the 100 runs, and these values are correlated with the system properties using Kendall's τ, a nonparametric rank correlation that accommodates tied ranks [41]. The sample size is therefore the number of systems, fourteen, and not the number of runs.

The properties are homophily $\mathcal{H}$, the proportion of static dependencies whose source and target files belong to the same ground-truth module [89], and average module size $\langle \mathcal{M} \rangle$, the mean number of files per ground-truth module, which characterizes the granularity of the target decomposition. Table 5.4 reports both properties and the normalized TurboMQ of the ground-truth decomposition.

Table 5.4: Properties of the benchmark systems, with the normalized TurboMQ of the ground-truth decomposition computed on the dependency-count-weighted graph. Shaded cells mark the highest value in each column.

System	Normalized TurboMQ TMQ_{norm}	Homophily $\mathcal{H}$	Avg. module size $\langle \mathcal{M} \rangle$
ArchStudio4	40.77	0.67	10.23
ArgoUML	59.27	0.55	54.71
Commons-Imaging	50.93	0.61	15.67
Hadoop	18.58	0.34	8.82
JabRef	31.78	0.56	168.83
Lucene	73.90	0.68	150.57
OODT	16.92	0.27	4.06
SweetHome-3D	13.60	0.36	18.56
TeamMates	9.17	0.34	51.87
Bash	45.29	0.58	20.86
Chromium	53.76	0.58	265.84
HDC	77.82	0.63	18.82
HDF	48.02	0.35	17.00
Libxml	35.08	0.21	4.82

Papers II and IV reported TMQ_{norm} without aggregating the multiplicity of member-level dependencies into a single file-pair edge weight, so the values here are not directly comparable.

Chapter 6

Research Findings and Synthesis

Contents

6.1 Results of the Cross-Study Evaluations 41
6.1.1 Recovery: Learning Objectives and Graph Inputs 41
6.1.2 Mapping: Predictors and Dependency Types 44
6.2 Synthesis of Research Findings 46
6.2.1 Objective 1: Representation Learning for Recovery and Mapping 46
6.2.2 Objective 2: Representations of Implementation Signals 47
6.2.3 Objective 3: Implementation Signals and Graph Structure 49

This chapter presents and synthesizes the findings against the three research objectives. It first reports the cross-study evaluation results, then combines them with the findings of the five studies to answer the research questions.

6.1 Results of the Cross-Study Evaluations

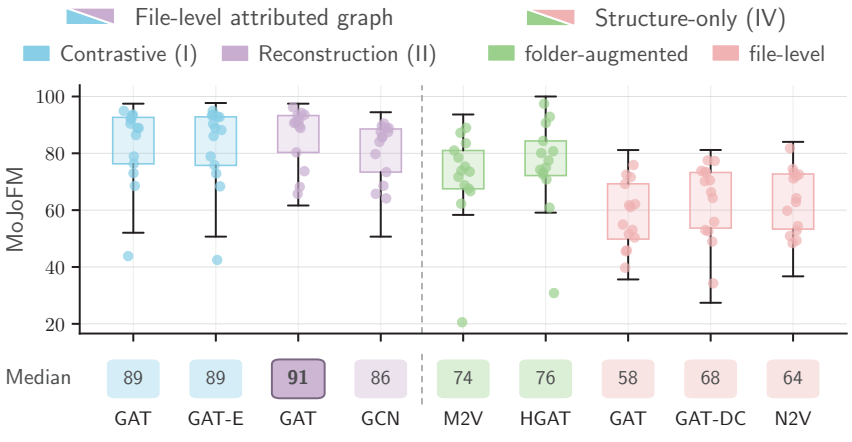
This section reports four cross-study evaluations in architecture recovery and C2A mapping. Boxplots pool 100 runs per method and system across all systems. They show the pooled distribution of outcomes without separating within-system from between-system variation. System-wise tests complement these plots.

6.1.1 Recovery: Learning Objectives and Graph Inputs

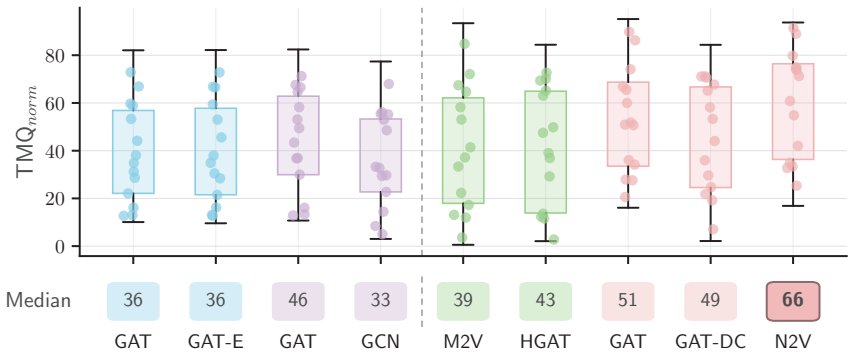
The two recovery evaluations cover all fourteen benchmark systems. The first compares the contrastive approach of Paper I with the reconstruction-based approach of Paper II using GAT and common node features. It also includes GAT-E, the edge-attribute variant of Paper I. The approaches retain their respective training settings, so the comparison does not isolate the learning objective alone.

The second compares the attributed-graph GCN and GAT of Paper II with the structure-only methods of Paper IV. These comprise GAT, GAT-DC, and Node2Vec on the file-level dependency graph, and MetaPath2Vec and HGAT on the folder-augmented graph.

MoJoFM, ARI, and $C2C_{cvg}$ measure agreement with the reference decomposition, while TMQ_{norm} measures dependency cohesion independently of that reference. Figure 6.1 presents the results using MoJoFM and TMQ_{norm}. The comparisons use a common clustering procedure, as described in Section 5.6.



(a) MoJoFM.



(b) Normalized TurboMQ

Figure 6.1: Recovery results across studies: agreement with the reference decomposition (a) and dependency cohesion (b). The attributed-graph methods are the contrastive (Paper I) and reconstruction (Paper II) variants; the structure-only methods (Paper IV) use the folder-augmented or the file-level graph. Boxplots pool all runs per system; dots mark system medians, and the boxed value indicates the median of these medians.

Agreement with the reference decomposition. In Figure 6.1(a), the contrastive and reconstruction variants have similar median MoJoFM values, but the contrastive variants show a wider spread toward lower scores. System-wise comparisons significantly favor the reconstruction-based GAT over the contrastive GAT on both MoJoFM and ARI for nine of the fourteen systems. Effects are large for ArchStudio4, Bash, Chromium, Libxml, and OODT, and small to medium for Commons-Imaging, HDF, SweetHome-3D, and TeamMates. Differences are not significant for ArgoUML, HDC, JabRef, or Lucene, while contrastive learning performs better only on Hadoop.

Adding edge attributes within the contrastive setup provides no consistent advantage. Mann–Whitney U tests find a significant difference between GAT and GAT-E only for MoJoFM on OODT, with a small effect favoring GAT.

Comparing graph inputs, attributed GCN and GAT achieve a higher median MoJoFM than all structure-only methods. ARI and $C2C_{cvg}$ give broadly similar rankings to MoJoFM when methods are ordered by their median scores across systems. The exceptions are that GCN ranks above the contrastive GAT variants under ARI, while GAT-DC and Node2Vec exchange places under $C2C_{cvg}$.

Dependency cohesion. Figure 6.1(b) shows a different ordering from reference agreement. The file-level structure-only methods achieve higher median TMQ_{norm} than the attributed methods, with Node2Vec attaining the highest value. Among the attributed methods, reconstruction-based GAT scores above the contrastive variants, while GCN has the lowest median of all methods. Reconstruction-based GAT also scores above the folder-augmented HGAT and MetaPath2Vec.

Because TMQ_{norm} varies widely across systems, each method is also compared with the reference architecture of the same system. Table 6.1 reports how many systems have significantly lower or higher scores, using one-sample Wilcoxon signed-rank tests. GCN falls below the reference on eleven of fourteen systems. The contrastive and folder-augmented methods fall below it on seven or eight systems, whereas the remaining methods do so on five or fewer. Node2Vec exceeds the reference on thirteen systems and falls below it only on Libxml.

Together, these comparisons show that agreement with the reference decomposition and dependency cohesion favor different methods. The attributed methods achieve higher reference agreement, while the file-level structure-only methods produce more dependency-cohesive partitions.

The common clustering procedure changes Node2Vec's results relative to its original k-means configuration. Using Ward linkage lowers the median ARI across systems from 0.293 to 0.266 and MoJoFM from 68.9 to 63.5, while increasing normalized TurboMQ from 63.9 to 66. Changes in reference agreement vary in direction across systems, whereas normalized TurboMQ increases on thirteen of the fourteen systems.

Table 6.1: Number of the fourteen systems on which a method's TMQ_{norm} is significantly below or above that of the ground truth (one-sample Wilcoxon signed-rank test, $p < 0.05$); on the remaining systems, the difference is not significant.

	Attributed graph (Papers I & II)				**Structure-only (Paper IV)**				
	Contrastive		Reconstruction		Folder-augmented		File-level		
	GAT	GAT-E	GAT	GCN	M2V	HGAT	GAT	GAT-DC	N2V
Below	8	8	5	11	8	7	2	5	1
Above	4	4	7	3	6	5	11	9	13

6.1.2 Mapping: Predictors and Dependency Types

The two mapping evaluations cover the ten-system mapping benchmark and use macro- and micro-F1. The first compares HGT with the MLP, GCN, and GAT of Paper III using the same file features and labeled sets. The graph-based predictors use a file-level graph with separate dependency relations.

The second compares typed and merged GAT using the four file representations from Paper V. It examines whether the benefit of preserving dependency types extends beyond the combined lexical and folder-path features used in Paper III.

Predictor comparison. Figure 6.2 shows that GAT has the highest median macro- and micro-F1. HGT's median micro-F1 lies between GCN and GAT, but its median macro-F1 is the lowest of all four predictors.

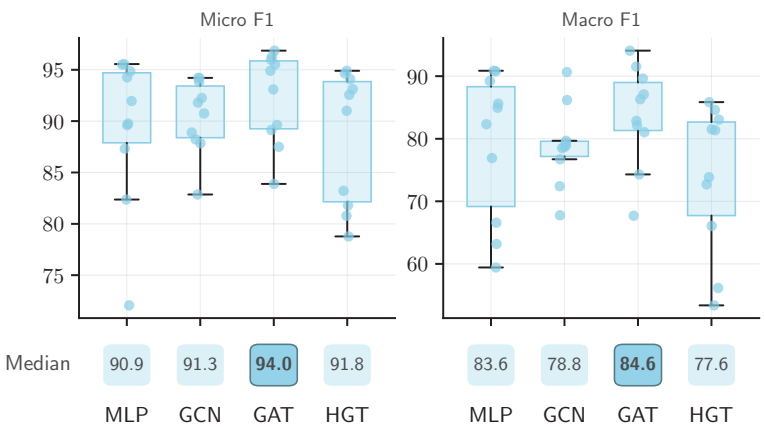


Figure 6.2: Macro- and micro-F1 for MLP, GCN, GAT, and HGT on the same input features and labeled-set splits. Boxplots pool all runs per system; dots mark system medians, and the boxed value indicates the median of these medians.

System-wise comparisons significantly favor GAT over HGT on nine of ten systems for macro-F1 and eight for micro-F1, with mostly large effects. HGT significantly outperforms GAT only on Chromium's micro-F1, by 1.9 points with a large effect.

In the system-wise tests, HGT does not generally outperform MLP and GCN. Compared with MLP, it scores significantly lower on seven systems for each metric, with small-to-large effects, and higher on Chromium and JabRef for both. Compared with GCN, it scores significantly lower on six systems for macro-F1 and five for micro-F1, and higher on two and four systems, respectively.

Effects of dependency type modeling. Table 6.2 compares typed and merged GAT using Word2Vec, Node2Vec, and zero-shot and fine-tuned UniXcoder representations. Both variants use the GAT architecture of Paper III with source-to-target aggregation. None of the input representations includes folder information.

Preserving dependency types yields positive median F1 differences across systems for all four representations. The largest median gain is for Word2Vec in macro-F1, at 5.8 points, and for fine-tuned UniXcoder in micro-F1, at 2.8 points. System-wise tests show significant gains on seven of ten systems for Word2Vec under both metrics, and on seven systems for fine-tuned UniXcoder in macro-F1 and nine in micro-F1.

The benefit is not uniform. Every representation also shows significant losses on some systems. Bash is a recurring exception, with significant losses for Word2Vec and both UniXcoder representations under both metrics. Zero-shot UniXcoder shows particularly mixed effects, with large gains on SweetHome-3D and HDF and large losses on Bash and Chromium.

Table 6.2: Typed minus merged GAT across the ten systems, by input vector and metric. Δ: median F1 difference in points. +ve: number of systems with a positive difference. ↑/↓: number of systems with a significant gain/loss (paired Wilcoxon signed-rank test, Holm-corrected across the ten systems for each vector and metric, $\alpha = 0.05$). N/S/M/L: number of systems with negligible, small, medium, or large Cliff's δ between the typed and merged scores.

	Macro-F1					Micro-F1				
Vector	Δ	+ve	↑	↓	N/S/M/L	Δ	+ve	↑	↓	N/S/M/L
N2V	+0.6	6	5	1	5/4/0/1	+0.8	7	7	3	4/4/0/2
W2V	+5.8	8	7	1	2/0/1/7	+1.4	9	7	1	1/4/1/4
UXC_{ZS}	+0.1	5	4	4	2/3/1/4	+0.1	5	5	3	2/2/2/4
UXC_{FT}	+2.6	7	7	3	1/3/1/5	+2.8	9	9	1	2/1/2/5

6.2 Synthesis of Research Findings

This section combines the findings of the five studies with the cross-study results in Section 6.1 to answer the research questions. The synthesis follows the three research objectives, examining the use of representation learning for recovery and mapping, the information provided by representations of individual implementation signals, and the contribution of additional graph structure.

6.2.1 Objective 1: Representation Learning for Recovery and Mapping

Objective 1 examines how graph-based representation learning supports architecture recovery and C2A mapping. The findings address the combination of implementation signals (RQ1.1), the treatment of dependency types (RQ1.2), and the effects of learning objective, encoder architecture, and depth (RQ1.3).

RQ1.1: Joint representation of implementation signals. Papers I–III combine dependencies, code identifiers, and folder organization to learn representations for recovery and mapping. These methods improve on their respective baselines in reference agreement for recovery and F1 for mapping. The results support the combined representations, but do not establish how much each signal contributes.

In Paper III, an MLP using the same file features also outperforms the Naive Bayes baseline on every system under both F1 metrics. Improvement over that baseline therefore does not, by itself, demonstrate a benefit from dependency-based message passing.

RQ1.2: Modeling dependency types. Papers I–III represent dependency types as separate relations, each processed by its own encoder channel, and combine the outputs by summation. Paper III compares this design with merging all dependencies into a single relation. With lexical and folder-token features, separate relations significantly improve both GCN and GAT across all evaluated systems. However, the separate channels also add parameters, so the comparison does not isolate dependency typing from model capacity. Papers I and II do not test a merged alternative.

The cross-study mapping comparison extends this evaluation to the folder-free representations of Paper V. Separate relations generally benefit GAT, but the benefit is less consistent than in Paper III and varies across representations and systems, with significant losses on some systems (Section 6.1.2). HGT does not match GAT on the typed dependency graph, showing that the use of dependency types alone does not determine predictor performance.

Recovery provides less consistent support for separate relations. In Paper IV, preliminary experiments on file-level dependency graphs showed markedly lower performance with separate relations, so the reported dependency-only

comparisons use merged dependencies. With folder nodes and containment relations included, preserving dependency types gives modest benefits that vary across systems and evaluation measures.

Overall, separate dependency relations benefit mapping more consistently than recovery in the evaluated settings. Their effects vary with the input representations and encoder, but differences between the studies prevent these factors from being isolated.

RQ1.3: Learning objective, encoder architecture, and depth. The studies examine how these three choices affect recovery and mapping.

- *Learning objective:* The cross-study evaluation in Section 6.1.1 compares the contrastive approach of Paper I with the reconstruction-based approach of Paper II using common node features and a GAT encoder. Reconstruction generally achieves better agreement with the reference decomposition, although the advantage is not universal. Because the approaches retain different training settings, the comparison supports the reconstruction-based approach without isolating the learning objective alone.
- *Encoder architecture:* GAT generally outperforms GCN across recovery and mapping (Papers II and III), and HGT in the cross-study mapping comparison. GCN uses degree-normalized aggregation, whereas GAT learns weights for neighboring representations. However, the models also differ in parameterization, so these results cannot be attributed to attention alone. In mapping, GAT usually outperforms the MLP, while GCN frequently underperforms it. Access to dependencies therefore does not guarantee better predictions.
- *Encoder depth:* Paper III finds no consistent benefit from increasing message-passing depth. A second layer improves performance on some systems and reduces it on others.

Summary of Objective 1

Combining dependency, lexical, and folder signals improves on the evaluated baselines in recovery and mapping, although the contribution of each signal is not isolated (RQ1.1). Preserving dependency types benefits mapping more consistently than recovery, with effects varying across inputs and encoders (RQ1.2). The reconstruction-based approach generally achieves better reference agreement than the contrastive approach, but their training settings also differ. GAT generally outperforms GCN across both tasks and HGT in the cross-study mapping comparison, while increasing encoder depth has no consistent benefit (RQ1.3).

6.2.2 Objective 2: Representations of Implementation Signals

Objective 2 (Section 3.2.2) examines the architectural information provided by representations of individual implementation signals. The findings address re-

covery from dependency structure alone (RQ2.1), dependency-guided adaptation of contextual code representations (RQ2.2), and the comparison of structural, lexical, and contextual file representations (RQ2.3).

RQ2.1: Recovery from dependency structure. Paper IV shows that dependency structure alone supports recovery. Node2Vec and GAT-DC outperform the classical recovery baselines, while plain GAT is generally weaker than both.

Structural reach helps explain this difference. Node2Vec and GAT-DC achieve better reference agreement than the two-layer GAT on most systems. Applying diffusion to the GAT input graph closes much of the gap to Node2Vec, suggesting that access to a wider dependency neighborhood contributes to recovery performance.

The evaluation criterion matters as well (Section 6.1.1). Node2Vec produces the most dependency-cohesive partitions, while GAT-DC improves reference agreement over plain GAT on most systems without a corresponding advantage in dependency cohesion.

RQ2.2: Dependency-guided adaptation of code representations. Paper V finds that dependency-guided fine-tuning of UniXcoder improves mapping performance over its zero-shot form on nearly all systems. Both representations are held fixed during classifier training and evaluated using the same MLP and labeled sets, without folder information or additional message passing.

Fine-tuning uses a contrastive objective with positive pairs drawn mainly from two-hop dependency neighborhoods. This choice follows the observation in Paper IV that structural context beyond direct neighbors can support recovery. With the classifier and evaluation conditions held fixed, the gains support adapting the pretrained representation to the target system using dependency information.

However, the comparison does not isolate the contribution of dependency-based pair selection, since it includes no fine-tuning alternative using pairs selected without dependency information.

RQ2.3: Comparing file representations. Paper V compares structural, lexical, and contextual file representations using the same graph-free MLP and labeled sets, without folder information or additional message passing. Representations that encode dependency information generally perform better. Node2Vec captures dependencies through random walks, while fine-tuned UniXcoder incorporates them through dependency-guided adaptation. Both generally outperform Word2Vec and zero-shot UniXcoder, although there are exceptions, including JabRef, where Word2Vec is among the strongest. Word2Vec and zero-shot UniXcoder show no consistent ordering, with each significantly outperforming the other on several systems.

Neither Node2Vec nor fine-tuned UniXcoder is uniformly best. Node2Vec performs significantly better on six systems under both F1 metrics, while fine-tuned

UniXcoder performs significantly better on three systems under both metrics and additionally on HDC under macro-F1.

Together with the improvement from zero-shot to fine-tuned UniXcoder (RQ2.2), these findings suggest that dependency information contributes to the stronger representations. However, the representations also differ in content and training procedure, so the contribution of dependency information alone cannot be isolated.

Summary of Objective 2

Dependency structure alone supports recovery, with performance depending on structural reach and how closely dependencies follow the reference modules. Reference agreement and dependency cohesion favor different methods (RQ2.1). Dependency-guided fine-tuning improves UniXcoder over its zero-shot form on nearly all systems, although the contribution of dependency-based pair selection is not isolated (RQ2.2). Node2Vec and fine-tuned UniXcoder generally outperform Word2Vec and zero-shot UniXcoder in mapping without folder information or additional message passing, but neither is uniformly best. Differences in content and training prevent the advantage from being attributed solely to dependency information (RQ2.3).

6.2.3 Objective 3: Interaction Between Implementation Signals and Graph Structure

Objective 3 (Section 3.2.3) examines how graph structure contributes beyond the initial file representations. The findings address dependency-neighbor aggregation and its direction (RQ3.1), folder organization (RQ3.2), and the additional detail provided by member nodes and edge attributes (RQ3.3).

RQ3.1: Dependency neighbors and aggregation direction. Paper V examines the contribution of dependency neighbors by comparing HGT, GAT, and a neighbor MLP with and without neighbor aggregation, holding file representations fixed. The neighbor MLP appends the mean of neighboring files' vectors to each file's own vector. These comparisons assess neighbor access within each predictor, avoiding the change in predictor architecture involved in comparing a GNN with a graph-free MLP.

Dependency neighbors mostly improve mapping beyond the information already present in the file representations, but the gains vary across predictors and inputs. For GAT and the neighbor MLP, gains are larger for Word2Vec and zero-shot UniXcoder, which do not encode dependencies, than for Node2Vec. However, fine-tuned UniXcoder benefits more than Node2Vec. GAT is generally the strongest predictor, while HGT gains the least from neighbor access. With GAT, fine-tuned UniXcoder scores higher than Node2Vec in micro-F1 on seven of the ten systems with either aggregation direction, up from three without aggregation (RQ2.3), whereas in macro-F1 the count rises only from four to five.

Aggregation direction also affects performance. Source-to-target aggregation updates a file using information from files that depend on it, while undirected aggregation permits information to flow in both directions. In Paper V, source-to-target aggregation lowered median macro-F1 with Node2Vec, which already encodes dependency context, whereas undirected aggregation improved it. The preferred direction varied across systems rather than across predictors or input representations. Systems that combine low homophily with high average degree favored source-to-target aggregation, while several sparser systems favored undirected aggregation. However, making the graph undirected also adds reverse edges, so the comparison does not separate direction from neighborhood size, and the ten systems do not separate connectivity from homophily.

Paper III examines aggregation direction for GAT and GCN and finds source-to-target aggregation generally preferable for GAT. This preference agrees with Paper V on some systems but differs on others. The studies also differ in dependency typing, node features, depth, and training, so neither the benefit of aggregation nor the preferred direction can be assumed to transfer across settings.

These comparisons isolate neighbor access in mapping. The dependency-only recovery methods have no corresponding condition without dependency context. Their results instead show how recovery varies with structural reach and how closely dependencies follow reference-module boundaries (RQ2.1).

RQ3.2: Folder organization. The studies represent folder organization either through directory tokens in file features (Papers I–III) or through folder nodes and containment relations (Papers IV and V). In recovery, methods that include organizational information achieve better agreement with the reference decomposition, particularly where dependencies provide weak evidence of module boundaries. Dependency-only methods instead produce more dependency-cohesive partitions (Section 6.1.1).

In mapping, Paper V finds that adding folder nodes produces the largest and most consistent gains across the evaluated additions to the graph, with Word2Vec benefiting most. Across these configurations, folder nodes use the same folder-token features. With fine-tuned UniXcoder, the folder-aware model approaches the mapping performance of a rule based on directory hierarchy and the initial labeled set on most systems, exceeding it only on Lucene.

The overall gains do not establish a consistent benefit for files that the directory-based rule assigns incorrectly. On these files, the folder-aware model is more accurate than the model without folder nodes on some systems, similar on others, and less accurate on others, and on most of the systems where the rule makes errors it still assigns most of them incorrectly. The analysis does not isolate files whose module differs from the majority module of their directory. Folder location is therefore informative for the reference mappings, while whether the model handles such files reliably remains untested. These mapping findings apply to HGT with source-to-target dependency aggregation and hold with one or two layers.

RQ3.3: Member nodes and edge attributes. Neither member-level nodes nor edge-level interaction attributes consistently improve performance in the evaluated settings.

In Paper V, adding member nodes to the file-only graph lowers the median F1 for all three representations evaluated with HGT. When folder nodes are also present, the effect of adding member nodes varies with the representation. These additions change both the graph structure and how features enter the model, so their effects cannot be attributed to structure alone.

Paper I reports small gains from edge attributes on some systems. However, the cross-study evaluation finds no significant improvement over the same model without these attributes (Section 6.1.1). Paper I attributes the limited benefit on TeamMates to edge attributes amplifying noisy cross-module interactions, suggesting that additional interaction detail may be less useful where dependencies poorly reflect module boundaries.

Summary of Objective 3

Dependency-neighbor aggregation improves mapping beyond the initial file representations, most reliably for GAT and the neighbor MLP. The magnitude of improvement depends on both the predictor and the input representation, with fine-tuned UniXcoder gaining more than Node2Vec, while the preferred aggregation direction varies across systems (RQ3.1). Folder organization provides the strongest additional signal in the evaluated mapping configurations, but does not consistently help correct files misclassified by the directory-based rule. In recovery, higher agreement with the reference architecture can coincide with weaker dependency cohesion (RQ3.2). Adding finer-grained structure does not extend these benefits consistently. Member-level nodes give no consistent improvement in C2A mapping, and edge-level attributes give no significant gain in the architecture recovery (RQ3.3).

Chapter 7

Discussion

Contents

7.1 Interpretation of the Findings 53
7.1.1 Reference Alignment and Dependency Cohesion 53
7.1.2 System Properties and Recovery Performance 54
7.1.3 Structural Reach and Neighborhood Aggregation 55
7.2 Limitations 56

This chapter interprets the findings across the five studies and presents an exploratory analysis of how system properties relate to recovery performance. It examines differences between recovery-method rankings under reference agreement and dependency cohesion, the relationship between dependency homophily and recovery performance, and the role of structural reach in recovery and mapping. It concludes with the limitations of the work. Unless directly tested, the proposed explanations remain interpretations of the observed patterns.

7.1 Interpretation of the Findings

The findings raise three questions for interpretation: why evaluation criteria favor different methods, how system properties relate to recovery performance, and how the available dependency context affects recovery and mapping. The following subsections examine these questions across the studies.

7.1.1 Reference Alignment and Dependency Cohesion

In the cross-study recovery comparison (Section 6.1.1), attributed GCN and GAT show the strongest agreement with the reference decomposition, whereas file-level structure-only methods produce more dependency-cohesive partitions. Node2Vec has the highest median normalized TurboMQ. Several factors may explain these different rankings.

TurboMQ rewards dependency concentration within clusters, while file-level structure-only methods learn exclusively from the dependency graph. Although their objectives do not explicitly optimize cohesion or coupling, learning from dependency proximity may favor partitions that score well on this measure.

Attributed representations also include folder paths and identifiers, which reflect how developers organize and describe responsibilities. These signals may support agreement with the reference decomposition without increasing dependency cohesion.

The reference decomposition itself need not maximize dependency modularity. Utility or cross-cutting code may receive dependencies from several modules, so matching its reference assignment need not produce the most dependency-cohesive partition.

The compared methods differ in graph construction, input information, and learning configuration. These explanations are therefore plausible interpretations of the observed rankings, but the comparisons do not isolate their individual contributions.

7.1.2 System Properties and Recovery Performance

We examine associations between recovery performance and benchmark-system properties. For each method in the cross-study recovery evaluation, we correlate per-system median scores with dependency homophily and average module size using Kendall's τ across the fourteen systems (Figure 7.1). Given the small sample and multiple tests, these correlations are treated as exploratory associations rather than independent effects.

Dependency homophily shows the clearest association with performance, particularly for structure-only methods. The structure-only GAT, GAT-DC, and Node2Vec all show significant positive associations with homophily for MoJoFM and TMQ_{norm}, while only the GAT does so for ARI. The attributed GCN and GAT show weaker and less consistent associations. Lexical and file-location attributes may provide complementary evidence when dependencies cross module boundaries, but these comparisons do not isolate their contribution from other methodological differences.

The Node2Vec results are consistent with its reliance on dependency structure alone, so its sensitivity to homophily may be a property of the method and not only of the benchmark. Random walks define the contexts from which file representations are learned, without module labels to distinguish within-module from cross-module dependencies. When dependencies frequently cross module boundaries, files from different modules may therefore appear in similar walk contexts. The positive associations with MoJoFM and TMQ_{norm} are consistent with this interpretation but do not establish the mechanism. Because homophily is computed from the reference decomposition, it provides a retrospective explanation and cannot guide the choice of a representation in advance.

Associations involving TMQ_{norm} need a separate reading. Homophily describes dependency concentration within reference modules, whereas TMQ_{norm} rewards dependency concentration within recovered clusters and is normalized by the number of clusters, so its associations with homophily and module size are partly built into the metric. They describe the structural quality that TMQ_{norm} measures and do not independently support an effect on recovery accuracy.

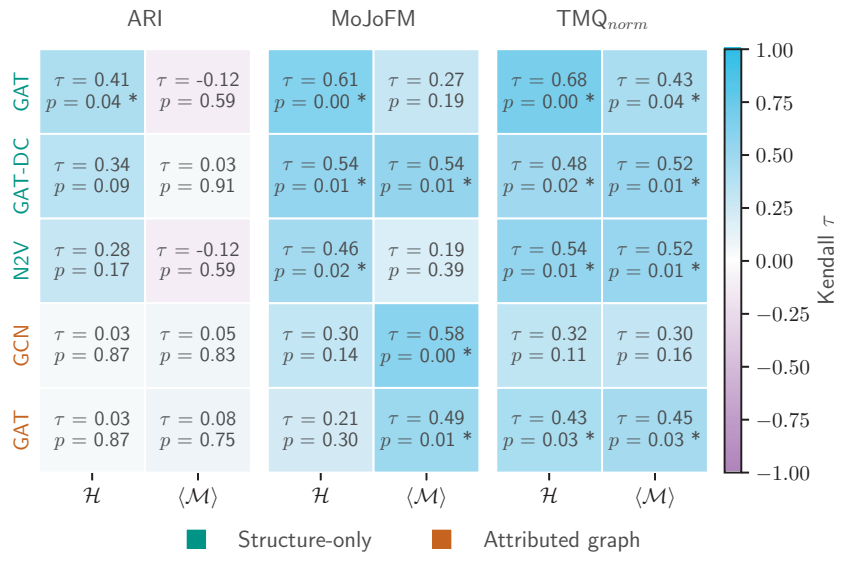


Figure 7.1: Kendall's τ correlation between recovery outcomes (ARI, MoJoFM, TMQ_{norm}) and two system properties, homophily $\mathcal{H}$ and average module size $\langle\mathcal{M}\rangle$, per method. GAT, GAT-DC, and N2V (structure-only) operate on the homogeneous dependency graph alone; GCN and GAT (attributed graph) operate on the multi-relational dependency graph with node features derived from file location and code identifiers.

Average module size is positively associated with MoJoFM for three of the five methods, but with ARI for none. This pattern is difficult to interpret independently of other system properties: OODT and Libxml have both the smallest modules and the lowest homophily, while larger modules tend to occur in larger systems. With fourteen systems, these associations cannot be separated, so no independent effect of module size is inferred.

7.1.3 Structural Reach and Neighborhood Aggregation

We use structural reach to describe the extent of the dependency neighborhood from which a file's representation can incorporate information through message passing, diffusion, or random walks. The studies vary these mechanisms rather than measuring reach directly.

In dependency-only recovery, Paper IV finds that GAT-DC and Node2Vec outperform plain GAT overall. GAT-DC makes diffusion-close nodes directly accessible to the encoder, while Node2Vec learns from contexts formed by random walks. These results suggest that the two-hop neighborhood available to plain GAT provides insufficient context on some systems.

However, GAT-DC also uses system-specific settings for depth and the number of retained diffusion neighbors. Paper IV examines sensitivity to the retained-neighbor count, but the comparison with plain GAT does not isolate the contribution of structural reach under matched settings.

Increasing depth offers another way to access more distant nodes, but it is not necessarily equivalent to diffusion or random walks. Additional layers repeatedly aggregate information over the same graph, diffusion changes which nodes are directly connected, and random walks define the contexts used to learn representations. Paper III finds mixed effects from adding layers in mapping, with neither connectivity nor homophily alone explaining when greater depth helps. These mechanisms were not compared under matched conditions across tasks.

Where dependencies cross module boundaries, additional layers may mix information from different modules. Repeated aggregation may also make node representations less distinguishable through over-smoothing [49]. Neither mechanism was measured directly in these studies, so both remain possible explanations for the mixed effects of depth.

In mapping, Paper V shows that encoding dependency context in the input representation does not remove the potential benefit of neighbor aggregation during prediction. With file vectors held fixed, adding neighborhood aggregation produces different gains across predictors and directions, including when the input vectors already encode dependencies through Node2Vec. The usefulness of additional context therefore depends on how it is combined with the existing representation.

Aggregation direction also varies in usefulness across settings. Papers III and V favor different directions on several systems, but also differ in dependency typing, features, depth, and training. These differences were not individually tested as explanations. Moreover, undirected aggregation adds reverse edges, so its comparison with source-to-target aggregation does not separate direction from neighborhood size.

Taken together, the studies suggest that both the available dependency context and the mechanism used to incorporate it matter. Wider reach can support recovery and mapping, but its benefit depends on the initial representation, the predictor, and how closely dependencies follow module boundaries.

7.2 Limitations

The conclusions are limited by the systems, architectural representations, and methodological choices examined. Some limitations apply across all five studies, while others concern particular graph constructions, model configurations, or evaluation protocols.

Benchmark scope and external validity. The studies evaluate overlapping subsets of open-source Java, C, and C++ systems with published reference decompositions. The findings may not generalize to other programming languages,

application domains, build environments, or architectural styles. Although the methods may be applied without a reference decomposition, agreement with an intended architecture cannot be assessed without a comparable reference. Chromium is the only large-scale system evaluated, so evidence of scalability is limited to a single system. Each system is evaluated at a particular version; the studies therefore do not establish robustness to refactoring or changes in dependency structure across releases.

The exploratory analysis across fourteen systems does not establish whether dependency homophily affects recovery independently of system size, module granularity, or other system properties.

Dependency graph. Static dependencies are extracted automatically with the Depends [62] tool. The resulting graphs may omit runtime dependencies or dependencies the tool cannot resolve, and may include edges not realized at runtime. Recovery and mapping results therefore depend on the extracted graph's accuracy and coverage. Within this thesis, this limitation may affect dependency-derived signals, including dependency-guided fine-tuning of contextual embeddings. Folder and containment relations, lexical embeddings, and zero-shot contextual embeddings do not depend on this extraction. The studies neither independently validate extraction accuracy nor assess sensitivity to extraction errors.

Reference architectures and architectural representation. The reference decompositions were produced by system experts, industry partners, or researchers reconstructing architecture from documentation, and only one reference is available per system. They are not uniquely determined ground truths: reasonable disagreement may arise when module boundaries are unclear, files implement several concerns, or the intended architecture differs from the implementation. Source revisions and reference mappings are not always aligned, and build configuration determines which files are analyzed, so assignments may be outdated or missing. In the mapping studies, labels are sampled from the same reference used for evaluation, so an incorrect assignment can misdirect training and penalize a plausible prediction.

Architecture granularity is a modeling choice: the same system can be represented by a few large modules or by more, smaller submodules. JabRef, for example, has six modules of very uneven size, so one of the larger ones could plausibly be split into three, whereas the 15 modules of TeamMates could be merged into six higher-level ones according to their name prefixes. Merging modules keeps existing internal dependencies within modules and may make inter-module dependencies internal as well, so homophily can only increase or stay unchanged; splitting has the opposite effect. Even with the same dependency graph, the chosen granularity can therefore change homophily, the reference partition's TurboMQ, and potentially the recovery and mapping scores.

More generally, both tasks represent architecture as a flat, exclusive partition of files into modules. Files implementing multiple concerns must be assigned to a

single module; overlapping responsibilities, cross-cutting infrastructure, and hierarchical decompositions are not evaluated. The reported scores therefore measure agreement with one file-level architectural view, not whether the recovered or predicted architecture fully represents the system.

Mapping protocol. The random labeled sample comprises 5% of files, bounded between 30 and 200, with five systems at the 30-file minimum, so label count and proportion vary with system size and were not varied independently. The mapping models are also transductive, since all system files are available during training.

Organizational signals and shared provenance. Folder organization is the strongest additional signal identified under Objective 3, but it introduces a validity concern. Folder hierarchies and reference decompositions may reflect shared design decisions: developers may organize folders around intended module boundaries, and researchers may use folder structure when constructing reference decompositions. A directory-based baseline in the mapping study agrees closely with the reference mappings, but the extent of this overlap was not otherwise quantified. Folder paths also inform file-node features in Papers I–III and initialize folder nodes in Paper V. The gains from folder information may therefore partly reflect shared provenance between inputs and reference labels rather than independent architectural evidence. Systems with reorganized, flattened, or architecture-inconsistent directory structures were not evaluated.

Representation, objective, and comparison choices. The studies use graph abstractions that encode different notions of structural relevance. Several models use binary file-level adjacency, which omits dependency frequency. Paper I includes member-level interaction counts and types as edge attributes, whereas Paper IV uses PPR weights to represent topology-derived proximity. These represent different notions of edge strength and were not compared across encoders and tasks. Aggregation direction was varied only in mapping, so its effect on recovery remains untested. Member-level information was evaluated only as an aggregate node per file and with one encoder, HGT, so the results do not show that it is uninformative under other designs or encoders.

Recovery objectives optimize proxies rather than agreement with the intended architecture, and results also depend on the clustering method and, in the recovery studies, on the number of clusters, which was searched over a range. The cross-study evaluations fix the cluster count to the reference count, which improves comparability but supplies information generally unavailable in practice.

Between-method comparisons involve differences in graph construction, inputs, training, depth, and parameter count, which limit attribution of performance differences to individual choices. Hyperparameter selection on the evaluation systems using reference-based metrics may also favor more extensively tuned methods.

Stochasticity and reproducibility. Results are summarized by medians over 100 runs per configuration in all five studies. Input representations were computed once and reused, except for Node2Vec in recovery, which was recomputed for each run. Reported variation therefore excludes the stochasticity of the fixed representations. With 100 runs, even small differences can reach statistical significance without being practically meaningful. Effect sizes are therefore reported alongside p-values, and interpretation should consider effect magnitude rather than significance alone.

In the mapping studies, each run pairs a labeled set with a training seed, so their contributions to variation are not separated in the main evaluation. When Paper III varied the training seed with the labeled set held fixed, the ranking of a few configurations changed across seeds, whereas some advantages were robust. These results indicate seed sensitivity, but the limited comparisons do not characterize it fully. Close rankings should therefore be interpreted cautiously.

Chapter 8

Conclusion and Future Work

Contents

8.1 Conclusion 61
8.2 Future Work 63

This chapter concludes the thesis by synthesizing findings on graph-based representation learning for architecture recovery and C2A mapping. It summarizes the conclusions across the three research objectives and outlines directions for further research arising from the findings and limitations of the work.

8.1 Conclusion

This thesis examined how graph-based representation learning can support software architecture recovery and C2A mapping, and what different implementation signals contribute to these tasks. Recovery was studied as an unsupervised task and mapping as a supervised task with limited labeled data. Across five studies, the work progressed from learning joint representations of static dependencies, folder organization, and source-code content to examining their individual contributions and their interactions with graph structure.

The first objective examined whether dependency, lexical, and organizational signals could be combined in learned representations for recovery and mapping, and how performance depended on the learning objective and model configuration. The joint representations supported both tasks, although the evidence differed between them. In recovery, the approaches of Papers I and II improved on or matched the evaluated baselines across most comparisons. In mapping, however, improvement over the Naive Bayes baseline was not specific to the GNN models. Both the GNN models of Paper III and an MLP using the same features outperformed that baseline. This improvement therefore does not by itself establish a benefit from message passing.

In the recovery comparison using a common GAT encoder and input features, the reconstruction-based approach generally produced better agreement with the reference decompositions than the contrastive approach. The approaches retained different training settings, so the comparison does not isolate the learning objective alone. Encoder choice also mattered: GAT generally outperformed

GCN in both tasks. Modeling dependency relations separately improved mapping with lexical and folder-token features (Paper III), although the encoder also had more parameters. With file vectors without folder-path features, the benefit was smaller and varied across representations and systems, as shown in the cross-study mapping evaluation. Separate relations gave no benefit in preliminary tests in dependency-only recovery (Paper IV). Relation-specific modeling was therefore useful under some input representations and tasks, rather than consistently beneficial.

The joint-signal comparisons did not separate the contributions of the individual signals. The second objective therefore examined what architectural information static dependencies and source-code content carry, and how representation choices make that information available. In recovery from static dependencies alone, the results suggest that the extent of dependency context encoded in a file's representation (structural reach) matters. Node2Vec extends this reach through random walks and GAT-DC through PPR diffusion. Both outperformed the classical baselines on most systems, whereas the two-layer GAT, limited to its two-hop neighborhood, was generally weaker. Agreement with the reference decomposition was lower where within-module and cross-module dependencies were similarly dense, whereas the dependency cohesion of the recovered partitions frequently matched or exceeded that of the reference. Dependency structure alone therefore supported recovery to a bounded extent: its usefulness depended on structural reach, on how well the reference modules were separated by dependencies, and on the evaluation criterion.

In mapping with an MLP, Word2Vec and zero-shot UniXcoder showed no consistent ordering, so the pretrained code transformer was not consistently better than a lexical representation. Dependency-guided fine-tuning improved UniXcoder on nearly all systems, supporting adaptation of the pretrained representation to the target system using dependency information. Node2Vec and fine-tuned UniXcoder generally outperformed the other two representations. Neither of the two was uniformly best: Node2Vec most often performed best, while fine-tuned UniXcoder was significantly stronger on some systems. Dependency information was therefore useful in mapping as in recovery, regardless of representation family, although differences in content and training prevent attributing the performance differences to dependency information alone.

The information carried by individual file representations did not establish what additional graph context could contribute. The third objective therefore examined how dependency-based message passing, folder organization, and member-level detail changed their usefulness. With undirected aggregation and fixed file representations, GAT and an MLP supplied with mean neighbor vectors improved on most systems for every representation, with no significant losses. Gains were largest for zero-shot UniXcoder and smallest for Node2Vec. Fine-tuned UniXcoder gained more than Node2Vec and had the highest median macro-F1 across systems for every predictor. The benefit of aggregation therefore depended on how dependency context was already encoded.

Aggregation direction also mattered. Source-to-target aggregation lowered median macro-F1 with Node2Vec, and the preferred direction depended on the system rather than the predictor or representation. Papers III and V agreed on some systems but differed on others, so neither the benefit nor the preferred direction can be assumed to transfer across setups. Undirected aggregation also added reverse edges, so direction and neighborhood size were not separated.

Folder organization supplied architectural evidence beyond static dependencies alone. Across the recovery studies, methods using organizational information agreed more closely with the reference decomposition, particularly in Paper IV where dependencies gave limited evidence of module boundaries, but produced less dependency-cohesive partitions than file-level dependency-only methods. Reference agreement and dependency cohesion therefore favored different configurations. In mapping with HGT and source-to-target dependency aggregation, folder nodes improved every file representation, but the model with fine-tuned UniXcoder did not consistently outperform the directory-based baseline, limiting the evidence for a benefit beyond directory organization itself (Paper V). Folder information and the reference decomposition may also overlap because directories are consulted during reference construction; the extent of this overlap was not quantified.

Member-level detail gave no consistent improvement in the evaluated configurations. With HGT and source-to-target dependency aggregation, member nodes lowered median mapping F1 for all three representations without folders and had representation-dependent effects with folders (Paper V). Since member nodes also changed how features entered the model, their effects cannot be attributed to the added structure alone. In the cross-study recovery evaluation, member-level edge attributes gave no significant improvement over the same model without these attributes. These findings therefore concern the evaluated encoders and representations, not the usefulness of member-level information in general.

Taken together, dependency information remained useful, but encoding it in file representations did not make message passing redundant, and folder organization supplied further information. The dependence on encoder choice observed under Objective 1 recurred as a dependence on predictor and aggregation direction, and across the studies no representation or configuration was uniformly preferable: the contribution of each signal depended on how it was represented and combined with other evidence.

8.2 Future Work

A first direction is to determine which aspects of graph context account for the observed performance differences. Undirected aggregation changed both propagation direction and neighborhood size, while member nodes changed graph structure and feature placement. Comparing incoming, outgoing, and combined neighborhoods at matched sizes could help distinguish direction from neigh-

borhood size, while member-level comparisons should preserve feature content. Parameter-matched comparisons of relation-specific and collapsed-relation models could distinguish relational detail from model capacity.

Paper I included interaction frequency and type together as member-level edge attributes. The cross-study recovery evaluation found no significant improvement from these attributes, but did not isolate the contribution of frequency. Whether frequency adds architectural information, and how it should enter representation learning through edge weights, attention terms, or relation encodings, remains open for both recovery and mapping.

A second direction concerns configuration selection and evaluation on unfamiliar and evolving systems. The benefit and preferred direction of neighbor aggregation varied across systems, but dependency homophily, used to interpret these differences, requires reference module assignments. Future work should examine whether properties observable in the dependency graph alone can guide configuration selection, with selection rules evaluated on systems not used to develop them. Folder-aware methods could also be assessed across systems that differ in how closely directories match the reference modules. Reporting performance separately for files whose module differs from that of their directory would help establish whether the methods provide useful predictions beyond directory organization.

The mapping evaluations made all system files available, leaving performance on subsequently introduced files untested. Future work could evaluate whether a model trained on mapped files from one version can assign new and changed files in later versions without retraining. The version history could also supply evolutionary signals, such as files that change together, which may carry architectural information that dependencies, folders, and code content do not. Practitioner studies could then assess whether the predictions support architecture maintenance.

Bibliography

[1] Nour Ali, Sean Baker, Ross O'Crowley, Sebastian Herold, and Jim Buckley. "Architecture consistency: State of the practice, challenges and requirements". In: *Empirical Software Engineering* 23 (2018), pp. 224–258.

[2] Domenico Amalfitano, Marco De Luca, Tiziano Santilli, Patrizio Pelliccione, and Anna Rita Fasolino. "Automated Software Architecture Design Recovery from Source Code Using LLMs". In: *European Conference on Software Architecture*. Springer. 2025, pp. 73–89.

[3] Periklis Andritsos, Panayiotis Tsaparas, Renée J Miller, and Kenneth C Sevcik. "LIMBO: Scalable clustering of categorical data". In: *International Conference on Extending Database Technology*. Springer. 2004, pp. 123–146.

[4] Nicolas Anquetil and Timothy C Lethbridge. "Recovering software architecture from the names of source files". In: *Journal of Software Maintenance: Research and Practice* 11.3 (1999), pp. 201–221. DOI: 10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1.

[5] Roberto Almeida Bittencourt, Gustavo Jansen de Souza Santos, Dalton Dario Serey Guerrero, and Gail C. Murphy. "Improving Automated Mapping in Reflexion Models Using Information Retrieval Techniques". In: *17th Working Conference on Reverse Engineering*. 2010, pp. 163–172.

[6] Joao Brunet, Roberto Almeida Bittencourt, Dalton Serey, and Jorge Figueiredo. "On the evolutionary nature of architectural violations". In: *2012 19th Working Conference on Reverse Engineering*. IEEE. 2012, pp. 257–266.

[7] Max Brunsfeld and contributors. *Tree-sitter: An Incremental Parsing System for Programming Tools*. https://tree-sitter.github.io/tree-sitter/. Accessed: 2026-04-22. 2018.

[8] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. "NEGAR: network embedding guided architecture recovery for software systems". In: *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE. IEEE, 2022, pp. 367–376. DOI: 10.1109/apsec57359.2022.00049.

[9] Andreas Christl, Rainer Koschke, and Margaret-Anne Storey. "Automated clustering to support the reflexion method". In: *Information and Software Technology* 49.3 (2007), pp. 255–274.

[10] Paul Clements, David Garlan, Reed Little, Robert Nord, and Judith Stafford. "Documenting software architectures: views and beyond". In: *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE. 2003, pp. 740–741.

[11] Norman Cliff. "Dominance statistics: Ordinal analyses to answer ordinal questions." In: *Psychological bulletin* 114.3 (1993), p. 494.

[12] Anna Corazza, Sergio Di Martino, Valerio Maggio, and Giuseppe Scanniello. "Investigating the use of lexical information for software system clustering". In: *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE. 2011, pp. 35–44. DOI: 10.1109/csmr.2011.8.

[13] Lakshitha De Silva and Dharini Balasubramaniam. "Controlling software architecture erosion: A survey". In: *Journal of Systems and Software* 85.1 (2012), pp. 132–151. DOI: 10.1016/j.jss.2011.07.036.

[14] Yuxiao Dong, Nitesh V Chawla, and Ananthram Swami. "metapath2vec: Scalable representation learning for heterogeneous networks". In: *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2017, pp. 135–144.

[15] Vijay Prakash Dwivedi, Chaitanya K Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. "Benchmarking graph neural networks". In: *Journal of Machine Learning Research* 24.43 (2023), pp. 1–48.

[16] Ramtin Ehsani, Ahmed Aljohani, Hyunsook Do, and Yuanfang Cai. "Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development". In: *ACM Transactions on Software Engineering and Methodology* (2025). DOI: 10.1145/3820165.

[17] Michael P Fay and Michael A Proschan. "Wilcoxon-Mann-Whitney or t-test? On assumptions for hypothesis tests and multiple interpretations of decision rules". In: *Statistics Surveys* 4 (2010), p. 1.

[18] Alexander Florean, Laoa Jalal, Zipani Tom Sinkala, and Sebastian Herold. "A Comparison of Machine Learning-Based Text Classifiers for Mapping Source Code to Architectural Modules". In: *15th European Conference on Software Architecture*. 2021.

[19] Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. "A comparative analysis of software architecture recovery techniques". In: *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE. 2013, pp. 486–496. DOI: 10.1109/ase.2013.6693106.

[20] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. "Obtaining ground-truth software architectures". In: *35th International Conference on Software Engineering (ICSE)*. IEEE. 2013, pp. 901–910. DOI: 10.1109/icse.2013.6606639.

[21] Joshua Garcia, Duc Le, Daniel Link, AS Pooyan Behnamghader, Eder Figueroa Ortiz, and Nenad Medvidovic. "An empirical study of architectural change and decay in open-source software systems". In: *USC-CSSE, Tech. Rep* (2014).

[22] Joshua Garcia, Daniel Popescu, Chris Mattmann, Nenad Medvidovic, and Yuanfang Cai. "Enhancing architectural recovery using concerns". In: *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE. 2011, pp. 552–555. DOI: 10.1109/ase.2011.6100123.

[23] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. "Diffusion improves graph learning". In: *Advances in neural information processing systems* 32 (2019).

[24] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. "Neural message passing for quantum chemistry". In: *International conference on machine learning*. PMLR. 2017, pp. 1263–1272.

[25] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.

[26] Aditya Grover and Jure Leskovec. "node2vec: Scalable feature learning for networks". In: *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 2016, pp. 855–864.

[27] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. "UniXcoder: Unified Cross-Modal Pre-training for Code Representation". In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2022, pp. 7212–7225. DOI: 10.18653/v1/2022.acl-long.499.

[28] Hao He, Courtney Miller, Shyam Agarwal, Christian Kästner, and Bogdan Vasilescu. "Speed at the cost of quality: How Cursor AI increases short-term velocity and long-term complexity in open-source projects". In: *Proceedings of the 23rd international conference on mining software repositories*. 2026, pp. 181–193.

[29] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. "Heterogeneous graph transformer". In: *Proceedings of the web conference 2020*. 2020, pp. 2704–2710.

[30] Lawrence Hubert and Phipps Arabie. "Comparing partitions". In: *Journal of classification* 2.1 (1985), pp. 193–218. DOI: 10.1007/bf01908075.

[31] Rakhshanda Jabeen, Morgan Ericsson, Yuyao Duan, and Anna Wingkvist. *What Carries Architectural Signal? A Code-to-Architecture Mapping Study*. 2026. arXiv: 2610.41628 [cs.SE]. URL: https://arxiv.org/abs/2610.41628.

[32] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery". In: *Proceedings of the ACM on Software Engineering* 3.FSE (July 2026). DOI: 10.1145/3808187.

[33] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph convolutional networks for mapping source code entities to architectural modules". In: *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE. 2025, pp. 472–481.

[34] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Embeddings for Software Architecture Recovery". In: *Nordic Machine Intelligence* (2026). DOI: 10.5617/nmi.13153. URL: https://journals.uio.no/NMI/article/view/13153.

[35] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Representation Learning for Software Architecture Recovery". In: *ESANN*. 2026.

[36] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. *When Does the Dependency Graph Help Code-to-Architecture Mapping?* 2026. arXiv: 2610.19769 [cs.SE]. URL: https://arxiv.org/abs/2610.19769.

[37] Nafiseh Sadat Jalali, Habib Izadkhah, and Shahriar Lotfi. "Multi-objective search-based software modularization: Structural and non-structural features." In: *Soft Computing-A Fusion of Foundations, Methodologies & Applications* 23.21 (2019). DOI: 10.1007/s00500-018-3666-z.

[38] Nils Johansson and Mauro Caporuscio. "Mapping source code to software architecture using embedding models: Insights from an analysis of data properties". In: *Information and Software Technology* 198 (2026), p. 108200. ISSN: 0950-5849. DOI: https://doi.org/10.1016/j.infsof.2026.108200. URL: https://www.sciencedirect.com/science/article/pii/S0950584926001898.

[39] Nils Johansson, Mauro Caporuscio, and Tobias Olsson. "Mapping Source Code to Software Architecture by Leveraging Large Language Models". In: *European Conference on Software Architecture*. Springer. 2024, pp. 133–149.

[40] Masoud Kargar, Ayaz Isazadeh, and Habib Izadkhah. "Semantic-based software clustering using hill climbing". In: *2017 international symposium on computer science and software engineering conference (CSSE)*. IEEE. 2017, pp. 55–60. DOI: 10.1109/csicsse.2017.8320117.

[41] Maurice G Kendall. "A new measure of rank correlation". In: *Biometrika* 30.1-2 (1938), pp. 81–93.

[42] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: *arXiv preprint arXiv:1412.6980* (2014).

[43] Thomas N Kipf and Max Welling. "Semi-supervised classification with graph convolutional networks". In: *arXiv preprint arXiv:1609.02907* (2016). DOI: 10.48550/arXiv.1609.02907.

[44] Thomas N Kipf and Max Welling. "Variational graph auto-encoders". In: *arXiv preprint arXiv:1611.07308* (2016). DOI: 10.48550/arXiv.1611.07308.

[45] Philippe Kruchten, Robert L. Nord, and Ipek Ozkaya. "Technical Debt: From Metaphor to Theory and Practice". In: *IEEE Software* 29.6 (2012), pp. 18–21. URL: https://philippe.kruchten.com/wp-content/uploads/2012/08/kruchten-120821-techdebt.pdf.

[46] Duc Minh Le, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shahbazian, and Nenad Medvidovic. "An empirical study of architectural change in open-source software systems". In: *2015 IEEE/ACM 12th working conference on mining software repositories*. IEEE. 2015, pp. 235–245. DOI: 10.1109/msr.2015.29.

[47] Dong-Hyun Lee et al. "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks". In: *Workshop on challenges in representation learning, ICML*. Vol. 3. 2. Atlanta. 2013, p. 896.

[48] Jörg Lenhard, Martin Blom, and Sebastian Herold. "Exploring the suitability of source code metrics for indicating architectural inconsistencies". In: *Software Quality Journal* 27.1 (2019), pp. 241–274.

[49] Qimai Li, Zhichao Han, and Xiao-Ming Wu. "Deeper insights into graph convolutional networks for semi-supervised learning". In: *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018.

[50] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidović, and Robert Kroeger. "Measuring the impact of code dependencies on software architecture recovery techniques". In: *IEEE Transactions on Software Engineering* 44.2 (2017), pp. 159–181. DOI: 10.1109/tse.2017.2671865.

[51] Onaiza Maqbool and Haroon Atique Babri. "The weighted combined algorithm: A linkage algorithm for software clustering". In: *Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings.* IEEE. 2004, pp. 15–24. DOI: 10.1109/csmr.2004.1281402.

[52] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. "Distributed representations of words and phrases and their compositionality". In: *Advances in Neural Information Processing Systems* 26 (2013).

[53] Janardan Misra, KM Annervaz, Vikrant Kaulgud, Shubhashis Sengupta, and Gary Titus. "Software clustering: Unifying syntactic and semantic features". In: *2012 19th Working Conference on Reverse Engineering*. IEEE. 2012, pp. 113–122. DOI: 10.1109/wcre.2012.21.

[54] Brian S Mitchell and Spiros Mancoridis. "On the automatic modularization of software systems using the bunch tool". In: *IEEE Transactions on Software Engineering* 32.3 (2006), pp. 193–208. DOI: 10.1109/tse.2006.31.

[55] Wiem Mkaouer, Marouane Kessentini, Adnan Shaout, Patrice Koligheu, Slim Bechikh, Kalyanmoy Deb, and Ali Ouni. "Many-objective software remodularization using NSGA-III". In: *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24.3 (2015), pp. 1–45. DOI: 10.1145/2729974.

[56] Sina Mohammadi and Habib Izadkhah. "A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code". In: *Information and Software Technology* 105 (2019), pp. 252–256. DOI: 10.1016/j.infsof.2018.09.001.

[57] Gail C Murphy, David Notkin, and Kevin Sullivan. "Software reflexion models: Bridging the gap between source and high-level models". In: *3rd ACM SIGSOFT Symposium on Foundations of Software Engineering*. 1995, pp. 18–28.

[58] Fionn Murtagh and Pierre Legendre. "Ward's hierarchical agglomerative clustering method: which algorithms implement Ward's criterion?" In: *Journal of classification* 31 (2014), pp. 274–295.

[59] S. M. Naim, Kostadin Damevski, and M. Shahriar Hossain. "Reconstructing and evolving software architectures using a coordinated clustering framework". In: *Automated Software Engineering* 24.3 (2017), pp. 543–572. DOI: 10.1007/s10515-017-0211-8.

[60] Rashid Naseem, Mustafa Mat Deris, Onaiza Maqbool, and Sara Shahzad. "Euclidean space based hierarchical clusterers combinations: an application to software clustering". In: *Cluster Computing* 22.Suppl 3 (2019), pp. 7287–7311. DOI: 10.1007/s10586-017-1408-0.

[61] Rashid Naseem, Onaiza Maqbool, and Siraj Muhammad. "Cooperative clustering for software modularization". In: *Journal of Systems and Software* 86.8 (2013), pp. 2045–2062. DOI: 10.1016/j.jss.2013.03.080.

[62] Hung Viet Nguyen. *Depends: Multilingual Dependency Analysis Tool*. https://github.com/multilang-depends/depends. Accessed: 2025-08-17. 2023.

[63] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. "s4rdm3x: A Tool Suite to Explore Code to Architecture Mapping Techniques". In: *Journal of Open Source Software* 6.58 (2021).

[64] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. "To automatically map source code entities to architectural modules with Naive Bayes". In: *Journal of Systems and Software* 183 (2022), p. 111095. DOI: 10.1016/j.jss.2021.111095.

[65] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. "Representation Learning with Contrastive Predictive Coding". In: *arXiv preprint arXiv:1807.03748* (2018).

[66] Rusheng Pan, Bingcheng Mao, Tianyi Ma, and Zhenhua Ling. "ArchAgent: Scalable Legacy Software Architecture Recovery with LLMs". In: *2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2026, pp. 19357–19361. DOI: 10.1109/ICASSP55912.2026.11462552. URL: https://arxiv.org/abs/2601.13007.

[67] Marios Papachristou. "Software clusterings with vector semantics and the call graph". In: *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2019, pp. 1184–1186. DOI: 10.1145/3338906.3342483.

[68] David Lorge Parnas. "On the criteria to be used in decomposing systems into modules". In: *Communications of the ACM* 15.12 (1972), pp. 1053–1058.

[69] Saeed Parsa and Omid Bushehrian. "A new encoding scheme and a framework to investigate genetic clustering algorithms". In: *Journal of Research and Practice in Information Technology* 37.1 (2005), pp. 127–143.

[70] Babak Pourasghar, Habib Izadkhah, Ayaz Isazadeh, and Shahriar Lotfi. "A graph-based clustering algorithm for software systems modularization". In: *Information and Software Technology* 133 (2021), p. 106469. DOI: 10.1016/j.infsof.2020.106469.

[71] K. Praditwong, Mark Harman, and Xin Yao. "Software module clustering as a multi-objective search problem". In: *IEEE Transactions on Software Engineering* 37.2 (2011), pp. 264–282. DOI: 10.1109/tse.2010.26.

[72] Shiva Prasad Reddy Puchala, Jitender Kumar Chhabra, and Amit Rathee. "Ensemble clustering based approach for software architecture recovery". In: *International Journal of Information Technology* 14.4 (2022), pp. 2013–2019. DOI: 10.1007/s41870-021-00846-0.

[73] Amit Rathee and Jitender Kumar Chhabra. "Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering". In: *Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers*. Springer. 2017, pp. 94–106.

[74] Michele Risi, Giuseppe Scanniello, and Genoveffa Tortora. "Using fold-in and fold-out in the architecture recovery of software systems". In: *Formal Aspects of Computing* 24.3 (2012), pp. 307–330. DOI: 10.1007/s00165-011-0199-y.

[75] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek. "Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen'sd for evaluating group differences on the NSSE and other surveys". In: *annual meeting of the Florida Association of Institutional Research*. Vol. 177. 34. 2006.

[76] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. "Deductive software architecture recovery via chain-of-thought prompting". In: *44th International Conference on Software Engineering: New Ideas and Emerging Results*. 2024, pp. 92–96. DOI: 10.1145/3639476.3639776.

[77] Zipani Tom Sinkala and Sebastian Herold. "InMap: Automated Interactive Code-to-Architecture Mapping Recommendations". In: *18th International Conference on Software Architecture*. 2021, pp. 173–183.

[78] Ioanna Stavropoulou, Marios Grigoriou, and Kostas Kontogiannis. "Case study on which relations to use for clustering-based software architecture recovery". In: *Empirical Software Engineering* 22.4 (2017), pp. 1717–1762. DOI: 10.1007/s10664-016-9459-z.

[79] Navid Teymourian, Habib Izadkhah, and Ayaz Isazadeh. "A fast clustering algorithm for modularization of large-scale software systems". In: *IEEE Transactions on Software Engineering* 48.4 (2020), pp. 1451–1462. DOI: 10.1109/tse.2020.3022212.

[80] Vassilios Tzerpos and Richard C Holt. "ACDC: an algorithm for comprehension-driven clustering". In: *Proceedings Seventh Working Conference on Reverse Engineering*. IEEE. 2000, pp. 258–267. DOI: 10.1109/wcre.2000.891477.

[81] Vassilios Tzerpos and Richard C Holt. "The orphan adoption problem in architecture maintenance". In: *Working Conference on Reverse Engineering*. IEEE. 1997, pp. 76–82.

[82] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. "Graph attention networks". In: *arXiv preprint arXiv:1710.10903* (2017). DOI: 10.48550/arXiv.1710.10903.

[83] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. "Deep graph infomax". In: *arXiv preprint arXiv:1809.10341* (2018).

[84] Zhihua Wen and Vassilios Tzerpos. "An effectiveness measure for software clustering algorithms". In: *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004.* IEEE. 2004, pp. 194–203. DOI: 10.1109/wpc.2004.1311061.

[85] Frank Wilcoxon. "Individual comparisons by ranking methods". In: *Biometrics bulletin* 1.6 (1945), pp. 80–83. DOI: 10.2307/3001968.

[86] Kaiyuan Yang, Junfeng Wang, Zhiyang Fang, Peng Wu, and Zihua Song. "Enhancing software modularization via semantic outliers filtration and label propagation". In: *Information and Software Technology* 145 (2022), p. 106818. DOI: 10.1016/j.infsof.2021.106818.

[87] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. "Software architecture recovery with information fusion". In: *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2023, pp. 1535–1547. DOI: 10.1145/3611643.3616285.

[88] Wenting Zhao, Wuxia Jin, Yiran Zhang, Ming Fan, Haijun Wang, Li Li, Yang Liu, and Ting Liu. "Software Architecture Recovery Augmented with Semantics". In: *IEEE Transactions on Software Engineering* (2025).

[89] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. "Beyond homophily in graph neural networks: Current limitations and effective designs". In: *Advances in neural information processing systems* 33 (2020), pp. 7793–7804.

Paper I

Graph Representation Learning for Software Architecture Recovery

I

Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Representation Learning for Software Architecture Recovery". In: *ESANN*. 2026.

Graph Representation Learning for Software Architecture Recovery

Rakhshanda Jabeen[1], Morgan Ericsson[1], Jonas Nordqvist[2] and Anna Wingkvist[1]

[1]Linnaeus University, Computer Science and Media Technology, Växjö, Sweden
[2]Linnaeus University, Mathematics, Växjö, Sweden

Abstract.

Software architecture recovery aims to infer a system's modular organization from source code, bridging the gap between design intent and implementation structure. Traditional techniques rely on handcrafted heuristics and often fail to capture deeper architectural relationships. We investigate whether GNNs can recover these relationships by framing the task as unsupervised representation learning over a multi-relational software graph. Our approach learns node embeddings that reflect architectural boundaries, offering a promising alternative to existing recovery methods.

1 Introduction

Software architecture defines a system's key components, their relationships, and the rules governing their interactions. Maintaining accurate architectural documentation is challenging, as modern systems evolve rapidly and often diverge from their intended designs [1]. Software architecture recovery (SAR) aims to reconstruct a system's high-level design from its implementation artifacts.

Recovering architectural modules is challenging because the source code expresses many overlapping design signals. However, dependency relationships (e.g, *Call*, *Import*, or *inheritance*), folder hierarchies, and code semantics all reflect how responsibilities are organized in practice. Dependencies capture how entities interact, folder locations mirror modular boundaries, and code identifiers encode developer intent and concepts [2, 3]. Together, these signals provide a foundation for inferring architectural structure from the implementation.

Traditional SAR approaches rely on hand-crafted heuristics or manually weighted combinations of structural, textual, and directory signals [3]. While effective in some settings, such methods struggle to generalize across systems with diverse styles and dependency patterns, motivating the use of learning-based techniques that can integrate multiple cues directly from the source code. Graph neural networks (GNNs) naturally operate on relational graphs and update node representations by aggregating information from neighbors via message passing [4]. They combine structural and semantic signals into a unified representation suitable for architecture recovery.

Building on GNNs, we introduce ARL (Architecture Representation Learning), an unsupervised method that learns architecture-aware embeddings of source code entities. In its base form, ARL treats each dependency type as a separate message-passing channel, enabling the encoder to distinguish among

different interaction patterns. We also propose ARL-E, which augments this representation with edge attributes capturing the member-level interactions (e.g., method calls, field accesses, import-driven references) that give rise to each dependency. These attributes summarize (i) which program elements triggered the dependency and (ii) how frequently the interaction occurs. The motivation is that architectural cohesion is shaped not only by the existence of a dependency, but also by its semantic origin and interaction frequency.

To obtain node embeddings, we use a heterogeneous Graph Attention Network (GAT) [5] trained with contrastive learning [6]. The resulting representations are then clustered to recover an architectural decomposition of the system.

2 Background and Related Work

Knowledge graphs (KGs) represent entities and their typed relationships as directed, labeled multi-graphs, or equivalently as sets of triples (h, r, t), where h and t are entities and r is a relation [7]. This abstraction naturally aligns with multi-relational heterogeneous graphs used in machine learning. GNNs operate directly on such relational structures via iterative message passing, shaping node embeddings based on neighborhood semantics and topology. They have shown strong performance across relational reasoning tasks, including computer vision, software engineering, and link prediction [4].

Software artifacts fit directly into this view: code files or classes serve as primary nodes, while internal members, such as functions and variables, generate typed static dependencies (e.g., calls, inheritance, imports) that capture rich relational structure. Modeling software systems as multi-relational graphs, therefore frames SAR as a representation-learning problem in which GNNs can derive embeddings that reveal the system's cohesive modular components.

Recent SAR methods have evolved from using structural dependencies or text similarity alone to combining multiple signals. SARIF [3] represents a recent state-of-the-art fusion approach that integrates structural, semantic, and directory features through hand-crafted weighting schemes. NEGAR [8] adopts node2vec-style representation learning for dependency graphs but disregards the heterogeneous relations or semantic cues from source code. LLM-based approaches have also been explored, using chain-of-thought prompting to map entities to predefined architectures [9]. While GNNs have been applied to several software engineering tasks, their use for unsupervised architecture recovery remains largely unexplored. To the best of our knowledge, no prior work applies GNN-based relational representation learning to SAR.

3 Methodology

The overall pipeline is shown in Figure 1. We now describe each component in detail, beginning with the construction of the multi-relational software graph.

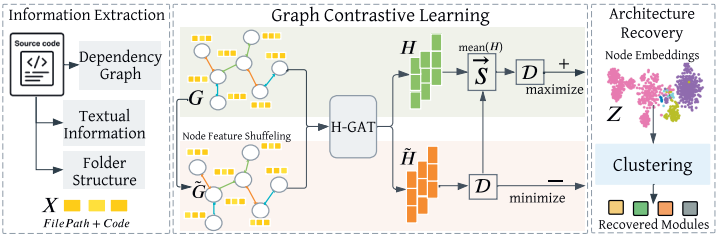


Fig. 1: Overview of ARL: (1) multi-relational software graphs are constructed with node and edge features; (2) a heterogeneous GAT encodes the graph and is optimized using contrastive learning; (3) the learned node embeddings are clustered to recover architectural components.

Graph Construction. We construct a multi-relational graph $G = (\mathcal{V}, X, A_{\mathcal{R}})$, where the nodes $\mathcal{V}$ represent source code entities, such as files or classes. Each relation type $\{A^{(r)} \mid r \in \mathcal{R}\}$ defines a separate directed adjacency matrix based on dependency information extracted using Depends [10]. Node features (X) are generated by combining tokens from the folder hierarchy and code semantics derived from file names and symbol identifiers that are not involved in inter-dependencies. Identifiers are split according to standard naming conventions, filtered for low-information tokens, and encoded as binary vectors, followed by l_2 normalization.

To extend the relational structure, we incorporate edge attributes that describe how each dependency arises. A dependency between two files may originate from interactions between different kinds of program elements (e.g., function-function, function-var, file-file). For each source–target member pair, we create a token capturing this interaction type and encode it as a binary vector followed by l_2 normalization. We additionally record the frequency of the dependency using a log-normalized count of how often the interaction occurs between the two files. The final edge attribute vector is the concatenation of the member-level interaction encoding and the dependency count.

Heterogeneous Graph Encoder and Contrastive Learning. Software dependencies vary in how strongly they reflect architectural cohesion, and heterogeneous GNNs outperform homogeneous ones on multi-relational software graphs [11]. We therefore use a heterogeneous GAT [5], which performs relation-specific message passing and applies attention to weight each connection. Each relation type has its own transformation and attention heads, and the outputs from all transformations are aggregated to produce the node embeddings.

We train the encoder using a contrastive objective inspired by Deep Graph Infomax (DGI) [6]. A corrupted view, $\tilde{G}$, is created by shuffling node features row-wise. The encoder generates embeddings H and $\tilde{H}$ for G and $\tilde{G}$, which

are contrasted using a discriminator $\mathcal{D}(\vec{h}, \vec{s}) = \vec{h}^{\top} W \vec{s}$, where $\vec{s}$ is a summary vector obtained by averaging the node embeddings of original graph, and W is a learnable weight matrix. The model is trained to score original node–summary pairs higher than corrupted ones using a binary cross-entropy objective.

Clustering. The resulting node embeddings (Z) are clustered using agglomerative clustering with Ward linkage. The number of clusters k is chosen from a search range by maximizing the silhouette score to obtain the final modules.

3.1 Experiments and Evaluation

Datasets. We evaluate CLAR on four open-source systems with available ground-truth (GT) architectures. Table 1 reports their basic properties and homophily ratio ($\mathcal{H}$)[1]. For JabRef and TeamMates, GT labels are produced by system experts at the SAEroCon workshops [11], while Bash and HDF follow the mappings provided in prior SAR studies [3].

Table 1: Dataset statistics and homophily.

System	Language	LoC	Files	Dependencies	Modules	$\mathcal{H}$
JabRef	Java	59K	1 131	18 426	6	0.54
TeamMates	Java	55K	778	17 818	15	0.29
Bash	C	115K	292	1 222	14	0.61
HDF	C	153K	153	1 239	15	0.45

Evaluation Metrics. We assess the alignment between the recovered and ground truth (GT) architectures using several standard architecture similarity measures: MoJoFM, A2A, and $\mathrm{C2C}_{\mathrm{cvg}}$. We also report the Adjusted Rand Index (ARI), a clustering agreement measure that ranges from -0.5 to 1.

MoJoFM measures the normalized number of move and join operations required to transform the recovered architecture into the GT. A2A builds on this by considering a broader range of edit operations for entities and modules. $\mathrm{C2C}_{\mathrm{cvg}}$ quantifies the proportion of GT clusters that are covered by clusters in the recovered architecture, based on a specified coverage threshold [3]. Higher values for these metrics indicate a better alignment between the architectures.

Experimental Configuration. We use a single heterogeneous GAT layer with one attention head, trained for 50 epochs using Adam with a learning rate of 10^{-3}. After training, we cluster the node embeddings, selecting k from [5, 20]. Each experiment is repeated 100 times with different random seeds. $\mathrm{C2C}_{\mathrm{cvg}}$ is computed using a coverage threshold of 0.5, counting a module as matched only when more than half its entities appear in one cluster.

For comparison, we use SARIF [3] as a baseline. SARIF integrates dependency types, textual similarity, and folder co-location into a weighted graph and

[1] $\mathcal{H}$ measures the fraction of dependencies whose endpoints lie in the same cluster.

applies modularity-based community detection. We run the official implementation[2] with default parameters.

4 Results and Analysis

Table 2 reports the median scores of 100 runs across all datasets. On average across all systems and metrics, ARL improves over SARIF by 13.4 points, while ARL-E (with edge attributes) yields a slightly higher gain of 13.9 points.

Table 2: Similarity of recovered architectures to GT. Metrics: MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. Δ = ARL − SARIF and Δ-E = ARL-E − SARIF. The top value per metric is highlighted.

Tech.	Bash				HDF				Jabref				TeamMates			
	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
SARIF	76	84	13	39	62	87	27	37	65	80	0	7	76	87	33	39
ARL	92	96	69	76	77	88	32	47	91	85	12	24	83	87	33	33
ARL-E	91	96	69	76	78	88	32	48	91	86	15	29	84	87	33	31
Δ	+16	+12	+56	+37	+15	+1	+5	+10	+26	+5	+12	+18	+7	0	0	-6
Δ-E	+15	+12	+56	+37	+16	+1	+6	+11	+26	+5	+15	+22	+8	0	0	-8

Dataset-Specific Behavior. Bash and JabRef show the most considerable improvements, consistent with their moderate homophily ratios of 0.54 and 0.61, respectively. Most dependencies occur within the same module, providing strong signals for GAT. Consequently, MoJoFM increases from 76 to 92 in Bash and from 65 to 91 in JabRef, with corresponding gains in ARI. HDF shows a similar pattern, but with smaller margins, exhibiting an increase of 15-16 in MoJoFM. This suggests that dependency information is moderately informative here; edge attributes add some benefit but do not dominate performance.

TeamMates exhibits limited improvements. Its low homophily (0.29) indicates strong inter-module coupling, meaning dependency-based features provide weak architectural cues. Consequently, ARL yields modest improvements in MoJoFM, and other metrics remain close to SARIF. ARL-E does not help further, as edge attributes amplify noisy cross-module interactions.

Metrics Behavior. The largest gains appear in MoJoFM and ARI, especially in more homophilic systems. $C2C_{cvg}$ is inherently unstable because it counts a ground truth module as matched only when a single recovered cluster contains at least half of its entities at a coverage threshold of 0.5. Even a small split can cause the cluster to fall below this threshold, leading to a sharp drop in the score—particularly in systems with a few large modules. A2A remains uniformly high across methods and therefore provides limited discriminative value.

[2]https://github.com/anonymous2f4a9d/SARIF_FSE23

5 Conclusion

We presented ARL, a software architecture recovery approach that learns node embeddings using a heterogeneous GAT encoder over a multi-relational dependency graph. Clustering these embeddings yields coherent modular structures that align well with the ground truth. We also evaluated ARL-E, which extends ARL with edge attributes capturing member-level interaction types and interaction strength. While ARL-E provides small additional gains in some systems, its benefits remain limited in more heterotrophic systems where cross-module interactions dominate.

Future work will focus on lifting these learned representations into an explainable, symbolic layer to support the derivation of architectural constraints and provide justifiable reasoning. This direction aims to integrate neural and symbolic techniques for neuro-symbolic architectural analysis.

References

[1] Steffen Herbold, Christoph Knieke, Andreas Rausch, and Christian Schindler. Neurosymbolic architectural reasoning: Towards formal analysis through neural software architecture inference. *arXiv preprint arXiv:2503.16262*, 2025.

[2] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. Obtaining ground-truth software architectures. In *2013 35th International Conference on Software Engineering (ICSE)*, pages 901–910. IEEE, 2013.

[3] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. Software architecture recovery with information fusion. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, pages 1535–1547, 2023.

[4] Luís C Lamb, Artur Garcez, Marco Gori, Marcelo Prates, Pedro Avelar, and Moshe Vardi. Graph neural networks meet neural-symbolic computing: A survey and perspective. *arXiv preprint arXiv:2003.00330*, 2020.

[5] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[6] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. Deep graph infomax. *arXiv preprint arXiv:1809.10341*, 2018.

[7] Federico Bianchi, Gaetano Rossiello, Luca Costabello, Matteo Palmonari, and Pasquale Minervini. Knowledge graph embeddings and explainable ai. *arXiv preprint arXiv:2004.14843*, 2020.

[8] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. Negar: network embedding guided architecture recovery for software systems. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*, pages 367–376. IEEE, 2022.

[9] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. Deductive software architecture recovery via chain-of-thought prompting. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*, pages 92–96, 2024.

[10] Hung Viet Nguyen. Depends: Multilingual dependency analysis tool. `https://github.com/multilang-depends/depends`, 2023. Accessed: 2025-08-17.

[11] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. Graph convolution networks for mapping source code entities to architectural modules. In *22nd IEEE International Conference on Software Architecture (ICSA 2025), Odense, Denmark, March 31-April 4, 2025*. IEEE, 2025.

Paper II

GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery

II

Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery". In: *Proceedings of the ACM on Software Engineering* 3.FSE (July 2026). DOI: 10.1145/3808187.

GAER: Graph Auto-encoders for Unsupervised Software Architecture Recovery

RAKHSHANDA JABEEN, Electrolux Professional, Sweden
MORGAN ERICSSON, Linnaeus University, Sweden
JONAS NORDQVIST, Linnaeus University, Sweden
ANNA WINGKVIST, Linnaeus University, Sweden

Recovering the modular architecture of software systems from source code remains challenging when documentation is incomplete or outdated. Manual recovery is labor-intensive and does not scale to large systems. Although automated techniques have been proposed, many rely on handcrafted heuristics. They may focus on a limited set of inputs (such as dependencies or text), or combine multiple inputs using fixed rules, rather than integrating structural and semantic cues in a data-driven way.

We introduce GAER, an unsupervised architecture recovery approach based on graph autoencoders. GAER models a system as a heterogeneous dependency graph with typed relations and combines dependency information, folder hierarchy, and code semantics as node features. We study two factors that influence recovery outcomes: the choice of encoder (Graph Attention Network [GAT] vs. Graph Convolution Network [GCN]) and the number of clusters used for the final decomposition. We evaluate GAER across 10 open-source systems and benchmark it against established architecture recovery baselines using standard recovery measures against ground truth mappings. Overall, GAER is competitive, with strong baselines and often achieving higher agreement with ground truth, and the GAT variant generally performs best. By integrating multiple architectural cues in a graph learning framework, GAER produces architectural views that support system comprehension and reduce the effort required to maintain architectural documentation.

CCS Concepts: • **Software and its engineering → Software architectures**; **Software reverse engineering**; • **Computing methodologies → Unsupervised learning**.

Additional Key Words and Phrases: software architecture recovery, graph neural networks, graph auto-encoders

ACM Reference Format:
Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. GAER: Graph Auto-encoders for Unsupervised Software Architecture Recovery. *Proc. ACM Softw. Eng.* 3, FSE, Article FSE180 (July 2026), 22 pages. https://doi.org/10.1145/3808187

1 Introduction

Software systems evolve rapidly, making it increasingly difficult to maintain accurate and up-to-date architectural documentation [6]. The original design documents often become outdated or lost as a software project undergoes frequent refactoring, fast development cycles, and codebase changes. This process leads to architectural drift and erosion, where the implemented system diverges from its intended design, complicating maintenance and evolution due to a lack of reliable architectural knowledge [9].

Authors' Contact Information: Rakhshanda Jabeen, Electrolux Professional, Ljungby, Sweden, rakhshanda.jabeen@electroluxprofessional.com; Morgan Ericsson, Linnaeus University, Växjö, Sweden, morgan.ericsson@lnu.se; Jonas Nordqvist, Linnaeus University, Växjö, Sweden, jonas.nordqvist@lnu.se; Anna Wingkvist, Linnaeus University, Växjö, Sweden, anna.wingkvist@lnu.se.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2994-970X/2026/7-ARTFSE180
https://doi.org/10.1145/3808187

Software Architecture Recovery (SAR) addresses this challenge by automatically inferring a system's modular decomposition from its source code. SAR supports program understanding and re-engineering, since manually organizing hundreds or thousands of files into coherent modules is labor-intensive and does not scale. For example, recovering the architecture of a medium-sized system (70K–280K lines of code) can require hundreds of hours of expert effort [11].

Software dependency graphs capture relations among source code entities, and files that implement related functionality often appear close in the graph. Many traditional architecture recovery techniques, therefore, exploit static dependencies and use clustering or search-based optimization to produce decompositions with high cohesion and low coupling [1, 25, 27, 49]. However, shared utilities and layered interactions may introduce many cross-module dependencies, so "connected by an edge" does not necessarily imply "belongs to the same module" [46]. Moreover, dependency relations differ in how strongly they reflect architectural boundaries [24, 47]. The recent dependency-only representation learning method, NEGAR [4], uses node2vec-style embeddings to capture higher-order structural context beyond immediate neighbors, but recovery quality can be sensitive to the choice of dependency types.

Directory structure can reflect how developers organize the codebase into subsystems, while textual cues from identifiers and comments can capture domain concepts and developer intent [8, 11, 13]. Accordingly, some architecture recovery approaches incorporate textual information (e.g., source code and comments) via topic modeling or latent semantic indexing to group artifacts by semantic similarity [8, 13, 43]. These semantic cues can be particularly helpful when structural dependencies are noisy or dominated by cross-cutting utilities, as they provide complementary evidence about intended responsibilities.

More recent techniques combine both structural and non-structural cues, but often integrate them using hand-crafted similarity measures and manually tuned weighting schemes rather than integrating signals in a data-driven way [38, 56]. For example, SARIF [56] augments dependency information with directory structure and textual similarity, but combines these cues using manually designed similarity functions and tuned weights, including dependency-type weights. Since dependency patterns and architectural styles differ across projects, fixed heuristics may not generalize consistently across systems.

We illustrate these challenges using two systems from our study. JabRef contains widely used utility code that creates many cross-module dependencies, making structure-only signals ambiguous; here, directory context and identifier-level semantics can provide complementary evidence for recovering a meaningful decomposition. Chromium, in contrast, is substantially larger and more complex, where fixed fusion rules and manually tuned weights are brittle across diverse dependency patterns; in such settings, learning-based embeddings can better capture global structural context. These observations motivate a representation-learning approach that jointly learns from both structural and non-structural signals, without manually tuning weights across dependency types or information sources.

We propose GAER[1] [18], an unsupervised SAR approach that models the software system as a multi-relational dependency graph and learns relation-aware embeddings of software entities using a heterogeneous Graph Neural Network (GNN) within a Graph Auto-Encoder (GAE) framework. The learned node embeddings are clustered to form the architectural decomposition.

A key design choice in GAER is the encoder. We study two variants, a graph convolution network (GCN) and a graph attention network (GAT), to assess their effectiveness in SAR. We also analyze how varying architectural granularity (i.e., the number of clusters) influences the

[1]The source code and data used are also available at https://github.com/sse-lnu/GAER.git (v.1.1.0).

quality of recovered decompositions. For evaluation, we compare GAER against several established techniques. The primary contributions of this work can be summarized as follows:

(1) **Typed Dependency Graph and Heterogeneous Encoder.** We model each system as a multi-relational dependency graph, where nodes are software entities (files/classes), edges are typed static dependencies, and node features combine semantic signals from code identifiers and directory context from file paths. We use a relation-specific (heterogeneous) GNN encoder with type-specific parameters, so different dependency types are modeled explicitly rather than merged into a single weighted graph.

(2) **Unsupervised Architecture Recovery via GAE.** We train a graph auto-encoder to reconstruct observed dependency edges under a single objective, so the learned embeddings preserve the dependency structure. The resulting node embeddings are then clustered to recover architectural modules.

(3) **Empirical Evaluation.** We evaluate GAER on 10 systems with available ground-truth mappings against established baselines using standard architecture-recovery metrics. We further analyze the impact of encoder choice (GCN vs. GAT), cluster granularity, and run-to-run variability of stochastic methods.

The paper is structured as follows. Section 2 introduces the background concepts underlying our approach. Section 3 describes GAER, including heterogeneous graph construction, the GAE-based learning framework, and the clustering procedure. Section 4 introduces the datasets and baselines, details the experimental setup, and defines the evaluation metrics. Section 5 reports and analyzes the results. Section 6 reviews related work, Section 7 discusses threats to validity, and Section 8 concludes the paper and outlines future directions.

2 Background

Software architecture recovery (SAR) can be viewed as a graph representation learning problem. A software system can be represented as a set of software entities connected by static dependencies such as calls, imports, and inheritance. Together, these dependencies define a multi-relational graph. In addition to the dependency structure, each entity can be associated with attributes such as code identifiers and directory information. The goal is to learn representations that capture architectural similarity, so that entities within the same module are close in the learned vector space.

GNNs are well-suited to SAR because they learn node representations by combining node attributes with graph structure through message passing. At a high level, each node updates its representation by aggregating information from its neighbors along the graph edges [20]. In a software graph, this allows an entity representation to reflect both its own local attributes and the dependency context in which it appears. Moreover, different dependency types can carry different architectural meanings. For example, function-call edges may indicate stronger modular boundaries than type-usage edges [47]. For this reason, relation-specific processing can be used to handle edge types separately rather than merging them into a single graph.

Graph Auto-Encoders (GAEs) [21] provide an unsupervised way to learn such representations. A GAE consists of an encoder, which maps nodes to latent embeddings in a vector space, and a decoder, which attempts to reconstruct observed graph connections from those embeddings. Training encourages the model to produce embeddings that preserve useful structural patterns in the graph while incorporating the input node attributes. After training, these embeddings can be clustered to recover architectural modules.

3 Methodology

Our methodology decomposes software architecture recovery into three stages: (i) constructing a heterogeneous, multi-relational graph from static dependencies, code identifiers, and directory structure; (ii) learning embeddings of software entities with a heterogeneous GNN encoder within a GAE framework; and (iii) clustering the learned embeddings to infer architectural modules. Figure 1 illustrates the end-to-end pipeline of GAER.

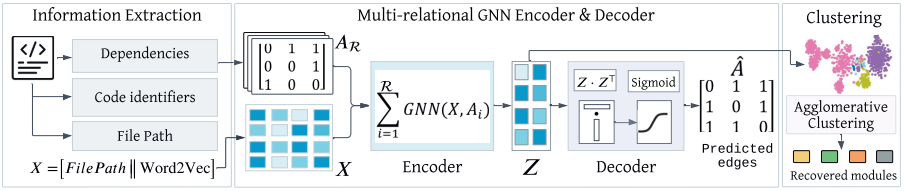


Fig. 1. Overview of the GAER pipeline. Source code is transformed into a multi-relational dependency graph with node features derived from Word2Vec-embedded code identifiers and binary directory-path vectors. A multi-relational heterogeneous GNN encoder learns node embeddings Z within the GAE framework, which are clustered to recover architectural modules.

3.1 Graph Construction

We integrate three sources of information: static dependencies, code identifiers, and directory structure. Dependencies are represented as a directed multi-relational graph, whereas identifiers (class, method, and variable names) and file paths are encoded as node features. Together, these signals provide the structural and contextual information needed for architecture recovery [41, 56].

Formally, we represent the system as a multi-relational directed graph $G = (X, A_{\mathcal{R}})$ with n entities (e.g., files or classes), where $X \in \mathbb{R}^{n \times F}$ is the node feature matrix and $A_{\mathcal{R}} = \{A^{(r)}\}_{r \in \mathcal{R}}$ is the set of relation-specific adjacency matrices. For each relation $r \in \mathcal{R}$, an entry in the adjacency matrix is defined as:

$$A_{ij}^{(r)} = \begin{cases} 1 & \text{if a dependency of type } r \text{ exists from } i \text{ to } j, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

We use the static analysis tool *Depends* [36] to extract implemented code entities (e.g., classes and files), their associated program elements (e.g., methods, variables, etc), and typed dependencies from the subject systems. The extracted dependency types (e.g., calls, inheritance, interface implementation, imports, field usage, exceptions) define $\mathcal{R}$ and populate the adjacency matrices $\{A^{(r)}\}_{r \in \mathcal{R}}$.

Node features for each file are derived from textual identifiers and file paths. We normalize code identifiers by splitting *camelCase*, *PascalCase*, and *snake_case*, and remove low-information tokens (stopwords, very short tokens, and tokens appearing in over 90% of files). The remaining tokens are embedded into 100-dimensional vectors using the skip-gram variant of Word2Vec [28], trained for 10 epochs with a context window of 5. The textual representation $\mathcal{T}_i$ of an entity i is then computed by averaging its Word2Vec embeddings and then normalizing them using the ℓ_2 norm.

To encode directory structure, file paths are tokenized and represented as binary bag-of-words vectors. The final feature vector for node i is the concatenation of its binary file-path vector and the Word2Vec embeddings as follows:

$$x_i = \left[\text{Binary(FilePath)}_i \,\middle\|\, \text{Word2Vec}(\mathcal{T}_i)\right] \tag{2}$$

3.2 GAER Model

Given the heterogeneous graph constructed in Section 3.1, GAER learns node embeddings by training a graph auto-encoder to reconstruct observed dependencies. The encoder maps the attributed multi-relational graph to an embedding matrix Z, where each row of Z corresponds to the learned embedding of a software entity:

$$Z = f_\theta(X, A_\mathcal{R}) \tag{3}$$

The decoder then uses these embeddings to estimate which pairs of entities should be connected.

3.2.1 Relation-Specific GNN Encoder. We instantiate the encoder with a relation-specific GNN operating on the software graph $G = (X, A_\mathcal{R})$, where X encodes node features (code identifiers and file paths) and $A_\mathcal{R}$ encodes typed dependencies. We use a single encoder layer, so each node updates its representation using messages from its one-hop neighbors. This produces embeddings that reflect both node attributes and local dependency structure.

We consider two encoder variants, GCN [20] and GAT [52], to compare fixed degree-normalized aggregation with learnable neighbor weighting. Because different dependency types can reflect architectural boundaries with different strengths [4, 47], we use a heterogeneous encoder with type-specific parameters, allowing each relation type to be handled separately. Such heterogeneous encoders have also been shown to outperform homogeneous counterparts in related tasks [17].

Intuitively, each relation $r \in \mathcal{R}$ is processed by its own message-passing channel with separate learnable parameters, and the relation-specific outputs are summed to obtain node embeddings. Formally, the encoder produces an embedding matrix $Z \in \mathbb{R}^{n \times F'}$. Let $\mathbf{h}_i^{(l)}$ denote the embedding of node i at layer l, $\mathcal{N}_i^r$ denote the neighbors of node i under relation r, and let d_i^r denote the degree of node i under relation r. In our setup, we use a single encoder layer, i.e., $\mathbf{h}_i^{(0)} = x_i$ and $z_i = \mathbf{h}_i^{(1)}$, where z_i denotes the final F'-dimensional embedding of node i. For each dependency type r, we apply a learnable relation-specific linear transformation $W^r \in \mathbb{R}^{F' \times F}$ before aggregating neighbor messages.

Heterogeneous GCN. In the heterogeneous GCN variant, node i is updated by averaging degree-normalized messages from its neighbors for each dependency type. Messages from relation-specific neighbors $j \in \mathcal{N}_i^r$ are first linearly transformed using W^r and normalized by the degrees d_i^r and d_j^r. The aggregated message is then passed through a non-linear activation function (ReLU) to obtain the updated node representation:

$$\mathbf{h}_i^{(1)} = \mathrm{ReLU}\left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_i^r} \frac{1}{\sqrt{d_i^r d_j^r}} W^r \mathbf{h}_j^{(0)} \right) \tag{4}$$

Degree normalization controls the influence of high-degree nodes and stabilizes aggregation across different dependency types.

Heterogeneous GAT. In the heterogeneous GAT variant, degree-based normalization is replaced by learnable attention weights inspired by the self-attention mechanism [51]. The attention weights quantify the importance of each neighbor $j \in \mathcal{N}_i^r$ when updating node i under dependency type r. For each relation r, node features are first linearly transformed using W^r, and attention scores e_{ij}^r are computed and normalized with a softmax function to obtain α_{ij}^r. The attention-weighted messages are then aggregated and passed through a non-linear activation function (ELU) to produce

the updated node representation:

$$\mathbf{h}_i^{(1)} = \mathrm{ELU}\left(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r} \alpha_{ij}^r \, W^r \mathbf{h}_j^{(0)}\right) \tag{5}$$

3.2.2 ***Decoder and Reconstruction Loss.*** The decoder g_ϕ produces an estimated adjacency matrix $\hat{A} = g_\phi(Z)$, where each entry $\hat{A}_{ij}$ represents the predicted probability of an edge between the node pair (i, j). It is computed by taking the inner product of the corresponding node embeddings and applying a logistic sigmoid function as shown in Eq. 6, where $(\cdot)^\top$ denotes transposition.

$$\hat{A}_{ij} = \frac{1}{1 + \exp\left(-z_i^\top z_j\right)} \tag{6}$$

For each dependency type $r \in \mathcal{R}$, let $E_r = \{(i, j) \mid A_{ij}^{(r)} = 1\}$ denote the observed edge set. Since the decoder reconstructs a single symmetric adjacency matrix, we treat dependencies as undirected for reconstruction by ignoring edge direction. The positive edge set is therefore defined as the union of observed edges across all relation types, $E^+ = \bigcup_{r\in\mathcal{R}} E_r$.

Negative edges are node pairs $(i, j) \notin E^+$. Because the number of such pairs is large, we sample a set of negative pairs S^- uniformly at random from the complement of E^+, matching the number of sampled negatives to the number of positive edges. The reconstruction loss then encourages high scores for observed edges and low scores for sampled negative edges using the following binary cross-entropy objective:

$$\mathcal{L} = -\sum_{(i,j)\in E^+} \log \hat{A}_{ij} - \sum_{(i,j)\in S^-} \log\left(1 - \hat{A}_{ij}\right) \tag{7}$$

By learning embeddings that reconstruct observed dependency patterns, the GAE captures structural regularities in the software graph while also exploiting the node attributes provided as input. These embeddings are then used for downstream software architecture recovery.

3.3 Clustering for Architecture Recovery

To recover architectural modules from node embeddings learned from the GAE, we apply Agglomerative Hierarchical Clustering (AHC) [33] with Ward linkage. AHC starts with each entity as its own cluster and iteratively merges the closest clusters. Rather than analyzing the full hierarchy, we output a flat partition by selecting the number of clusters k.

GAER supports either a user-defined k or automatic selection, where k is chosen from a candidate search range $\mathcal{K}$ using the elbow criterion. For each $k \in \mathcal{K}$, we compute a Within-Cluster Sum of Squares (WCSS) value from the resulting AHC partition, defined as the sum of squared Euclidean distances from each entity to the mean (centroid) of its assigned cluster. Since WCSS decreases monotonically as k increases, we select k using a standard distance-to-chord heuristic on the WCSS curve, i.e., the k that maximizes the perpendicular distance to the line connecting the smallest and largest candidate k. For scalability, on large systems, the WCSS curve is estimated on a random subsample of 2 000 entities, and the final AHC clustering is then applied to the full dataset using the selected k. Algorithm 1 summarizes the GAER algorithm.

4 Experiments

We evaluate GAER and a set of representative baseline methods on a collection of open-source software systems with known ground truth (GT) labels. This section describes the subject systems, the baseline techniques and their configuration, the GAER experimental setup, and the evaluation

Algorithm 1: GAER

Input: Software system with n entities, their file paths, identifier tokens, and typed dependencies $\{E_r\}_{r\in\mathcal{R}}$;
Embedding dimension d; training epochs ϵ; optimizer;
Clustering: user-defined k or candidate range $\mathcal{K}$
Output: Recovered architecture (module assignment) $\mathcal{A}$

Step 1: Construct Multi-relational Adjacency Matrices and Node Features
$X \leftarrow \left[\textsc{Binary}(\text{file paths}) \,\|\, \textsc{Word2Vec}(\text{code})\right]$; // node features $X \in \mathbb{R}^{n\times F}$
Construct $A_{\mathcal{R}} = \{A^{(r)}\}_{r\in\mathcal{R}}$ from $\{E_r\}_{r\in\mathcal{R}}$; // binary adjacency matrices

Step 2: Representation Learning (GAE)
Initialize encoder f_θ (het. GCN/GAT) and decoder g_ϕ (inner-product);
$E^+ \leftarrow \bigcup_{r\in\mathcal{R}} E_r$; // observed dependencies
for $t = 1$ **to** ϵ **do**
  Sample S^- from the complement of E^+;
  $Z \leftarrow f_\theta(X, A_{\mathcal{R}})$; // node embeddings, $Z \in \mathbb{R}^{n\times F'}$
  $\hat{A} \leftarrow g_\phi(Z)$; // edge probabilities, $\hat{A} \in \mathbb{R}^{n\times n}$
  Compute loss $\mathcal{L}(E^+, S^-)$;
  Update f_θ, g_ϕ using optimizer to minimize $\mathcal{L}$;

Step 3: Clustering if k *is user-defined* **then**
  $k^* \leftarrow k$;
else
  $k^* \leftarrow \textsc{Elbow}\Big(\text{WCSS}(\text{AHC}(Z,k))_{k\in\mathcal{K}}\Big)$;
$\mathcal{A} \leftarrow \text{AHC}(Z, k^*)$; **return** $\mathcal{A}$

procedures and metrics used to benchmark performance. Our experimental study is structured to address the following research questions:

- **RQ1:** How well does GAER perform in terms of established architecture recovery measures compared to existing baselines?
- **RQ2:** What is the impact of the number of clusters (granularity) on the recovered decomposition and recovery metrics?
- **RQ3:** How stable are stochastic architecture recovery techniques across repeated runs in terms of evaluation metric variability?

4.1 Subject Systems

We evaluate our approach on ten open-source systems that have been widely studied in prior work on SAR and Code-to-Architecture (C2A) mapping [4, 37, 41, 56]. The systems cover different domains and vary in size. Using published GT architectures is common in SAR as producing reliable C2A mappings requires substantial manual effort and expert knowledge [11]. These established GT architectures also allow for direct comparability with previous studies. Table 1 summarizes the key characteristics of the systems, including the number of architectural components k.

For ArchStudio (AS4), Hadoop, OODT, and Bash, we use the C2A mapping curated by Garcia et al. [11]. These mappings were produced using a certification process in which researchers first recovered an initial architectural decomposition from documentation and source code, and then refined it through iterative validation with system experts. The reported average number of person-hours to obtain the mapping was approximately 107 per system for these medium-sized systems. The same line of work also provides the GT mapping for Chromium ($\approx$ 11M LoC), which required long-term collaboration with developers over roughly two years [23].

JabRef and TeamMates (T.Mates) were studied in the Software Architecture Erosion and Architectural Consistency (SAEroCon) workshops in 2016 and 2017, where the system experts provided

Table 1. Properties and versions of software systems.

Name	Version	Language	LoC	|Files|	|Dependencies|	k	Domain
ArchStudio	4	Java	280K	583	9.1K	57	Architecture Modeling
Hadoop	0.19.0	Java	200K	591	24.7K	67	Distributed Processing
JabRef	v3.7	Java	59K	1 180	26.3K	6	Reference Management
OODT	0.2	Java	180K	876	18K	216	Data Management
TeamMates	v5.11	Java	55K	778	49.7K	15	Peer Feedback System
Chromium	svn-171054	C++	11M	18 343	713.5K	67	Web Browser
Bash	4.2	C	115K	292	1.8K	14	Shell
HDC	46ff87	C++	26K	207	2.5K	11	Camera Interface
HDF	0e196f	C	153K	153	2.8K	9	Driver Subsystem
Libxml	2.4.22	C	81K	82	9.3K	17	XML Parser

the reference architecture and the corresponding C2A mapping [37]. The GT C2A mappings for HDF, HDC, and Libxml were developed in collaboration with industry partners and are provided in the SARIF benchmark suite [56], with labels distributed via its GitHub[2] repository.

4.2 Baseline Methods

We compare GAER against established SAR techniques that use different information sources and recovery strategies. We include dependency-driven methods (ACDC [49], WCA [27], LIMBO [1], FCA [48], NEGAR [4]), a semantic-similarity method (ARC [13]), and hybrid methods that combine structural and semantic cues (SADE [38], SARIF [56]). A broader overview of these techniques is provided in the Table 8. We use SARIF and NEGAR as primary reference methods in our analysis.

SARIF [56] integrates three information sources: dependency graph, textual features extracted from code and comments, and directory structure into a unified weighted graph. Dependency types are assigned weights tuned to maximize modularity, while additional edge weights are derived from TF-IDF similarity and topic modeling of code identifiers and comments, as well as from directory co-location. The resulting weighted graph is clustered using the Louvain community detection algorithm [5], with the resolution parameter γ controlling the clustering granularity.

NEGAR [4] learns node embeddings from an unweighted dependency graph. To do so, it applies node2vec [14] and generates a fixed number of second-order random walks for each node i. The return parameter p and in-out parameter q control the balance between local and outward exploration. Each walk yields a node sequence, and the collection of sequences is treated as a corpus. A word2vec-style SkipGram [28] model is then trained to learn embeddings by predicting the neighborhood of node i within a context window along these sequences. Finally, k-means clustering is applied to the embeddings to recover architectural modules.

4.3 Experimental Configuration

In preliminary experiments on a subset of systems, we explored different encoder depths (1–2 layers), embedding dimensions (64, 128, and 256), and training lengths (30, 50, and 100 epochs). Increasing the number of epochs yielded only minor differences in recovery metrics, so we use 30 epochs and a consistent GAER configuration across all datasets. Specifically, we use a single-layer GCN or GAT encoder with 128-dimensional embeddings (a single attention head for GAT), trained with Adam at a learning rate of 1e-4. For very small systems (<100 entities), we use a learning rate of 1e-3 and train for 50 epochs; this applies to Libxml.

[2] https://github.com/anonymous2f4a9d/SARIF_FSE23

We reuse baseline results (ACDC, ARC, WCA, LIMBO, FCA, and SADE) from the SARIF GitHub[2]. Methods that require a user-specified number of clusters (WCA, LIMBO, ARC) are evaluated for $k \in [10, 100]$ with a step of 5; ARC additionally varies the number of topics from 50 to 150 with a step of 10. We report the best result observed for each metric across the evaluated settings. We run SARIF using the same replication package across all datasets, with the default resolution parameter ($\gamma = 1.7$) reported by the authors. We observed no run-to-run variation across repeated executions, so we report a single run per dataset.

We apply NEGAR following Chen et al.'s recommended configuration and tuning setup (Table II in [4]). We tune on AS4/Bash/Hadoop and use the reported Chromium setting; for the remaining datasets, we keep a consistent configuration (128 dimensions, walk length 30, context size 15, 200 walks per node, in-out and return parameters $p = q = 1$, 5 negative samples, and 5 training epochs). While NEGAR evaluates dependency types via separate pipelines, we merge all dependency types into a single dependency graph, achieving comparable performance. To support our unknown-k evaluation, we additionally select k from the same user-defined search range described in Sec. 3.3 using k-means on the learned embeddings. We use the same dependencies extracted by Depends across GAER, NEGAR, and SARIF.

All experiments are conducted on a server with 2x AMD EPYC 7413 CPUs (24 cores, 2.65GHz base clock), 512GB of RAM, and an NVIDIA A100 GPU (SMX4, 40GB RAM).

4.4 Performance Evaluation

Multiple reasonable architectural decompositions may exist for the same software system, since experts can disagree on architectural boundaries and design intent [11]. Consequently, a recovered decomposition may differ from the particular GT architecture used for evaluation. To avoid reliance on any single criterion, we report a set of complementary metrics using standard definitions and computation procedures, as summarized in prior architecture-recovery evaluations [24]. Specifically, we report MoJoFM, A2A, and $C2C_{cvg}$ as GT-alignment accuracy measures commonly used in architecture recovery, and TurboMQ as a GT-independent modularization-quality metric. In addition, we report the Adjusted Rand Index (ARI) [16], a widely used clustering-agreement measure that ranges from −1 to 1 (0 indicates chance-level agreement and 1 a perfect match). We implemented the metrics in Python based on descriptions and previous implementations.

MoJoFM [53] is defined by the formula,

$$\mathrm{MoJoFM}(A_1, A_2) = \left(1 - \frac{mno(A_1, A_2)}{\max(mno(\forall A_1, A_2))}\right) \times 100 \tag{8}$$

where $mno(A_1, A_2)$ is the minimum number of *move* and *join* operations required to transform the recovered architecture A_1 into the GT architecture A_2 at the cluster-level, and the denominator is the maximum possible number of such operations used for normalization. MoJoFM is reported to be more accurate than other measures; however, it is less sensitive to over-fragmentation because of its relatively cheap *join* operations. Thus, many small clusters can still yield a high score when they largely align with GT modules [24].

Architecture-to-Architecture (A2A) [22] extends MoJoFM by incorporating operations for adding, removing, and moving entities and clusters. It is defined as follows:

$$\mathrm{A2A}(A_1, A_2) = \left(1 - \frac{mto(A_1, A_2)}{aco(A_1) + aco(A_2)}\right) \times 100 \tag{9}$$

where mto is the number of edits required to convert A_1 into A_2, and aco is the cost of constructing the respective architecture from scratch. One limitation of A2A is that it can assign high similarity

scores even when two architectures are structurally quite different, thereby overestimating their true similarity [23].

Cluster-to-Cluster Coverage $C2C_{cvg}$) [12] measures the percentage of GT clusters that are sufficiently matched by clusters in the recovered architecture. It is defined as follows:

$$C2C_{cvg}(A_1, A_2) = \frac{|\text{simC}(A_1, A_2)|}{|A_2|} \times 100 \tag{10}$$

where, $\text{simC}(A_1, A_2)$ denotes the number of clusters in A_2 that overlap with some cluster in A_1 beyond a user-defined coverage threshold th_{cvg}. By convention, $th_{cvg} = 0.10,\ 0.33$, and 0.50 correspond to weak, moderate, and majority matches.

Normalized TurboMQ [24] is independent of the GT architecture and assesses modular quality based on the dependency graph by rewarding intra-cluster cohesion and penalizing inter-cluster coupling. Let k be the number of recovered clusters. For each cluster i, let μ_i denote the total weight of intra-cluster dependencies, and let ε_{ij} denote the total weight of dependencies from cluster i to cluster j. TurboMQ is defined as:

$$\text{TurboMQ} = \sum_{i=1}^{k} \frac{\mu_i}{\mu_i + 0.5 \times \sum_{j \neq i} (\varepsilon_{ij} + \varepsilon_{ji})} \tag{11}$$

TurboMQ can favor decompositions with many clusters [24]. To reduce this bias, we report only the normalized TurboMQ score (TurboMQ/k).

5 Results and Analysis

This section presents the empirical evaluation of GAER with respect to the three research questions and includes additional analyses of scalability. We first compare GAER to established baselines using standard architecture recovery metrics (RQ1), then analyze the impact of architectural granularity (RQ2), and finally examine the stability of the stochastic methods across repeated runs (RQ3). Since GAER and NEGAR involve stochastic components (e.g., random initialization and negative sampling), we repeat each configuration 100 times with different random seeds to explicitly quantify run-to-run variability and assess stability. Unless stated otherwise, we report the median performance across runs.

5.1 Evaluation and Comparison of Recovered Architectures (RQ1)

To address RQ1, we evaluate GAER by comparing recovered architectures against the GT using MoJoFM, A2A, $C2C_{cvg}$, and ARI. We compare the GAER variants (GCN and GAT) with ACDC, LIMBO, FCA, SADE, SARIF, two WCA variants, and NEGAR. Table 2 reports results in the unknown-k setting, where k is selected from the practical range 10–30 using the fixed selection criterion described in Sec. 3.3 (applied to GAER and NEGAR).

The metrics in Table 2 capture different aspects of alignment with the GT mapping. Overall, GAER achieves the strongest results across most datasets and metrics, with GAT often best and GCN close behind. OODT is the main exception because its reference decomposition has more than 200 modules, which lie far outside our $\mathcal{K}$ range (10-30); ACDC, ARC, and FCA achieve the highest scores there at much finer granularity. While we report all baselines for completeness, several are competitive only on a subset of systems or metrics; we therefore focus the discussion on the most informative and comparable reference techniques, SARIF and NEGAR, and highlight cases where other approaches are most competitive.

MoJoFM shows a clear trend: GAER often achieves higher GT alignment than the baselines, with GAT usually best and GCN close behind. Compared with SARIF, GAER improves MoJoFM on several datasets (e.g., Chromium, Hadoop, HDF, and T.Mates). NEGAR is also a strong structure-only

Table 2. Similarities between recovered and GT architecture. The metrics MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. The top 2 values per metric are highlighted.

Tech.	ArchStudio				Bash				Chromium				Hadoop				OODT				HDC				HDF				Libxml				JabRef				T.Mates			
	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
ACDC	73	86	37	20	50	81	10	13	48	82	1	7	46	82	0	13	44	85	39	45	49	82	0	24	38	79	4	13	34	83	25	13	–	–	–	–	–	–	–	–
ARC	57	86	20	32	28	80	5	3	33	83	0	2	47	83	43	24	30	82	50	12	37	81	4	8	36	80	0	12	29	83	26	6	–	–	–	–	–	–	–	–
FCA	60	82	19	10	41	80	7	13	58	76	0	3	49	82	19	11	50	86	42	23	53	78	2	10	32	78	3	15	37	84	24	8	–	–	–	–	–	–	–	–
LIMBO	27	80	2	0	27	80	2	0	31	78	0	0	17	80	1	2	12	77	0	0	25	80	5	1	20	81	0	-53	22	84	24	2	–	–	–	–	–	–	–	–
WCA_{UE}	31	84	0	3	40	81	2	8	31	82	0	2	16	79	13	0	12	78	32	0	51	80	2	11	45	82	7	26	32	84	15	9	–	–	–	–	–	–	–	–
WCA_{UEM}	31	84	0	3	39	80	2	7	58	82	0	2	16	79	13	0	12	78	32	0	48	80	1	10	44	82	5	23	27	83	14	6	–	–	–	–	–	–	–	–
SADE	42	83	29	11	45	86	17	21	58	87	35	26	41	83	44	23	21	78	43	13	72	88	17	41	49	88	33	30	34	84	17	22	–	–	–	–	–	–	–	–
SARIF	66	87	56	35	76	84	13	39	60	86	23	27	47	84	13	29	30	80	41	20	89	89	33	53	62	87	27	37	47	88	45	26	89	83	7	17	76	87	33	39
NEGAR	63	89	45	53	65	87	22	33	64	88	48	39	51	84	27	27	19	76	13	12	71	87	17	33	57	86	16	31	41	86	29	18	77	82	0	9	67	85	7	25
GCN	66	86	41	40	90	92	53	53	73	91	58	59	57	86	33	40	24	77	13	17	89	88	28	45	88	92	44	72	66	92	55	44	90	85	9	21	87	89	29	48
GAT	70	87	53	39	91	92	63	57	76	91	61	60	24	77	16	17	89	87	26	42	92	92	50	70	66	91	53	41	92	85	11	22	95	90	44	46				

baseline and often reaches MoJoFM values around the 60s; it can exceed SARIF on some systems, including Hadoop and Chromium. Libxml is a notable exception for NEGAR: its dependency graph is small and sparse, so random-walk embeddings have limited structural context to learn from.

A2A remains high across most datasets and methods, which limits its ability to clearly separate methods. Still, the overall ranking is broadly consistent with MoJoFM, with GAER usually strongest for both encoders. NEGAR is also competitive and even surpasses SARIF on A2A for Bash, AS4, and Chromium (e.g., Bash: 87 vs. 84).

$C2C_{cvg}$ is more sensitive to splitting GT modules than MoJoFM and A2A. It checks whether each GT module is covered by at least 50% by a single recovered cluster (with $th_{cvg} = 0.5$), so it can drop sharply when GT modules are split across multiple clusters. For example, JabRef has only six GT modules, and searching k in 10–30 can produce finer-grained clusters that split these modules, leading to low $C2C_{cvg}$ values. Overall, GAER achieves strong coverage on Bash, Chromium, Hadoop, HDF, Libxml, and T.Mates, indicating that many GT modules are captured by a dominant recovered cluster. $C2C_{cvg}$ also highlights cases where GAER does not lead; SARIF attains higher values on ArchStudio and HDC.

ARI is also sensitive to fragmentation, but it is more stable than $C2C_{cvg}$ because it does not use a fixed coverage threshold. GAER achieves strong ARI on Bash, Chromium, Hadoop, HDF, Libxml, and T.Mates. AS4 is a major exception, in which NEGAR achieves the highest ARI, suggesting that AS4 aligns particularly well with structural signals. Another exception is HDC, where SARIF has a higher ARI. These cases highlight that different metrics can favor distinct decompositions, motivating our subsequent analyses of the number of clusters (RQ2) and stability (RQ3).

Overall, the results show two consistent patterns. First, using information beyond the dependency graph can help recover decompositions that align better with the intended architecture. Across several datasets, GAER and SARIF often outperform structure-only baselines such as NEGAR, suggesting that directory and semantic cues provide a useful signal when dependencies are noisy or incomplete. Second, Chromium highlights a different limitation. On this large system with diverse dependency patterns, the representation-learning methods (GAER and NEGAR) outperform SARIF and all other techniques across all metrics, indicating that fixed fusion rules and manually tuned weights do not generalize well at scale. Together, these findings motivate learning-based integration of multiple cues and show that no single information source or fusion strategy is best for every system.

Statistical Significance. We conduct statistical significance tests to verify that the observed improvements are systematic rather than due to stochasticity in training. Since GAT is the stronger variant of GAER, we focus on pairwise comparisons between GAT, NEGAR, and SARIF. Since metric

values from repeated runs are not normally distributed, we use nonparametric tests. We apply the one-sample Wilcoxon signed-rank test [54] when comparing the 100-run distributions of GAER and NEGAR against SARIF's single score, and the Mann–Whitney U test [26] when comparing independent 100-run distributions between the stochastic methods.

We also report Cliff's delta (δ)[7] to quantify effect size. δ ranges from −1 to 1, and its sign indicates which method tends to achieve higher scores, with positive values favoring the first method and negative values favoring the second. We interpret the effect size using standard thresholds: negligible ($|\delta| < 0.147$), small ($|\delta| < 0.33$), medium ($|\delta| < 0.474$), and large ($|\delta| \geq 0.474$). Table 3 summarizes these pairwise comparisons (median differences, significance, and effect sizes) across the datasets and metrics included in this analysis.

Table 3. Median differences of methods for all metrics across datasets. Positive values indicate that the first method outperforms the second. Effect size magnitude based on $|\delta|$: N (negligible), S (small), M (medium), L (large) are shown in parentheses. Entries marked with † are not statistically significant ($p \geq 0.05$), and highlighted cells indicate cases where the second method performs significantly better.

	GAT – GCN				GAT – SARIF				GAT – NEGAR				SARIF – NEGAR			
Dataset	MoJoFM	A2A	$C2C_{cvg}$	ARI	MoJoFM	A2A	$C2C_{cvg}$	ARI	MoJoFM	A2A	$C2C_{cvg}$	ARI	MoJoFM	A2A	$C2C_{cvg}$	ARI
AS4	+3.19(L)	+0.5(M)	+11.5(L)	+0.2(N)†	+3.7(L)	−0.4(L)	−3.4(S)	+4.8(L)	+6.6(L)	−0.9(M)	+2.6(S)†	−13.0(L)	+2.9(L)	−0.5(N)	+6.0(L)	−17.9(L)
Bash	+1.1(L)	+0.3(M)	+9.6(L)	+3.1(L)	+15.1(L)	+8.1(L)	+49.5(L)	+17.5(L)	+26.6(L)	+4.6(L)	+40.3(L)	+23.8(L)	+11.5(L)	−3.5(L)	−9.2(L)	+6.3(L)
Chrome	+3.1(L)	+0.5(M)†	+2.7(S)	+1.0(N)†	+16.1(L)	+5.4(L)	+37.6(L)	+32.8(L)	+12.6(L)	+3.6(L)	+12.5(L)	+20.7(L)	−3.4(M)	−1.8(L)	−25.0(L)	−12.1(L)
Hadoop	+8.6(L)	+1.9(L)	+25.0(L)	+12.4(L)	+19.0(L)	+3.7(L)	+45.4(L)	+23.4(L)	+14.6(L)	+3.9(L)	+31.9(L)	+25.9(L)	−4.4(L)	+0.1(M)	−13.5(L)	+2.6(L)
HDC	+0.3(S)†	−0.8(M)	−1.5(S)	−2.8(L)	+0.39(S)	−2.1(L)	−6.7(L)	−11.6(L)	+18.4(L)	−0.3(N)†	+9.6(L)	+9.2(L)	+18.0(L)	+1.9(S)	+16.3(L)	+19.8(L)
HDF	+4.9(L)	−0.7(M)	+6.3(L)	−2.3(S)	+30.3(L)	+5.2(L)	+23.0(L)	+32.6(L)	+35.4(L)	+6.1(L)	+34.2(L)	+38.2(L)	+5.1(L)	+0.8(M)	+11.2(L)	+5.7(L)
Libxml	+0.0(N)†	−0.3(S)	−2.4(S)	−3.1(S)	+18.8(L)	+3.5(L)	+7.6(L)	+15.3(L)	+24.7(L)	+5.2(L)	+24.1(L)	+23.6(L)	+5.9(L)	+1.7(L)	+16.4(L)	+8.3(L)
JabRef	+3.2(L)	+0.17(N)†	+2.4(S)†	+0.4(N)	+3.9(L)	+1.9(L)	+4.4(L)	+4.9(L)	+16.3(L)	+3.0(L)	+11.41(L)	+13.3(L)	+12.3(L)	+1.0(L)	+7.0(L)	+8.4(L)
T.Mates	+7.1(L)	+0.5(S)	+15.2(L)	−1.7(M)	+18.5(L)	+2.7(L)	+10.8(L)	+6.8(L)	+27.1(L)	+4.5(L)	+36.3(L)	+21.0(L)	+8.6(L)	+1.7(M)	+25.6(L)	+14.2(L)
Average	+3.50	+0.23	+7.64	+0.80	+13.98	+3.11	+18.69	+14.06	+20.26	+3.30	+22.55	+18.08	+6.28	+0.16	+3.87	+3.92

Table 3 shows that many differences are systematic; several comparisons are statistically significant and often have medium-to-large effect sizes. Since we select k from a range (10–30), both GAER encoders produce decompositions at a manageable granularity, rather than extremely fine-grained clusterings. Overall, the gap between the two GAER encoders is modest. On average, GAT improves over GCN by +3.5 MoJoFM and +7.64 $C2C_{cvg}$, while differences on A2A (+0.23) and ARI (+0.80) are small. ARI is mixed, favoring GCN on HDC, HDF, Libxml, and T.Mates, which is consistent with GCN sometimes producing more compact partitions. In contrast, GAT more often improves MoJoFM and $C2C_{cvg}$.

Compared with SARIF, GAER-GAT yields positive results across most datasets and metrics, often with large effects. The clearest exceptions are AS4 and HDC. On AS4, SARIF is higher on A2A and $C2C_{cvg}$, while on HDC, it is higher on A2A, $C2C_{cvg}$, and ARI. Overall, the aggregate row shows that GAT's gains over SARIF are largest on MoJoFM, $C2C_{cvg}$, and ARI, while A2A differences are smaller.

The GAT vs. NEGAR comparison shows a consistent advantage for GAER across most datasets and metrics, with only a few exceptions (e.g., AS4 on ARI). In contrast, SARIF vs. NEGAR is mixed: SARIF has a modest overall advantage on MoJoFM, $C2C_{cvg}$, and ARI, but not on A2A, and it trails NEGAR on systems such as Chromium and Hadoop. Overall, the results support two conclusions: both GAER variants are competitive, and GAER-GAT is the stronger default setting.

Scalability. To assess scalability, we report end-to-end runtime for GAER variants and compare it with NEGAR and SARIF (Table 4). We report source code processing separately, since extracting text and dependencies (via ctags [50] and Depends) is performed once per system and reused across methods. For GAER, total runtime includes graph construction, Word2Vec embedding generation,

graph auto-encoder training, agglomerative hierarchical clustering, and selecting the best k within the 10–30 search range. We also include metric computation time. The runtime measurements confirm that our approach is practical and scalable for large-scale architecture recovery tasks.

Table 4. Total time consumption (in seconds) for the GAER (GCN and GAT), NEGAR, and SARIF pipelines. Data extraction is the time required to extract text and dependencies from the source code (using ctags + Depends). We used the default configuration for GAER and SARIF. For NEGAR, we found that 16 parallel tasks provided the best balance across the datasets.

Technique	ArchStudio	Bash	Chromium	Hadoop	OODT	HDC	HDF	Libxml	JabRef	T.Mates
SARIF	48.43	50.05	1 976.79	70.06	53.80	13.19	25.24	36.48	77.36	56.35
NEGAR	4.26	19.12	289.48	5.14	16.89	15.42	7.69	0.41	8.95	8.19
GAER-GCN	1.83	0.57	36.78	1.62	18.84	0.61	0.49	0.61	1.96	1.62
GAER-GAT	2.09	0.74	37.29	1.90	13.08	0.84	0.67	0.81	2.25	1.91
Data extraction	53.12	11.08	2 796.84	58.00	21.36	12.35	36.49	10.81	27.01	30.65

Answer to RQ1: Across most systems, GAER achieves higher (or comparable) alignment with the GT architecture than the baselines on standard recovery metrics. GAT is often best, with GCN usually close behind. The clearest gains appear in MoJoFM and ARI. Improvements in $C2C_{cvg}$ are also common, but this metric is sensitive to how modules are split or merged. In contrast, A2A remains high across most methods, thereby separating methods less clearly. Overall, approaches that integrate multiple information sources or use representation learning tend to be the most competitive, although the best choice can vary by system.

5.2 Impact of Architectural Granularity (RQ2)

Architectural granularity is a design choice rather than a single objectively correct answer, and systems are often organized at different levels of modular abstraction. Our approach recovers architectures by clustering node embeddings into a flat k-partition, so k directly determines the resolution of the recovered decomposition and can substantially influence the GT-alignment metrics.

Several techniques in our comparison also decide the number of clusters on their own and return a partition without a user-chosen k. While this avoids fixing a single resolution in advance, the number of clusters produced by the method can vary widely across systems (and, for stochastic methods, across runs). As shown in Table 6, some approaches yield very fine-grained decompositions of large systems, such as Chromium, producing hundreds or thousands of clusters, which are difficult to interpret as an architectural overview. We therefore evaluate all recovered decompositions as flat partitions and treat granularity as a controlled modeling choice, rather than cutting the hierarchical dendrogram at a fixed distance threshold.

To answer RQ2, we evaluate GT alignment under two granularity views as shown in Table 5. First, we vary the cluster search ranges $\mathcal{K}$ (10–20, 10–30, 10–50, and 10–100) for GAER-GAT to study how widening the search interval shifts the recovered architecture toward coarser or finer decompositions. Second, we report metrics for both GAER variants and NEGAR when $k = k_{GT}$ to assess their potential for alignment with GT at the reference resolution.

Across datasets, widening $\mathcal{K}$ primarily affects performance due to granularity mismatch. When the range is too small for a system with many GT modules, the recovered architecture becomes too coarse, resulting in multiple GT modules being merged. When the range is too large for a system with few GT modules, the recovered architecture tends to be overly fine-grained, with GT modules split across multiple clusters. MoJoFM can stay high even under such splits, while ARI and $C2C_{cvg}$ typically decrease more clearly when fragmentation increases.

Table 5. Performance under varying architectural granularity. The top block shows GAER-GAT results evaluated with k selected from progressively increasing search ranges $\mathcal{K}$. The bottom block reports results for $k = k_{GT}$ in GAER (GAT, GCN) and NEGAR. Each entry is the median over 100 runs; MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. The top scores per dataset are highlighted.

$\mathcal{K}$	ArchStudio				Bash				Chromium				Hadoop				OODT				HDC				HDF				Libxml				JabRef				T.Mates			
	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
10–20	67	86	55	43	91	93	65	58	72	91	64	61	62	87	57	49	20	76	12	14	91	90	43	55	94	95	57	79	62	91	53	41	92	86	14	24	93	90	47	51
10–30	70	87	53	40	91	92	63	57	76	91	61	60	66	88	59	52	24	77	16	17	89	87	26	42	92	92	50	70	66	91	53	41	92	85	11	22	95	90	44	46
10–50	71	86	42	30	90	90	45	47	81	91	52	53	68	87	50	47	29	79	25	20	88	85	14	35	88	88	29	55	66	87	41	36	93	83	4	17	96	89	42	39
10–100	75	87	44	29	85	87	30	38	87	89	45	41	68	87	33	37	37	81	40	29	82	82	6	26	76	81	12	30	64	83	31	29	93	81	0	12	97	88	32	35
NEGAR^k	72	86	39	18	63	89	36	34	74	84	30	16	60	85	16	23	60	89	46	38	69	89	27	35	53	88	22	30	42	88	28	18	64	85	0	-7	67	87	7	28
GCN^k	73	87	32	24	90	93	57	58	89	87	32	30	64	86	19	30	69	92	52	61	89	92	64	62	88	96	78	76	66	92	56	44	83	87	17	22	86	90	33	48
GAT^k	80	88	42	27	91	93	64	58	91	87	33	30	69	86	30	28	74	93	64	61	89	92	64	62	90	97	78	84	66	92	56	42	91	90	33	34	94	90	47	51

For ArchStudio, Hadoop, Chromium, and especially OODT, the GT decompositions contain many modules; for example, OODT has $k_{GT} > 200$, whereas smaller ranges, such as 10–20 and 10–30, enforce a much coarser partition. In these systems, expanding $\mathcal{K}$ generally increases MoJoFM as the recovered architecture becomes more comparable to the GT. In OODT, as $\mathcal{K}$ widens, all metrics increase, indicating that the smaller ranges are too coarse for the reference granularity; the exact-k setting further confirms this. This also shows that the learned representations can support finer-grained decompositions when the target granularity requires it. ArchStudio and Chromium follow the same direction in MoJoFM, and Hadoop shows a similar trend in MoJoFM, while ARI and $C2C_{cvg}$ can change more noticeably when $\mathcal{K}$ becomes very wide. Since $C2C_{cvg}$ is coverage threshold-based, we interpret it together with ARI.

In contrast, systems with comparatively small GT module counts are more sensitive to over-fragmentation. JabRef illustrates this clearly; as $\mathcal{K}$ widens, MoJoFM remains high, but $C2C_{cvg}$ drops from 14 to 0 and ARI drops from 24 to 12, indicating that the recovered decomposition becomes too fine-grained relative to the reference modules. Under exact-k, both metrics increase again (ARI 34, $C2C_{cvg}$ 33), showing that the embeddings can also support a coarse decomposition when the target granularity is known.

Bash, HDC, and HDF show the same fragmentation trend as $\mathcal{K}$ widens. ARI and $C2C_{cvg}$ decrease while MoJoFM remains comparatively high. For example, on HDF, ARI drops from 79 to 30, and $C2C_{cvg}$ drops from 57 to 12. The exact-k results in the second block of Table 5 further show that setting $k = k_{GT}$ can improve GT alignment on several systems, especially when k_{GT} is small or far from the 10–30 range.

To complement the GT alignment results in Table 2 with an internal modularization view, Table 6 reports the normalized TurboMQ scores, number of recovered clusters, and includes the GT scores as a reference. The table shows that higher internal modularity does not necessarily imply better GT alignment, because TurboMQ measures cohesion and separation within the recovered decomposition, whereas GT metrics assess similarity to the reference architecture. For example, NEGAR often achieves high normalized TurboMQ, consistent with its structure-only design based on second-order random walks. SADE and SARIF can also achieve high scores when they produce relatively compact decompositions. For this reason, we interpret normalized TurboMQ alongside the cluster count and the GT-alignment metrics.

Table 6. Normalized TurboMQ (T) and recovered cluster count $\tilde{k}$. The first row reports the GT decomposition. For GAER and NEGAR, results are medians of T and $\tilde{k}$ over 100 runs with $\mathcal{K} = [10, 30]$.

	AS4		Bash		Chromium		Hadoop		OODT		HDC		HDF		Libxml		JabRef		T.Mates	
Tech.	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$	T	$\tilde{k}$
GT	25	57	24	14	46	67	19	67	79	216	38	11	31	9	31	9	36	6	11	15
ACDC	14	100	28	77	26	784	13	83	36	133	60	17	6	51	36	8	—	—	—	—
FCA	7	160	87	21	29	2 360	9	137	26	222	43	63	5	62	24	21	—	—	—	—
SADE	7	26	67	6	82	34	40	9	92	23	70	12	34	9	30	6	—	—	—	—
SARIF	62	25	16	68	79	22	55	16	72	29	64	11	67	15	41	11	45	29	25	15
NEGAR	87	13	65	18	79	20	64	17	88	17	70	13	48	16	22	15	61	18	31	20
GCN	47	18	33	17	61	18	41	19	69	17	59	18	28	16	33	19	38	17	5	18
GAT	58	18	46	16	72	18	52	18	76	17	60	18	39	17	37	19	48	17	11	18

Answer to RQ2: Varying the number of clusters strongly affects the recovered decomposition and the resulting GT-alignment scores. Smaller k values typically yield higher-level structures that are easier to inspect, whereas larger k values expose more detail and can improve MoJoFM when the reference decomposition is itself detailed. However, increasing k also increases the risk of splitting reference modules, which often lowers ARI and $C2C_{cvg}$. Overall, the number of clusters should be treated as a modeling choice based on the desired level of architectural detail, since it directly affects both the recovered decomposition and its agreement with the reference architecture.

5.3 Stability of Stochastic Architecture Recovery Methods (RQ3)

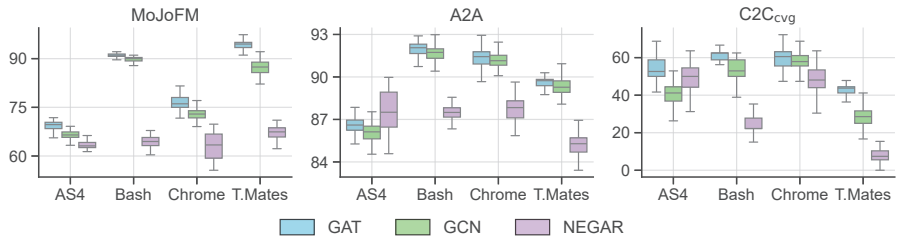
RQ3 evaluates the run-to-run stability of the stochastic methods in our study, GAER (GAT/GCN) and NEGAR, across different random seeds. For each dataset, we run each method 100 times in the range search setting $\mathcal{K} = (10, 30)$ and record the resulting metrics. We compare variability using variance ratios, $\mathrm{VR} = \mathrm{Var}(A)/\mathrm{Var}(B)$, and test for significant differences in variance using the Brown-Forsythe test [3], which compares the average absolute deviation from the median. Table 7 reports pairwise variance ratios for GAT vs. GCN and GAT vs. NEGAR.

Table 7. Run-to-run variability over 100 runs using variance ratios (VR). We report $\mathrm{VR} = \mathrm{Var}(A)/\mathrm{Var}(B)$ for two method pairs, where VR > 1 means A is more variable than B and VR < 1 indicates the opposite. Entries marked with † are not statistically significant ($p \geq 0.05$). Highlighted cells (VR> 1) indicate cases where the second method is significantly less variable than the first.

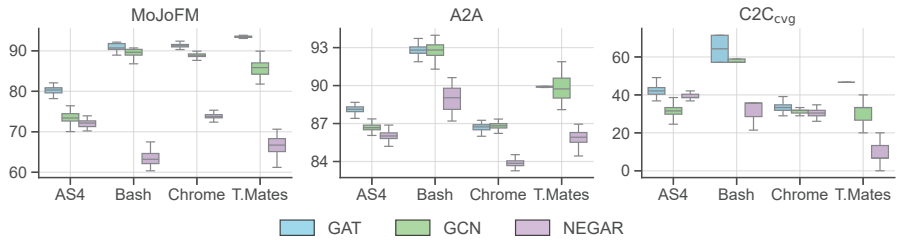
	GAT / GCN				GAT / NEGAR			
Dataset	MoJoFM	A2A	$C2C_{cvg}$	ARI	MoJoFM	A2A	$C2C_{cvg}$	ARI
AS4	0.78†	0.70†	0.87†	1.31†	1.41†	**0.17**	1.01†	**0.20**
Bash	0.46†	**0.09**	**0.31**	**0.05**	**0.09**	**0.03**	**0.14**	**0.05**
Chromium	0.91†	**0.61**	0.73†	0.76†	**0.68**	**0.69**	0.75†	1.01†
Hadoop	0.82†	0.80†	1.05†	**1.45**	0.79†	0.95†	**0.58**	**2.49**
OODT	**1.66**	**1.61**	1.27†	**1.90**	**0.37**	**0.35**	**0.24**	**0.41**
HDC	**0.58**	**0.71**	**0.75**	**0.78**	**0.12**	**0.48**	1.63†	0.99†
HDF	0.67†	1.18†	**2.09**	1.11†	**0.33**	1.05†	0.97†	**4.56**
Libxml	**2.15**	**1.73**	**2.87**	**3.20**	**1.39**	1.18†	0.68†	1.06†
JabRef	**2.47**	**0.62**	0.88†	**0.60**	1.52†	**0.66**	0.98†	**2.65**
T.Mates	**0.15**	0.59†	**0.40**	**1.41**	**0.14**	**0.58**	1.16†	1.00†

Overall, GAT is as stable as, or more stable than, GCN across many datasets and metrics. The clearest reductions in variability are observed for Bash and HDC, where GAT exhibits significantly lower variability across several metrics. There are also dataset- and metric-specific exceptions in which GCN is significantly less variable, most notably on Libxml (across metrics) and in a few isolated cases, such as JabRef (MoJoFM) and Hadoop/T.Mates (ARI). Compared to NEGAR, GAT often shows lower variability on MoJoFM and A2A (e.g., Bash, Chromium, and T.Mates), while differences on $C2C_{cvg}$ and ARI are more mixed. Lower variability does not necessarily imply better alignment with the GT architecture; a method can be stable across runs while still producing a consistently weaker decomposition.

Figure 2 shows box plots across 100 runs for three metrics (MoJoFM, A2A, and $C2C_{cvg}$) on four representative systems. The plots show that MoJoFM is comparatively stable across methods, whereas the largest spread is typically observed for $C2C_{cvg}$, which is sensitive to fragmentation and to small changes in the recovered partition. The bottom row shows that fixing granularity to the GT cluster count makes the distributions noticeably tighter across metrics and methods, indicating that a large part of the observed variability arises from selecting k within the search range rather than from stochastic training alone. Across the shown systems, GAT is typically the most consistent, with tighter spreads than GCN and NEGAR, especially on A2A and $C2C_{cvg}$. In practice, when k is unknown, running the pipeline a few times (e.g., 2–3) and selecting the solution corresponding to the median-selected k yields a stable estimate of the decomposition.



(a) Cluster range search ($k \in [10, 30]$).



(b) Fixed Clusters ($k = k_{GT}$).

Fig. 2. Run-to-run variability over 100 seeds on four representative systems. Top: k selected via range search. Bottom: k fixed to the reference module count.

Answer to RQ3: GAER shows strong run-to-run stability across datasets. GAER-GAT is often as stable as or more stable than GAER-GCN, with some dataset- and metric-specific exceptions. Compared to NEGAR, both GAER variants are typically less variable on MoJoFM and A2A. Variability is highest for the fragmentation-sensitive metric $C2C_{cvg}$, especially when k is selected from a search range, because small changes in granularity can split or merge reference modules. Fixing the number of clusters reduces variability for all methods, since allowing k to vary across runs introduces additional variation in the recovered partitions and the resulting metrics.

6 Related Work

Architecture recovery aims to infer higher-level architectural modules from implementation artifacts. Prior work differs mainly in the information sources it exploits (dependencies, code semantics, directory structure) and in how these signals are combined to derive a decomposition (search-based optimization, clustering, heuristic fusion, representation learning, or LLM-guided mapping). Table 8 summarizes representative techniques by information source and recovery strategy.

Traditional SAR models treat software artifacts as a dependency graph and recover modules using several related paradigms. A first group comprises search-based methods that optimize modularization objectives. Within this group, some methods optimize a single modularization quality (MQ) objective, favoring high cohesion within modules and low coupling across modules [25, 27, 30, 39], while others use multi-objective or evolutionary formulations to jointly optimize multiple criteria [15]. A second group comprises clustering-based methods, which merge artifacts using dependency-derived similarity or information-theoretic criteria [1, 34, 35]. In addition, some structural methods emphasize practicality, determinism, or scalability in recovery [32, 40, 48]. In practice, these approaches can be effective when dependencies reflect architectural structure, but recovery quality remains sensitive to how the dependency graph is constructed and how dependency relations are represented or weighted.

Beyond structural approaches, semantic approaches use identifiers and comments to cluster artifacts and may outperform dependency-only methods in certain scenarios [10]. Representative techniques cluster artifacts in a latent semantic space learned from source code (e.g., topic models or LSI) or directly from lexical similarity features [8, 13, 43]. Kargar et al. [19] construct a semantic dependency graph from lexical cues and optimize the decomposition via local search. Furthermore, Anquetil et al. [2] proposed a directory-based recovery approach that uses file names and directory paths as proxies for the architecture. However, these non-structural signals can be noisy in isolation because widely reused terminology and utility directories mix entities from many modules.

Hybrid approaches combine multiple information sources to improve robustness. Prior work includes weighted similarity fusion with search-based partitioning and many-objective remodularization, which augment structural objectives with semantic and evolution-related criteria [29, 31, 42, 45]. Puchala et al. [41] combine multiple clustering results into a consensus solution, while Yang et al. [55] used semantic cues to guide propagation over the dependency graph. Papachristou et al. [38] and Zhang et al. [56] fuse signals by constructing weighted dependency graphs and applying community detection. Overall, most hybrid techniques rely on fixed fusion rules, heuristic weighting, or late-stage aggregation, rather than learning a unified representation that can jointly model structural and semantic signals.

Representation learning has also been explored for SAR. Chen et al. [4] proposed NEGAR, which learns embeddings from static dependency graphs via random walks and clusters the resulting embeddings to find architectural decompositions. They report that recovery quality varies across dependency types, motivating methods that can jointly exploit multiple relations. However, NEGAR runs a separate pipeline for each dependency type and does not incorporate non-structural signals.

Table 8. Overview of SAR techniques grouped by information source and recovery strategy. Dep: static dependencies; Sem: textual features from code/comments; Dir: directory structure/package names.

Technique	Dep.	Sem.	Dir.	Recovery Strategy/Clustering Method
1) Dependency Graph-Based Approaches				
Bunch–NAHC/SAHC [30]	✓	✗	✗	Local-search based graph partitioning; hill-climbing variants; optimize MQ.
WCA [27]	✓	✗	✗	Weighted hierarchical clustering; feature-vector similarity; optimize MQ.
MCA & ECA [15]	✓	✗	✗	Multi-objective and evolutionary clustering (genetic algorithm); optimize MQ.
DAGC [39]	✓	✗	✗	Global search partitioning; genetic algorithm+permutation encoding; optimize MQ.
ACDC [49]	✓	✗	✗	Pattern-driven clustering: builds a subsystem skeleton and uses orphan adoption.
CCT [35]	✓	✗	✗	Cooperative hierarchical clustering using multiple similarity measures.
LIMBO [1]	✓	✗	✗	Information-bottleneck hierarchical clustering (information-theoretic merging).
FCA [48]	✓	✗	✗	Deterministic fast clustering on neighborhood degree matrix; designed for scalability.
Mohammadi et al. [32]	✓	✗	✗	Graph-guided heuristic clustering based on neighborhood structure.
HCC [34]	✓	✗	✗	Ensemble hierarchical clustering; combine multiple dendrograms into consensus.
GMA [40]	✓	✗	✗	Graph-similarity clustering with k-medoids (iterative refinement).
2) Directory-Based Approaches				
Anquetil et al. [2]	✗	✗	✓	Cluster using file names and directory paths (substring/n-gram similarity).
3) Semantic-Based Approaches				
ARC [13]	✗	✓	✗	LDA extracts topics ("concerns") from code/comments; cluster by topic distributions.
ZBR [8]	✗	✓	✗	TF-IDF lexical zones with weights learned via expectation maximization → hierarchical clustering.
Risi et al. [43]	✗	✓	✗	Cluster in LSI latent space with k-means; fold-in/fold-out updates without recomputing the model.
Kargar et al. [19]	✗	✓	✗	Build a semantic dependency graph from lexical cues → hill-climbing search to optimize MQ.
4) Hybrid / Multi-source Approaches				
Misra et al. [29]	✓	✓	✗	Automated feature weighting → weighted graph partitioning (multi-objective modularity criterion).
NSGA-III [31]	✓	✓	✗	Multi-objective optimization, semantics, and change-history consistency.
EoD/CGH/CGoH [45]	✓	✓	✗	Optimize a multi-objective fitness function combining multiple signals with various search strategies.
ESCRC [42]	✓	✓	✗	Combines structural coupling & conceptual text similarity and applies hierarchical clustering.
Puhala et al. [41]	✓	✓	✓	Generate diverse base clusterings across Dep/Sem/Dir → evidence-accumulation consensus clustering.
EVOL [55]	✓	✓	✗	Filter semantic outliers → build clustering skeleton → label propagation on dependency graph.
SADE [38]	✓	✓	✗	Semantic weighting → weighted call graph → community detection.
SARIF [56]	✓	✓	✓	Heuristic information fusion → weighted graph → Louvain community detection.
5) Representation Learning				
NEGAR [4]	✓	✗	✗	Random-walk graph embeddings (node2vec/SkipGram) → k-means clustering.
6) LLM-Based Approaches				
Rukmono et al. [44]	✗	✓	✗	GPT-4-guided deductive mapping to the reference architecture; chains-of-thought prompting.

Complementary to bottom-up approaches, Rukmono et al. [44] use chains-of-thought prompting on GPT-4 to map implementation units to a predefined reference architecture. However, this approach requires an available architectural model and raises scalability concerns.

7 Threats to Validity

External Validity. Our evaluation relies on ten open-source systems in C, C++, and Java, each with a published ground-truth C2A mapping. While these systems provide a solid benchmark for comparing approaches, ten projects across three languages represent only a small sample relative to the diversity of real-world software and do not cover all architectural styles or sizes.

Internal Validity. We made a strong effort to match the system versions used in prior studies. However, some benchmarks are more than 10 years old, so an exact match is not always verifiable (e.g., Chromium). Some entities in the ground truth are generated during the build process, so the build environment must also be configured consistently. We reproduced values comparable to those reported in the SARIF and NEGAR study, suggesting that any remaining dataset differences are unlikely to affect our conclusions.

The dependencies were extracted using a static analysis tool, which may miss some interactions due to language limitations, dynamic features, or incomplete parsing. Although this affects all dependency-based recovery techniques, it can lead to an under-representation of actual cohesion and coupling in the input graph. We decided to drop one system used in previous studies, ITK, because we could not reliably extract dependencies from it, and we could not reproduce prior baseline results on that system.

The reliability of the ground-truth architectures themselves is another concern. Humans create these labels, which may be subject to interpretation, since system experts do not always agree on every assignment. For some systems (e.g., JabRef, TeamMates, HDC, HDF, and Libxml), the mappings were provided directly by system experts or industry partners. For others (e.g., Archstudio, Bash, Chromium, Hadoop, and OODT), researchers derived them from documentation and implementation details. Although prior studies have shown such reconstructions to be consistent and stable over time, the possibility of errors or biases remains, which may influence the evaluation of recovery techniques.

A further threat concerns the extent to which benchmark-based evaluation reflects real-world architecture recovery practice. We compare GAER with strong recent baselines, including SARIF and NEGAR, using standard recovery metrics and identical experimental conditions across datasets. To reduce reliance on a single benchmark suite and a single style of ground-truth construction, we also include systems with expert-provided architectures (JabRef and TeamMates; Section 4.1). This setup supports a fair and reproducible comparison, although conclusions may still depend on the characteristics of the available benchmarks.

From a statistical perspective, with 100 runs per method, tests can detect even small systematic differences, so very low p-values may occur even when practical differences are minor. This can occur, for example, for A2A when scores are uniformly high across methods, yet small non-random gaps remain. We therefore emphasize effect sizes alongside p-values and report Cliff's delta to indicate whether differences are practically meaningful rather than only statistically detectable.

8 Conclusion

We presented GAER, a graph auto-encoder framework for unsupervised recovery of software architectures. We studied two encoder variants, GAER-GAT and GAER-GCN. Our approach models software systems as heterogeneous, multi-relational dependency graphs and integrates structural dependencies, directory hierarchy, and code semantics as node features. By training the encoder to reconstruct observed dependencies, it learns architecture-aware embeddings without requiring handcrafted fusion heuristics or labeled data. Architectural boundaries are then recovered by clustering the learned node representations.

Across the evaluated systems, GAER yields decompositions that align well with the reference architectures and are competitive with strong established baselines. GAT is often the stronger encoder, with GCN typically close behind. On average, GAER-GAT improves over SARIF by about +14 MoJoFM, +3 A2A, +19 $C2C_{cvg}$, and +14 ARI, and over NEGAR by about +20 MoJoFM, +3 A2A, +23 $C2C_{cvg}$, and +18 ARI (Table 3).

We also found that the number of clusters used to form the final decomposition substantially influences the recovered architecture and the reported metrics. Smaller cluster counts typically yield

higher-level structures that are easier to inspect, whereas larger cluster counts reveal more detail and can improve MoJoFM when the reference decomposition is itself fine-grained. At the same time, larger cluster counts increase the risk of splitting reference modules, thereby reducing ARI and $C2C_{cvg}$. Finally, repeated-run analysis indicates that GAER is generally stable across datasets.

Overall, GAER demonstrates the potential of graph representation learning as a scalable, data-driven alternative to manual and heuristic-based recovery techniques. By reducing the effort needed to maintain architectural documentation, GAER supports a more effective understanding and evolution of complex software systems.

Future research could focus on integrating the clustering objective directly into the GAE training process to jointly learn embeddings and architectural boundaries. Another promising direction is to investigate sparsification strategies to handle noisy or redundant dependencies in dense graphs, thereby improving performance and efficiency. Finally, extending the framework to semi-supervised settings, in which limited expert input guides recovery, could improve interpretability and alignment with the intended design. A natural extension is to explore automatic naming or interpretation of recovered clusters using language models or code summarization techniques to enhance comprehensibility.

References

[1] Periklis Andritsos, Panayiotis Tsaparas, Renée J Miller, and Kenneth C Sevcik. 2004. LIMBO: Scalable clustering of categorical data. In *International Conference on Extending Database Technology*. Springer, 123–146. https://doi.org/10.1007/978-3-540-24741-8_9

[2] Nicolas Anquetil and Timothy C Lethbridge. 1999. Recovering software architecture from the names of source files. *Journal of Software Maintenance: Research and Practice* 11, 3 (1999), 201–221. https://doi.org/10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1

[3] Morton B. Brown and Alan B. Forsythe. 1974. Robust Tests for the Equality of Variances. *J. Amer. Statist. Assoc.* 69, 346 (1974), 364–367. https://doi.org/10.1080/01621459.1974.10482955

[4] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. 2022. NEGAR: network embedding guided architecture recovery for software systems. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, IEEE, 367–376. https://doi.org/10.1109/apsec57359.2022.00049

[5] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical Review E—Statistical, Nonlinear, and Soft Matter Physics* 70, 6 (2004), 066111. https://doi.org/10.1103/physreve.70.066111

[6] Paul Clements, David Garlan, Reed Little, Robert Nord, and Judith Stafford. 2003. Documenting software architectures: views and beyond. In *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 740–741. https://doi.org/10.1109/icse.2003.1201264

[7] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin* 114, 3 (1993), 494. https://doi.org/10.1037/0033-2909.114.3.494

[8] Anna Corazza, Sergio Di Martino, Valerio Maggio, and Giuseppe Scanniello. 2011. Investigating the use of lexical information for software system clustering. In *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE, 35–44. https://doi.org/10.1109/csmr.2011.8

[9] Lakshitha De Silva and Dharini Balasubramaniam. 2012. Controlling software architecture erosion: A survey. *Journal of Systems and Software* 85, 1 (2012), 132–151. https://doi.org/10.1016/j.jss.2011.07.036

[10] Joshua Garcia, Igor Ivkovic, and Nenad Medvidovic. 2013. A comparative analysis of software architecture recovery techniques. In *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 486–496. https://doi.org/10.1109/ase.2013.6693106

[11] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. 2013. Obtaining ground-truth software architectures. In *2013 35th International Conference on Software Engineering (ICSE)*. IEEE, 901–910. https://doi.org/10.1109/icse.2013.6606639

[12] Joshua Garcia, Duc Le, Daniel Link, AS Pooyan Behnamghader, Eder Figueroa Ortiz, and Nenad Medvidovic. 2014. An empirical study of architectural change and decay in open-source software systems. *USC-CSSE, Tech. Rep* (2014).

[13] Joshua Garcia, Daniel Popescu, Chris Mattmann, Nenad Medvidovic, and Yuanfang Cai. 2011. Enhancing architectural recovery using concerns. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 552–555. https://doi.org/10.1109/ase.2011.6100123

[14] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864. https://doi.org/10.1145/2939672.2939754

[15] Mark Harman and Xin Yao. 2010. Software module clustering as a multi-objective search problem. *IEEE Transactions on Software Engineering* 37, 2 (2010), 264–282. https://doi.org/10.1109/tse.2010.26

[16] Lawrence Hubert and Phipps Arabie. 1985. Comparing partitions. *Journal of classification* 2, 1 (1985), 193–218. https://doi.org/10.1007/bf01908075

[17] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2025. Graph Convolution Networks for Mapping Source Code Entities to Architectural Modules. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 472–481. https://doi.org/10.1109/icsa-c65153.2025.00071

[18] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. *Reproduction package for GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery*. https://doi.org/10.5281/zenodo.19534384

[19] Masoud Kargar, Ayaz Isazadeh, and Habib Izadkhah. 2017. Semantic-based software clustering using hill climbing. In *2017 international symposium on computer science and software engineering conference (CSSE)*. IEEE, 55–60. https://doi.org/10.1109/csicsse.2017.8320117

[20] Thomas N Kipf and Max Welling. 2016. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016). https://doi.org/10.48550/arXiv.1609.02907

[21] Thomas N Kipf and Max Welling. 2016. Variational graph auto-encoders. *arXiv preprint arXiv:1611.07308* (2016). https://doi.org/10.48550/arXiv.1611.07308

[22] Duc Minh Le, Pooyan Behnamghader, Joshua Garcia, Daniel Link, Arman Shahbazian, and Nenad Medvidovic. 2015. An empirical study of architectural change in open-source software systems. In *2015 IEEE/ACM 12th working conference on mining software repositories*. IEEE, 235–245. https://doi.org/10.1109/msr.2015.29

[23] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidovic, and Robert Kroeger. 2015. Comparing software architecture recovery techniques using accurate dependencies. In *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, Vol. 2. IEEE, 69–78. https://doi.org/10.1109/icse.2015.136

[24] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidović, and Robert Kroeger. 2017. Measuring the impact of code dependencies on software architecture recovery techniques. *IEEE Transactions on Software Engineering* 44, 2 (2017), 159–181. https://doi.org/10.1109/tse.2017.2671865

[25] Spiros Mancoridis, Brian S Mitchell, Yihfarn Chen, and Emden R Gansner. 1999. Bunch: A clustering tool for the recovery and maintenance of software system structures. In *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).'Software Maintenance for Business Change'(Cat. No. 99CB36360)*. IEEE, 50–59. https://doi.org/10.1109/icsm.1999.792498

[26] Henry B Mann and Donald R Whitney. 1947. On a test of whether one of two random variables is stochastically larger than the other. *The annals of mathematical statistics* (1947), 50–60. https://doi.org/10.1214/aoms/1177730491

[27] Onaiza Maqbool and Haroon Atique Babri. 2004. The weighted combined algorithm: A linkage algorithm for software clustering. In *Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings.* IEEE, 15–24. https://doi.org/10.1109/csmr.2004.1281402

[28] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems* 26 (2013).

[29] Janardan Misra, KM Annervaz, Vikrant Kaulgud, Shubhashis Sengupta, and Gary Titus. 2012. Software clustering: Unifying syntactic and semantic features. In *2012 19th Working Conference on Reverse Engineering*. IEEE, 113–122. https://doi.org/10.1109/wcre.2012.21

[30] Brian S Mitchell and Spiros Mancoridis. 2006. On the automatic modularization of software systems using the bunch tool. *IEEE Transactions on Software Engineering* 32, 3 (2006), 193–208. https://doi.org/10.1109/tse.2006.31

[31] Wiem Mkaouer, Marouane Kessentini, Adnan Shaout, Patrice Koligheu, Slim Bechikh, Kalyanmoy Deb, and Ali Ouni. 2015. Many-objective software remodularization using NSGA-III. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 24, 3 (2015), 1–45. https://doi.org/10.1145/2729974

[32] Sina Mohammadi and Habib Izadkhah. 2019. A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. *Information and Software Technology* 105 (2019), 252–256. https://doi.org/10.1016/j.infsof.2018.09.001

[33] Fionn Murtagh and Pierre Legendre. 2014. Ward's hierarchical agglomerative clustering method: which algorithms implement Ward's criterion? *Journal of classification* 31 (2014), 274–295. https://doi.org/10.1007/s00357-014-9161-z

[34] Rashid Naseem, Mustafa Mat Deris, Onaiza Maqbool, and Sara Shahzad. 2019. Euclidean space based hierarchical clusterers combinations: an application to software clustering. *Cluster Computing* 22, Suppl 3 (2019), 7287–7311. https://doi.org/10.1007/s10586-017-1408-0

[35] Rashid Naseem, Onaiza Maqbool, and Siraj Muhammad. 2013. Cooperative clustering for software modularization. *Journal of Systems and Software* 86, 8 (2013), 2045–2062. https://doi.org/10.1016/j.jss.2013.03.080

[36] Hung Viet Nguyen. 2023. Depends: Multilingual Dependency Analysis Tool. https://github.com/multilang-depends/depends. Accessed: 2025-08-17.
[37] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2022. To automatically map source code entities to architectural modules with Naive Bayes. *Journal of Systems and Software* 183 (2022), 111095. https://doi.org/10.1016/j.jss.2021.111095
[38] Marios Papachristou. 2019. Software clusterings with vector semantics and the call graph. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1184–1186. https://doi.org/10.1145/3338906.3342483
[39] Saeed Parsa and Omid Bushehrian. 2005. A new encoding scheme and a framework to investigate genetic clustering algorithms. *Journal of Research and Practice in Information Technology* 37, 1 (2005), 127–143.
[40] Babak Pourasghar, Habib Izadkhah, Ayaz Isazadeh, and Shahriar Lotfi. 2021. A graph-based clustering algorithm for software systems modularization. *Information and Software Technology* 133 (2021), 106469. https://doi.org/10.1016/j.infsof.2020.106469
[41] Shiva Prasad Reddy Puchala, Jitender Kumar Chhabra, and Amit Rathee. 2022. Ensemble clustering based approach for software architecture recovery. *International Journal of Information Technology* 14, 4 (2022), 2013–2019. https://doi.org/10.1007/s41870-021-00846-0
[42] Amit Rathee and Jitender Kumar Chhabra. 2017. Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. In *Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers*. Springer, 94–106. https://doi.org/10.1007/978-981-10-5780-9_9
[43] Michele Risi, Giuseppe Scanniello, and Genoveffa Tortora. 2012. Using fold-in and fold-out in the architecture recovery of software systems. *Formal Aspects of Computing* 24, 3 (2012), 307–330. https://doi.org/10.1007/s00165-011-0199-y
[44] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. 2024. Deductive software architecture recovery via chain-of-thought prompting. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 92–96. https://doi.org/10.1145/3639476.3639776
[45] Nafiseh Sadat Jalali, Habib Izadkhah, and Shahriar Lotfi. 2019. Multi-objective search-based software modularization: Structural and non-structural features. *Soft Computing-A Fusion of Foundations, Methodologies & Applications* 23, 21 (2019). https://doi.org/10.1007/s00500-018-3666-z
[46] Anthony Savidis and Crystalia Savaki. 2022. Software Architecture Mining from Source Code with Dependency Graph Clustering and Visualization.. In *VISIGRAPP (3: IVAPP)*. 179–186. https://doi.org/10.5220/0010896800003124
[47] Ioanna Stavropoulou, Marios Grigoriou, and Kostas Kontogiannis. 2017. Case study on which relations to use for clustering-based software architecture recovery. *Empirical Software Engineering* 22, 4 (2017), 1717–1762. https://doi.org/10.1007/s10664-016-9459-z
[48] Navid Teymourian, Habib Izadkhah, and Ayaz Isazadeh. 2020. A fast clustering algorithm for modularization of large-scale software systems. *IEEE Transactions on Software Engineering* 48, 4 (2020), 1451–1462. https://doi.org/10.1109/tse.2020.3022212
[49] Vassilios Tzerpos and Richard C Holt. 2000. ACDC: an algorithm for comprehension-driven clustering. In *Proceedings Seventh Working Conference on Reverse Engineering*. IEEE, 258–267. https://doi.org/10.1109/wcre.2000.891477
[50] Universal Ctags Team. 2024. *Universal Ctags*. https://github.com/universal-ctags/ctags Accessed: 2026-01-25.
[51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
[52] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017). https://doi.org/10.48550/arXiv.1710.10903
[53] Zhihua Wen and Vassilios Tzerpos. 2004. An effectiveness measure for software clustering algorithms. In *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004.* IEEE, 194–203. https://doi.org/10.1109/wpc.2004.1311061
[54] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83. https://doi.org/10.2307/3001968
[55] Kaiyuan Yang, Junfeng Wang, Zhiyang Fang, Peng Wu, and Zihua Song. 2022. Enhancing software modularization via semantic outliers filtration and label propagation. *Information and Software Technology* 145 (2022), 106818. https://doi.org/10.1016/j.infsof.2021.106818
[56] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. 2023. Software architecture recovery with information fusion. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1535–1547. https://doi.org/10.1145/3611643.3616285

Received 2026-02-25; accepted 2026-03-24

Paper III

When Does the Dependency Graph Help Code-to-Architecture Mapping?

III

Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. *When Does the Dependency Graph Help Code-to-Architecture Mapping?* 2026. arXiv: 2610.19769 [cs.SE]. URL: https://arxiv.org/abs/2610.19769. *Under review.*

When Does the Dependency Graph Help Code-to-Architecture Mapping?

Rakhshanda Jabeen[a,c,*], Morgan Ericsson[a], Jonas Nordqvist[b] and Anna Wingkvist[a]

[a]Department of Computer Science, Linnaeus University, Växjö, Sweden
[b]Department of Mathematics, Linnaeus University, Växjö, Sweden
[c]Electrolux Professional AB, Ljungby, Sweden

ARTICLE INFO

Keywords:
Software Architecture
Code-to-Architecture Mapping
Graph Neural Networks
Semi-Supervised Learning

ABSTRACT

Architectural technical debt arises when a system's implementation drifts from its intended modular decomposition, while the mapping between source files and architectural modules is missing, incomplete, or costly to maintain manually. We formulate code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, with files as nodes, typed dependencies as edges, and lexical features derived from code identifiers and directory layout. Starting from a small labeled sample, an iterative self-training procedure expands the mapping to the remaining files. We compare a graph convolutional network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP), using identical features and training procedures for the learned models, against a Naive Bayes reference. Across 100 runs on ten open-source Java and C/C++ systems, all three neural models outperform the reference on both F1 metrics. GAT achieves the highest averages of system-level median macro and micro F1, at 83.5 and 92.2, respectively, whereas MLP remains competitive or superior on several systems. On Chrome, the largest system with 18,343 files, GAT and GCN exceed MLP by 22 and 19 macro-F1 points, respectively, and have substantially fewer persistently misclassified files. Connectivity and homophily are associated with where neural message passing helps, while multi-relational aggregation outperforms a single merged edge type on every system for both GCN and GAT. A follow-up study finds that several model rankings depend on the training seed, qualifying smaller advantages without removing the large graph benefit observed on Chrome.

1. Introduction

Software architecture defines a system's high-level structure and the dependencies among its principal elements, and it provides the basis for reasoning about maintainability, security, and other quality attributes [7]. That reasoning requires the implementation to stay aligned with the intended design. When it does not, the gap is architectural technical debt [21]: cheap to introduce, but diffuse and relational rather than local to a single file, and therefore expensive to detect and repay once it accumulates. Large language models (LLMs) are increasingly used to generate or assist with code commits [10], motivating renewed attention to whether local code changes remain consistent with a system's module structure. An empirical study of commercial AI code generators found clones between generated functions and existing open-source code [38]. These observations motivate conformance checking, but do not establish that LLM-generated code causes architectural erosion. Architectural erosion more generally concerns drift between the intended design and its implementation [9].

Detecting this drift requires comparing the intended architecture against the implemented one. Static Architecture Conformance Checking (SACC) addresses this comparison, and reflexion modeling is a SACC technique that classifies relations between the intended and implemented architectures as convergent, divergent, or absent [26, 27]. However, reflexion modeling depends on a mapping between source code entities and architectural modules, and such mappings are often missing, incomplete, or expensive to maintain manually [1]. Even for medium-sized systems, recovering or maintaining the architectural decomposition can require substantial expert effort [13]. Automated code-to-architecture (C2A) mapping is therefore an important prerequisite for practical conformance checking.

Prior work on C2A mapping has explored supervised and recommendation-based approaches. Supervised approaches compute *attraction values* that quantify the association strength between source code entities and architectural modules [6, 3]. Olsson et al. [29] proposed a semi-supervised Naive Bayes mapper that combines syntactic and semantic information to assign source code entities to architectural modules. InMap [33] took a recommendation-based view instead, using information retrieval over the codebase to suggest candidate mappings with minimal architectural documentation and no training data.

A separate line of work targets architecture recovery, where the decomposition itself must be discovered rather than assumed. Recent recovery approaches include graph neural networks (GNNs) combined with contrastive learning and autoencoders [15, 17], along with language models applied directly to source code for recovering design

*Corresponding author
rakhshanda.jabeen@electroluxprofessional.com (R. Jabeen); morgan.ericsson@lnu.se (M. Ericsson); jonas.nordqvist@lnu.se (J. Nordqvist); anna.wingkvist@lnu.se (A. Wingkvist)
ORCID(s): 0009-0002-0430-8179 (R. Jabeen); 0000-0003-1173-5187 (M. Ericsson); 0000-0002-0510-6782 (J. Nordqvist); 0000-0002-0835-823X (A. Wingkvist)

structures [2], for agent-based recovery of large legacy systems [30], and for extracting architectural entities to support traceability [12]. The distinction from C2A mapping matters here: the module set is unknown in recovery, whereas in mapping it is given by the intended architecture, and the task is to assign each file to one of those modules. It also matters for evaluation, since many recovery benchmarks are open source, and a decomposition reproduced by a pretrained model may reflect code understanding, prior exposure to the system, or both. Mapping methods that learn from a system's own structure and can be evaluated against a known ground truth therefore remain important.

In this research, we study C2A mapping[1] as semi-supervised node classification over a multi-relational directed dependency graph. Source files are nodes, typed static dependencies between them are edges, and node features encode code identifiers via Word2Vec embeddings together with directory context. A small set of labeled files is expanded into a complete mapping through iterative self-training. We compare three encoders under the same features and training procedure: a graph convolutional network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP). Holding the feature representation, the classifier head, and the self-training loop fixed is intended to isolate what the dependency graph contributes beyond the node features alone, though the encoders themselves differ in architecture beyond the presence or absence of message passing.

We evaluate the approach on ten open-source systems written in Java and C/C++, reporting mapping accuracy and its variation across runs. This paper extends our earlier study [14] in five respects: it broadens the benchmark by adding C/C++ systems; replaces the earlier bag-of-words representation of code identifiers with Word2Vec embeddings; adds GAT to assess the effect of attention-based aggregation on the mapping task; introduces an MLP baseline to assess the contribution of dependency information; and corrects a label-assignment defect in the earlier self-training pipeline. We further evaluate the corrected self-training procedure, conduct new ablation studies of model depth and edge-aggregation direction, and examine the sensitivity of selected comparisons to the training seed.
Our study is guided by the following research questions:

- **RQ1.** How effectively does GNN-based C2A mapping perform relative to non-graph baselines, and how stable is its performance across runs?
- **RQ2.** What does the dependency graph contribute beyond textual features under an identical training procedure, and which system properties are associated with where it helps?
- **RQ3.** How sensitive is the approach to modeling dependency types separately, aggregation depth, and edge direction?

[1] Source code is available at https://github.com/sse-lnu/GCN-SAEroCon25.

2. Related Work

Tzerpos and Holt [35] introduced the orphan adoption problem, in which source code entities that are not yet part of the documented architecture are incrementally assigned to modules. Christl et al. [6] built on this idea with the Human-Guided Clustering Method (HuGMe), a semi-automatic approach that recommends mappings through attraction functions and leaves the final decision to the architect. Bittencourt et al. [3] studied attraction functions derived from information retrieval, where an entity is assigned to the module whose textual content it most resembles. These approaches rely on two signals that persist in later work: lexical similarity between an entity and a module and dependency proximity to entities that are already mapped, but they combine them through fixed, manually specified rules.

Subsequent work replaced these rules with learned models. Olsson et al. [29] proposed a semi-supervised Naive Bayes mapper that combines lexical information from source code with dependency-derived information through Concrete Dependency Abstraction, and reported that 15–20% of the entities must be mapped beforehand to reach acceptable performance. Florean et al. [11] compared text classifiers for the same task and showed that both the classifier and the text representation affect mapping quality. InMap [33] instead ranks candidate mappings interactively, using natural language module descriptions together with source code, which removes the need for a pre-mapped training set. In all of these, dependencies enter the model only after being flattened into features attached to individual entities, so the relational structure of the system is consumed rather than represented.

Language models have recently been applied to the same task. Rukmono et al. [32] use chain-of-thought prompting with GPT-4 to deductively assign implementation units to a predefined reference architecture. Johansson et al. [19] evaluate language model embeddings alongside TF-IDF and GloVe as input to method-level classifiers. Follow-up work links variation across projects most strongly to class separability and density in the embedding space [18]. A parallel line applies language models to architecture recovery, where the module set is inferred rather than given [2, 30]. Pretrained models capture semantic cues that engineered text features miss. ArchAgent, for example, also incorporates reference graphs and related-file context when summarizing code. Our approach instead models dependency types directly through relation-specific message passing.

Graph representation learning uses dependencies as structure rather than as features. NEGAR [5] showed that random-walk embeddings over the dependency graph, followed by clustering, recover meaningful architectural structure. GAER [15] extended this to heterogeneous graphs, comparing GCN and GAT encoders trained within a graph autoencoder and modeling each dependency type as a separate relation, and [17] reports related results using contrastive objectives. Both are unsupervised recovery methods, so the module set is discovered by clustering rather than fixed in advance as it is in mapping. Our earlier work [14] took this

step and evaluated GCN-based models on homogeneous and dependency-type-aware graphs, showing that a small labeled set can be expanded through self-training. That study was limited to Java systems of moderate size, used a single graph-based encoder family, and did not isolate how much of the observed performance came from the dependency graph as opposed to the node features.

The present work addresses these limitations. We evaluate GNN encoders together with a graph-free neural baseline that receives identical features and an identical self-training loop, to examine the contribution of dependency information under a common input representation and self-training schedule. Differences in encoder depth, dropout, and parameter count remain potential confounders.

3. Methodology

This section presents our approach to C2A mapping as semi-supervised node classification over a directed multi-relational dependency graph. We first formulate the mapping task, then describe the graph and feature construction, introduce the encoders compared in this study, and finally the self-training procedure used to grow an initial labeling into a full mapping. Figure 1 illustrates the heterogeneous GNN encoder at the core of this pipeline.

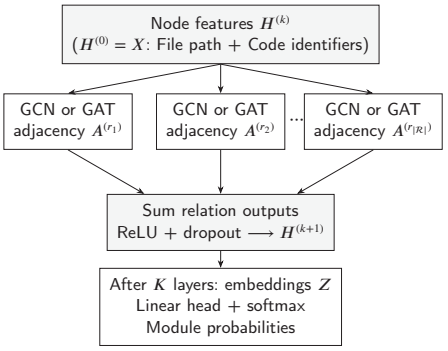


Figure 1: Overview of the heterogeneous GNN encoder. Node features X (file-path indicator + Word2Vec identifiers) and relation-specific adjacency matrices A_R are passed through a relation-specific encoder (GCN or GAT), whose outputs are summed and passed through a non-linear activation function (ReLU) to produce embeddings $Z \in \mathbb{R}^{|\mathcal{F}| \times d_h}$, where d_h is the number of hidden channels. A linear layer maps Z to per-module logits, and a final softmax produces the module probabilities used for classification and pseudo-labeling.

3.1. Problem Formulation

Given a software system with a set of files $\mathcal{F}$ and a fixed set of architectural modules $\mathcal{C}$, a C2A mapping assigns each file $i \in \mathcal{F}$ to a module $y_i \in \mathcal{C}$. A small subset $L \subset \mathcal{F}$ is labeled with its ground-truth module; the remaining files $U = \mathcal{F} \setminus L$ are unlabeled, and the task is to recover the assignment for every file in U. We consider a transductive setting in which the lexical node features and dependency structure of all files are available during training, while module labels are available only for files in L.

Two sources of information are available for each file. The first is intrinsic and consists of what the file itself declares (identifiers) and where it sits in the source tree (directory structure), and it is available for every file regardless of how the file is connected to the rest of the system. The second is relational: the representations of its neighbors in the dependency graph, propagated through message passing and available only where the file has neighbors. We frame the task as representation learning so the two sources can be treated separately: every encoder maps a file i to an embedding z_i, and a shared classification head maps z_i to a module, so the encoders differ in what evidence enters the encoder to generate z_i and not in how it is used.

3.2. Graph Construction

We represent a software system as a heterogeneous graph $G = (X, A_{\mathcal{R}})$ over its files, following the construction introduced for unsupervised recovery in GAER [15]. Code identifiers and directory paths are file-level properties and are encoded in the feature matrix X. Static dependencies are file-to-file relations, each belonging to a dependency type r drawn from the set of relation types $\mathcal{R}$, and encoded as edges in the multi-relational adjacency $A_{\mathcal{R}} = \{A^{(r)}\}_{r \in \mathcal{R}}$.

3.2.1. Node Features

The intrinsic evidence for a file comes from two conventions developers follow when they write and organize code. Identifiers name the concepts a file works with, so class, method, and variable names carry the vocabulary of the domain the file serves and, with it, an indication of the responsibility it implements [40]. Directory paths record how developers chose to group files, and that grouping frequently tracks the subsystem decomposition the architecture describes [13]. The two are complementary where structure alone is ambiguous, since a widely used utility file may depend on code throughout the system while its name and its location still indicate what it is for. Both signals are available after identifier extraction, without requiring additional dependency resolution during feature construction.

We normalize code identifiers by splitting on *camelCase*, *PascalCase*, and *snake_case* boundaries and removing low-information tokens (stopwords, very short tokens, tokens appearing in over 90% of files), then train a 128-dimensional skip-gram Word2Vec [25] embedding over the remaining tokens for 20 epochs with a context window of 5, a minimum token count of 5, and a vocabulary-pruning limit of 2,000 during vocabulary construction. We use one document per file, a fixed stopword list, and a single Word2Vec worker with a fixed seed. Let T_i denote the resulting sequence of normalized tokens for file i; its textual representation is the ℓ_2-normalized average of the Word2Vec embeddings of the tokens in T_i. File paths are tokenized separately and represented as a binary bag-of-words vector. Common

leading directories and the generic components `src`, `main`, and `java` are removed. Java files use their remaining directory tokens, falling back to the file name when none remain; C/C++ files also include base-name tokens. Tokens are lowercased and vectorized using the default word-token pattern of `CountVectorizer`. For file i, the two are concatenated:

$$x_i = \left[\text{Binary(FilePath)}_i \,\middle\|\, \text{Word2Vec}(T_i)\right]. \quad (1)$$

Stacking x_i over all files gives the feature matrix X. Every encoder in this study receives the same X, so the intrinsic evidence available to each of them is identical by construction, although the encoders also differ in depth and parameter count.

3.2.2. Static Dependencies (Edges)

Files that implement related functionality tend to depend on one another directly or through short dependency paths, which is what makes the dependency graph informative for architecture recovery [16]. Dependency types are not interchangeable in this respect, and call edges have been argued to indicate tighter coupling than type-usage edges [5, 34]. Collapsing them into one adjacency matrix removes the distinction between dependency types, so we keep one adjacency matrix per type and let the encoder learn a separate transformation for each. Section 5 evaluates this relation-typed design against a homogeneous graph for both GNN encoders.

We extract dependencies with the multilingual static-analysis tool *Depends* [39], which resolves the typed relations listed in Table 1 between code entities across our Java and C/C++ systems. Because the mapping task operates at file granularity, member-level dependencies such as a call between two methods are aggregated into edges between their enclosing files, and repeated dependencies of the same type between the same ordered pair collapse into a single edge, so $A^{(r)}$ records whether a relation holds and not how often it occurs.

Table 1
Static dependencies used in the software graph.

Dependency	Description
`Annotation`	Annotation relation to an annotated entity.
`Call`	Function or method invocation.
`Cast`	Conversion of an entity to another type.
`Contain`	Definition of a variable or field of another type.
`Create`	Creation of an instance or program entity.
`Extend`	Inheritance or parent–child type relation.
`Implement`	A class implements an interface or abstraction.
`Import`	A file imports other classes, methods, or variables.
`Parameter`	Use of a type as a method or function parameter.
`Return`	Use of a type as a return type.
`Throw`	A type is used as an exception.
`Use`	Use or assignment of variables.

`Annotation` and `Throw` are not observed in the C/C++ systems.

A static dependency is asymmetric, and $A^{(r)}$ preserves that direction: an edge points from the file where it originates to the file it refers to. For file i under relation r, we distinguish its in-neighborhood, the files that depend on it, from its out-neighborhood, the files it depends on. A file with a large in-neighborhood and a small out-neighborhood provides functionality, the profile of an interface or utility, and its clients may be spread across many modules. A file with the opposite profile consumes functionality, like orchestration or entry-point code, and what it draws on says more about what it coordinates than about which module it belongs to. Which of the two an encoder aggregates over, or whether it discards the direction altogether and treats the edge as undirected, is therefore a modeling choice rather than a fixed property of the graph; Section 3.3 defines it as a parameter of message passing, and Section 5 tests its impact on mapping.

3.3. Representation Learning and Mapping

We compare three encoders that operate on the same node features X. The graph-based models, GCN and GAT, use the relation-specific adjacency A_R to aggregate information from neighboring files, while the non-graph baseline uses X alone. All models take $h_i^{(0)} = x_i$ as input.

The GNNs stack K message-passing layers to produce $z_i = h_i^{(K)}$, while the MLP uses one hidden layer. Each encoder feeds its embedding into a linear classification head followed by softmax to obtain module probabilities. The encoders differ in their use of neighbors, aggregation weights, depth, and parameter count.

3.3.1. Graph Neural Networks (GNNs)

The GNNs update the representation of each file by aggregating messages from its neighbors under each dependency type r, a mechanism known as message passing. The neighborhood $\mathcal{N}_i^{r,\pi}$ is determined by relation r and propagation scheme π, as introduced in Section 3.2.2. Forward propagation is source-to-target: a file aggregates from files that depend on it. Reverse propagation aggregates from files it depends on, while undirected propagation adds each edge in both directions.

A relation- and layer-specific transformation $W^{r,(k)}$ is applied to each neighbor's message, so different dependency types, such as calls and inheritance, contribute through separate learned transformations. A K-layer encoder incorporates information from up to K dependency steps away. We add self-loops so each file's own features contribute to its update, even when it has no other neighbors. The neighborhoods in Equations 2 and 3 include these self-loops.

GCN: We use a relation-specific variant of the graph convolutional layer [20], which updates each file's representation by aggregating its neighbors' messages, weighted by the inverse square root of the product of the endpoint degrees under each relation:

$$h_i^{(k+1)} = \text{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \frac{1}{\sqrt{d_i^r d_j^r}} W^{r,(k)} h_j^{(k)} \Big), \quad (2)$$

where $W^{r,(k)}$ is the relation- and layer-specific transformation, and d_i^r and d_j^r denote the sizes of $\mathcal{N}_i^{r,\pi}$ and $\mathcal{N}_j^{r,\pi}$,

respectively. These degrees follow the propagation scheme π: incoming degrees for forward propagation and outgoing degrees for reverse propagation, both including self-loops. The implementation also learns a bias per relation and layer, omitted here for readability. ReLU [28] is the non-linear activation. Degree normalization reduces the influence of high-degree files, such as widely used base types, on the aggregation.

GAT: We use a relation-specific variant of the graph attention layer [36], which replaces fixed degree-based weighting with learned attention coefficients. For each relation r, the features of file i and its neighbors are projected using $W^{r,(k)}$ and used to compute attention scores. A softmax over $\mathcal{N}_i^{r,\pi}$ normalizes these scores to obtain the attention weights α_{ij}^r. The layer update for file i is:

$$h_i^{(k+1)} = \mathrm{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \alpha_{ij}^r \, W^{r,(k)} h_j^{(k)} \Big). \tag{3}$$

Equation 3 shows a single head; the implementation concatenates four heads per relation before summing relation outputs and applying ReLU. Learned attention weights increase flexibility but introduce additional parameters. Changing the propagation scheme π changes both the aggregated neighbors and their degree-normalization weights in GCN, whereas in GAT it changes the neighborhood over which attention is normalized.

3.3.2. Multi-layer Perceptron (MLP)

To assess the contribution of graph-neighborhood information, we use an MLP that operates on x_i alone and does not aggregate messages from other files. For each file i, the layer update rule is defined as:

$$h_i^{(k+1)} = \mathrm{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big). \tag{4}$$

Unlike the GNNs, whose representations incorporate information from neighboring files over multiple dependency hops, the MLP constructs each representation from the file's own node features only. Performance differences between the MLP and the GNNs are therefore consistent with a contribution of dependency-graph information, but do not isolate graph aggregation, since the encoders also differ in depth, dropout, and parameter count, with GAT introducing additional attention parameters.

3.3.3. Classification head

All three encoders end in the same kind of output stage. The file embedding $z_i \in \mathbb{R}^{d_h}$, which is the output of the last message-passing layer for GCN and GAT and of the hidden layer for the MLP, is passed to a single linear layer that maps it to one score per architectural module. A softmax then converts these scores into a probability distribution over the modules:

$$\begin{aligned} p_i &= \mathrm{softmax}\big(W_c z_i + b_c\big), \\ \hat{y}_i &= \arg\max_m p_{i,m}, \qquad \mathrm{confidence}_i = \max_m p_{i,m}, \end{aligned} \tag{5}$$

where $W_c \in \mathbb{R}^{|C|\times d_h}$, $b_c \in \mathbb{R}^{|C|}$, and $|C|$ is the number of architectural modules. The module with the highest probability, $\hat{y}_i$, is the model's prediction for file i, and the self-training procedure in Section 3.4 compares its probability, $\mathrm{confidence}_i$, against the threshold τ. The same softmax output enters the cross-entropy loss during training.

3.4. Iterative Learning Framework

GCN and GAT receive the same dependency graph and node features, differing only in how they aggregate and weight messages from neighboring files; the MLP uses the node features alone. Despite these differences, all three models follow the same self-training procedure, in which the confidence from Equation 5 decides which files are pseudo-labeled.

Training begins with a fixed set of labeled files L, while the remaining files form the unlabeled set U. After an initial warm-up phase on L, each round scores every file. Any pseudo-label that the current prediction contradicts with confidence above the threshold τ is revised. Files from U whose confidence exceeds τ are promoted into the mapped set $\mathcal{M}$, which initially equals L, with their predicted module as label. Ground-truth seed labels never change. The classifier is then trained further on $\mathcal{M}$ using full-batch cross-entropy, retaining the model parameters and Adam optimizer state across rounds. The procedure stops without further training when no new file exceeds τ, or after R rounds. Algorithm 1 summarizes this loop, identical across encoders except for the input representation and forward computation.

Algorithm 1: Iterative Mapping Algorithm.

Input: Input representation $\mathcal{I}$, files $\mathcal{F}$, labeled files L, confidence threshold τ, warm-up epochs E_w, self-training epochs E_s, max rounds R
Output: Trained classifier f, mapped files $\mathcal{M}$
$\mathcal{M} \leftarrow L$; $U \leftarrow \mathcal{F} \setminus L$;
Train f on $(\mathcal{I}, \mathcal{M})$ for E_w epochs;
for $k \leftarrow 1$ **to** R **do**
  Score all files in $\mathcal{F}$ with f using $\mathcal{I}$;
  Revise any pseudo-label in $\mathcal{M} \setminus L$ for which f predicts a different module with confidence $> \tau$;
  $\mathcal{P} \leftarrow \{o \in U \mid \mathrm{confidence}(o) > \tau\}$;
  if $\mathcal{P} = \emptyset$ **then**
    break;
  Assign each $o \in \mathcal{P}$ its current highest-probability module as a pseudo-label;
  $\mathcal{M} \leftarrow \mathcal{M} \cup \mathcal{P}$; $U \leftarrow U \setminus \mathcal{P}$;
  Retrain f on $(\mathcal{I}, \mathcal{M})$ for E_s epochs;
return f, $\mathcal{M}$;

4. Experiments

This section describes the experimental design used to answer the three research questions: the benchmark systems, configuration, baseline, and evaluation metrics.

4.1. Benchmark Systems

We use existing ground-truth (GT) mappings rather than constructing new ones, since a reliable C2A mapping requires substantial architectural knowledge, manual effort, and often expert validation [13]. Table 2 lists the ten Java and C/C++ systems in our evaluation.

Alongside system sizes, Table 2 reports graph properties, including average degree and file-level dependency homophily $\mathcal{H}$, the proportion of edges whose two files belong to the same GT module:

$$\mathcal{H} = \frac{|\{(i,j) \in \bar{E} : y_i = y_j\}|}{|\bar{E}|}, \tag{6}$$

where $\bar{E}$ is the set of merged edges, containing (i, j) if file i depends on file j under at least one dependency type. Homophily ranges from 0 (all dependencies cross modules) to 1 (all dependencies remain within modules).

Table 2
Properties of the software systems†. $|C|$ denotes the number of modules in the GT architecture, edges E are distinct typed dependencies between files, and the average typed out-degree is computed as $\langle k \rangle = |E|/|\text{Files}|$.

	System	Version	$\lvert C\rvert$	|Files|	$\lvert E\rvert$	$\mathcal{H}$	$\langle k \rangle$
Java	A.UML	r13713	14	766	9 491	0.55	12.4
	C.Img	v1.0a2	21	329	4 561	0.61	13.9
	JabRef	v3.7	6	1 180	17 557	0.58	14.9
	Lucene	r1075001	7	1 054	18 595	0.67	17.6
	SH-3D	r002382	9	167	3 912	0.36	23.4
	T.Mates	v5.11	15	778	17 818	0.34	22.9
C	Bash	4.2	13	292	1 222	0.60	4.2
	HDF	0e196f	9	153	1 239	0.35	8.1
C++	Chrome	svn-171054	69	18 343	284 435	0.58	15.5
	HDC	46ff87	11	207	1 117	0.63	5.4

†The file, module, and edge counts reflect the data used in our experiments, which for some systems omits a small number of files or edges from the raw pre-processed dataset.

The Java systems are drawn from the same source as our earlier work [14], assembled by Olsson et al. [29] from systems previously studied at SAEroCon and by Brunet et al. [4] and Lenhard et al. [22], with mappings distributed through the s4rdm3x GitHub repository[2].

For Bash, we use the C2A mapping curated by Garcia et al. [13], produced by deriving an initial decomposition from documentation and source code and then validating it with system experts. The same line of work provides the GT mapping for Chrome, established through roughly two years of collaboration with the system's developers [24].

The GT mappings for HDF and HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [40], distributed through the SARIF replication package[3].

[2] https://github.com/h0bb3/s4rdm3x
[3] https://github.com/anonymous2f4a9d/SARIF_FSE23

4.2. Experimental Configuration

GCN and GAT use two message-passing layers; the one-layer ablation is examined separately in Section 5. Both use 256 hidden channels. GAT concatenates four attention heads with 64 channels per head at each layer. The MLP has one hidden layer with 256 units. GCN and GAT apply dropout of 0.01 to node activations after each ReLU; the MLP uses no dropout. All three encoders are followed by the same kind of linear classification head. Thus, the encoders share the features, classifier form, and self-training schedule, but are not matched in depth or parameter count. All three models are trained with the Adam optimizer at a learning rate of 0.001, minimizing cross-entropy loss over the architectural modules on the labeled and pseudo-labeled files.

All models use the same iterative learning procedure: a warm-up phase of $E_w = 100$ epochs on the initial labeled set, followed by at most $R = 4$ self-training rounds of $E_s = 30$ epochs each. The initial labeled set contains 5% of a system's files, bounded between 30 and 200 files. The lower bound provides sufficient supervision in small systems, where 5% alone would yield too few labeled files, while the upper bound limits the manual labeling effort in large systems such as Chrome. After sampling, we add one randomly selected file from each module still missing from the initial labeled set, so every module has at least one labeled file before training. During self-training, only predictions with confidence above $\tau = 0.95$ are added as pseudo-labels. These settings are fixed across the neural models. We use the first 100 supplied fixed splits, with run IDs 0–99, for all methods and ablations. The initial 200-file cap applies before adding representatives of missing modules; the final seed set can therefore exceed 200 files.

The neural models were trained on an NVIDIA GB10 GPU using Python 3.11, PyTorch 2.9.1, and PyTorch Geometric 2.7.0; NBA ran on the CPU. Node features were generated once per system and reused across runs and neural configurations. The training seed was set to $42 + r$ for split r, independently of feature generation. The complete software environment, configurations, and split identities are recorded in the replication package.

To examine sensitivity to initialization and training randomness, we also conducted an exploratory follow-up using the first ten splits crossed with five training seeds, keeping the features fixed. The selected comparisons cover close model rankings and depth or direction effects, with the large Chrome graph advantage included as a reference. A subset was repeated under deterministic GPU execution after a same-seed discrepancy was observed on A.UML. These follow-ups are reported separately from the primary 100-run experiment.

4.3. Baseline: Naive Bayes

We compare against *NBAttract* [29] (NBA), a multinomial Naive Bayes classifier originally proposed for semi-supervised code-to-architecture mapping. NBA represents

each entity using text features extracted from code identifiers and package names, combined with module-level Concrete Dependency Abstraction (CDA) sentences, document terms that encode cross-module structural relations, e.g., *A-Inherits-B* when an entity in module *A* inherits from one in module *B*. Olsson et al. report that NBA outperforms *CountAttract* [6] and the information retrieval methods *IRAttract* and *LSIAttract* [3] on the same benchmark [29], so we treat NBA rather than those earlier methods as the reference point for prior work.

NBA was originally proposed and evaluated on Java systems only; we extend it to C/C++ systems in our benchmark and re-implement the algorithm in Python, following the original iterative procedure, including its 0.90 confidence threshold for adding pseudo-labels and the same initial labeled set used by the other methods. Self-training stops either when no remaining entity can be mapped with sufficient confidence or when all entities have been mapped. Code for our NBA re-implementation and C/C++ extension is available in the GitHub repository 1. For the Python baseline, Java node names retain the qualified entity names supplied by the extractor; for C/C++, directory-qualified file base names provide the analogous naming context. Identifier tokens use English Snowball stemming. We use binary term occurrences, multinomial Naive Bayes with Laplace smoothing ($\alpha = 1$), and class priors from the currently mapped files. Architectural module names also contribute training documents. This is a Python adaptation using scikit-learn, not a claim of numerical equivalence to the original Java/Weka implementation.

4.4. Performance Evaluation

Module distributions in the benchmark systems are highly imbalanced, with several modules containing only one or two files. We use macro-F1 to expose performance on smaller modules, which overall accuracy can obscure. We also report micro-F1, which equals accuracy for this single-label, exhaustive classification task. F1 is the harmonic mean of precision and recall. Precision measures the proportion of files assigned to a module that actually belong to it, whereas recall measures the proportion of files that truly belong to a module and are correctly assigned.

We report macro- and micro-averaged F1 scores. Macro-F1 is the unweighted average of per-module F1 scores over modules represented in the evaluated ground-truth subset. A module whose files all belong to the seed set has no held-out examples and is excluded from that macro average. Undefined precision and recall are assigned zero. Micro-F1 considers all predictions, including predictions into modules absent from the test ground truth, and reflects overall file-level mapping performance.

5. Results

We report results from 100 runs per model and system, using the same fixed labeled sets for all methods and ablations. All primary metrics cover the full test set, excluding the initial labeled files. For the neural models, each test file receives the final model's prediction, whether or not it was confidently mapped during self-training. NBA retains accepted assignments and assigns the remaining files using their last attraction scores. Thus, no method benefits from evaluation on a smaller subset. Results are summarized as the median across the 100 runs. This section addresses the three research questions in order.

We use two-sided Wilcoxon signed-rank tests [37] to assess statistical significance, pairing runs by their shared split ID. The compared neural configurations also share the training seed within each split. We apply Holm–Bonferroni correction at $\alpha = 0.05$, grouping comparisons by the question being tested: neural methods (60 tests), neural methods against NBA (60), relation types (40), depth (40), and direction (80). Each family includes both metrics and all ten systems. The tables report differences between the two medians, not medians of paired differences. Cliff's δ [8] describes the distributional effect size, classified by magnitude as negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), medium ($0.33 \leq |\delta| < 0.474$), or large ($|\delta| \geq 0.474$) [31].

RQ1. Overall Effectiveness of C2A Mapping

To answer RQ1, we compare NBA, MLP, GCN, and GAT over 100 runs per system, all initialized from the same labeled set; NBA follows its own self-training procedure (Section 4.3), while MLP, GCN, and GAT share the iterative learning loop of Section 3.4. Table 3 reports median macro and micro F1. All three neural methods outperform the Python NBA adaptation on both metrics across all ten systems, with statistically significant differences, although the size of the advantage varies. For example, MLP exceeds NBA by only 2.1 micro-F1 points on JabRef. We focus the remaining comparison on MLP, GCN, and GAT, with MLP assessing the contribution of node features without exploiting the dependency graph.

Using unrounded medians, GAT achieves the highest macro F1 on six systems and micro F1 on eight. MLP has the highest macro F1 on A.UML, SH-3D, HDF, and HDC, although several differences are small. On micro F1, MLP leads on HDF and GCN on HDC. HDF and HDC are among the smallest and least connected benchmark systems, whereas SH-3D is comparatively well connected but has low modular cohesion; in these systems, MLP either outperforms GAT or remains competitive. On Chrome, the only large-scale system in the benchmark, GAT and GCN exceed MLP by 21.8 and 19.0 macro-F1 points, and by 17.1 and 16.1 micro-F1 points, respectively. Because Chrome is the only large-scale system, this result alone cannot establish whether the graph advantage generalizes as system size increases. GCN consistently outperforms NBA but does not occupy a consistent position between MLP and GAT. RQ2 examines these system-level differences.

The same encoder and training settings are used for Java and C/C++ systems, with the language-dependent naming and path handling described in the feature construction. Bash, HDF, and HDC are the smallest and sparsest systems

Table 3
Median F1 scores (×100) across 100 runs. MA and MI denote macro- and micro-F1, respectively. The highest unrounded median for each system–metric pair is highlighted; highlighting does not imply statistical significance.

	A.UML		C.Img		JabRef		Lucene		SH-3D		T.Mates		Bash		HDF		Chrome		HDC	
Method	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
NBA	32.6	64.7	17.5	47.7	47.2	89.7	60.6	82.3	39.6	73.5	32.5	68.8	27.6	54.0	33.4	48.8	9.5	54.4	44.0	76.7
MLP	82.8	89.6	90.3	93.6	66.1	91.7	85.3	95.0	84.8	94.9	92.4	95.4	64.7	82.3	89.5	90.1	59.2	72.4	76.4	86.7
GCN	78.8	91.3	90.7	93.8	80.0	94.1	85.5	93.7	78.5	91.8	79.5	91.3	68.2	87.5	79.6	82.9	78.3	88.5	70.8	88.8
GAT	82.5	93.2	91.5	95.5	85.1	95.8	89.8	95.1	84.7	95.6	94.7	97.1	69.4	87.9	81.9	83.6	81.1	89.5	74.5	88.6

in the C/C++ subset, and MLP is competitive with or outperforms the GNNs on two of them.

Statistical significance of observed differences. Table 4 reports pairwise statistical comparisons of the three learned methods for the performance differences shown in Table 3. GAT significantly outperforms GCN on macro F1 for nine

Table 4
Differences of median F1 scores in points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor the first method. Cells in which the second method is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT − GCN		GCN − MLP		GAT − MLP	
System	MA	MI	MA	MI	MA	MI
A.UML	+3.7 (M)	+1.9 (L)	−4.0 (M)	+1.7 (S)	−0.3† (N)	+3.7 (L)
C.Img	+0.7 (S)	+1.7 (L)	+0.4† (N)	+0.2† (N)	+1.1 (S)	+1.9 (L)
JabRef	+5.0 (M)	+1.7 (L)	+13.9 (L)	+2.3 (L)	+19.0 (L)	+4.0 (L)
Lucene	+4.3 (M)	+1.4 (M)	+0.3† (N)	−1.3 (M)	+4.5 (M)	+0.0† (N)
SH-3D	+6.2 (M)	+3.8 (L)	−6.3 (L)	−3.1 (L)	−0.2† (N)	+0.7† (N)
T.Mates	+15.2 (L)	+5.9 (L)	−12.9 (L)	−4.1 (L)	+2.3 (S)	+1.8 (M)
Bash	+1.2† (N)	+0.4† (N)	+3.5† (S)	+5.2 (L)	+4.7 (S)	+5.6 (L)
HDF	+2.2 (S)	+0.7† (N)	−9.8 (L)	−7.2 (L)	−7.6 (L)	−6.5 (L)
Chrome	+2.8 (L)	+1.1 (S)	+19.0 (L)	+16.1 (L)	+21.8 (L)	+17.1 (L)
HDC	+3.6 (S)	−0.2† (N)	−5.5 (S)	+2.0† (N)	−1.9† (N)	+1.9† (N)

systems and on micro F1 for seven, with effects ranging from small to large. Neither difference is significant on Bash; on HDC, only the macro-F1 advantage is significant. GCN's difference from MLP varies substantially by system and metric. MLP significantly outperforms GCN on macro F1 on A.UML, SH-3D, T.Mates, HDF, and HDC, whereas GCN significantly outperforms MLP on JabRef and Chrome on both metrics. On Bash, GCN does not significantly outperform MLP on macro F1 but does on micro F1, with a large effect.

GAT significantly outperforms MLP on macro F1 for C.Img, JabRef, Lucene, T.Mates, Bash, and Chrome, and on micro F1 for A.UML, C.Img, JabRef, T.Mates, Bash, and Chrome. The differences on SH-3D are not significant on either metric. In contrast, MLP significantly outperforms GAT on both metrics only for HDF, with large effects. Despite its low connectivity, Bash favors GAT on both metrics; this exception is discussed in RQ2.

How stable is mapping across runs? Each of the 100 runs uses a different labeled partition and training seed; Figure 2 therefore shows F1-score variability arising from both sources. Micro F1 is generally stable, with interquartile ranges below 6 points for seven of ten systems. Macro F1 varies more because changes in the initial labeled set disproportionately affect small modules. This pattern is shared across MLP, GCN, and GAT. HDC is the most variable system, with macro-F1 interquartile ranges of 13–14 points, compared with about 8 points for micro-F1.

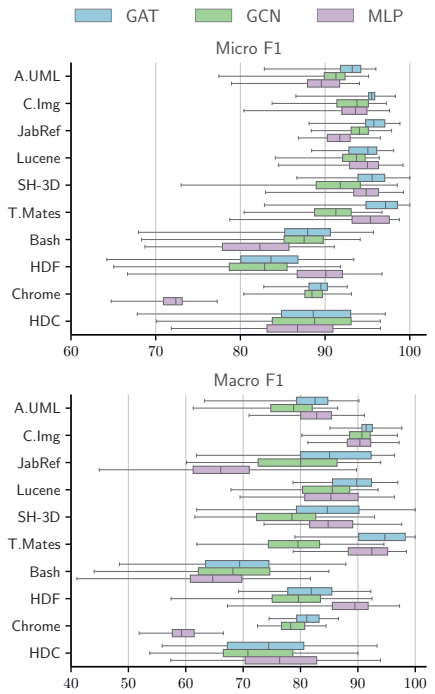


Figure 2: Distribution of micro F1 (top) and macro F1 (bottom), in points, across 100 runs for each model and system. Boxes show the median and interquartile range; whiskers span the observed range.

Variability is associated with the number of initially labeled files and, for some systems, graph connectivity, rather than system size or language. HDC, HDF, Bash, and SH-3D start near the 30-file floor, leaving only one or two labeled files per module. HDC, HDF, and Bash are also sparse, further limiting the structural signal available to graph-based models, whereas SH-3D is comparatively well connected. C.Img provides a contrast: despite its small size, it is dense and cohesive and remains stable. Once the initial training set reaches roughly 50 files, interquartile ranges are generally narrow regardless of scale; C.Img and Chrome are among the most stable systems on macro F1. Remaining macro-F1 variability is associated with module-size skew: systems with both very large and small modules, such as JabRef, retain wider distributions despite adequate supervision. RQ3 fixes the labeled partition across repeated runs to isolate the contribution of the training seed.

Answer to RQ1.

All three neural methods outperform NBA on both metrics across all ten systems. GAT achieves the highest averages of system-level median macro F1 (83.5) and micro F1 (92.2), leading on six and eight systems, respectively. GCN improves over NBA but does not consistently outperform the graph-free MLP baseline. Run-to-run variability is largest on macro F1 for the smallest systems. RQ2 examines the graph contribution beyond lexical features, while RQ3 isolates variability from the training seed.

RQ2. Contribution of the Dependency Graph

RQ1 shows that no learned method dominates throughout the benchmark. RQ2 examines what the dependency graph and GNNs contribute beyond lexical features: which system properties are associated with the benefit of message passing, and how often each model correctly classifies individual files across runs, a question median F1 cannot answer since it aggregates performance at the system level. All three neural models use identical lexical features and training loops, although their encoder architectures and capacities differ.

Which system properties are associated with where the graph helps? Connectivity describes how much information is available from neighboring files; homophily measures how often dependencies connect files in the same ground-truth module [41]. Bash, HDC, and HDF have the lowest average typed out-degrees in the benchmark (4.2, 5.4, and 8.1, respectively). MLP remains competitive with GAT on HDC and significantly outperforms it on HDF, but GAT significantly outperforms MLP on Bash. Sparse connectivity alone therefore does not determine whether message passing helps. These systems also differ in homophily, module composition, and the supervision available per module.

On the low-homophily systems SH-3D ($\mathcal{H} = 0.36$) and T.Mates ($\mathcal{H} = 0.34$), GCN performs significantly worse than MLP with large effect sizes. GAT is more competitive: neither difference from MLP is significant on SH-3D, while T.Mates favors GAT by 2.3 macro-F1 and 1.8 micro-F1 points. One possible explanation is that attention can assign less weight to uninformative cross-module neighbors than degree-normalized aggregation. We did not measure this mechanism directly, however, and the crossed follow-up shows seed-dependent rankings on both systems. On HDF ($\mathcal{H} = 0.35$), both GNNs perform significantly worse than MLP. Together, these observations are compatible with connectivity and homophily influencing the value of message passing, but do not establish a rule for selecting an encoder.

Which files does the graph help most, and where does it fail? For each file, we compute its misclassification rate as the proportion of its test-set evaluations in which its predicted module differs from the ground-truth module, and refer to files with a rate of at least 0.9 as *persistently wrong*. Files that appear only in labeled sets are excluded. Figure 3 contrasts Chrome against the pooled distribution of the other nine systems, giving each evaluated file equal weight.

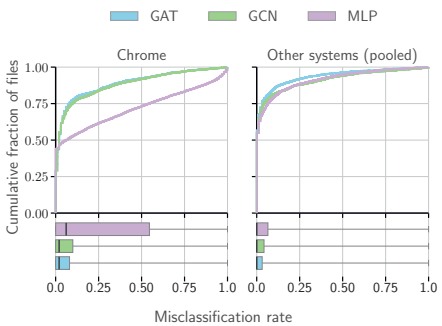


Figure 3: Empirical cumulative distribution of per-file misclassification rates across 100 runs, for GAT, GCN, and MLP, on Chrome (left) and pooled across the remaining nine systems (right). The y-axis is the cumulative proportion of test-set files whose misclassification rate is at or below the value on the x-axis, so a curve that rises earlier and higher indicates more reliable per-file decisions. The box plot below each panel shows the median, interquartile range, and full range of the misclassification rate for each model.

Across the nine small-to-medium-sized systems, the pooled distributions are concentrated near low error rates for all three models. Chrome shows a much larger contrast: MLP has a wider misclassification-rate distribution and a heavier tail of files misclassified in nearly every run. Among the 18,342 Chrome files evaluated at least once, 1,913 are persistently wrong under MLP (10.4%), compared with 213 under GCN (1.2%) and 204 under GAT (1.1%). Thus, the graph advantage on Chrome is visible not only in aggregate F1 but also in the frequency of repeated file-level errors.

Across all ten systems, 241 files meet the persistent-error criterion under default GAT, of which 204 are in Chrome.

These counts describe the current 100-run experiment; they can change with the labeled sets, training seeds, and error threshold. RQ3 examines how the persistent-error counts change when edge direction is altered.

Answer to RQ2.

Connectivity and homophily provide plausible context for the system-level differences, but neither alone determines whether a GNN improves on lexical features. GAT is more competitive than GCN on the low-homophily systems SH-3D and T.Mates, while MLP significantly outperforms both GNNs on HDF. On Chrome, both GNNs substantially improve aggregate F1 and reduce the share of persistently wrong files relative to MLP. These results support a contribution from dependency information, without isolating it from encoder capacity or establishing a causal model-selection rule.

RQ3. Sensitivity to Design Choices

RQ1 and RQ2 establish where the approach performs well and what the dependency graph contributes. RQ3 examines whether modeling dependency types separately is necessary and how sensitive the approach is to two message-passing design choices: the number of layers and the direction of aggregation. A crossed follow-up then tests whether the training seed alone can change the rankings observed in RQ1 and the ablation comparisons above. Because the benchmark systems have highly imbalanced module sizes, macro F1 can be volatile: it gives each module equal weight, so reassigning only a few files in a small module can substantially change the score. Micro F1 instead weights each file equally and may remain largely unchanged by the same reassignments. We therefore report both metrics throughout and regard a significant difference in either metric as evidence that the design choice affects performance.

Does modeling dependency types separately matter? Our encoders keep one adjacency matrix per relation type and learn a separate transformation for each, rather than collapsing all dependencies into a single edge type. Figure 4 compares this multi-relational design against a uni-relational variant, in which all dependency types are merged into one undifferentiated adjacency matrix before message passing, for both GAT and GCN across the ten benchmark systems.

Both GAT and GCN perform significantly better with multiple relation types than with merged edges on every system and both metrics. Median macro-F1 gains range from 6.6 to 40.2 points for GAT and from 8.1 to 32.7 points for GCN. The smallest gain occurs on C.Img for both encoders, and the largest on T.Mates. This is consistent with different dependency types carrying different structural signals about module membership: a call edge and an inheritance edge need not carry the same evidence that two files share a module. The multi-relational encoder also has more parameters than the uni-relational one, so the experiment does not isolate relation typing from added capacity.

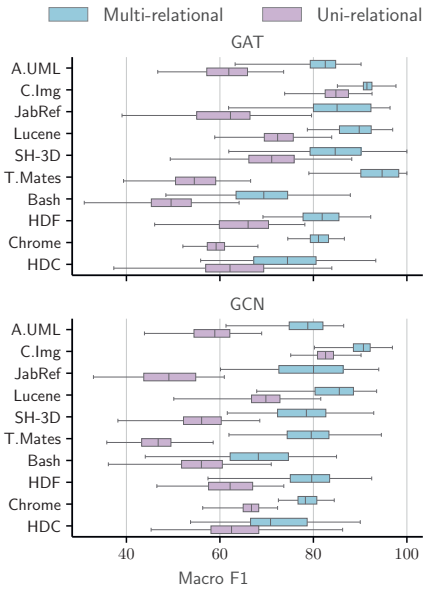


Figure 4: Distribution of macro F1, in points, across 100 runs for the multi-relational and uni-relational variants of GAT and GCN. Boxes show the median and interquartile range; whiskers span the observed range.

How many message-passing layers does the graph support? Additional GNN layers enlarge each node's receptive field, which is the L-hop neighborhood that can influence its representation after L message-passing layers. However, when repeated aggregation mixes information from a large portion of the graph, node representations can become increasingly similar and lose the discriminative information needed for classification, an effect known as over-smoothing [23]. We therefore explore only one and two layers, and Table 5 reports the comparison. According to Table 5, two layers significantly improve GAT's macro F1 on JabRef, T.Mates, Bash, and Chrome, and its micro F1 on JabRef, Bash, and Chrome. They significantly reduce macro F1 on HDC (-6.0 points) and micro F1 on Lucene (-1.1 points). Neither difference is significant on SH-3D. For GCN, two layers significantly improve both metrics on JabRef and Bash, and micro F1 on HDF and Chrome. They significantly reduce micro F1 on C.Img and SH-3D, both metrics on T.Mates, and macro F1 on HDC. The HDF micro-F1 gain does not remain significant when all neural comparisons are corrected together.

These results support system-dependent depth choices. The gains on JabRef are compatible with informative two-hop neighborhoods, while the losses on T.Mates for GCN

Table 5
Differences of median F1 scores in points for macro-F1 (MA) and micro-F1 (MI), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor two layers; cells in which one layer is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT (2L − 1L)		GCN (2L − 1L)	
System	MA	MI	MA	MI
A.UML	+0.4† (N)	−0.4† (N)	−0.3† (N)	−0.6† (N)
C.Img	+0.1† (N)	+0.0† (N)	−0.4† (N)	−1.1 (S)
JabRef	+3.2 (S)	+1.3 (M)	+5.1 (M)	+1.6 (M)
Lucene	+2.4† (N)	−1.1 (S)	+2.8† (S)	−0.5† (N)
SH-3D	+0.9† (N)	+0.7† (N)	+0.0† (N)	−0.9 (S)
T.Mates	+2.4 (S)	−0.3† (N)	−5.6 (L)	−2.6 (L)
Bash	+5.0 (M)	+4.6 (L)	+6.1 (S)	+3.8 (M)
HDF	−2.0† (N)	−0.9† (N)	−0.1† (N)	+1.6 (N)
Chrome	+2.9 (L)	+3.0 (L)	+0.1† (N)	+1.3 (L)
HDC	−6.0 (S)	−0.5† (N)	−9.1 (M)	−0.2† (N)

are compatible with greater mixing across modules. These mechanisms were not measured directly, however, and neither connectivity nor homophily alone explains every result. Two GAT layers benefit several systems in this benchmark, but are not uniformly preferable.

Does edge direction matter? The heterogeneous encoder aggregates source-to-target by default, so a file's representation is shaped by the neighbors that depend on it rather than by the neighbors it depends on. Table 6 compares this against an undirected variant, which symmetrizes every edge, and a reverse variant, which aggregates target-to-source. As shown in Table 6, source-to-target aggregation significantly outperforms the reversed version in macro F1 across six systems: C.Img, JabRef, SH-3D, T.Mates, Chrome, and HDC for GAT. Reversing the aggregation notably enhances macro F1 in A.UML and Lucene by 1.5 and 4.2 points, respectively, and raises micro F1 in Lucene by 1.2 points. No significant difference is observed between original and reversed aggregation on Bash or HDF. Compared to undirected aggregation, the directed default significantly improves at least one metric on nine systems, although no difference is significant on Bash.

GCN is more mixed: the default has a significant advantage over at least one alternative on six systems, and a significant disadvantage on six, with some systems in both groups. On JabRef, reversal reduces macro F1 but improves micro F1; the latter difference is not significant under the combined correction across all neural comparisons. These metric-dependent effects do not support one direction for every system.

At the file level, reversal reduces the number of persistently wrong GAT files from 11 to 6 on Lucene and leaves it at 9 on A.UML. Chrome illustrates a different trade-off: reversal reduces the count from 204 to 103 even though median macro F1 falls by 4.3 points; undirected aggregation raises the count to 322. Aggregate F1 and the size of the persistent-error tail can therefore move in different directions. These counts do not establish that the same individual errors are resolved without introducing new ones.

How robust are these rankings to the training seed? The 100-run comparisons in RQ1 and the preceding ablations vary both the labeled partition and the training seed. We therefore conduct an exploratory follow-up to test whether changing only the training seed can alter model rankings. For each of ten fixed splits, we train each configuration with five seeds. We summarize each comparison using the mean paired difference, first averaging across seeds within each split. To quantify uncertainty, we use a crossed bootstrap that resamples splits and seeds separately. We apply a Bonferroni adjustment to the 95% intervals for the 24 comparisons in the broader follow-up and retain this adjustment for the deterministic subset. Thus, the 50 runs per configuration are not treated as 50 independent splits.

Table 7 reports the selected comparisons repeated under deterministic execution. Even with the labeled files held fixed, the macro-F1 ranking changes across seeds on six of ten splits for GAT versus MLP on SH-3D, five for the same comparison on T.Mates, and eight for GAT versus GCN on HDC. None of these three intervals rules out the possibility of no difference. This does not overturn RQ1's significant advantages on T.Mates and HDC, but it shows those advantages are less consistent once the effect of seed is isolated. For the reversal comparison, Lucene holds up under this check, while A.UML does not. GAT's advantage over MLP on Chrome is the most robust result: it wins in all 50 split-seed combinations, with a large and consistent margin of about 22.6 points and an adjusted interval of [19.1, 26.6].

The deterministic follow-up was prompted by an A.UML run that changed when repeated with the same seed. Deterministic execution produced identical results in all twelve tested configuration-level repeat checks. However, it changed the selected mean comparisons by less than 0.1 F1 points and left all seed-ranking flip counts unchanged. Thus, the observed seed sensitivity cannot be explained by numerical nondeterminism alone.

Answer to RQ3.

Multi-relational aggregation improves both metrics on every system for both encoders, although part of the gain may reflect added capacity. Depth and direction are more system-dependent: two GAT layers benefit several systems, while reversed aggregation improves macro F1 on A.UML and Lucene. The deterministic follow-up supports the Lucene and Chrome advantages but shows that several closer rankings, on SH-3D, T.Mates, and HDC, depend on the training seed. No depth or direction is uniformly best, and direction changes can reduce persistent errors even when aggregate macro F1 decreases, as on Chrome.

Table 6
Differences of median F1 scores in points for macro-F1 (MA) and micro-F1 (MI) under default directed (D) source-to-target aggregation, compared with UD (undirected, symmetrized edges) and RD (reversed, target-to-source), with Cliff's δ magnitude in parentheses (N/S/M/L). Positive values favor D; cells where the alternative is significantly better are highlighted. † marks a difference that is not statistically significant.

	GAT				GCN			
	D − UD		D − RD		D − UD		D − RD	
System	MA	MI	MA	MI	MA	MI	MA	MI
A.UML	+2.7 (S)	+1.5 (M)	−1.5 (S)	−0.2† (N)	−0.9 (S)	−1.0 (S)	−3.3 (M)	−1.5 (M)
C.Img	+2.9 (L)	+1.4 (L)	+5.8 (L)	+2.1 (L)	+3.8 (M)	+0.3† (S)	+9.3 (L)	+1.1 (S)
JabRef	+10.7 (L)	+1.4 (M)	+8.3 (L)	+0.2† (N)	+3.6† (N)	−0.3† (N)	+10.6 (L)	−0.8 (S)
Lucene	+3.1 (M)	+1.2 (S)	−4.2 (L)	−1.2 (M)	−2.0† (S)	−0.6 (S)	−7.4 (L)	−1.4 (M)
SH-3D	+18.6 (L)	+7.4 (L)	+14.5 (L)	+5.9 (L)	+8.7 (L)	+3.6 (L)	+16.6 (L)	+5.2 (L)
T.Mates	+8.2 (L)	+1.4 (S)	+9.5 (L)	−0.1† (N)	−0.9† (S)	−2.3 (M)	+0.8† (N)	−4.8 (L)
Bash	−1.6† (N)	+0.4† (N)	−1.5† (N)	+0.1† (N)	−3.2 (S)	−0.7† (N)	−3.7 (S)	−0.4† (N)
HDF	+3.6 (M)	+4.2 (L)	−3.0† (S)	−1.5† (N)	+3.1 (S)	+4.4 (M)	−2.5 (S)	+0.2† (N)
Chrome	+6.2 (L)	+2.0 (L)	+4.3 (L)	−0.1† (N)	+2.4 (M)	−0.1† (N)	+2.6 (L)	−0.1† (N)
HDC	+8.9 (M)	+1.3 (N)	+10.9 (L)	−0.2† (N)	+3.1 (S)	+0.8† (N)	+11.1 (L)	+1.8† (N)

Table 7
Exploratory macro-F1 comparisons under deterministic execution. Effects are mean paired differences in F1 points, with family-adjusted 95% crossed-bootstrap intervals. *Flips* counts the number of splits in which both positive and negative differences occur across the five seeds. RD and D denote reversed and default directions.

System	Comparison	Mean	Interval	Flips
A.UML	GAT RD − D	+2.4	[−1.9, 6.4]	3/10
Lucene	GAT RD − D	+6.7	[2.1, 11.8]	3/10
SH-3D	GAT − MLP	+1.0	[−5.2, 7.0]	6/10
T.Mates	GAT − MLP	+2.1	[−2.7, 6.9]	5/10
Chrome	GAT − MLP	+22.6	[19.1, 26.6]	0/10
HDC	GAT − GCN	+4.5	[−1.1, 11.3]	8/10

6. Threats to Validity

External Validity: The ten benchmark systems span Java, C, and C++, but they do not represent the full range of architectural styles, application domains, development practices, or programming languages. They were selected because usable ground-truth mappings were available. Chrome is the only system at industrial scale; consequently, observations about scalability rest on a single system rather than a broader trend. None of the systems consists primarily of AI-generated code, so it remains unknown whether such code yields dependency structures and module cohesion comparable to those in the studied systems.

Our approach is transductive and assumes that the graph is fixed during training. Adding files or changing dependencies requires reconstructing the graph and retraining the model. The approach is therefore more suitable for periodic architecture analyses and maintenance workflows than for incremental mapping at commit time.

Internal Validity: The dependency graph is derived through static analysis. Static extraction can miss dependencies that arise only at runtime, for example through reflection, function pointers, conditional compilation, or macro expansion. It can also introduce imprecise edges when references cannot be resolved unambiguously. Because message passing depends on the extracted graph, missing and incorrect edges can directly affect the model's predictions.

A further threat arises from version alignment between the analyzed source code and the published ground-truth mapping. Several benchmark mappings were created years before the source versions used in this study were obtained. In addition, the files included in a system can depend on its build configuration. The exact file set in the analyzed revision may therefore differ from that represented in the ground-truth mapping.

Files can plausibly belong to more than one module, and different annotators may assign such files differently. In older systems, architectural drift may also weaken the correspondence between a file and its assigned ground-truth module. Since the initial labeled set is sampled from the same mapping used for evaluation, an incorrect ground-truth assignment can both provide misleading supervision during training and penalize a prediction that is architecturally plausible.

The primary unit of analysis is the file, following the granularity of the available ground-truth mappings. A file that implements multiple concerns must nevertheless receive one module label. Neither the GNNs nor the compared baselines can represent multiple simultaneous module assignments.

The 100 initial labeled sets and training seeds are fixed and reused across neural configurations. Since both change between runs, the primary experiment does not separate their contributions to variability. The crossed follow-up holds each partition fixed while varying the training seed and reveals seed-dependent rankings on several systems. It covers only ten splits and five seeds, with comparisons

selected after the main experiment, and should therefore be interpreted as exploratory rather than as a replacement for the primary analysis.

The primary experiment did not enforce deterministic GPU operations. Same-seed controls exposed a numerical discrepancy on A.UML that was amplified during training and pseudo-label selection. Deterministic execution produced identical results in the tested repeat controls, but did not remove the seed-dependent rankings or materially change the selected average comparisons. This check does not establish repeatability on every software or hardware configuration.

Construct Validity: A fixed confidence threshold determines whether an unlabeled file receives a pseudo-label. This threshold captures how concentrated the model's softmax output is, rather than mapping quality alone. We evaluate predictions over the complete test set, preventing a model that pseudo-labels fewer files from benefiting from evaluation on an easier subset.

Directory paths are included in the node features, and in several benchmark systems the folder hierarchy closely reflects the reference module decomposition. Where this alignment is strong, performance may be optimistic because folder-path features provide partial information about the target module, rather than requiring the model to infer the mapping from identifiers and dependencies alone.

The encoders share features and a training schedule but differ in depth, dropout, and parameter count. In particular, the multi-relational GNNs have more parameters than their merged-relation counterparts. Their advantages cannot therefore be attributed solely to graph structure or relation typing. The NBA baseline is a Python adaptation extended to C/C++, not a numerically verified reproduction of the original Java implementation.

Statistical significance depends in part on the comparison family. Combining all neural comparisons into one correction family leaves most conclusions unchanged, but removes significance for GCN's micro-F1 depth gain on HDF and reversal gain on JabRef. Persistent-error counts also depend on the chosen threshold and the files' test-set appearances; they describe this experiment rather than a seed-invariant set of difficult files.

7. Conclusion

We studied code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, comparing GAT and GCN against an MLP baseline that receives the same lexical features and passes through the same self-training loop. Across ten systems in Java and C/C++ and 100 runs per system, all three neural models outperform the Naive Bayes reference on both metrics. GAT achieves the highest averages of system-level median macro F1 (83.5) and micro F1 (92.2), although MLP remains competitive or superior on several systems. On Chrome, the largest system in the benchmark at 18,343 files, GAT and GCN outperform MLP by 22 and 19 macro-F1 points, respectively. With only one large-scale system in the benchmark, this gap cannot establish whether the graph's advantage scales with system size more generally.

Two system properties are plausibly associated with where this advantage appears: connectivity sets a baseline for how much neighborhood information a file can aggregate, and homophily measures how often dependency edges connect files in the same ground-truth module. These associations are observational and do not establish a causal selection rule. At the file level, Chrome again shows the clearest contrast: MLP persistently misclassifies 10.4% of the files observed in a test set, compared with 1.2% for GCN and 1.1% for GAT.

Letting the encoder learn a separate transformation per relation outperforms a single merged edge type on every system, although this also increases model capacity. Depth and direction have more system-dependent effects. Two GAT layers benefit several systems but are not uniformly preferable, and reversed aggregation improves macro F1 on A.UML and Lucene. Direction can also trade aggregate performance against persistent errors: on Chrome, reversal reduces the persistent-error count despite a lower macro F1.

The crossed seed study supports the large graph advantage on Chrome and the reversal advantage on Lucene, while showing that several other rankings vary with the training seed. Deterministic execution removes discrepancies in the tested same-seed controls without resolving this sensitivity. Average advantages should therefore not be read as guarantees for an individual labeled set or training run.

Three directions could clarify and extend the contribution of dependency information. First, an additional attention mechanism over relation types could replace the unweighted sum of relation outputs when different dependencies provide conflicting evidence; the current GAT already learns separate neighbor-attention parameters for each relation. Second, the graph could incorporate finer-grained architectural information, including folder hierarchies and class members, to better reflect how architects conceptualize modules. Third, separating identifier-based lexical features from path-based features would clarify how much of the MLP baseline's signal comes from naming versus directory structure, and how much of that overlaps with what the dependency graph already encodes.

CRediT authorship contribution statement

Rakhshanda Jabeen: Conceptualization, Methodology, Software, Investigation, Writing (original draft, review & editing). **Morgan Ericsson:** Conceptualization, Software review, Supervision, Writing (review & editing). **Jonas Nordqvist:** Supervision, Writing (review & editing). **Anna Wingkvist:** Supervision, Writing (review & editing).

Declaration of use of AI-assisted technologies

We used various AI tools to review the manuscript and the source code used for the experiments. We used the

output from these tools to manually update the manuscript and fix any issues found in the source code. We take full responsibility for the content of the manuscript and the source code provided in the reproduction package.

Funding

Rakhshanda Jabeen was supported by the Industrial Graduate School "Data Intensive Applications," funded by the Knowledge Foundation, Sweden (grant no. 20190336).

The Knowledge Foundation had no role in the study design; collection, analysis, or interpretation of data; preparation of the manuscript; or decision to submit the article for publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data Availability

The replication package, including source code, experimental splits, and results, is available at https://github.com/sse-lnu/GCN-SAEroCon25, together with instructions for obtaining the benchmark data and reproducing the experiments. Upon acceptance, the version corresponding to this article will be archived on Zenodo, and its DOI will be included in the final manuscript.

References

[1] Ali, N., Baker, S., O'Crowley, R., Herold, S., Buckley, J., 2018. Architecture consistency: State of the practice, challenges and requirements. Empirical Software Engineering 23, 224–258.

[2] Amalfitano, D., De Luca, M., Santilli, T., Pelliccione, P., Fasolino, A.R., 2025. Automated software architecture design recovery from source code using llms, in: European Conference on Software Architecture, Springer. pp. 73–89.

[3] Bittencourt, R.A., Santos, G.J.d., Guerrero, D.D.S., Murphy, G.C., 2010. Improving automated mapping in reflexion models using information retrieval techniques, in: 17th Working Conference on Reverse Engineering, pp. 163–172.

[4] Brunet, J., Bittencourt, R.A., Serey, D., Figueiredo, J., 2012. On the evolutionary nature of architectural violations, in: 2012 19th Working Conference on Reverse Engineering, IEEE. pp. 257–266.

[5] Chen, J., Wang, Z., Jiang, Y., Pang, J., Zhang, T., Pan, M., Sun, J., 2022. NEGAR: network embedding guided architecture recovery for software systems, in: 2022 29th Asia-Pacific Software Engineering Conference (APSEC), IEEE. pp. 367–376. doi:10.1109/apsec57359.2022.00049.

[6] Christl, A., Koschke, R., Storey, M.A., 2007. Automated clustering to support the reflexion method. Information and Software Technology 49, 255–274.

[7] Clements, P., Garlan, D., Little, R., Nord, R., Stafford, J., 2003. Documenting software architectures: views and beyond, in: 25th International Conference on Software Engineering, 2003. Proceedings., IEEE. pp. 740–741.

[8] Cliff, N., 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological bulletin 114, 494–509. doi:10.1037/0033-2909.114.3.494.

[9] De Silva, L., Balasubramaniam, D., 2012. Controlling software architecture erosion: A survey. Journal of Systems and Software 85, 132–151. doi:10.1016/j.jss.2011.07.036.

[10] Ehsani, R., Rawal, S., Cai, Y., Chatterjee, P., 2026. Faster code, deeper debt? A multivocal literature review on technical debt and its early signs in LLM-assisted software development. ACM Transactions on Software Engineering and Methodology doi:10.1145/3820165.

[11] Florean, A., Jalal, L., Sinkala, Z.T., Herold, S., 2021. A comparison of machine learning-based text classifiers for mapping source code to architectural modules, in: Companion Proceedings of the 15th European Conference on Software Architecture. URL: https://ceur-ws.org/Vol-2978/saerocon-paper1.pdf.

[12] Fuchß, D., Liu, H., Hey, T., Keim, J., Koziolek, A., 2026. Exarch: Enabling architecture traceability by llm-based architecture component name extraction, in: SE2026, Gesellschaft für Informatik, Bonn. pp. 61–62. doi:10.18420/se2026_20.

[13] Garcia, J., Krka, I., Mattmann, C., Medvidovic, N., 2013. Obtaining ground-truth software architectures, in: 35th International Conference on Software Engineering (ICSE), IEEE. pp. 901–910. doi:10.1109/icse.2013.6606639.

[14] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2025. Graph convolutional networks for mapping source code entities to architectural modules, in: 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), IEEE. pp. 472–481.

[15] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2026a. GAER: Graph auto-encoders for unsupervised software architecture recovery. Proceedings of the ACM on Software Engineering 3, 4094–4115. doi:10.1145/3808187.

[16] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2026b. Graph embeddings for software architecture recovery. Nordic Machine Intelligence 6, 20–36. URL: https://journals.uio.no/NMI/article/view/13153, doi:10.5617/nmi.13153.

[17] Jabeen, R., Ericsson, M., Nordqvist, J., Wingkvist, A., 2026c. Graph representation learning for software architecture recovery., in: ESANN, pp. 457–462. doi:10.14428/esann/2026.ES2026-316.

[18] Johansson, N., Caporuscio, M., 2026. Mapping source code to software architecture using embedding models: Insights from an analysis of data properties. Information and Software Technology 198, 108200. URL: https://www.sciencedirect.com/science/article/pii/S0950584926001898, doi:https://doi.org/10.1016/j.infsof.2026.108200.

[19] Johansson, N., Caporuscio, M., Olsson, T., 2024. Mapping source code to software architecture by leveraging large language models, in: Software Architecture. ECSA 2024 Tracks and Workshops, Springer. pp. 133–149. doi:10.1007/978-3-031-71246-3_13.

[20] Kipf, T.N., Welling, M., 2017. Semi-supervised classification with graph convolutional networks, in: International Conference on Learning Representations (ICLR). URL: https://openreview.net/forum?id=SJU4ayYgl.

[21] Kruchten, P., Nord, R.L., Ozkaya, I., 2012. Technical debt: From metaphor to theory and practice. IEEE Software 29, 18–21. URL: https://philippe.kruchten.com/wp-content/uploads/2012/08/kruchten-120821-techdebt.pdf.

[22] Lenhard, J., Blom, M., Herold, S., 2019. Exploring the suitability of source code metrics for indicating architectural inconsistencies. Software Quality Journal 27, 241–274.

[23] Li, Q., Han, Z., Wu, X.M., 2018. Deeper insights into graph convolutional networks for semi-supervised learning, in: Proceedings of the AAAI conference on artificial intelligence, pp. 3538–3545. doi:10.1609/aaai.v32i1.11604.

[24] Lutellier, T., Chollak, D., Garcia, J., Tan, L., Rayside, D., Medvidović, N., Kroeger, R., 2018. Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Transactions on Software Engineering 44, 159–181. doi:10.1109/tse.2017.2671865.

[25] Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality. Advances in Neural Information Processing Systems 26.

[26] Murphy, G.C., Notkin, D., Sullivan, K., 1995. Software reflexion models: Bridging the gap between source and high-level models, in: 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering, pp. 18–28.

[27] Murphy, G.C., Notkin, D., Sullivan, K.J., 2001. Software reflexion models: Bridging the gap between design and implementation. IEEE Transactions on Software Engineering 27, 364–380. doi:10.1109/32.917525.

[28] Nair, V., Hinton, G.E., 2010. Rectified linear units improve restricted boltzmann machines, in: Proceedings of the 27th international conference on machine learning (ICML-10), pp. 807–814.

[29] Olsson, T., Ericsson, M., Wingkvist, A., 2022. To automatically map source code entities to architectural modules with naive bayes. Journal of Systems and Software 183, 111095. doi:10.1016/j.jss.2021.111095.

[30] Pan, R., Mao, B., Ma, T., Ling, Z., 2026. ArchAgent: Scalable legacy software architecture recovery with LLMs, in: 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 19357–19361. doi:10.1109/ICASSP55912.2026.11462552, arXiv:2601.13007.

[31] Romano, J., Kromrey, J.D., Coraggio, J., Skowronek, J., 2006. Appropriate statistics for ordinal level data: Should we really be using t-test and cohen's d for evaluating group differences on the nsse and other surveys, in: annual meeting of the Florida Association of Institutional Research. URL: https://web.archive.org/web/20070716131907id_/http://www.florida-air.org/romano06.pdf.

[32] Rukmono, S.A., Ochoa, L., Chaudron, M., 2024. Deductive software architecture recovery via chain-of-thought prompting, in: 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp. 92–96. doi:10.1145/3639476.3639776.

[33] Sinkala, Z.T., Herold, S., 2021. Inmap: Automated interactive code-to-architecture mapping recommendations, in: 18th International Conference on Software Architecture, pp. 173–183.

[34] Stavropoulou, I., Grigoriou, M., Kontogiannis, K., 2017. Case study on which relations to use for clustering-based software architecture recovery. Empirical Software Engineering 22, 1717–1762. doi:10.1007/s10664-016-9459-z.

[35] Tzerpos, V., Holt, R.C., 1997. The orphan adoption problem in architecture maintenance, in: Working Conference on Reverse Engineering, IEEE. pp. 76–82.

[36] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y., 2018. Graph attention networks, in: International Conference on Learning Representations (ICLR). URL: https://openreview.net/forum?id=rJXMpikCZ.

[37] Wilcoxon, F., 1992. Individual comparisons by ranking methods, in: Breakthroughs in statistics: Methodology and distribution. Springer, pp. 196–202.

[38] Wu, W., Hu, H., Fan, Z., Qiao, Y., Huang, Y., Li, Y., Zheng, Z., Lyu, M., 2025. An empirical study of code clones from commercial AI code generators. Proc. ACM Softw. Eng. 2, 2874–2896. doi:10.1145/3729397.

[39] Zhang, G., Jin, W., 2018. Depends: A fast, comprehensive code dependency analysis tool. https://github.com/multilang-depends/depends. Software repository, accessed: 2026-08-28.

[40] Zhang, Y., Xu, Z., Liu, C., Chen, H., Sun, J., Qiu, D., Liu, Y., 2023. Software architecture recovery with information fusion, in: Proceedings of the 31st ACM Joint ESEC/FSE, pp. 1535–1547. doi:10.1145/3611643.3616285.

[41] Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., Koutra, D., 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. Advances in neural information processing systems 33, 7793–7804.

Paper IV

Graph Embeddings for Software Architecture Recovery

IV

Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. "Graph Embeddings for Software Architecture Recovery". In: *Nordic Machine Intelligence* (2026). DOI: 10.5617/nmi.13153. URL: https://journals.uio.no/NMI/article/view/13153. Reprinted under the Creative Commons Attribution 4.0 International License (CC BY 4.0). Copyright © 2026 Nordic Machine Intelligence.

NORDIC MACHINE INTELLIGENCE

Nordic Machine Intelligence, vol. 6, pp. 20–36, 2026
Received 15 Feb 2026 / published 17 Jun 2026
https://doi.org/10.5617/nmi.13153

Graph Embeddings for Software Architecture Recovery

Rakhshanda Jabeen[1,2] Morgan Ericsson[1], Jonas Nordqvist[3], and Anna Wingkvist[1]

1. *Department of Computer Science, Linnaeus University, Växjö, Sweden*
2. *Electrolux Professional AB, Ljungby, Sweden*
3. *Department of Mathematics, Linnaeus University, Växjö, Sweden*

Abstract

A software system's source code is organized into files, but the way those files group into architectural modules is often undocumented or has drifted from the original design. Software architecture recovery (SAR) reconstructs this module view to help developers understand and maintain large systems.

We study SAR from file-level static dependency graphs by learning file embeddings in a self-supervised manner and clustering them into modules. Using node2vec as a baseline, we train graph autoencoders via link reconstruction and compare dependency-based representations (a graph attention encoder, with and without personalized PageRank diffusion) against heterogeneous ones that also incorporate the project's folder structure. We evaluate eleven open-source Java, C, and C++ systems against ground-truth architectures using complementary alignment and modularity metrics. Recovery from a dependency structure alone is useful but limited, bounded more by how much structural context is available to the model than by the choice of encoder. Folder organization helps where dependency cohesion is weak, but heterogeneous methods often gain alignment by drifting from pure dependency modularity, a trade-off between structural cohesion and the developer-intended decomposition. No single method wins on every system and metric — node2vec and the diffusion model are comparable on dependencies alone, while folder-aware methods are worth their added complexity mainly when cohesion is weak. All representations scale well and are practical to run and tune, so representation learning is not the bottleneck for adoption.

Keywords: Variational graph autoencoders, Graph attention networks, Software architectures

1 Introduction

Software architecture describes the high-level structure of a system in terms of its principal elements, their responsibilities, and the relations among them [1]. In implementation, this structure is often reflected as a modular decomposition: a partition of files or classes into modules that group related functionality [2]. Such decompositions are important for program understanding, maintenance, onboarding, and refactoring, but documented architecture is often missing, incomplete, or quickly outdated as systems evolve [1]. The problem is becoming more pressing as AI coding assistants increase the volume of code they generate, with the intended module structure seldom recorded.

Software Architecture Recovery (SAR) aims to infer this modular decomposition directly from implementation artifacts [3, 4]. Traditional SAR methods often rely on the software dependency graph (SDG), where files or classes are connected by static relations such as calls, imports, and inheritance. The underlying assumption is that files implementing related functionality tend to be more densely connected within modules than across module boundaries [5]. Existing approaches exploit this signal through hierarchical clustering [6, 7], search-based optimization guided by cohesion and coupling objectives [8, 9, 10], or complementary sources such as lexical semantics [11, 12], multi-source fusion [13, 14], and large language models [15].

Graph representation learning provides another way to use these signals. Instead of directly clustering the SDG, it learns dense embeddings of source files and clusters the resulting representations. NEGAR [5] showed that node2vec-style random walks can recover useful architectural structure from dependency context. GAER [4] extended this direction by using Graph Neural Network (GNN) encoders over heterogeneous dependency graphs, typed edges, and semantic node initialization, demonstrating that graph neural models with semantic features can improve recovery. These results leave open an important question: how far can graph representation learning go from dependencies alone, and when are additional signals needed?

We study graph representation learning for SAR by separating the contributions of dependency structure and

project organization. First, we evaluate dependency-only representations on the homogeneous file-level SDG, including random-walk embeddings and Graph Attention Network (GAT) [16] encoders trained within a Variational Graph Autoencoder (VGAE) [17] objective. In this setting, the GAT learns file embeddings from dependency neighborhoods, and the VGAE objective trains the embeddings to reconstruct dependency edges. Second, we augment the SDG with folder nodes and folder–file containment edges to test whether project layout provides an architectural signal beyond dependencies. We compare meta-path-guided random walks with heterogeneous GAT encoders, and include a typed-dependency variant to test whether preserving dependency types adds value within the heterogeneous setting. Finally, we evaluate the practical cost of these methods in terms of runtime and hyperparameter sensitivity[1].

Our empirical study is guided by the following research questions:

RQ1: How effectively can software architecture be recovered from dependency structure alone?

RQ2: Does augmenting the dependency graph with project organizational structure improve recovery?

RQ3: What are the practical costs and tuning sensitivities of these methods as system size grows?

The paper is organized as follows: Section 2 reviews related work, and Section 3 formulates the learning objective and describes our methodology. Section 4 details the experimental configurations and evaluation metrics, and Section 5 presents and analyzes the results. Section 6 discusses threats to validity, and Section 7 concludes the paper and outlines future directions.

2 Related Work

Prior SAR work leverages different signals, including static dependencies [18, 19, 8], project organization, such as directory structure and file paths [20], textual information, such as identifiers and comments [21, 22], and hybrid approaches that integrate multiple signals [23, 24, 25].

SDG-based approaches use structural information for modularization and fall into two groups. Search-based methods cast recovery as the optimization of a modularization objective, such as Modularization Quality (MQ), which rewards cohesion and penalizes coupling. Bunch and its hill-climbing variants (NAHC/SAHC) are representative [9], and MCA and ECA add multi-objective genetic optimization [26]. Hierarchical methods instead merge clusters greedily by dependency-derived similarity: WCA uses dependency-based feature vectors [10], LIMBO summarizes the SDG using information theory before merging [7], and CCT [27] and HCC [6] combine multiple similarity views or dendrograms into a consensus decomposition. Altınışık et al. [28] proposed an approach that treats each dependency model separately during clustering and then aggregates the resulting decompositions. A further line targets scalability, including FCA's matrix-based clustering for large systems [29], the deterministic neighborhood-tree method of Mohammadi and Izadkhah [30], and GMA, which derives similarity from the depth of dependency relations [31].

Beyond dependencies, many approaches used directory structure and semantic signals from code and documentation for SAR [20]. ARC [11] uses topic modeling, while ZBR [12] constructs weighted lexical zones via expectation maximization on textual features to support clustering. Hybrid methods combine structural and non-structural sources using automated feature weighting [32]; SADE and SARIF fuse multiple signals by constructing weighted dependency graphs and applying community detection [14, 13]. More recently, LLM-assisted approaches have explored mapping implementation units to a reference architecture using chain-of-thought prompting and deductive reasoning [15]. SemArc [33] uses an LLM to interpret code and architectural semantics based on canonical patterns, then clusters components using an anchor-based strategy.

Representation learning offers an alternative: it learns dense embeddings of source-code files and clusters them to recover modules. NEGAR [5] uses node2vec-style random walks with a Skip-gram objective to place files with similar structural contexts close together, then clusters the embeddings. GAER [4] extended this with GNN encoders, comparing GCN and GAT encoders within a graph autoencoder. It models each dependency type as a separate relation and combines structural dependencies with semantic signals from source code, such as file paths and code identifiers, and reports improved recovery over node2vec and state-of-the-art SAR methods. Jabeen et al. [34] used heterogeneous GCNs for a closely related task, mapping source-code files to architectural modules.

Learning embeddings that reconstruct graph structure and then partitioning them is a common clustering approach well beyond SAR, used in community detection [35], financial risk analysis [36], and time-series clustering [37]. GAER applies this approach to SAR, combining dependency, organizational, and semantic signals without separating them. Building on this line of work, we fix the model, a GAT encoder trained within a VGAE, and vary only the input graph: the file-level SDG alone, or with folder nodes and containment edges added. This isolates what dependencies recover on their own from what organizational structure contributes.

3 Methodology

This section describes the architecture-recovery pipeline used in the study. The pipeline first constructs a graph from source-code dependencies and, where applicable,

[1]Source code and data are available at github.com/sse-lnu/StructSAR.

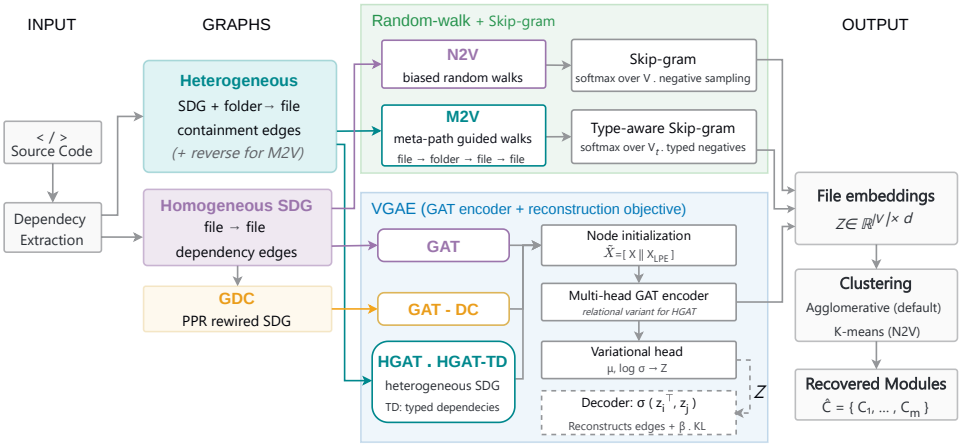


Figure 1: Schematic workflow of studied methods. Static file-level dependencies are extracted from source code and, for the heterogeneous methods, augmented with folder nodes and containment edges. The resulting graph feeds one of the representation learners: biased random walks with Skip-gram (N2V), meta-path-guided walks with type-aware Skip-gram (M2V), or a GAT encoder trained within a VGAE objective on the homogeneous graph (GAT), on a PPR-diffused homogeneous graph (GAT-DC), or on the heterogeneous graph with relation-specific message passing (HGAT), with a variant (HGAT-TD) that keeps typed dependency edges. The resulting file embeddings Z are then clustered to recover the architectural decomposition $\hat{\mathcal{C}}$.

folder containment relations. It then learns an embedding for each source-code file and clusters the file embeddings to recover a modular decomposition.

The graph construction follows the first two research questions. For RQ1, we use a homogeneous file-level SDG, in which nodes are files and edges represent static dependencies. For RQ2, we augment this graph with folder nodes and folder–file containment edges, yielding a heterogeneous graph that represents both dependency structure and project organization. We also include a typed-dependency variant of the heterogeneous model, HGAT-TD, which keeps dependency types separate and tests whether typed dependency relations add value within the heterogeneous setting.

The learning methods vary along two axes: the input graph and the representation-learning strategy. The random-walk variants use a Skip-gram objective, while the GAT-based variants use a graph attention encoder trained with a VGAE reconstruction objective. The learned file embeddings are then clustered to recover a modular decomposition. RQ3 evaluates the runtime and tuning sensitivity of the same representations. Figure 1 summarizes the pipeline.

3.1 Graph Construction

Static dependencies capture structural relationships between source-code files. Files that implement related functionality often depend on one another directly, or are connected through short dependency paths, making the dependency graph a useful signal for architecture recovery [5]. Dependency types may also carry different architectural meanings; for example, call edges can indicate tighter coupling than type-usage edges [38]. Project organization provides a complementary signal: folders often reflect how developers decompose a system into responsibilities and group related functionality [39, 13].

We extract static dependencies using Depends [40]. Although Depends can report finer-grained method- or variable-level information, we aggregate dependencies to the file level because the ground-truth architectures used in this study are labeled per file. We construct two graph forms from the same extracted dependencies: a homogeneous dependency graph used for RQ1, and a heterogeneous graph that augments dependencies with folder containment relations for RQ2.

Homogeneous software graph. The homogeneous graph $G = (V, A)$ contains one node per source-code file, so $V = V_{\texttt{file}}$. A single undirected edge connects two files if at least one static dependency exists between them, regardless of dependency type. Formally, $A_{ij} = 1$ if files i and j are connected by one or more dependencies, and $A_{ij} = 0$ otherwise. All dependency types are therefore merged into one file–file relation.

Heterogeneous software graph. The heterogeneous graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}, \mathcal{R})$ augments the file-level dependency graph with project layout information. The node set $\mathcal{V} = V_{\texttt{file}} \cup V_{\texttt{folder}}$ contains both file nodes and folder nodes, where each directory in the source tree is represented as

one folder node. The node-type function $\phi : \mathcal{V} \to \mathcal{T}$ maps each node to its type, with $\mathcal{T} = \{\texttt{file}, \texttt{folder}\}$.

The edge set $\mathcal{E}$ is typed by a relation function $\psi : \mathcal{E} \to \mathcal{R}$. The relation set $\mathcal{R}$ contains:

1. $\texttt{file} \xrightarrow{r} \texttt{file}$: a static dependency of type r;
2. $\texttt{folder} \xrightarrow{contains} \texttt{file}$: a containment relation from a folder to each file it directly contains;
3. $\texttt{file} \xrightarrow{in} \texttt{folder}$: the reverse containment relation, used by M2V walks.

For M2V and HGAT, all file–file dependency types are merged into a single `depends_on` relation matching the homogeneous graph. HGAT-TD is a typed-dependency variant: it keeps the file–file dependency types as separate relations while using the same folder containment structure. Thus, all methods use the same extracted dependencies; they differ in whether they also use folder organization and whether dependency types are merged or preserved.

3.2 Learning Objective

Given the constructed graph $\mathcal{G}$, our goal is to recover an architectural decomposition from its source-code files using graph representation learning. Here, the input graph is G for homogeneous methods and $\mathcal{G}$ for heterogeneous methods. Concretely, we learn an encoder:

$$f_\theta : \mathcal{G} \longrightarrow Z \in \mathbb{R}^{|V_{\texttt{file}}| \times d}, \tag{1}$$

that maps each source-code file $i \in V_{\texttt{file}}$ to a d-dimensional embedding $z_i \in \mathbb{R}^d$, where θ denotes the learnable parameters of the representation-learning method. The resulting file embeddings Z are then clustered to recover the architectural decomposition:

$$\hat{\mathcal{C}} = \{C_1, \ldots, C_m\},$$

a partition of the source-code files into m disjoint modules. Folder nodes, when present, are used only to support representation learning and are not included in the recovered decomposition.

3.3 Random-Walk Based Methods

Random-walk based methods learn file embeddings from node sequences sampled over the graph. Nodes that appear in similar walk contexts are encouraged to have similar embeddings through a Skip-gram objective. We study two variants: Node2Vec (N2V), which samples biased walks on the homogeneous dependency graph, and MetaPath2Vec (M2V), which guides walks over the heterogeneous graph using a meta-path schema. The following subsections describe the two methods.

3.3.1 Node2Vec (N2V) on Homogeneous Graph

We apply node2vec [41] on the homogeneous dependency graph, following the architecture-recovery setup of NEGAR [5]. Node2vec samples biased random walks on G and feeds the resulting node sequences to a Word2Vec-style Skip-gram model [42]. Walk bias is controlled by parameters p and q, which adjust transition probabilities to favor either staying near the starting node or exploring farther regions, so the model can capture both local connectivity and longer-range structural patterns.

Given the generated sequences, Skip-gram forms training pairs by treating nodes within a fixed context window centered on a target node i as observed context nodes j. Let $z_i \in \mathbb{R}^d$ denote the embedding of node i. The model assigns a probability to observing context node j given i using a softmax over dot-product similarities:

$$p(j \mid i) = \frac{\exp(z_i^\top z_j)}{\sum_{k \in V} \exp(z_i^\top z_k)}$$

Full softmax over all nodes is expensive, so training uses negative sampling. For each observed pair (i, j), we sample η negative nodes $\{n_m\}_{m=1}^{\eta}$ (nodes not observed in the context of i) and update embeddings to increase similarity for (i, j) while decreasing similarity for (i, n_m), by maximizing the following objective:

$$\log \frac{1}{1 + e^{-z_i^\top z_j}} + \sum_{m=1}^{\eta} \log \frac{1}{1 + e^{z_i^\top z_{n_m}}}. \tag{2}$$

3.3.2 MetaPath2Vec (M2V) on Heterogeneous Graph

MetaPath2Vec [43] extends random-walk-based embedding to heterogeneous graphs by guiding the walk with a meta-path schema $\mathcal{P}$ – a fixed sequence of allowed node and edge types. We use the file-folder containment links to guide the walker, so the sampled sequences reflect both folder co-membership and file dependency relations. The meta-path alternates between file and folder nodes as:

$$\texttt{file} \xrightarrow{in} \texttt{folder} \xrightarrow{contains} \texttt{file} \xrightarrow{\texttt{depends_on}} \texttt{file}, \ldots$$

At each step, the walker selects the next node uniformly from neighbors that match the required next type in $\mathcal{P}$, and the schema repeats cyclically. Two files therefore appear in the same walk either because they depend on each other, because they share a folder, or through a longer chain combining both relations.

Given the sampled sequences, nodes c_t of type t appearing within a fixed window around target node i form observed context pairs. MetaPath2Vec models their conditional probability using a type-restricted softmax:

$$p(c_t \mid i) = \frac{\exp(z_i^\top z_{c_t})}{\sum_{k \in V_t} \exp(z_i^\top z_k)}, \tag{3}$$

where V_t contains nodes of the same type as c_t. We optimize the negative-sampling objective in Eq. 2, drawing negative nodes from V_t so that each positive pair is contrasted against same-type candidates.

3.4 Graph Attention Network-Based Methods

Random-walk methods learn from sampled sequences and co-occurrence statistics. GNNs take a different route: they compute node representations through message passing, iteratively updating each node from its neighbors [44]. We use the GAT as the encoder and train it within the VGAE framework [17], which reconstructs the observed dependency edges from the learned embeddings, thereby providing a self-supervised training objective.

However, a shallow L-layer GNN can only aggregate information from nodes within L hops. Increasing the number of layers may capture longer-range dependencies, but it also risks over-smoothing, where node representations become less discriminative [45]. We therefore study structural inputs that are prepared before message passing: Laplacian positional encoding (LPE) [46], which augments node features with graph-position information, and Graph Diffusion Convolution (GDC) [47], which rewires the input graph so that diffusion-close nodes become direct neighbors.

3.4.1 Node Initialization

Each variant starts from a trainable embedding table learned with the rest of the model. For the homogeneous variants, a single table $X \in \mathbb{R}^{|V_{\texttt{file}}| \times d_0}$ covers all file nodes, with row x_i giving the initial representation of file i; the heterogeneous variants maintain one table per node type, $X^{(t)} \in \mathbb{R}^{|V_t| \times d_0}$ for $t \in \{\texttt{file}, \texttt{folder}\}$.

A randomly initialized embedding carries no information about where a file sits in the dependency graph, so two files in opposite regions of the system look identical to the encoder before message passing begins. Augmenting GAT inputs with positional information is known to help on graphs where neighbors are not necessarily similar [48]. We therefore inject position information into the initial features via Laplacian positional encoding (LPE) [46], taking the first ζ non-trivial eigenvectors of the symmetric normalized graph Laplacian of the file–file graph as a feature matrix $X_{\mathrm{LPE}} \in \mathbb{R}^{|V_{\texttt{file}}| \times \zeta}$. The encoder input is the row-wise concatenation of the trainable embedding and the LPE features:

$$\tilde{X} = [\, X \,\|\, X_{\mathrm{LPE}} \,] \in \mathbb{R}^{|V_{\texttt{file}}| \times (d_0 + \zeta)}, \tag{4}$$

where $\|$ denotes concatenation. Hence, each file enters the encoder with a position-aware feature $\tilde{x}_i$. For HGAT and HGAT-TD, LPE is computed on the file–file subgraph and applied only to file nodes; folder nodes use their trainable embedding alone.

3.4.2 Encoder

We use GAT [16] to refine the initial representations through message passing on the input graph, assigning a learned importance weight to each neighbor before aggregation. Let $\mathcal{N}(i)$ denote the one-hop neighborhood of node i including a self-loop, and let $h_i^{(\ell)}$ be its representation after ℓ GAT layers, with $h_i^{(0)} = \tilde{x}_i$.

For each neighbor $j \in \mathcal{N}(i)$, GAT computes an unnormalized attention score:

$$e_{ij}^{(\ell)} = \mathrm{LeakyReLU}\Big(a^{(\ell)\top} [\, W^{(\ell)} h_i^{(\ell)} \,\|\, W^{(\ell)} h_j^{(\ell)} \,] \Big), \tag{5}$$

where $\mathrm{LeakyReLU}(\cdot)$ is the leaky rectified linear unit [49], $W^{(\ell)}$ is a learnable linear transformation, and $a^{(\ell)}$ is a learnable attention vector. The final attention coefficient $\alpha_{ij}^{(\ell)}$ is obtained by softmax normalization of $e_{ij}^{(\ell)}$ over all neighbors in $\mathcal{N}(i)$.

For the homogeneous variants (GAT, GAT-DC), each layer updates a node's representation as a weighted aggregation of transformed neighbor features:

$$h_i^{(\ell+1)} = \mathrm{ELU}\left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij}^{(\ell)} W^{(\ell)} h_j^{(\ell)} \right), \tag{6}$$

where $\mathrm{ELU}(\cdot)$ is the exponential linear unit [50]. We use multi-head attention and average across heads; stacking L layers yields the embedding matrix $H^{(L)} \in \mathbb{R}^{|V| \times d}$.

The heterogeneous variants (HGAT, HGAT-TD) instead perform relation-specific message passing, aggregating over the relations in $\mathcal{R}$:

$$h_i^{(\ell+1)} = \mathrm{ELU}\left(\sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \alpha_{ij}^{(r,\ell)} W_r^{(\ell)} h_j^{(\ell)} \right), \tag{7}$$

where $\mathcal{N}_r(i)$ is the neighborhood of i under relation r, $W_r^{(\ell)}$ is a relation-specific weight matrix, and $\alpha_{ij}^{(r,\ell)}$ are attention weights normalized over $\mathcal{N}_r(i)$.

3.4.3 Structural Augmentation with Graph Diffusion Convolution

Files in the same architectural module are often close in the dependency graph but not directly connected [5], so with only one or two GAT layers, they never exchange messages. GDC [47] addresses this by replacing the raw adjacency A with a diffused adjacency $\tilde{A}$ that links diffusion-close nodes directly and assigns each new edge an importance weight.

We compute $\tilde{A}$ using Personalized PageRank (PPR) with restart probability $\alpha \in (0, 1)$, connecting each node i to the top-k nodes a walker restarting at i visits most frequently and retaining the visit probabilities as edge weights w_{ij}. Files connected via many short paths thus receive higher weight than those reached via long or cross-cluster routes. We pass w to the encoder as a scalar edge attribute that biases the attention logits of Eq. 5:

$$e_{ij}^{(\ell)} = \mathrm{LeakyReLU}\Big(a^{(\ell)\top} \big[W^{(\ell)} h_i^{(\ell)} \,\|\, W^{(\ell)} h_j^{(\ell)} \,\|\, w_{ij}\, b^{(\ell)} \big] \Big), \tag{8}$$

where $b^{(\ell)}$ is a learnable projection of the scalar edge weight, so diffusion-close neighbors receive a stronger prior in the attention distribution. The reconstruction objective remains on the original edges E, while GDC changes only the message flow within the encoder.

3.4.4 Variational Projection and Reconstruction Decoder

The final GAT layer produces file embeddings $H_{\texttt{file}}^{(L)}$; for HGAT, folder embeddings shape file representations during message passing but are not reconstructed. In all GAT-based variants, the decoder reconstructs only file–file dependency edges, so the loss in Eq. 11 remains the same. Two linear projection heads map these file embeddings to the mean (μ) and log-standard deviation ($\log\sigma$) of a Gaussian posterior for each file node i. During training, the model samples the latent embedding z_i by adding Gaussian ϵ noise scaled by σ_i:

$$z_i = \mu_i + \sigma_i \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

At inference time, we remove this randomness and use the posterior mean μ_i as the final file embedding.

A dot-product decoder reconstructs the file-level dependency graph by estimating the probability of a dependency edge between files i and j:

$$\hat{A}_{ij} = \text{sigmoid}(z_i^\top z_j). \tag{9}$$

Since observed dependencies represent only a small subset of all possible file pairs, we use negative sampling. For each set of positive dependency edges E, we sample absent file pairs S^- with $|S^-| = \rho|E|$, where ρ is the negative sampling ratio. The reconstruction loss is binary cross-entropy over positive and sampled negative pairs:

$$\mathcal{L}_{\text{recon}} = -\sum_{(i,j)\in E} \log \hat{A}_{ij} - \sum_{(i,j)\in S^-} \log\big(1 - \hat{A}_{ij}\big) \tag{10}$$

The full VGAE objective combines reconstruction with a KL-divergence regularizer that keeps the learned posterior close to a standard normal prior:

$$\mathcal{L}_{\text{VGAE}} = \mathcal{L}_{\text{recon}} + \frac{\beta}{|V_{\texttt{file}}|} \sum_{i\in V_{\texttt{file}}} \text{KL}(q(z_i) \,\|\, \mathcal{N}(0, I)), \tag{11}$$

where β is a scalar that controls the strength of the regularization. All GAT-based variants are trained with this objective and optimized with Adam [51].

3.5 Clustering and Architecture Recovery

Architectural granularity is a design choice rather than a single objectively correct answer, since systems can be organized at different levels of modular abstraction [4]. We therefore recover architectures by clustering the learned file embeddings into a flat m partition, where each source file is assigned to exactly one recovered module. The value of m can either be specified directly or selected from a candidate range $\mathcal{K}$ using the silhouette score [52], which balances intra-cluster cohesion and inter-cluster separation.

We apply agglomerative clustering with Ward linkage to all methods except N2V. For N2V, we follow the original setup of NEGAR [5] and use K-means clustering on the node2vec embeddings.

4 Experiments

We evaluate all methods studied on a collection of open-source software systems with known ground-truth (GT) labels. This section describes the benchmark systems, the experimental configurations for all techniques, and the evaluation metrics used to assess performance.

4.1 Benchmark Systems

We evaluate our approach on 11 open-source systems that are widely studied in software architecture recovery [5, 13, 53]. We use published ground-truth (GT) architectures to ensure direct comparability with prior work, as producing reliable mappings is resource-intensive and does not scale well [39]. Table 1 summarizes properties and the number of GT modules $|\mathcal{C}|$ of benchmark systems.

Table 1: Properties of the software systems. Static dependencies between files yield edges, and the average degree is computed as $\langle k\rangle = 2|E|/|V_{\texttt{file}}|$.

	System†	Version	$\lvert\mathcal{C}\rvert$	$\lvert V_{\texttt{file}}\rvert$	$\lvert V_{\texttt{folder}}\rvert$	|Edges|	$\langle k\rangle$
Java	A.UML	r13713	14	766	58	3,754	9.8
	AS4	4	57	583	115	1,826	6.3
	C.Img	v1.0a2	21	329	41	1,565	9.5
	Hadoop	0.19.0	63	591	55	3,648	12.3
	JabRef	v3.7	6	1,180	127	6,126	10.4
	Lucene	r1075001	7	1,054	188	6,424	12.2
	T.Mates	v5.11	15	778	42	6,721	17.3
C/C++	Bash	4.2	13	292	11	1,010	6.9
	Chrome	svn-171054	69	18,343	999	145,492	15.9
	HDC	46ff87	11	207	87	543	5.2
	HDF	0e196f	9	153	26	535	7.0

†Datasets are available at `https://github.com/sse-lnu/StructSAR`.

For AS4, Hadoop, Bash, and Chromium, we use the GT labels curated by Garcia et al. [39, 3], produced through a certification process in which researchers derived an initial architectural decomposition from documentation and source code and then refined it through validation with system experts. Chromium was developed through long-term collaboration with the project's developers over roughly two years.

The GT labels for HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [13], distributed through the SARIF replication package on GitHub[2].

The other five Java systems, ArgoUML (A.UML), Lucene, Common-Imaging (C.Img), JabRef, and TeamMates (T.Mates), were studied by Olsson et al. [53] for code-to-architecture mapping and are available through the s4rdm3x repository[3]. The GT mappings for JabRef and TeamMates were produced by system experts at the Software Architecture Erosion and Architectural Consistency Workshop (SAEroCon)[4] in 2016 and 2017. The

[2] https://github.com/anonymous2f4a9d/SARIF_FSE23
[3] https://github.com/h0bb3/s4rdm3x
[4] https://github.com/sebastianherold/SAEroConRepo

C.Img mapping was produced by its developers as a benchmark system for Olsson's study.

4.2 Classical Dependency-only Baselines

For context, we also evaluate four classic SAR methods that only use the static dependency-graph as input: ACDC [8], FCA [29], LIMBO [7], and WCA [10] variants(WCA_{UE} and WCA_{UENM}). We use the authors' original implementations for FCA, and the ARCADE framework for ACDC, LIMBO, and WCA [54]. The runner scripts are available in a replication package on GitHub[5]. ACDC and FCA automatically determine the number of clusters, whereas LIMBO and WCA stop at the ground-truth module count. For all methods, we use the same dependency graph, extracted using Depends [40], to ensure a consistent comparison.

4.3 Experimental Configuration

We use a single configuration per method across all benchmark systems, with per-system overrides only for GAT-DC (PPR top-k and layer count) and for Chrome. The selection rationale and sensitivity analysis are discussed in Section 5.3.

Node2vec. We adopt the Node2vec search space reported by Chen et al. (Table II in [5]). We ran the search on three benchmark systems also used in their study (AS4, Bash, and Hadoop) and selected one configuration that performed consistently across them: 128 embedding dimensions, walk length 30, context size 15, 100 walks per node, $p = q = 1$, five negative samples, and five training epochs. This configuration applies to all systems. For Chrome, we use the setting explicitly published by Chen et al. (one epoch and one negative sample). Unlike Chen et al., who studied each dependency type separately, we merge all types into a single edge for N2V, consistent with the common-input design across all methods; this reproduces recovery comparable to their reported results.

MetaPath2vec. We use an embedding dimension of 64, walk length 15, context size 5, and 40 walks per node. Skip-gram training uses SparseAdam with one negative sample, a learning rate of 0.05, a batch size of 32, and 20 training epochs.

Variational graph autoencoders. All GAT-based variants initialize each node type with a 128-dimensional learnable identifier embedding. For file-level nodes, this embedding is concatenated with a 16-dimensional LPE vector. Unless stated otherwise, the encoder uses 256 hidden channels, two attention heads, and is trained for 100 epochs with Adam at a learning rate of 10^{-3}. The variant-specific parameters and Chrome-specific scalability overrides are summarized in Table 2.

Chrome is substantially larger than the other benchmark systems, with about 18k file nodes. Full-batch training degraded the recovery scores of the homogeneous GAT variants (GAT and GAT-DC) in preliminary experiments, so we use link-level neighborhood sampling and reduced training settings for Chrome, as listed in Table 2. HGAT and HGAT-TD remained stable during full-batch training on the heterogeneous graph and were trained with the default capacity, including one additional GAT layer. These settings were selected for stable and competitive recovery rather than exhaustive tuning; their scalability implications are discussed in Section 5.3.

For GAT-DC, PPR diffusion expands the effective neighborhood, so larger systems use one GAT layer while smaller systems use two. The layer rule and per-system top-k values are given in Table 2 (footnotes † and ‡); the top-k sensitivity analysis is discussed in Section 5.3.

Table 2: Hyperparameters for the GAT-based variants. Values are shared within each block unless overridden below. Chrome overrides are listed only where they differ from the Default; a dash (—) means the Default applies.

	Hyperparameter	Default	Chrome
Homogen. shared	Hidden channels	256	128
	Attention heads	2	1
	β (KL)	0.1	—
	ρ (neg. ratio)	5	1
	Epochs	100	50
GAT	GAT layers	2	1
GAT-DC	GAT layers	1 or 2†	1
	PPR top-k	{16, 32, 64, 128}‡	32
	α (restart)	0.15	—
HGAT/ HGAT-TD	GAT layers	1	2
	β (KL)	0.2	—
	ρ (neg. ratio)	2	—
	Epochs	100	—

† GAT layers: 1 layer if $|V_{\text{file}}| > 500$ *and* $|C| > 10$; 2 otherwise (see Table 1).
‡ PPR top-k: 16 for HDC/HDF, 32 for C.Img, 128 for Lucene/T.Mates, 64 for others.

We run the experiments on an Ubuntu 24.04.4 LTS server with two Intel Xeon Platinum 8468 CPUs (96 physical cores), 1 TiB RAM, and eight NVIDIA H100 80GB GPUs. Each reported run uses one H100 GPU, and threaded methods are capped at 24 CPU threads. GAT-based methods mainly benefit from the GPU, while node2vec benefits most from CPU parallelism during random-walk generation and embedding training. These settings were selected from preliminary runs to give stable runtimes without oversubscribing shared resources.

4.4 Performance Evaluation

Multiple reasonable architectural decompositions may exist for the same software system, since experts can disagree on design decisions [39]. As a result, a recovered decomposition may differ from the particular GT architecture used for evaluation. To avoid relying on any single criterion, we report a set of complementary metrics using the standard definitions and computation procedures as presented in [3].

[5] https://github.com/sse-lnu/sar-baselines

MoJoFM [55] is a primary evaluation metric in SAR. It measures how many *move* and *join* operations are needed to transform the recovered architecture A_1 into the GT architecture A_2 at the cluster level:

$$\text{MoJoFM}(A_1, A_2) = \left(1 - \frac{mno(A_1, A_2)}{\max(mno(\forall A_1, A_2))}\right) \times 100, \quad (12)$$

where *mno* is the minimum edit cost, and the denominator normalizes by its maximum possible value. However, its *join* operations carry a low cost, so a recovered partition with many small clusters can still score high if those clusters align with GT modules [56, 4].

Architecture-to-Architecture (A2A) [57] extends the edit model to also include adding, removing, and moving entities or clusters:

$$\text{A2A}(A_1, A_2) = \left(1 - \frac{mto(A_1, A_2)}{aco(A_1) + aco(A_2)}\right) \times 100, \quad (13)$$

where mto is the edit distance between the two architectures and aco is the cost of building each from scratch. A2A can return high similarity even when two architectures differ substantially in structure [3].

Cluster-to-Cluster Coverage (C2C_{cvg}) [58] counts the fraction of GT clusters that are sufficiently matched by some recovered cluster:

$$C2C_{\text{cvg}}(A_1, A_2) = \frac{|\text{simC}(A_1, A_2)|}{|A_2|} \times 100, \quad (14)$$

where $\text{simC}(A_1, A_2)$ is the set of GT clusters whose overlap with some recovered cluster exceeds a coverage threshold (th_{cvg}).

Normalized TurboMQ (TMQ_{norm}) [56] is independent of GT and measures modular quality from the dependency graph by rewarding intra-cluster cohesion and penalizing inter-cluster coupling. For k recovered clusters with intra-cluster edge weight μ_i for cluster i and inter-cluster weight ε_{ij} from cluster i to cluster j:

$$\text{TurboMQ} = \sum_{i=1}^{k} \frac{\mu_i}{\mu_i + 0.5 \times \sum_{j \neq i} (\varepsilon_{ij} + \varepsilon_{ji})}. \quad (15)$$

The TMQ_{norm} is obtained by dividing the TurboMQ by the number of recovered clusters.

In addition to the SAR-specific metrics, we also report the Adjusted Rand Index (ARI) [59], a standard clustering metric that measures agreement between the recovered and GT partitions corrected for chance, in $[-1, 1]$, with 1 indicating perfect agreement.

5 Results and Analysis

This section presents the results of our study and addresses the research questions. Our primary evaluation uses Exact-k clustering, where the number of recovered clusters is fixed to the number of GT modules. We also report *Search* results, in which the number of clusters is automatically selected within the range $\mathcal{K} = [5, 20]$, with wider per-system ranges for C.Img ($[10, 25]$), AS4 and Hadoop ($[40, 70]$), and Chrome ($[50, 75]$); these results appear in the lower block of Tables 3 and 5. Since all methods are stochastic, we run each dataset–method configuration 100 times with different seeds and report the median scores. Run-to-run variability is shown with boxplots and summarized using IQR (interquartile range).

For pairwise method comparisons, we test significance with the Mann–Whitney U test [60], which suits independent, non-Gaussian samples, and report Cliff's δ [61] as the effect size: negligible ($|\delta| < 0.147$), small ($0.147 \leq |\delta| < 0.33$), medium ($0.33 \leq |\delta| < 0.474$), and large ($|\delta| \geq 0.474$).

The metrics capture complementary aspects of recovery, so we interpret MoJoFM, A2A, C2C_{cvg}, and ARI together, with TMQ_{norm} providing a GT-independent view of structural modularity. Throughout the results, C2C_{cvg} uses a fixed coverage threshold of $th_{\text{cvg}} = 0.5$, corresponding to majority matching[56], and can therefore be more volatile near the threshold. The metrics also respond differently to cluster granularity: MoJoFM tolerates over-fragmentation, whereas ARI and C2C_{cvg} penalize it. A2A is the least discriminative of the GT-alignment metrics. We therefore rely mainly on MoJoFM and ARI to distinguish between methods.

5.1 RQ1 – Effectiveness of Dependency-only Graph Representations for Architecture Recovery

RQ1 examines how effectively software architecture can be recovered from dependency structure alone, before adding organizational or semantic information. This establishes the baseline that RQ2 builds on. We compare three learned representations on the homogeneous file–file dependency graph, N2V, GAT, and GAT-DC, together with four classic methods that rely solely on the dependency graph: ACDC, FCA, LIMBO, and variants of WCA.

All homogeneous methods use a single collapsed `depends_on` edge. In preliminary experiments, modeling dependency types separately in the GAT encoder did not improve recovery, so we revisit typed edges only in RQ2 (HGAT-TD). Table 3 reports the median GT-alignment scores for Java and C/C++ systems over 100 runs per (system, method) configuration, with the classic methods in the top block, and Table 4 reports the pairwise significance tests for the learned methods (median differences and Cliff's δ). Overall, N2V and GAT-DC achieve comparable median performance, whereas GAT is generally weaker.

No single representation dominates across all systems and metrics. Among the learned methods, GAT-DC has a slight edge over MoJoFM and ARI; N2V remains competitive on C2C_{cvg} and A2A; and GAT is generally weaker. A2A varies little across the learned methods, so MoJoFM, C2C_{cvg}, and ARI drive the discrimination between them.

ACDC and FCA decide the number of clusters themselves, while the other take it as input. ACDC achieves relatively high MoJoFM across several systems, but it can return far more clusters than a developer would consider

Table 3: Recovered-vs-GT similarity on the homogeneous file–file SDG. MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. In the top block, ACDC and FCA select their own cluster count while all other methods use Exact-k (the GT module count); the lower block selects the count automatically (Search). Learned-method scores are medians across 100 runs, and the best top-block score for each (system, metric) pair is highlighted.

		Java																								**C/C++**											
		A.UML				AS4				C.Img				Hadoop				JabRef				Lucene				T.Mates				Bash				Chrome				HDC				HDF			
	Method	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
Exact-k	ACDC	72	82	3	22	74	85	41	17	82	89	34	41	42	82	9	11	76	81	1	20	68	76	0	1	48	82	4	2	59	81	10	20	68	80	2	13	57	87	12	24	47	85	6	26
	FCA	59	78	1	11	61	80	18	6	63	80	10	11	44	81	13	8	62	73	0	2	65	75	0	8	36	74	0	0	52	78	6	10	53	73	0	2	63	76	6	12	50	81	6	22
	LIMBO	59	86	21	15	66	86	37	19	59	87	19	30	48	83	11	13	62	87	17	0	50	86	14	12	63	85	20	17	58	86	31	17	51	87	7	21	55	86	9	20	48	87	22	21
	WCA_{cut}	55	86	14	24	71	85	30	16	73	86	24	29	51	84	5	21	60	85	0	−10	25	82	0	0	65	85	7	21	41	86	23	12	46	83	0	21	64	90	27	42	48	88	22	30
	WCA_{inter}	52	87	21	24	70	85	33	17	72	86	29	26	51	84	11	20	60	85	0	−10	25	82	0	0	53	83	0	14	41	86	23	12	46	83	3	20	62	90	45	40	51	88	22	31
	N2V	75	90	43	40	72	86	40	19	80	90	43	47	52	84	11	17	72	86	17	7	73	91	57	52	66	86	7	27	63	89	31	33	74	84	30	17	71	89	18	38	55	88	33	32
	GAT	55	87	29	26	72	87	44	29	76	90	43	48	50	84	8	17	72	86	17	5	62	89	43	36	46	84	0	11	61	89	31	32	69	83	21	14	62	88	18	29	46	87	22	23
	GAT-DC	74	89	43	40	70	88	28	29	77	90	48	48	53	85	8	26	78	86	17	12	74	92	57	58	66	84	7	23	64	88	23	35	71	83	25	14	73	89	27	39	56	89	44	35
Search	N2V	77	89	30	38	72	86	40	17	79	90	50	50	51	84	11	17	75	85	7	9	73	90	33	47	67	86	8	26	58	89	60	31	72	84	36	20	72	88	17	35	58	86	18	32
	GAT	59	88	25	28	71	88	49	38	77	90	38	48	48	83	10	17	72	85	14	3	74	88	15	40	46	84	0	9	61	89	31	33	69	83	21	14	68	86	10	31	44	86	20	22
	GAT-DC	74	89	38	39	70	87	29	28	76	91	57	58	53	85	9	27	73	86	20	3	72	93	80	57	63	87	14	28	61	90	50	37	71	83	23	14	76	87	12	36	57	89	30	35

a single architecture. For example, 1,061 clusters, compared to a GT of 69, and across systems, it produces 1.7× to 30× the GT count. Because MoJoFM tolerates fine partitions, but ARI and $C2C_{cvg}$ compare partitions at a fixed granularity and drop when the count is far from the GT. This is why ACDC and FCA trail on ARI and $C2C_{cvg}$, where N2V and GAT-DC lead by about +15 to +18 ARI and +10 to +24 $C2C_{cvg}$ on average, and their lower TMQ_{norm} in Table 7 is consistent with this. The gap is not only about cluster count. LIMBO and the WCA variants use the GT count, so granularity is matched, yet the learned methods still lead them by about +12 MoJoFM and +16 ARI. HDC is the clearest exception, where the WCA variants achieve a higher ARI by a few points. Since the cluster count has such a strong effect, we treat it as a modeling choice and recover a flat partition at a chosen k, one that a developer can reason about.

The Search block broadly follows the same pattern as Exact-k: median MoJoFM stays close on most systems, so automatic k selection does not substantially change the overall ranking. The largest differences appear in $C2C_{cvg}$, whose values can shift sharply in either direction, because partitions with a different cluster count cross the fixed $C2C_{cvg}$ threshold in different ways. The Search results, therefore, mainly show $C2C_{cvg}$'s sensitivity to cluster count rather than a consistent change in recovery.

Why do GAT-DC and N2V outperform GAT? The pattern in Table 4 is consistent across the four metrics. GAT trails N2V on average, most clearly on MoJoFM (−7.4) and ARI (−6.0), while GAT-DC recovers most of that gap. Against N2V, GAT-DC shows small average gains on MoJoFM and ARI, ties on A2A, and is slightly behind on $C2C_{cvg}$. So GAT itself is not the main limitation; the limitation is applying a shallow GAT to the raw dependency graph. GAT aggregates only two-hop neighborhoods and can miss the long-range structure that separates modules, since files in the same module are often linked through shared neighbors or dependency chains rather than direct edges. Both random walks and diffusion-based rewiring expand the structural context available during representation learning, so the key factor is not only the encoder but also its ability to capture long-range dependency structure.

What limits recovery from the SDG alone? Two factors beyond the encoder bound how well methods built on dependencies alone align with the GT: the cohesion and the granularity of the GT architecture. Table 7 shows that systems with cohesive GT decompositions, roughly in the low-40s to high-50s TMQ_{norm} range (A.UML, Lucene, Chrome, AS4, and HDC), align better under these methods. Hadoop and T.Mates are the opposite case: their GT TMQ_{norm} values are much lower, around 12 to 19, so cross-module dependencies are nearly as strong as intra-module ones. These methods stay limited here, with MoJoFM below about 53 on Hadoop and 66 on T.Mates and modest ARI. Recovery is therefore bounded by how well the GT architecture is reflected in the dependency structure.

The second factor is the number of modules in the GT decomposition (granularity). Fine-grained architectures have small modules, so local context and a shallow model suffice. Coarse-grained architectures have large modules spanning many files, so the encoder needs a wider reach to pull them into a tight embedding. For GAT-DC, this reach is set by the GAT layer count and the PPR top-k; for N2V, by the walk and context lengths. This is why these settings track system size and granularity (Table 2), which we analyze in Section 5.3. Recovery from dependencies alone is thus strongest when the GT decomposition is cohesive in the dependency graph *and* the encoder's reach matches the target granularity.

Variability. Figure 2 shows the boxplots of stochastic methods across runs for systems where the variability is most pronounced. GAT-DC has the smallest average spread across the four metrics, making it the most consistent representation. This is clearest on A.UML, JabRef, Lucene, and T.Mates, especially JabRef, where GAT-DC stays tight while N2V ranges from 0 to 17 at a comparable median. $C2C_{cvg}$ varies the most because its

Table 4: Pairwise comparison of methods across 100 runs per (system, metric) cell. Each entry reports the median difference between the two methods, with Cliff's delta effect-size class in parentheses (L: large, M: medium, S: small, N: negligible). Positive values favor the first method; negative values favor the second. All differences are statistically significant (p-value < 0.05) unless marked with a †, and cells in which the second method is significantly better than the first are shaded.

	GAT − N2V				**GAT-GDC − N2V**				**GAT-GDC − GAT**			
Dataset	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI
A.UML	−19.9(L)	−2.7(L)	−14.3(L)	−13.9(L)	−1.3(N)†	−0.9(L)	0.0(S)	−0.7(S)†	+18.7(L)	+1.7(L)	+14.3(L)	+13.0(L)
AS4	−0.7(M)	+0.8(L)	+3.5(M)	+10.6(L)	−2.1(L)	+1.7(L)	−12.3(L)	+9.7(L)	−1.4(L)	+0.8(L)	−15.8(L)	−0.9(S)
C.Img	−4.5(L)	+0.6(S)	0.0(N)†	+0.7(S)	−3.0(L)	+0.7(M)	+4.8(M)	+0.6(N)†	+1.4(M)	+0.1(N)†	+4.8(M)	0.0(S)
Hadoop	−1.7(L)	−0.1(M)	−3.4(L)	+0.2(N)†	+1.0(L)	+1.3(L)	−3.4(L)	+9.0(L)	+2.6(L)	+1.4(L)	0.0(S)	+8.9(L)
JabRef	+0.5(N)†	−0.3(S)	0.0(S)	+0.2(S)	+5.1(L)	−0.1(S)†	0.0(S)	+4.8(L)	+4.6(L)	+0.3(S)	0.0(N)†	+4.7(L)
Lucene	−10.8(L)	−2.0(L)	−14.3(L)	−16.2(L)	+1.2(L)	+0.5(S)	0.0(S)	+6.2(L)	+12.1(L)	+2.5(L)	+14.3(L)	+22.5(L)
T.Mates	−20.8(L)	−1.9(L)	−6.7(L)	−16.1(L)	−0.1(N)†	−1.4(L)	0.0(S)	−4.2(L)	+20.8(L)	+0.5(S)	+6.7(L)	+12.0(L)
Bash	−1.2(S)	+0.8(M)	0.0(S)	−0.5(N)†	+1.6(L)	−0.7(S)	−7.7(L)	+1.9(M)	+3.2(L)	−1.5(L)	−7.7(L)	+3.0(L)
Chrome	−4.4(L)	−0.8(L)	−9.4(L)	−2.7(L)	−3.1(L)	−0.5(L)	−5.8(L)	−2.7(L)	+1.3(L)	+0.3(L)	+3.6(L)	+0.1(N)†
HDC	−9.0(L)	−0.8(L)	0.0(S)	−9.1(L)	+2.0(M)	−0.1(N)†	+9.1(S)	+1.5(S)†	+11.1(L)	+0.7(L)	+9.0(M)	+10.6(L)
HDF	−9.0(L)	−1.5(L)	−11.1(L)	−8.9(L)	+1.0(M)	+0.7(L)	+11.1(M)	+3.3(L)	+10.0(L)	+2.2(L)	+22.2(L)	+12.2(L)
Average	−7.4	−0.7	−2.5	−6.0	+0.2	+0.1	−0.4	+2.7	+7.7	+0.9	+4.7	+7.8

fixed threshold lets small file movements flip a GT module between matched and unmatched, so much of the spread reflects threshold crossings rather than real differences in recovery. Under Search, automatic k selection adds a second source of randomness and affects the methods unevenly: GAT-DC stays stable in ARI (average IQR 2.2 to 2.4), whereas N2V widens from 4.0 to 5.7. This increase is concentrated in coarse-grained systems, where the silhouette score shows no sharp peak and the selected k varies across runs: the N2V ARI IQR rises from 8.7 to 19.1 on JabRef and from 4.7 to 18.2 on Lucene. Here, the instability for N2V likely arises from both embedding noise and uncertainty in the number of clusters. The spread is also small relative to the gap over the baselines: where N2V or GAT-DC lead ACDC on ARI, even their low runs stay ahead, except on JabRef where ACDC leads outright.

Answer to RQ1. Representations built on dependencies alone provide a useful but limited basis for SAR. N2V and GAT-DC achieve comparable performance, with GAT-DC slightly stronger on MoJoFM and ARI and the most stable, while GAT alone is generally weaker. This suggests that two-hop message passing on the raw dependency graph is often too local, and that both random-walk context and diffusion-based rewiring help expose longer-range dependency structure. However, recovery remains bounded by how clearly the GT architecture is reflected in the dependency graph, and by how well the model's use of dependency context matches the target granularity. The lead of GAT-DC and N2V over deterministic baselines on fragmentation-sensitive metrics (ARI and C2C$_{cvg}$) holds across runs.

5.2 RQ2 – Impact of Project Organization Signal on Architecture Recovery

In RQ2, we examine whether folder organization adds architectural information beyond the cohesion and coupling signals in the SDG. We model this using folder nodes and `folder-file` containment edges, and compare M2V, HGAT, and HGAT-TD against the homogeneous baselines N2V and GAT-DC from RQ1 (Table 5). We ask whether combining dependencies with folder co-membership improves alignment with the human-defined GT mapping, and when this signal is most useful. We also report GAER [4] as a reference point, since it additionally incorporates code semantics[6].

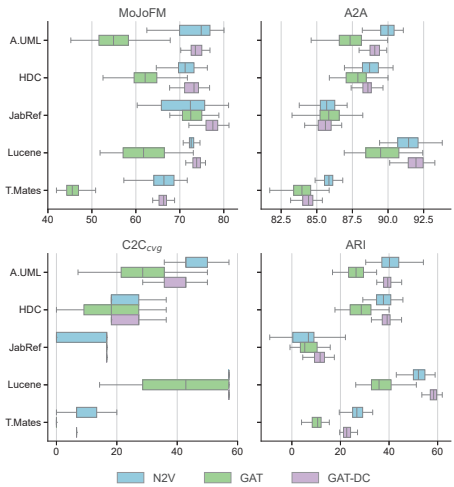


Figure 2: Score distributions over 100 runs for A.UML, HDC, JabRef, Lucene, and T.Mates.

Across both Java and C/C++ systems, the heteroge-

[6]For datasets shared in the GAER paper [4], we report the published numbers. For A.UML, C.Img, and Lucene, we ran GAER using the publicly released implementation [62].

Table 5: Recovered-vs-GT similarity on the heterogeneous graph (file–file SDG plus folder nodes and containment edges), median over 100 runs. Metrics MoJoFM (M), A2A (A), C2C$_{cvg}$ (C), and ARI (I) are scaled by 100. The upper block fixes the cluster count to the GT module count (Exact-k); the lower block selects it automatically (Search). The strongest dependency-only methods (N2V and GAT-DC) from RQ1 are repeated in the top block. GAER is shown in italics as a reference point, since it additionally uses source-code semantics. The best Exact-k result per (system, metric) is highlighted.

		Java																								**C/C++**											
		A.UML				AS4				C.Img				Hadoop				JabRef				Lucene				T.Mates				Bash				Chrome				HDC				HDF			
	Method	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
	GAER	*94*	*97*	*86*	*83*	*80*	*88*	*42*	*27*	*93*	*91*	*48*	*49*	*69*	*86*	*30*	*28*	*91*	*90*	*33*	*34*	*92*	*94*	*68*	*58*	*94*	*90*	*47*	*51*	*91*	*93*	*64*	*58*	*91*	*87*	*33*	*30*	*89*	*95*	*73*	*71*	*90*	*97*	*78*	*76*
Exact-k	N2V	75	90	43	40	72	86	40	19	80	90	43	47	52	84	11	17	72	86	17	7	73	91	57	52	66	86	7	27	63	89	31	33	74	84	30	17	71	89	18	38	55	88	33	32
	GAT-DC	74	89	43	40	70	88	28	29	77	90	48	48	53	85	8	26	78	86	17	12	74	92	57	58	66	84	7	23	64	88	23	35	71	83	25	14	73	89	27	39	56	89	44	35
	M2V	89	93	57	60	75	88	40	26	87	92	50	55	62	87	21	33	69	86	17	1	74	91	43	49	81	89	33	41	78	95	50	66	74	88	32	37	72	89	27	41	67	92	56	57
	HGAT	91	95	64	68	77	88	44	27	84	92	52	52	61	85	21	24	75	85	17	11	71	90	43	45	80	91	47	50	93	95	69	71	81	87	23	33	74	89	36	42	73	93	67	65
	HGAT-TD	89	94	57	62	78	88	44	26	82	91	48	50	64	86	23	29	75	85	17	10	71	90	43	45	79	90	40	46	93	96	77	75	83	87	26	34	74	89	36	42	74	93	67	60
Search	M2V	90	91	40	48	77	87	41	25	89	92	42	53	63	87	23	33	79	82	0	10	90	88	20	47	85	89	35	42	79	95	40	66	73	89	37	40	82	86	20	35	78	93	35	68
	HGAT	93	93	50	62	80	88	44	25	86	92	46	51	61	86	27	33	80	82	0	11	87	88	20	45	85	91	45	49	92	94	60	68	82	87	24	33	82	86	20	34	92	94	40	78
	HGAT-TD	92	93	50	59	80	88	44	25	85	91	46	50	65	87	27	36	80	82	0	11	87	88	20	46	86	91	40	48	92	96	67	71	84	87	25	33	82	85	20	33	88	92	35	71

neous methods outperform N2V and GAT-DC on most (system, metric) cells, with the largest gains where the dependency signal alone fell short. On Hadoop, MoJoFM rises from at most 53 (GAT-DC) to 62 (M2V) and 64 (HGAT-TD), and ARI from 26 to 33. On T.Mates, MoJoFM increases from 66 to 81 (M2V) and 80 (HGAT), and C2C$_{cvg}$ from 7 to 33 (M2V) and 47 (HGAT). The C/C++ results match: Bash improves from 63–64 to 93 under HGAT and HGAT-TD, and HDF from 55–56 to the low-70s. Folder containment yields the largest gains on systems with low TMQ$_{norm}$ on GT (Hadoop, T.Mates) or sparse dependency graphs relative to module count (Bash, HDF), suggesting that folder organization carries architectural information not present in the SDG alone.

Under Search, the heterogeneous methods behave differently from the homogeneous ones. When silhouette selection chooses more clusters than the GT, GT modules are split into finer parts, increasing MoJoFM and ARI but lowering C2C$_{cvg}$. HDF is the clearest example: although the GT has nine modules, silhouette selection always chooses 20 clusters, so HGAT's MoJoFM rises from 73 to 92, ARI also improves, and C2C$_{cvg}$ drops sharply because GT modules are distributed across multiple recovered clusters. HDC and Lucene show the same pattern. These gains are therefore real but metric-dependent, and remain meaningful because the cluster ranges are small: the finer partitions still provide a useful architectural view and can be merged if a coarser interpretation is preferred.

What does folder structure add, and where does it stop helping? Folder nodes shift the learning signal away from pure dependency graph toward the organizational decisions encoded in the directory tree. The improvements in Table 5 indicate that, on most systems, folder placement aligns reasonably well with the GT architecture, reflecting that incorporating folder co-membership better aligns with the human-defined mappings. Systems where the GT decomposition has very low cohesion in the dependency graph, or has many fine-grained modules relative to graph density, gain the most because the organizational signal supplies boundary information that is lacking in the dependencies.

However, folder location is not always enough. JabRef is the clearest counter-example: its source is split across 127 folders for 1,180 files, so any single `folder-file` link captures only a narrow slice of organizational context. All three heterogeneous methods stay below the GAT-DC baseline built on dependencies alone (78 MoJoFM): M2V at 69, HGAT and HGAT-TD at 75. GAER, which uses information beyond graph structure, reaches 91 on JabRef, but its advantage is neither uniform nor metric-independent. HGAT and HGAT-TD beat GAER on all four metrics for Bash, and M2V or HGAT variants match or beat GAER on A2A, C2C$_{cvg}$, or ARI for AS4, C.Img, Hadoop, T.Mates, and Chrome. GAER leads on all four metrics for A.UML, JabRef, HDC, and HDF, with its clearest edge on MoJoFM. Structural and organizational signals stay competitive in partition-level agreement and coverage, even where GAER is strongest on MoJoFM.

How do HGAT and M2V use folder structure? To assess whether HGAT's advantage over M2V is consistent across runs, Table 6 (right block) reports pairwise comparisons over 100 runs using median score differences and Cliff's δ, following the same protocol as RQ1 5.1.

Table 6: Pairwise comparison across 100 runs per (system, metric) cell: HGAT against HGAT-TD (left block) and HGAT against M2V (right block). Each entry reports the median difference with Cliff's delta effect sizes (L: large, M: medium, S: small, N: negligible). Positive values favor the first method. Differences are significant ($p < 0.05$) unless marked †, and cells where the second method significantly wins are shaded.

	HGAT − HGAT-TD				**HGAT − M2V**			
Dataset	MoJoFM	A2A	C2C$_{cvg}$	ARI	MoJoFM	A2A	C2C$_{cvg}$	ARI
A.UML	$+1.1(L)$	$+1.1(L)$	$+7.1(S)$	$+5.9(L)$	$+1.7(L)$	$+1.6(L)$	$+7.1(M)$	$+7.7(L)$
AS4	$-0.2(N)^{\dagger}$	$+0.3(M)$	$+0.0(S)$	$+0.8(L)$	$+2.1(L)$	$+0.7(L)$	$+3.5(L)$	$+1.1(L)$
C.Img	$+1.9(M)$	$+0.5(L)$	$+4.8(S)$	$+2.1(L)$	$-2.9(L)$	$-0.3(M)$	$+2.4(N)^{\dagger}$	$-2.3(L)$
Hadoop	$-3.6(L)$	$-1.1(L)$	$-1.9(L)$	$-5.3(L)$	$-1.4(L)$	$-2.6(L)$	$-0.3(M)$	$-9.5(L)$
JabRef	$-0.2(N)^{\dagger}$	$-0.3(N)^{\dagger}$	$+0.0(L)$	$+1.1(N)^{\dagger}$	$+6.3(L)$	$-0.7(L)$	$+0.0(N)^{\dagger}$	$+10.5(L)$
Lucene	$-0.1(N)^{\dagger}$	$-0.3(S)$	$+0.0(N)^{\dagger}$	$+0.6(N)^{\dagger}$	$-2.7(L)$	$-0.6(M)$	$+0.0(S)$	$-3.5(L)$
T.Mates	$+0.9(S)$	$+0.4(L)$	$+6.7(S)$	$+4.0(L)$	$-0.9(M)$	$+1.8(L)$	$+13.3(L)$	$+8.6(L)$
Bash	$+0.0(N)^{\dagger}$	$-1.9(L)$	$-7.7(L)$	$-4.5(L)$	$+14.4(L)$	$-0.3(S)^{\dagger}$	$+19.2(L)$	$+5.0(L)$
Chrome	$-2.3(L)$	$-0.2(M)$	$-2.9(L)$	$-0.9(M)$	$+6.8(L)$	$-0.9(L)$	$-8.7(L)$	$-3.4(L)$
HDC	$+0.2(N)^{\dagger}$	$-0.1(N)^{\dagger}$	$+0.0(S)$	$+0.6(N)^{\dagger}$	$+2.5(S)$	$-0.2(N)^{\dagger}$	$+9.1(M)$	$+1.3(S)^{\dagger}$
HDF	$-1.7(S)$	$-0.3(M)$	$+0.0(S)$	$+5.1(L)$	$+5.9(L)$	$+0.8(S)$	$+11.1(M)$	$+8.3(L)$
Average	−0.4	−0.2	+0.6	+0.9	+2.9	−0.1	+5.2	+2.2

HGAT and M2V use the folder signal differently. For M2V, folder structure is the learning objective: meta-path walks pull files that share a folder together through the Skip-gram loss, so the model learns folder co-membership directly, but only from the immediate parent folder. For HGAT, folder structure is a supporting context: folder nodes join relation-specific message passing and shape the file embeddings, while the loss reconstructs file-to-file dependency edges, so the encoder learns relation-specific weights directly without a fixed schema. This pattern is visible in the per-system results. HGAT is significantly better on systems with deeper or more fragmented folder hierarchies (A.UML, AS4, JabRef, Bash, HDF, HDC), where folder nodes in the message passing help the encoder use folder context alongside dependencies. M2V is significantly better on Hadoop, C.Img, and Lucene, where each file's parent folder closely matches its GT module, allowing the meta-path walker to connect co-located files directly.

On average across the eleven systems, HGAT exceeds M2V by +2.9 MoJoFM, +5.2 $C2C_{cvg}$, and +2.2 ARI, with A2A tied. The two methods are therefore not interchangeable: HGAT is the more general choice. M2V is also more sensitive to learning rate and embedding dimension than HGAT (Section 5.3). Chrome is the one configuration exception: HGAT uses two layers rather than one because the system is large enough (18,000 files, nearly 1,000 folders) that file context lies more than one hop away. This reflects the same reach-versus-size principle observed for GAT-DC in RQ1, where larger systems need wider context, and here the encoder supplies it via an extra layer rather than via PPR rewiring.

Does modeling dependency types separately help? We also test whether modeling dependency types separately adds value beyond the folder signal. HGAT-TD keeps dependency types such as calls and imports as separate relations, but changes little overall and provides only a modest, system-dependent benefit. It sometimes improves MoJoFM, but its effect on $C2C_{cvg}$ and ARI is mixed and often slightly negative, likely because splitting dependencies by type makes each relation sparser and can fragment clusters. The clearest exceptions are Hadoop and Bash, where separating dependency types appears to reduce cross-module noise and improve several metrics. Across systems, the average differences are small, so dependency typing appears to be a system-specific refinement rather than a reliable improvement.

Variability. Figure 3 compares the run-to-run distributions of MoJoFM and ARI across heterogeneous methods on selected systems. Bash is the most stable: HGAT and HGAT-TD are nearly flat while M2V spreads a few points wider. On HDC and HDF, all three are comparably variable, with HGAT variants the widest on MoJoFM, so no method is clearly consistent there, yet all three stay well above homogeneous methods. HGAT-TD tracks HGAT throughout the spread, so modeling the dependency types separately does not change stability. JabRef is an exception: M2V's ARI remains near zero, so its low spread reflects consistent failure, and all three heterogeneous methods fall below GAT-DC. Overall, variability does not weaken the main RQ2 result: across all other systems, the heterogeneous methods maintain medians and distributions above those of N2V and GAT-DC. Their spread also stays similar or decreases under Search, with the average IQR over MoJoFM and ARI dropping from 3.3 to 2.0 for HGAT, nearly unchanged for M2V (3.5 to 3.2), and flat for HGAT-TD (3.1).

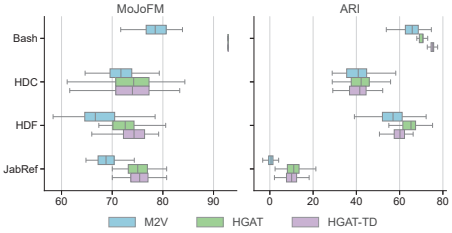


Figure 3: Distribution of scores over 100 runs for MoJoFM (left) and ARI (right) for Bash, HDC, HDF, and JabRef.

5.2.1 Internal Modularity

Table 7 reports TMQ_{norm} for the recovered decompositions alongside the GT decomposition. Since TMQ_{norm} measures dependency-based modularity, higher values mean stronger intra-cluster cohesion and weaker inter-cluster coupling. The dependency-only methods, especially N2V, reach the highest TMQ_{norm} on most systems, so random-walk embeddings capture dependency cohesion particularly strongly; the GAT-based homogeneous variants follow a similar pattern.

Table 7: Normalized TurboMQ scores. The GT row reports the score obtained by applying the published GT clustering to the dependency graph; the remaining rows report recovered scores per method. The upper block lists the homogeneous methods, and the lower block the heterogeneous methods and GAER. The best recovered score per dataset is highlighted.

	Java							C/C++			
Method	A.UML	AS4	C.Img	Hadoop	JabRef	Lucene	T.Mates	Bash	Chrome	HDC	HDF
GT	44	43	38	12	30	58	19	36	47	43	35
ACDC	20	36	39	24	13	28	15	34	31	65	41
FCA	18	29	22	20	18	22	11	32	27	57	23
LIMBO	37	43	31	31	43	59	19	48	12	48	44
WCA_{LIE}	39	39	41	22	67	14	14	54	6	59	41
WCA_{LIENM}	43	43	42	25	67	14	18	54	7	50	59
N2V	77	52	60	39	85	85	33	73	67	84	72
GAT	67	52	50	34	65	90	27	74	51	86	61
GAT-DC	71	36	44	22	65	71	25	53	58	71	68
GAER	*59*	*37*	*53*	*30*	*64*	*72*	*13*	*43*	*65*	*67*	*49*
M2V	65	37	58	22	85	72	13	33	53	67	41
HGAT	65	39	63	29	70	73	12	37	50	69	48
HGAT-TD	66	39	61	30	73	74	13	39	50	68	54

The heterogeneous methods and GAER generally obtain lower TMQ_{norm} than the strongest homogeneous

methods. This is not a weaker recovery but a shift in objective: TurboMQ rewards decompositions that minimize cross-cluster *dependency* edges, whereas M2V and HGAT also group files by folder containment, and GAER adds semantic signals from identifiers and naming. These signals move the recovered decomposition away from pure dependency cohesion, yet the improved GT alignment in Table 5 suggests the GT architecture is not based on dependencies alone. It also reflects how developers organize and name the system, and that organization may inform how experts draw the GT, so directory layout and the GT are not fully independent. Methods built on dependencies thus favor structurally cohesive partitions, whereas organization- and semantically aware methods better capture developer-defined boundaries, consistent with the view that no single decomposition is objectively correct [4, 3].

Answer to RQ2. Augmenting the dependency graph with folder nodes and containment edges improves GT alignment on most systems, most clearly where the dependency signal alone is weak, so project organization carries architectural information that dependencies do not capture. HGAT is the more robust choice, especially when the folder hierarchy is deep or fragmented, while M2V does better when a file's anchor folder already aligns with its GT module. Modeling dependency types separately (HGAT-TD) is a system-dependent refinement rather than a uniform gain, yielding only about a 1-point gain on HGAT on average. Overall, folder structure remains a partial signal: it complements dependencies but is insufficient when the layout diverges from the architecture, and its gains come with lower TMQ_{norm}, as recovery shifts from dependency modularity toward developer-defined boundaries.

5.3 RQ3 – Practical Considerations for Adoption

RQ3 examines two practical concerns: runtime as systems scale and per-system hyperparameter tuning. We report runtime first, then analyze sensitivity to the method-specific hyperparameters in Table 2.

5.3.1 Scalability

Table 8 reports end-to-end runtime under the Search setting, excluding dependency extraction since it is shared by all methods and measured after extraction with Depends [40]. All methods finish in under 60 seconds on every system except Chrome, where runtimes range from 57 seconds (M2V) to 289 seconds (N2V). M2V and HGAT are the fastest: they avoid both N2V's 100 random walks and GAT-DC's denser rewired graph, where keeping up to k neighbors per node typically gives the GAT encoder more edges than the raw dependency graph. HGAT-TD is slower than HGAT, since splitting the dependency edges by type raises the relation count and the per-layer message passing cost. N2V runs on the CPU via the Gensim library [63] to stay comparable to Chen et al. [5], while all other methods use a GPU. Even on Chrome, every method completes in under five minutes, so the studied representation learning methods are not the scalability bottleneck for SAR adoption.

Table 8: Average runtime comparison across the benchmark systems under the Search setting where k is searched over a predefined search range. The values are reported in seconds and averaged over the three runs.

System	N2V	GAT	GAT-DC	M2V	HGAT	HGAT-TD
A.UML	20.21	3.28	3.40	2.55	2.35	4.76
AS4	16.63	2.26	2.22	1.91	1.88	4.37
C.Img	8.17	1.44	1.50	1.00	1.21	3.75
Hadoop	18.56	3.25	3.18	2.15	2.40	4.96
JabRef	25.59	5.08	5.39	3.94	3.63	6.17
Lucene	29.17	5.18	5.59	3.45	3.62	5.93
T.Mates	22.06	4.92	5.32	2.77	3.24	5.70
Bash	5.69	1.13	1.21	0.90	0.94	2.95
Chrome	289.46	87.55	100.67	56.69	69.17	89.29
HDC	4.14	0.92	0.98	0.55	0.75	3.07
HDF	2.48	0.85	0.92	0.44	0.71	2.71

5.3.2 Hyperparameter Sensitivity

This section characterizes how the main hyperparameters affect recovery. We report it as a sensitivity analysis rather than an automatic selection rule: the per-system values in Table 2 were fixed during development, and the analysis below explains how they relate to system properties such as size and GT granularity.

The PPR top-k parameter controls how many diffused neighbors each node retains after rewiring: larger values broaden dependency context, while smaller values keep it local. Because the useful context depends on each system's dependency structure, we sweep $k \in 16, 32, 64, 128$ per system. Table 9 reports median scores over 100 runs for each (system, top-k) setting. Since MoJoFM favors smaller k and ARI favors larger k, we interpret them jointly rather than optimizing either metric alone.

Across systems, the best top-k tracks GT granularity. Finer-grained architectures split at a smaller top-k because each module has fewer files and needs less diffuse context, whereas coarse-grained systems require wider context. The two smallest and sparsest systems, HDC and HDF (fewer than 250 files), settle at top-$k = 16$. Lucene, with seven large modules over 1,054 files, and T.Mates, with the highest average degree (17.3), need the widest context and use top-$k = 128$. System size and average degree are secondary: C.Img and Bash are close in size (329 versus 292 files) but prefer top-$k = 32$ and 64, tracking module counts (21 versus 13) rather than size. For Chrome, $k > 32$ was infeasible, since the diffusion matrix grows too dense at roughly 18k nodes.

AS4 and Hadoop show why we read metrics jointly. Both have fine-grained GTs (57 and 63 modules) with MoJoFM nearly flat across top-k (within one point), so

Table 9: Median GT-alignment scores for GAT-DC across four top-k values, over 100 runs per (system, top-k) configuration. MoJoFM (M), A2A (A), $C2C_{cvg}$ (C), and ARI (I) are scaled by 100. The best value per metric is highlighted.

	top-k=16				top-k=32				top-k=64				top-k=128			
System	M	A	C	I	M	A	C	I	M	A	C	I	M	A	C	I
A.UML	70	89	43	34	73	89	43	37	74	89	43	40	74	89	36	41
AS4	71	86	32	15	70	85	25	17	70	88	28	29	68	87	26	27
C.Img	79	90	43	44	77	90	48	48	73	89	38	43	71	89	33	46
Hadoop	52	83	6	15	53	85	10	22	53	85	8	26	52	84	6	24
JabRef	75	85	17	9	77	85	17	10	78	86	17	12	76	85	17	7
Lucene	65	89	43	35	70	89	57	40	71	89	57	40	74	92	57	58
T.Mates	63	85	13	20	55	84	13	14	61	83	7	15	66	84	7	23
Bash	62	86	23	28	64	87	23	33	64	88	23	35	60	87	19	32
Chrome	69	83	22	14	71	83	25	14	—	—	—	—	—	—	—	—
HDC	73	89	27	39	70	88	18	40	61	86	9	31	62	87	27	36
HDF	56	89	44	35	55	88	33	34	55	88	44	33	47	87	22	22

MoJoFM alone does not distinguish between the settings. ARI breaks the tie: on AS4 it rises from 15 at top-$k = 16$ to 29 at top-$k = 64$, and on Hadoop it peaks at top-$k = 64$ (26, versus 15 at top-$k = 16$). We set top-$k = 64$ for both, as the gain in partition-level agreement outweighs the slight drop in MoJoFM. Layer count follows the same logic: GAT-DC uses one GAT layer on large, fine-grained systems, where PPR already widens each node's reach, and two layers on smaller or coarser systems.

Tuning M2V. M2V was harder to tune than the other methods and was sensitive to both the learning rate and the embedding dimension. A learning rate of 0.05 and an embedding dimension of 64 produced stable training across all 11 systems. We attribute this sensitivity to the strong folder-anchor signal from the minimal meta-path schema: files that share a folder often co-occur in short walks, which can make the embeddings folder-dominated. The schema also matters; as noted in Section 5.2, longer schemas required hand-designed traversal orders and reverse-edge handling, and underperformed compared to the minimal `file-folder-file-file` path. HGAT-TD adds no tuning beyond HGAT; it reuses the HGAT configuration and only splits dependency edges by type.

Answer to RQ3. Runtime is not a bottleneck: every method finishes in under a minute except on Chrome (∼18k nodes), which takes one to five minutes. Hyperparameter sensitivity is limited and predictable. For GAT-DC, the best top-k tracks target granularity: 16 for the smallest and sparsest systems, 32 to 64 for mid-sized ones, and 128 for the densest or most coarse-grained systems (Lucene and T.Mates). Large swings appear only when top-k is far from a system's scale. M2V is more sensitive than the GAT-based variants to learning rate and embedding dimension, but trains stably across all 11 systems with learning rate 0.05, embedding dimension 64, and the minimal `file-folder-file-file` meta-path.

6 Threats to Validity

External validity. Our evaluation covers eleven open-source systems in Java, C, and C++ with available ground-truth module mappings. These systems sample only part of the real-world variation in size, domain, and architectural style, so the findings may not generalize to all systems. The benchmark also excludes Python and other dynamically typed languages, in which dependency extraction relies less on static imports and more on runtime behavior. We expect the relative ordering of the embedding methods to hold, since they operate on the dependency graph rather than on source syntax, but confirming this needs systems and GT in those languages. We likewise do not test code written largely by AI coding assistants, a setting that is becoming common and may produce different dependency structures from human-generated code.

Internal Validity. We recover architecture at the file-level granularity, which gives a coarser view than class- or method-level analysis. We chose files because the published ground-truth mappings are defined over files and because file-level dependency graphs are cheaper to build and more stable across systems. The cost is that fine-grained relations within a file are not visible to the model, and a file that spans more than one architectural concern cannot be split. A further limitation is that dependencies are recovered using a static analysis tool, which may miss edges arising from dynamic features or incomplete parsing, and may introduce non-existent ones. This affects all dependency-based recovery methods, but an inaccurate input graph can under- or over-represent the true coupling between files.

The quality of the ground-truth decompositions is another threat. These mappings are created by people and can involve judgment calls, especially for borderline artifacts, so different experts may not always agree. For some systems (e.g., JabRef, TeamMates, and CommonImaging), the mappings were provided by system experts; for HDC and HDF, they were produced by industry partners; and for AS4, Hadoop, Bash, and Chrome, the mappings were reconstructed from documentation and implementation details as part of research projects. These datasets are widely used in SAR research, but small errors or subjective choices in the GT can affect the reported alignment scores.

The statistical analysis also introduces threats. We perform pairwise comparisons across systems, methods, and metrics, which increases the chance of observing some significant results by chance. In addition, each comparison uses 100 stochastic runs, so tests can detect small distributional differences that may not be practically meaningful. We therefore interpret p-values alongside median differences and Cliff's delta effect sizes, and treat metrics with uniformly small absolute differences, such as A2A, as secondary evidence rather than as decisive

method rankings.

Ethical considerations. A recovered architecture describes how a system is structured, not how well its developers perform. We therefore caution against using these decompositions to evaluate individual engineers or to drive productivity and KPI dashboards, since module boundaries reflect design and dependency history rather than developer effort. The input graphs are built from file paths and static dependencies, which can expose identifying details such as author names or internal directory layouts. We use only publicly available GT benchmarks and release no private code or data. Our implementation and processed graphs are available in the linked repository 1 to support replication.

7 Conclusion

We studied software architecture recovery from static dependency graphs by learning self-supervised file embeddings and clustering them into architectural modules, across eleven benchmark systems. Recovery from dependencies alone is useful but bounded, and the bound is the model's reach rather than the encoder: shallow message passing on the raw graph stays too local, while random-walk context and diffusion rewiring expose the longer-range structure that separates modules. Among these methods, GAT-DC and N2V are comparable, with GAT-DC slightly ahead and the most stable.

When dependency cohesion is weak, project organization supplies architectural information that dependencies do not capture, and it helps most exactly where recovery from dependencies alone struggles, with HGAT the most robust choice among the heterogeneous methods. The gain comes with a trade-off: these methods align better with the GT by moving away from pure dependency cohesion, so their TMQ_{norm} falls below the strongest methods built on dependencies. Maximizing dependency modularity and recovering the developer-intended architecture are therefore related but not identical goals.

The methods are also scalable. Every method finishes in under a minute except on Chrome (about 18k nodes, one to five minutes), so representation learning is not the bottleneck for SAR adoption, and tuning sensitivity is limited and follows predictable patterns.

In future work, we plan to model dependency types jointly rather than in isolation. HGAT-TD applies a separate GAT layer per type but does not learn across them. Treating typed dependencies as meta-paths and attending over them would let the model weigh relation types jointly, and a heterogeneous decoder that reconstructs typed and organizational edges would let relational detail shape the embeddings on both sides of the autoencoder rather than only the encoder.

References

1. Clements P, Garlan D, Little R, Nord R, and Stafford J. Documenting software architectures: views and beyond. *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE. 2003 :740–1
2. Parnas DL. On the criteria to be used in decomposing systems into modules. Communications of the ACM 1972; 15:1053–8
3. Lutellier T, Chollak D, Garcia J, Tan L, Rayside D, Medvidovic N, and Kroeger R. Comparing software architecture recovery techniques using accurate dependencies. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering.* Vol. 2. IEEE. 2015 :69–78
4. Jabeen R, Ericsson M, Nordqvist J, and Wingkvist A. GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery. Proceedings of the ACM on Software Engineering 2026 Jul; 3. DOI: 10.1145/3808187
5. Chen J, Wang Z, Jiang Y, Pang J, Zhang T, Pan M, and Sun J. Negar: network embedding guided architecture recovery for software systems. *2022 29th Asia-Pacific Software Engineering Conference (APSEC).* IEEE. IEEE, 2022 :367–76
6. Naseem R, Deris MM, Maqbool O, and Shahzad S. Euclidean space based hierarchical clusterers combinations: an application to software clustering. Cluster Computing 2019; 22:7287–311
7. Andritsos P, Tsaparas P, Miller RJ, and Sevcik KC. LIMBO: Scalable clustering of categorical data. *International Conference on Extending Database Technology.* Springer. 2004 :123–46
8. Tzerpos V and Holt RC. Accd: an algorithm for comprehension-driven clustering. *Proceedings Seventh Working Conference on Reverse Engineering.* IEEE. 2000 :258–67
9. Mitchell BS and Mancoridis S. On the automatic modularization of software systems using the bunch tool. IEEE Transactions on Software Engineering 2006; 32:193–208
10. Maqbool O and Babri HA. The weighted combined algorithm: A linkage algorithm for software clustering. *Eighth European Conference on Software Maintenance and Reengineering, 2004. CSMR 2004. Proceedings.* IEEE. 2004 :15–24
11. Garcia J, Popescu D, Mattmann C, Medvidovic N, and Cai Y. Enhancing architectural recovery using concerns. *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011).* IEEE. 2011 :552–5
12. Corazza A, Di Martino S, Maggio V, and Scanniello G. Investigating the use of lexical information for software system clustering. *2011 15th European Conference on Software Maintenance and Reengineering.* IEEE. 2011 :35–44

13. Zhang Y, Xu Z, Liu C, Chen H, Sun J, Qiu D, and Liu Y. Software architecture recovery with information fusion. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2023 :1535–47
14. Papachristou M. Software clusterings with vector semantics and the call graph. *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 2019 :1184–6
15. Rukmono SA, Ochoa L, and Chaudron M. Deductive software architecture recovery via chain-of-thought prompting. *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results*. 2024 :92–6
16. Veličković P, Cucurull G, Casanova A, Romero A, Lio P, and Bengio Y. Graph attention networks. arXiv preprint arXiv:1710.10903 2017
17. Kipf TN and Welling M. Variational graph auto-encoders. arXiv preprint arXiv:1611.07308 2016
18. Mancoridis S, Mitchell BS, Chen Y, and Gansner ER. Bunch: A clustering tool for the recovery and maintenance of software system structures. *Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99).'Software Maintenance for Business Change'(Cat. No. 99CB36360)*. IEEE. 1999 :50–9
19. Parsa S and Bushehrian O. A new encoding scheme and a framework to investigate genetic clustering algorithms. Journal of Research and Practice in Information Technology 2005; 37:127–43
20. Anquetil N and Lethbridge TC. Recovering software architecture from the names of source files. Journal of Software Maintenance: Research and Practice 1999; 11:201–21
21. Risi M, Scanniello G, and Tortora G. Using fold-in and fold-out in the architecture recovery of software systems. Formal Aspects of Computing 2012; 24:307–30
22. Kargar M, Isazadeh A, and Izadkhah H. Semantic-based software clustering using hill climbing. *2017 international symposium on computer science and software engineering conference (CSSE)*. IEEE. 2017 :55–60
23. Puchala SPR, Chhabra JK, and Rathee A. Ensemble clustering based approach for software architecture recovery. International Journal of Information Technology 2022; 14:2013–9
24. Sadat Jalali N, Izadkhah H, and Lotfi S. Multi-objective search-based software modularization: Structural and non-structural features. Soft Computing-A Fusion of Foundations, Methodologies & Applications 2019; 23
25. Rathee A and Chhabra JK. Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. *Advanced Informatics for Computing Research: First International Conference, ICAICR 2017, Jalandhar, India, March 17–18, 2017, Revised Selected Papers*. Springer. 2017 :94–106
26. Harman M and Yao X. Software module clustering as a multi-objective search problem. IEEE Transactions on Software Engineering 2010; 37:264–82
27. Naseem R, Maqbool O, and Muhammad S. Cooperative clustering for software modularization. Journal of Systems and Software 2013; 86:2045–62
28. Altınışık M, Sözer H, and Gürsun G. Software architecture recovery from multiple dependency models. *Proceedings of the 39th ACM/SIGAPP symposium on applied computing*. 2024 :1185–92
29. Teymourian N, Izadkhah H, and Isazadeh A. A fast clustering algorithm for modularization of large-scale software systems. IEEE Transactions on Software Engineering 2020; 48:1451–62
30. Mohammadi S and Izadkhah H. A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. Information and Software Technology 2019; 105:252–6
31. Pourasghar B, Izadkhah H, Isazadeh A, and Lotfi S. A graph-based clustering algorithm for software systems modularization. Information and Software Technology 2021; 133:106469
32. Misra J, Annervaz K, Kaulgud V, Sengupta S, and Titus G. Software clustering: Unifying syntactic and semantic features. *2012 19th Working Conference on Reverse Engineering*. IEEE. 2012 :113–22
33. Zhao W, Jin W, Zhang Y, Fan M, Wang H, Li L, Liu Y, and Liu T. Software Architecture Recovery Augmented with Semantics. IEEE Transactions on Software Engineering 2025
34. Jabeen R, Ericsson M, Nordqvist J, and Wingkvist A. Graph Convolution Networks for Mapping Source Code Entities to Architectural Modules. *22nd IEEE International Conference on Software Architecture (ICSA 2025), Odense, Denmark, March 31-April 4, 2025*. IEEE. 2025
35. Bedi P and Sharma C. Community detection in social networks. Wiley interdisciplinary reviews: Data mining and knowledge discovery 2016; 6:115–35
36. Turner E. Graph auto-encoders for financial clustering. arXiv preprint arXiv:2111.13519 2021
37. Afzali A, Hosseini H, Mirzai M, and Amini A. Clustering time series data with gaussian mixture embeddings in a graph autoencoder framework. *International Workshop on AI for Transportation*. Springer. 2025 :82–107
38. Stavropoulou I, Grigoriou M, and Kontogiannis K. Case study on which relations to use for clustering-based software architecture recovery. Empirical Software Engineering 2017; 22:1717–62
39. Garcia J, Krka I, Mattmann C, and Medvidovic N. Obtaining ground-truth software architectures. *2013 35th International Conference on Software Engineering (ICSE)*. IEEE. 2013 :901–10
40. Nguyen HV. Depends: Multilingual Dependency Analysis Tool. `https://github.com/multilang-depends/depends`. Accessed: 2025-08-17. 2023
41. Grover A and Leskovec J. node2vec: Scalable feature learning for networks. *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 2016 :855–64

42. Mikolov T, Sutskever I, Chen K, Corrado GS, and Dean J. Distributed representations of words and phrases and their compositionality. Advances in neural information processing systems 2013; 26

43. Dong Y, Chawla NV, and Swami A. metapath2vec: Scalable representation learning for heterogeneous networks. *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 2017 :135–44

44. Wu Z, Pan S, Chen F, Long G, Zhang C, and Yu PS. A comprehensive survey on graph neural networks. IEEE transactions on neural networks and learning systems 2020; 32:4–24

45. Li Q, Han Z, and Wu XM. Deeper insights into graph convolutional networks for semi-supervised learning. *Proceedings of the AAAI conference on artificial intelligence*. Vol. 32. 1. 2018

46. Dwivedi VP, Joshi CK, Luu AT, Laurent T, Bengio Y, and Bresson X. Benchmarking graph neural networks. Journal of Machine Learning Research 2023; 24:1–48

47. Gasteiger J, Weißenberger S, and Günnemann S. Diffusion improves graph learning. Advances in neural information processing systems 2019; 32

48. Ma L, Rabbany R, and Romero-Soriano A. Graph attention networks with positional embeddings. *Pacific-Asia conference on knowledge discovery and data mining*. Springer. 2021 :514–27

49. Maas AL, Hannun AY, and Ng AY. Rectifier Nonlinearities Improve Neural Network Acoustic Models. *ICML Workshop on Deep Learning for Audio, Speech, and Language Processing*. 2013. Available from: https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf

50. Clevert DA, Unterthiner T, and Hochreiter S. Fast and accurate deep network learning by exponential linear units (elus). arXiv preprint arXiv:1511.07289 2015; 4:11

51. Kingma DP and Ba J. Adam: A method for stochastic optimization. arXiv preprint arXiv:1412.6980 2014

52. Rousseeuw PJ. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. Journal of computational and applied mathematics 1987; 20:53–65

53. Olsson T, Ericsson M, and Wingkvist A. To automatically map source code entities to architectural modules with Naive Bayes. Journal of Systems and Software 2022; 183:111095

54. Laser MS, Medvidovic N, Le DM, and Garcia J. ARCADE: an extensible workbench for architecture recovery, change, and decay evaluation. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE '20. ACM, 2020 :1546–50. DOI: 10.1145/3368089.3417941

55. Wen Z and Tzerpos V. An effectiveness measure for software clustering algorithms. *Proceedings. 12th IEEE International Workshop on Program Comprehension, 2004*. IEEE. 2004 :194–203

56. Lutellier T, Chollak D, Garcia J, Tan L, Rayside D, Medvidović N, and Kroeger R. Measuring the impact of code dependencies on software architecture recovery techniques. IEEE Transactions on Software Engineering 2017; 44:159–81

57. Le DM, Behnamghader P, Garcia J, Link D, Shahbazian A, and Medvidovic N. An empirical study of architectural change in open-source software systems. *2015 IEEE/ACM 12th working conference on mining software repositories*. IEEE. 2015 :235–45

58. Garcia J, Le D, Link D, Behnamghader AP, Ortiz EF, and Medvidovic N. An empirical study of architectural change and decay in open-source software systems. USC-CSSE, Tech. Rep 2014

59. Hubert L and Arabie P. Comparing partitions. Journal of classification 1985; 2:193–218

60. Mann HB and Whitney DR. On a test of whether one of two random variables is stochastically larger than the other. The annals of mathematical statistics 1947 :50–60

61. Cliff N. Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological bulletin 1993; 114:494

62. Jabeen R, Ericsson M, Nordqvist J, and Wingkvist A. Reproduction package for GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery. Zenodo. Version 1.1.0. 2026. DOI: 10.5281/zenodo.19534384

63. RARE Technologies. Gensim: Topic Modelling for Humans. https://radimrehurek.com/gensim/. Python library. 2026

Paper V

What Carries Architectural Signal? A Code-to-Architecture Mapping Study

V

Rakhshanda Jabeen, Morgan Ericsson, Yuyao Duan, and Anna Wingkvist. *What Carries Architectural Signal? A Code-to-Architecture Mapping Study.* 2026. arXiv: 2610.41628 [cs.SE]. URL: https://arxiv.org/abs/2610.41628. *Under review.*

What Carries Architectural Signal? A Code-to-Architecture Mapping Study

Rakhshanda Jabeen
Electrolux Professional AB
Ljungby, Sweden
Linnaeus University
Växjö, Sweden
rakhshanda.jabeen@electroluxprofessional.com

Morgan Ericsson
Linnaeus University
Växjö, Sweden
morgan.ericsson@lnu.se

Yuyao Duan
Linnaeus University
Växjö, Sweden
yuyaoduan4@gmail.com

Anna Wingkvist
Linnaeus University
Växjö, Sweden
anna.wingkvist@lnu.se

Abstract

Code-to-architecture mapping assigns source files to modules of a reference architecture. Representations of implementation artifacts can capture code text, dependencies, and directory organization, but predictive gains alone do not establish the contribution of additional graph processing. Dependencies may already be encoded in these representations, while directories may closely match the reference decomposition. We investigate how artifact representations and graph context affect mapping on ten Java, C, and C++ systems with a limited labeling budget. We compare lexical, structural, and contextual file representations and assess each predictor with and without dependency neighbors. Using a Heterogeneous Graph Transformer (HGT), we compare file-level graphs extended with member nodes summarizing each file's program elements, folder nodes, or both, alongside a variant encoding folder paths directly in file representations. We compare folder-aware predictions with a directory rule using the same labeled files.

Representations encoding dependency information outperform lexical representations on most systems. Additional neighbor aggregation can improve mapping, depending on the predictor and aggregation direction. Folder-aware variants yield the largest and most consistent gains across the evaluated HGT configurations. However, the directory rule achieves per-system median accuracy of 89–100%, and adding folder information does not consistently improve predictions for files the rule misclassifies. Member nodes provide no consistent improvement over file-level graphs. These findings support assessing what file representations and directory organization already provide before attributing improvements to graph neural networks.

Keywords

Code-to-Architecture Mapping, Software Architecture, Graph Neural Networks, Heterogeneous Graphs, Representation Learning

1 Introduction

Maintaining a software architecture requires knowing which implementation elements belong to its modules [9, 47]. This correspondence can become incomplete as a system grows and architectural documentation falls behind the implementation. Code-to-architecture (C2A) mapping addresses this problem by assigning source files to the modules of a reference architecture. When only part of a system has been mapped, automated approaches can use these known assignments together with information from the code and project structure to infer the remaining assignments.

Automated C2A mapping can draw on several sources of evidence. Identifiers express domain concepts and implementation roles, while declarations and operations describe how those responsibilities are implemented. Static dependencies describe interactions between files, and directories record how developers organize a project [6, 14, 65]. These sources need not agree: a utility may interact with several modules, terminology may be shared across responsibilities, and a directory may contain files assigned to different architectural modules. This study examines what dependency-neighbor aggregation and richer graph structures add beyond the information available in file representations and directory organization.

Prior C2A mapping work combines lexical features with static dependencies in graph neural networks [21], with later work also incorporating folder organization [25]. Related work on architecture recovery, which infers a decomposition without a reference architecture, explores random-walk and graph-autoencoder representations of dependencies [6, 23] and encodings of identifier and folder tokens [22]. When several information sources and predictor mechanisms change together, comparisons of overall performance leave the contribution of additional graph context unclear.

A classifier may already receive dependency information through its input vectors, so the contribution of additional neighbor aggregation needs to be evaluated separately. Similarly, high agreement with a reference architecture may partly reflect agreement between that architecture and the directory structure. A folder-aware model can improve overall accuracy while reducing accuracy on files that a simple directory rule misclassifies. Aggregate

scores alone do not distinguish these outcomes. Representing members separately may expose context obscured in a single file-level representation.

We investigate these questions on ten Java, C, and C++ systems, using a bounded labeling budget with at least one labeled file from every module. The setting is transductive: features and graph structure are available for labeled and unlabeled files, while module-label supervision uses only the initially labeled files. We compare lexical Word2Vec, structural Node2Vec, and contextual UniXcoder representations. A multilayer perceptron (MLP) establishes their performance before additional neighbor aggregation. We use a Heterogeneous Graph Transformer (HGT) [20] to compare file, folder, and member configurations, and assess folder-aware predictions against a directory rule using the same labeled files. We also compare performance with neighbors enabled and disabled for HGT, a neighbor MLP, and a Graph Attention Network (GAT) [62], with aggregation in one or both directions.
The study is guided by three research questions:

- **RQ1.** How does mapping performance vary across lexical, structural and contextual file representations?
- **RQ2.** How does adding member and folder nodes to file-level HGT change mapping performance, and how do folder-aware predictions compare with a directory rule?
- **RQ3.** How does using dependency neighbors change mapping performance, and how do the predictor and aggregation direction affect this change?

The results show that file representations encoding dependency information provide strong starting points, while the benefit of further neighbor aggregation depends on the predictor and aggregation direction. Folder nodes yield the largest gains within the HGT configurations, but the directory rule reaches similar accuracy on most systems, and folder paths supplied directly as file features match or exceed folder nodes on most systems. These overall gains do not consistently extend to files the rule assigns incorrectly. Member nodes provide no consistent improvement with one or two HGT layers. Code and data are available on GitHub[1].

2 Methodology

This section presents the methodology for studying how architectural signals and graph design choices affect C2A mapping. We first formulate the mapping task and then describe the graphs and the structural, lexical, and contextual features used to initialize their nodes. We then present the MLP, HGT, and GAT predictors.

Each research question varies one of these components and holds the others fixed. RQ1 varies the file representation, using the MLP. RQ2 varies the graph variant for each representation, using HGT because it handles the node and relation types of the heterogeneous variants. RQ3 varies the predictor, comparing the neighbor MLP with GAT and HGT, and the dependency neighbors they receive.

2.1 Problem Formulation

We formulate the mapping task as node classification over a software graph $G = (V, E, \mathcal{A}, \mathcal{R})$, where V and E are the sets of artifact nodes and relations between them, respectively, and $\mathcal{A}$ and $\mathcal{R}$ denote the sets of node and relation types. The mappings $\tau : V \to \mathcal{A}$ and $\phi : E \to \mathcal{R}$ assign node and relation types. Let $V_f \subseteq V$ be the set of file nodes, which are the targets of the mapping task. A small subset $L \subset V_f$ is labeled with known architectural module assignments Y_L, and the remaining file nodes $U = V_f \setminus L$ are unlabeled. The task is to learn a model:

$$f_\theta : (G, L, Y_L) \to \hat{Y}_U, \tag{1}$$

where θ denotes the learnable parameters. The model is trained on the labeled files only, but all nodes of G, labeled and unlabeled, are part of the graph during training.

Each model computes a d-dimensional embedding z_i for every file $i \in V_f$, and the architectural module of a file is predicted from this embedding.

2.2 Graph Construction

We represent the software system as a graph of software artifacts connected by folder organization, member containment, and static dependencies.

To study the individual and combined contributions of folder and member information, we construct four graph variants that differ in the node types they contain. They are named after these types: F (file nodes only), FF (folder and file nodes), FM (file and member nodes), and FFM (folder, file, and member nodes). All variants retain the dependency relation. F is homogeneous, with one node type and one relation type, and the other three variants are heterogeneous. Figure 1 provides a visual overview.

The complete graph (FFM) has three node types. File nodes represent source files and are the targets of C2A mapping. Because logical components are often implemented through namespaces or directory structures that group source code providing a particular functionality [14, 52], folder nodes represent directory levels of the project, with one node per level. Each file has one summarized member node covering the methods, fields, constructors, and variables of that file. This retains member-level information while avoiding the large and sparse graphs produced by representing individual members.

Four relations connect these nodes, as listed in Table 1. Three are structural and encode organization and containment, and the fourth encodes static dependencies. Folder nodes are connected through *parent_of* edges, forming chains such as `app` → `app/ui` → `app/ui/views`, and *contains* edges link each folder to its files. A *belongs_to* edge links each member node to its enclosing file.

Messages along structural relations flow toward the file. Folder information passes down the hierarchy through *parent_of* and *contains* edges, and member information passes up to the file through *belongs_to* edges. No reverse edges are added, so folder and member nodes are not updated from files through structural relations.

depends_on encodes static dependencies, which provide direct evidence of how source files interact [6, 31]. These dependencies were extracted with the Depends tool [42], which analyzes the syntactic structure of the projects and identifies entities together with several categories of static relations. We merge all dependency types into a single relation, drop duplicate edges, and remove edges whose endpoints belong to the same file.

[1] https://github.com/sse-lnu/hetmap

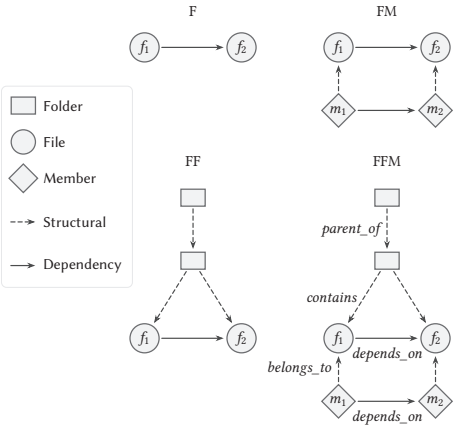


Figure 1: Graph variants F, FM, FF, and FFM. Files f_i are the prediction targets, and each m_i aggregates the members of file f_i. Arrows point from source to target: dashed arrows are structural relations and solid arrows are dependencies. Relation names are shown on FFM only. The panels omit further dependency edges and folder levels; Table 1 lists the relations with the node types they connect. Feature initialization also depends on the variant, as described in Section 2.3.

Table 1: Graph relations and node types.

Relation	Source → target	Description
parent_of	folder → folder	Directory hierarchy.
contains	folder → file	Folder contains a file.
belongs_to	member → file	Links a member node to its file.
depends_on	file → file	File-level dependencies.
	member → member	Member-level dependency in FM and FFM.

Dependency endpoints depend on the graph variant. In FM and FFM, dependencies connect either two files or two member nodes. Dependencies between a file and a member are resolved between their corresponding files. In F and FF, all member endpoints are replaced by their enclosing files, so all dependencies connect file nodes.

RQ1 uses no graph structure, and RQ3 uses the homogeneous file-level graph F. RQ2 uses all four variants to examine progressively richer graph structure, adding folder nodes in FF, member nodes in FM, and both in FFM.

2.3 Node Feature Initialization

Folder nodes are initialized in the same way in every setting. File and member nodes are initialized with one of the representations compared in RQ1: lexical embeddings from Word2Vec, structural embeddings from Node2Vec, or contextual embeddings from UniXcoder, used zero-shot or fine-tuned with dependencies.

2.3.1 *Folder Node Features*

Folder nodes use a binary bag-of-words representation of the folder tokens along their paths, after removing common project-root tokens. For example, `app/ui/views` is represented by `app`, `ui`, and `views`. Including the tokens of its ancestors preserves each folder's hierarchical context.

2.3.2 *Word2Vec (W2V): Lexical Features*

Source code identifiers encode domain concepts and implementation roles, making them useful signals for C2A mapping [65]. Following Jabeen et al. [22], we extract tokens from file names and from the identifiers of members, namely methods, fields, constructors, and variables. We split camelCase, PascalCase, and snake_case identifiers, convert tokens to lowercase, and remove stopwords and non-alphabetic tokens. Each node vector is the mean of the skip-gram Word2Vec embeddings of its associated tokens [34].

In FM and FFM, file nodes use file-name tokens, while member nodes use member identifiers. In F and FF, member identifiers are included in the file vector instead, and the MLP uses this same file vector. Member identifiers therefore remain available in every variant.

2.3.3 *Node2Vec (N2V): Structural Features*

Chen et al. [6] observe that files in the same architectural module can occupy a dense local dependency region without being directly connected. They use random-walk embeddings to capture this longer-range structure for architecture recovery. In our earlier study of dependency-only recovery, which follows their setup, Node2Vec was among the best-performing learned representations [23]. Therefore, we use Node2Vec [16] embeddings computed with uniform random walks on the undirected file-level dependency graph as a structural initialization. These embeddings encode dependency structure without using identifiers or folder location.

In FM and FFM, each summarized member node receives the Node2Vec embedding of its corresponding file.

2.3.4 *UniXcoder: Contextual Features*

For contextual initialization, we use UniXcoder [17], a pretrained code transformer. We parse each source file with Tree-sitter [5] using the corresponding Java, C, or C++ grammar. The syntax tree determines which source fragments are passed to the encoder, but is not itself an input to the model. Extraction produces two views. The file-level view contains type declarations, fields or global variables, and constructor signatures with short body excerpts where applicable. The member-level view contains methods or functions. Each view produces one vector per file. Member-level text is split into overlapping chunks whose embeddings are mean-pooled.

The zero-shot variant (UXC_{ZS}) uses the pretrained weights. UniXcoder's pretraining objectives do not target architectural grouping:

files with similar code can belong to different modules. We therefore also evaluate a dependency-guided variant (UXC_{FT}), fine-tuned separately for each system using an InfoNCE objective [61]. This objective brings dependency-based positive pairs closer than other in-batch examples treated as negatives. However, files that are close in the dependency graph need not belong to the same architectural module. During fine-tuning, we compute this loss using only the file-level views, without module labels. We then use the fine-tuned encoder to produce representations for both file- and member-level views.

Positive pairs are drawn from the two-hop neighborhood to capture longer-range dependency structure [23], with dependencies treated as undirected. For each anchor file, two positives are sampled at random from the eight two-hop candidates that share the most one-hop neighbors with it. Files without a two-hop candidate fall back to a direct neighbor. Other files in the batch serve as negatives, except directly connected files, which are masked unless selected as the positive pair.

Member information enters the model differently across representations. Word2Vec includes member identifiers in file vectors when member nodes are absent and places them in member nodes otherwise. UniXcoder provides member text only through member nodes, while Node2Vec assigns each member node its file's vector. Adding member nodes thus changes the placement or availability of information as well as the graph structure, so comparisons across graph variants are interpreted separately for each representation.

2.4 Mapping Models

We compare models that use file features alone with models that incorporate graph structure. All models use the same classification head to predict architectural modules. Throughout, $\mathcal{N}(\cdot)$ denotes the neighborhood of a node.

2.4.1 *Multilayer Perceptron (MLP)*

The MLP predicts module logits directly from a file-level input vector. Each hidden layer computes:

$$h_i^{(k+1)} = \text{ReLU}\big(W^{(k)} h_i^{(k)} + b^{(k)}\big), \tag{2}$$

with $h_i^{(0)} = x_i$ the file feature vector and $W^{(k)}, b^{(k)}$ the learnable parameters of layer k, and ReLU is the rectified linear unit [39]. The input x_i contains the single representation being evaluated.

For RQ3, we incorporate dependency neighborhood information in the MLP by concatenating each file's vector x_i with the mean $\bar{x}_{\mathcal{N}(i)}$ of its direct neighbors' vectors in the dependency graph, giving $[x_i \,\|\, \bar{x}_{\mathcal{N}(i)}]$. We refer to the MLP with this input as the neighbor MLP ($\mathcal{N}_{\text{MLP}}$). With neighbors disabled, the input is x_i alone, as in RQ1, so enabling neighbors also doubles the input width of the first layer.

2.4.2 *Heterogeneous Graph Transformer (HGT)*

HGT [20] conditions attention and message transformation on the source node type, edge type, and target node type. We describe an edge $e_{s,t}$ from node s to node t by the meta-relation $\langle \tau(s), \phi(e_{s,t}), \tau(t) \rangle$. For example, $\langle$folder, contains, file$\rangle$ captures project organization, whereas $\langle$file, depends on, file$\rangle$ captures code-level coupling.

Node types have different feature spaces depending on the graph variant and initialization strategy. We first map the input vector x_t of each node t into a shared hidden space using a type-specific linear transformation followed by ReLU, which gives its initial representation $h_t^{(0)}$. One HGT layer then updates node representations through multi-head attention over their neighbors.

For each attention head, the layer computes a message $M(s,e,t)$ and an attention weight $\alpha(s,e,t)$ for each source node s connected to a target node t by an edge e. We omit attention-head indices for brevity. The weighted messages are aggregated as:

$$\tilde{m}_t = \sum_{s \in \mathcal{N}(t)} \alpha(s,e,t)\, M(s,e,t), \tag{3}$$

$$M(s,e,t) = W^{\text{MSG}}_{\phi(e)}\, V_{\tau(s)}\, h_s,$$

$$\alpha(s,e,t) = \underset{s \in \mathcal{N}(t)}{\text{softmax}} \frac{\big(K_{\tau(s)} h_s\big)^{\top} W^{\text{ATT}}_{\phi(e)} \big(Q_{\tau(t)} h_t\big)\, \mu_{\langle \tau(s), \phi(e), \tau(t) \rangle}}{\sqrt{d_h}}.$$

where $\tau(\cdot)$ is the node type, $\phi(e)$ is the edge type, V, K, and Q are type-specific linear transformations, W^{MSG} and W^{ATT} are edge-type-specific matrices, μ is a learned prior for each meta-relation, and d_h is the head dimension. These parameters allow a *contains* edge from a folder and a *depends_on* edge from a dependent file to receive different attention weights.

In our implementation, the attention-head outputs are combined into a message m_t. We add this message to the node's projected input $h_t^{(0)}$ through a residual connection [19], followed by layer normalization:

$$h_t^{(1)} = \text{LayerNorm}\left(m_t + h_t^{(0)}\right). \tag{4}$$

2.4.3 *Graph Attention Network (GAT)*

GAT [62] aggregates neighbor representations using attention parameters shared across neighbors within each head, without the node-type- or edge-type-specific parameters of HGT. The outputs of the attention heads are concatenated. Our GAT implementation uses the same input projection, residual connection, and layer normalization as HGT, while replacing its neighbor aggregation mechanism. We add no self-loops; the residual connection retains the node's own representation.

2.4.4 *Classification Head*

All three predictors use the same classification head. The file embedding $z_i \in \mathbb{R}^d$, obtained from the last hidden layer of the MLP or the message-passing layer of GAT and HGT, is passed to a linear layer that produces one score per architectural module. A softmax converts these scores into a probability distribution:

$$\begin{aligned} p_i &= \text{softmax}\big(W_c z_i + b_c\big), \\ \hat{y}_i &= \arg\max_m p_{i,m}, \end{aligned} \tag{5}$$

where $W_c \in \mathbb{R}^{|M| \times d}$, $b_c \in \mathbb{R}^{|M|}$, and $|M|$ is the number of architectural modules. The predicted module $\hat{y}_i$ is the one with the highest probability. Training minimizes categorical cross-entropy over the labeled file nodes.

3 Experimental Setup

This section describes the benchmark systems, the experimental configuration, the directory-based reference predictor, and the evaluation procedure used to answer the three research questions.

3.1 Benchmark Systems

We evaluate the models on ten open-source systems with published ground-truth (GT) mappings, which have been used in prior work on architecture recovery and C2A mapping [22, 45]. They differ in programming language, domain, size, and architectural granularity. We use published mappings because reliable C2A mappings require substantial manual effort and expert validation [14], and because they allow comparison with previous studies.

Table 2 summarizes the system and graph properties. File-level dependency homophily $\mathcal{H}$ is the proportion of unique directed file pairs connected by a static dependency whose source and target files fall within the same GT module, from 0 (every dependency crosses a module boundary) to 1 (every dependency stays within a module). The mean degree $\langle k \rangle = 2|E|/|\text{Files}|$ counts each of the $|E|$ directed file-level dependency edges at both of its files.

Table 2: Subject systems and their file-level dependency graph properties. M: number of modules in the GT; $|E|$: number of directed file-level static dependencies; $\mathcal{H}$: dependency homophily; $\langle k \rangle$: mean degree.

	System	Version	M	Files	$\lvert E\rvert$	$\mathcal{H}$	$\langle k \rangle$
Java	A.UML	v0.24	14	766	3 754	0.55	9.80
	C.Img	v1.0a2	21	329	1 565	0.61	9.51
	JabRef	v3.7	6	1 013	6 126	0.56	12.09
	Lucene	r1075001	7	1 054	6 424	0.68	12.19
	SH-3D	r002382	9	167	1 220	0.36	14.61
	T.Mates	v5.11	15	778	6 721	0.34	17.28
C	Bash	4.2	14	292	1 010	0.58	6.92
	HDF	0e196f	9	153	535	0.35	6.99
C++	HDC	46ff87	11	207	543	0.63	5.25
	Chrome	svn-171054	69	18 343	145 492	0.58	15.86

The Java systems come from the benchmark of Olsson et al. [45]. JabRef and TeamMates (T.Mates) were studied at the SAEroCon workshop in 2016 and 2017, where system experts provided the architectures and mappings[2]. Lucene was studied by Lenhard et al. [30], and the Commons Imaging (C.Img) mapping was contributed as a test case for s4rdm3x [44] [3].

For Bash, we use the mapping curated by Garcia et al. [14], who recovered an initial decomposition from documentation and source code and refined it with system experts through a certification process. The same work provides the Chromium mapping, developed with developers over approximately two years [31].

The HDF and HDC mappings were developed with industry partners and are part of the SARIF benchmark of Zhang et al. [65], available in its replication package[4].

3.2 Experimental Configurations

We repeat each experiment 100 times, each with a different initial labeled set. In each run, we randomly sample 5% of the file nodes, with a minimum of 30 and a maximum of 200 files, and add one file from each GT module not represented in the sample. We use the same 100 initial labeled sets across all experiments and configurations, which allows paired comparisons. This procedure imposes a bounded labeling budget rather than a constant labeled proportion: before the module-coverage additions, the random sample covers approximately 9–20% of the files in the five systems subject to the floor (C.Img, SH-3D, Bash, HDF, and HDC) and approximately 1% in Chromium, the only system subject to the cap.

Input representations. For each system, Word2Vec embeddings are 128-dimensional vectors trained for 20 epochs with a context window of five on the file names and identifiers.

Node2Vec learns 128-dimensional vectors from uniform random walks ($p = q = 1$) on the undirected file-level dependency graph, with walk length 30, 200 walks per file, window 15, five skip-gram epochs, and five negative samples. Chromium uses 100 walks per file and one negative sample.

For UXC_{FT}, UniXcoder is fine-tuned separately for each system with an InfoNCE temperature of 0.05, AdamW with a learning rate of 2×10^{-5}, a batch size of eight, and two epochs with a fixed seed. Only the upper four of the twelve encoder layers are updated, while the token embeddings and the lower eight layers stay frozen. Both UniXcoder variants encode file inputs with a 512-token limit and member inputs in overlapping 256-token windows with a stride of 128, mean-pooled before normalization, producing 768-dimensional features.

All input representations are generated once per system and reused in every run and experiment.

Training models. All models are trained with Adam at a learning rate of 0.001 for 200 epochs. The MLP has two hidden layers of 256 dimensions, and GAT and HGT use one layer with 256 hidden dimensions and four attention heads.

Controlled comparisons. RQ1 trains the MLP on each of the four representations, which isolates the information in the file representation because the MLP uses no graph structure.

RQ2 trains HGT on all graph variants (Figure 1) with Word2Vec, Node2Vec, and UXC_{FT} features, using directed dependency edges. We additionally train all four HGT variants with two layers to test whether one layer limits what a file receives from its neighbors. We also train **FD**, an HGT model on graph F whose UXC_{FT} file vector is extended with a binary folder-path vector, including ancestor folders. This vector uses the same tokenizer and vocabulary as the folder-node features (Section 2.3), with one dimension per distinct folder token in the system. FD provides folder information without folder nodes.

RQ3 consists of controlled comparisons on the file graph, with all four representations. For each predictor (MLP, GAT, and HGT), we

[2] https://github.com/sebastianherold/SAEroConRepo

[3] Java labels are available at: https://github.com/h0bb3/s4rdm3x

[4] https://github.com/anonymous2f4a9d/SARIF_FSE23

compare settings with dependency neighbors disabled and enabled, and with different aggregation directions. For the neighbor MLP, enabling neighbors also increases the input width and parameter count. We keep the file representations, initial labeled sets, and training settings fixed and compare three settings. With neighbors *disabled*, each file uses only its own representation. With *directed* aggregation, a dependency edge from file u to file v, where u depends on v, carries information from u to v. Each file therefore receives information from the files that depend on it. With *undirected* aggregation, we also add the reverse edge, allowing each file to receive information from files it depends on. For the MLP, these settings determine which files contribute to the neighbor vector; for GAT and HGT, they determine which files send messages.

The MLP with neighbors disabled is the RQ1 classifier, and HGT with directed neighbors on the file graph is the F configuration of RQ2, so these runs are shared. In total, the experiments comprise 58,000 runs.

3.2.1 Directory Hierarchy Baseline

To assess how much folder-based configurations exploit files in the same directory sharing a module, we use a rule-based predictor based on the directory hierarchy and the initial labeled files. For each unlabeled file, it assigns the most common module among labeled files in the same directory. If none are present, it searches the directory subtree and then the subtrees of its parent directories until it finds labeled files. Ties are resolved by the module's frequency in the complete initial labeled set and then alphabetically. If no labeled files are found, the predictor assigns the most common module in the complete initial labeled set. It uses the directory hierarchy, but not directory-name text, source code, embeddings, dependencies, or test labels.

We apply the predictor to all 100 initial labeled sets and compare it with HGT using UXC_{FT} features on the directed FM and FFM graphs and with FD. For each system, we report the accuracy of the predictor, FM, FFM, and FD, which equals micro-F1 because each file has exactly one module label. For the files that the predictor assigns incorrectly, we also report the accuracy of each compared model on those files.

3.3 Performance Evaluation

The benchmark systems have imbalanced module sizes, so overall accuracy can hide poor performance on smaller modules. We therefore report both macro-F1 and micro-F1. F1 is the harmonic mean of precision and recall, which measure the correctness of predicted assignments and the coverage of true assignments, respectively. Macro-F1 averages the F1 scores of individual modules, giving each module equal weight regardless of size. Micro-F1 aggregates predictions across modules, giving each file equal weight. Reporting both captures performance across modules as well as overall file-level performance.

Because compared configurations use the same initial labeled sets, we pair runs by split. We use the two-sided Wilcoxon signed-rank test [63], a non-parametric test, to compare configurations separately for each system and metric without assuming normally distributed score differences. We apply Holm's correction across the ten systems for each metric and pair of compared configurations, using a significance level of $\alpha = 0.05$.

To quantify the magnitude of the differences, we also compute Cliff's δ [10] from the two sets of 100 scores. We classify its magnitude as negligible (N, $|\delta| < 0.147$), small (S, $0.147 \leq |\delta| < 0.33$), medium (M, $0.33 \leq |\delta| < 0.474$), or large (L, $|\delta| \geq 0.474$) [54].

4 Results

This section reports the results for code-to-architecture mapping across ten subject systems. RQ1 compares lexical, structural, and contextual file representations using an MLP. RQ2 examines how mapping performance changes when Word2Vec, Node2Vec, and UXC_{FT} initialize HGT with file, member, and folder context, and analyzes errors relative to directory-based predictions. RQ3 uses controlled experiments to examine how using dependency neighbors changes mapping performance, and how the predictor and aggregation direction affect this change.

For RQ1 and RQ2, we report median macro- and micro-F1 scores over the 100 runs per system and configuration, and differences between these medians in F1 percentage points, with Cliff's δ alongside the significance test. For RQ3, we report the median of the paired differences between conditions over the 100 splits. Summary tables give the median of these per-system values across the ten systems. F1 is computed only on the held-out test files of each split.

4.1 Comparing Lexical, Structural, and Contextual File Representations (RQ1)

RQ1 examines the mapping performance of each file representation before additional graph-based message passing. Table 3 reports the median macro-F1 and micro-F1 scores obtained using the MLP with Word2Vec, Node2Vec, and the two UniXcoder variants, UXC_{ZS} and UXC_{FT}.

Table 4 summarizes the pairwise differences across all ten systems, with the number of systems favoring each method and the number of Holm-significant differences in each direction.

Table 3: Median macro-F1/micro-F1 scores (%) across 100 runs for the MLP using W2V, UXC_{ZS}, UXC_{FT}, and N2V. Highlighted values indicate the highest score for each system and metric.

System	W2V	UXC_{ZS}	UXC_{FT}	N2V
A.UML	34 / 57	36 / 60	46 / 69	54 / 72
C.Img	23 / 39	33 / 52	42 / 62	71 / 80
JabRef	44 / 78	39 / 76	43 / 79	40 / 74
Lucene	57 / 71	54 / 66	63 / 76	66 / 79
SH-3D	22 / 41	48 / 64	47 / 67	43 / 58
T.Mates	30 / 55	37 / 68	51 / 79	41 / 68
Bash	7 / 29	23 / 42	27 / 49	42 / 61
HDF	8 / 22	42 / 44	46 / 48	53 / 57
Chrome	14 / 48	14 / 48	31 / 66	41 / 71
HDC	33 / 53	30 / 51	46 / 70	43 / 69

Table 3 shows that macro-F1 is lower than micro-F1 for every system and representation. Node2Vec achieves the highest median macro- and micro-F1 scores on six systems. On the other four systems, UXC_{FT} obtains the highest median micro-F1, while the

Table 4: Summary of pairwise differences in F1 points across ten systems. Δ is the median difference (first method minus second). +ve is the number of systems favoring the first method, followed in parentheses by the numbers of significant positive (↑) and negative (↓) differences.

	Macro F1		Micro F1	
Comparison	Δ	+ve (↑/↓)	Δ	+ve (↑/↓)
UXC_{ZS}−W2V	+5.2	6 (6/4)	+7.5	6 (6/3)
UXC_{FT}−UXC_{ZS}	+9.1	9 (9/0)	+9.2	10 (10/0)
UXC_{FT}−W2V	+17.5	9 (9/0)	+19.2	10 (9/0)
UXC_{FT}−N2V	−4.9	4 (4/6)	−3.3	3 (3/6)
N2V−W2V	+20.3	9 (9/1)	+16.2	9 (9/1)

highest median macro-F1 is held by UXC_{FT} on T.Mates and HDC, Word2Vec on JabRef, and UXC_{ZS} on SH-3D.

We first compare Word2Vec and UXC_{ZS}, the two representations that do not encode dependencies. They do not separate consistently: the significant differences fall in both directions, although UXC_{ZS} is favored on more systems. On macro-F1, UXC_{ZS} is significantly stronger on A.UML (medium effect), C.Img, SH-3D, T.Mates, Bash, and HDF (large effects), whereas Word2Vec is significantly stronger on JabRef (large effect), Lucene and Chrome (medium effects), and HDC (small effect). On micro-F1, UXC_{ZS} is significantly stronger on the same six systems, and Word2Vec is significantly stronger on JabRef and Lucene (large effects) and HDC (small effect); the difference in Chrome is not significant.

Fine-tuning significantly improves UniXcoder over UXC_{ZS} on micro-F1 for all ten systems and on macro-F1 for nine, predominantly with large effects. The effects are medium on HDF (both metrics) and on SH-3D micro-F1; the macro-F1 difference on SH-3D is not significant.

Both representations encoding dependency information are generally stronger than the lexical representation. Node2Vec significantly outperforms Word2Vec on both metrics for nine systems, with JabRef as the exception, where Word2Vec is significantly stronger. Similarly, UXC_{FT} significantly outperforms Word2Vec on both metrics for nine systems; on JabRef, the difference is not significant on either metric.

Node2Vec is significantly stronger than UXC_{FT} on both metrics for A.UML, C.Img, Lucene, Bash, HDF, and Chrome, whereas UXC_{FT} is significantly stronger on JabRef, SH-3D, and T.Mates; most of these differences have large effects. On HDC, UXC_{FT} is significantly stronger on macro-F1 with a small effect, and the micro-F1 difference is not significant.

4.2 Message Passing over Dependency Edges, Member Nodes, and Folder Nodes (RQ2)

RQ2 examines how mapping performance changes for each representation when message passing over the file dependency graph is added and the graph is extended with folder and member nodes. Figure 2 reports median macro- and micro-F1 for the MLP and HGT under F, FF, FM, and FFM. We use Word2Vec, Node2Vec, and the fine-tuned variant UXC_{FT}, which improves over UXC_{ZS} on nearly all systems in RQ1.

Answer to RQ1.

Representations that encode dependencies, either directly through random walks (Node2Vec) or through dependency-guided fine-tuning of UniXcoder, provide the strongest mapping performance overall. Fine-tuning consistently improves UniXcoder over its zero-shot representation and produces the best macro-F1 on two systems. Word2Vec and UXC_{ZS}, which do not encode dependencies, differ from each other in both directions, and Word2Vec is weaker than Node2Vec and UXC_{FT} on most systems; Word2Vec nevertheless remains stronger than Node2Vec on JabRef. Thus, no single file representation performs best across all systems and metrics.

Table 5 summarizes five steps: MLP to F, adding members (F to FM, FF to FFM), and adding folders (F to FF, FM to FFM). For each system, Δ is the difference between the two configuration medians over 100 runs. The table reports the median of these ten system-level differences, the number of systems with a positive difference, and the number of systems with a significant gain or loss.

File-level HGT versus the MLP. Moving from the MLP to file-level HGT improves Word2Vec and UXC_{FT} more than Node2Vec. For Word2Vec and UXC_{FT}, the improvement is positive on nine systems on both metrics, and the only significant loss is UXC_{FT} on HDC macro-F1. For Node2Vec, the changes are small, and significant losses occur on Lucene and HDC for both metrics and on HDF for macro-F1, but significant gains still outnumber losses on both metrics.

Member-level context. Adding member nodes to F lowers median macro-F1 and micro-F1 for every representation, most strongly for Word2Vec, with large effects on all ten systems. Significant macro-F1 losses outnumber gains for every representation. Adding member nodes also changes how features enter the model, depending on the representation (Section 2.3); for example, the UXC_{FT} member vectors come from an encoder fine-tuned on file-level text.

In addition, with one HGT layer, member-level dependencies cannot affect file predictions, so we test two layers in Section 4.2.1. These results therefore concern the evaluated member configuration, not member information in general.

Folder-level context. Adding folder nodes produces the largest and most consistent changes. FF improves over F, and FFM improves over FM, on both metrics for every system and representation. All 120 system–representation–metric comparisons are significant with a large effect, and each improves in at least 96 of the 100 paired runs.

Within the folder-containing configurations, the effect of adding member nodes depends on the representation. FFM improves over FF for Node2Vec and UXC_{FT} on both metrics, with small median gains, but lowers both metrics for Word2Vec. The significant macro-F1 losses for Node2Vec and UXC_{FT} occur on JabRef, Bash, and Chrome. For micro-F1, Node2Vec loses significantly only on Bash, and UXC_{FT} on no system.

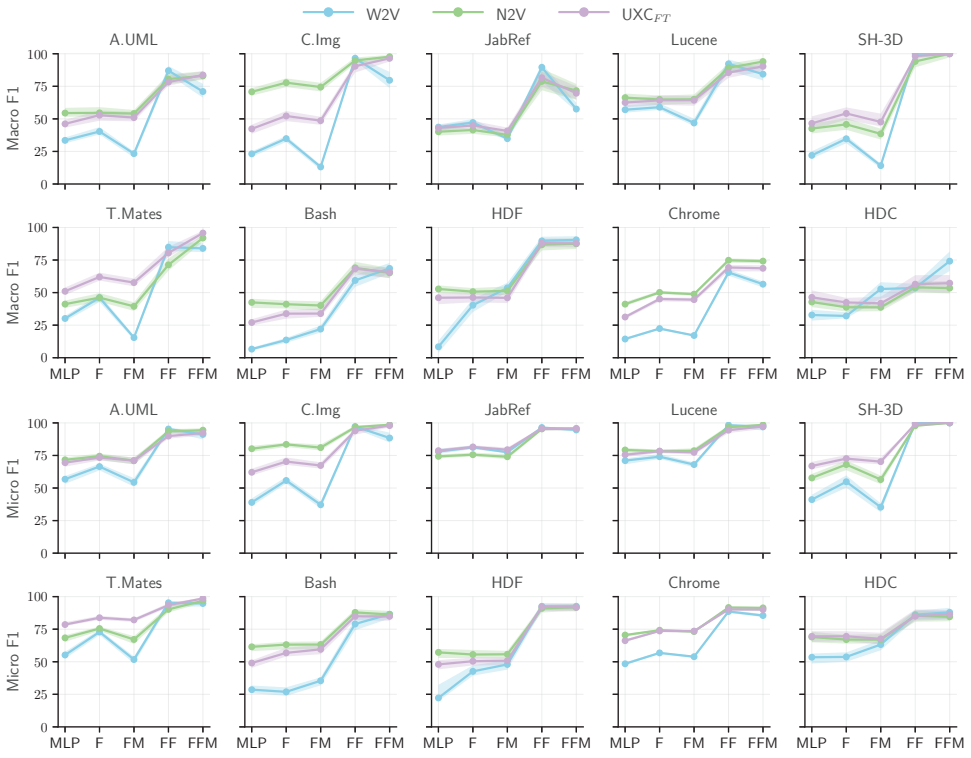


Figure 2: Median macro-F1 (top) and micro-F1 (bottom) for each representation, per system, across the MLP and HGT with increasing graph structure (F, FF, FM, FFM). Shaded areas denote interquartile ranges across the 100 runs.

Table 5: Change in F1 as graph structure is added, by representation. Each cell gives the median over ten systems of the difference between configuration medians in F1 points, followed in parentheses by the number of systems with a positive difference and the number of significant positive/negative differences. Blue shading indicates gains and purple indicates losses, with darker shades indicating larger magnitudes.

	W2V		N2V		UXC$_{FT}$	
Step	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
F − MLP	+7.5 (9; 9/0)	+9.0 (9; 8/0)	+0.8 (6; 5/3)	+2.2 (7; 7/2)	+6.7 (9; 8/1)	+4.6 (9; 9/0)
FM − F	−12.2 (3; 3/7)	−4.9 (3; 3/7)	−1.2 (2; 0/5)	−1.3 (3; 0/6)	−1.2 (2; 1/6)	−1.8 (2; 1/7)
FFM − FF	−4.4 (3; 2/5)	−1.0 (3; 2/5)	+1.6 (6; 5/3)	+0.8 (7; 5/1)	+1.3 (6; 5/3)	+1.1 (7; 5/0)
FF − F	+44.4 (10; 10/0)	+32.1 (10; 10/0)	+25.6 (10; 10/0)	+18.7 (10; 10/0)	+29.9 (10; 10/0)	+16.5 (10; 10/0)
FFM − FM	+43.0 (10; 10/0)	+39.9 (10; 10/0)	+28.9 (10; 10/0)	+22.3 (10; 10/0)	+32.3 (10; 10/0)	+20.6 (10; 10/0)

4.2.1 Depth of message-passing layers

To test whether using one HGT layer limits the results, we repeated F, FF, FM, and FFM with two layers, using the same labeled sets, representations, and training settings. Table 6 reports the results.

Table 6: Effect of a second message-passing layer on HGT, by input and graph variant. Δ is the median across ten systems of the change in median F1 (points) from one to two layers; positive favors two layers. +ve is the number of systems with a positive change, followed in parentheses by the numbers of significant gains (↑) and losses (↓). Shading marks at least five significant gains and at least twice as many gains as losses.

		Macro-F1		Micro-F1	
Input	Graph	Δ	+ve (↑/↓)	Δ	+ve (↑/↓)
W2V	F	+0.5	6 (3/4)	0.0	6 (4/3)
	FF	+2.4	8 (6/1)	+0.5	6 (6/0)
	FM	+3.5	6 (6/3)	+2.2	8 (7/2)
	FFM	+10.3	6 (6/3)	+1.1	6 (6/3)
N2V	F	−0.6	4 (2/5)	−0.1	4 (2/4)
	FF	+1.8	7 (5/2)	+0.4	6 (4/3)
	FM	0.0	5 (3/3)	+0.1	5 (3/3)
	FFM	+0.3	5 (4/3)	−0.2	3 (2/4)
UXC_{FT}	F	0.0	6 (2/1)	+0.8	8 (7/1)
	FF	+3.4	8 (8/2)	+1.4	10 (7/0)
	FM	−0.2	5 (1/2)	0.0	5 (3/3)
	FFM	+2.6	7 (7/1)	+1.0	8 (8/1)

For Word2Vec, the largest median macro-F1 improvements occur in FM and FFM. For UXC_{FT}, FF improves more than FFM, so the improvement is not specific to member nodes. Node2Vec shows little change in either member-containing configuration, but F is significantly lower on macro-F1 on five systems.

At matched two-layer depth, adding members to F still lowers median macro-F1 for all three representations. Adding members to FF remains negative for Word2Vec, while the small median gain for UXC_{FT} shrinks and that for Node2Vec becomes slightly negative. Folder nodes retain the largest gains: both folder additions improve both metrics on all ten systems for every representation, with significant large effects. Thus, additional depth improves some configurations but does not establish a consistent benefit from adding member nodes.

We also examined whether the macro-F1 change from one to two message-passing layers in FM and FFM relates to average file-level degree across the ten systems. Exploratory Kendall's τ analyses [29] show positive associations in FFM across all three representations, but in FM mainly with Word2Vec.

4.2.2 Organizational information and the directory baseline

The large gains from adding folder nodes show that directory organization is informative for the evaluated mappings. We first test whether these gains require folder nodes using FD, which adds folder-path features directly to the UXC_{FT} file vectors in graph F. We then examine whether the gains also help files misclassified by directory information alone, comparing FM, FFM, and FD against the directory hierarchy baseline (Section 3.2.1). FM and FFM differ only in the inclusion of folder nodes. Table 7 reports overall accuracy for the baseline and the three models, and model accuracy on files misclassified by the baseline. FD improves over F on every system with a large effect. Compared with FD, FF has lower median macro-F1 on eight systems, significantly so on six, and no significant gains. Folder nodes and relations therefore do not improve on supplying folder-path information directly as file features in this configuration.

Table 7: Accuracy of the directory baseline (Dir.) and of HGT with UXC_{FT} features on FM, FFM, and FD, median over 100 runs. n is the median number of test files per split that the baseline misclassifies; the columns under it give model accuracy on those files, over splits with at least one baseline error. Purity is the share of files that belong to the majority module of their directory.

	Accuracy (%)				Baseline misclassifications				
System	Dir.	FM	FFM	FD	n	FM	FFM	FD	Purity
A.UML	99	71	92	93	11	9.1	21.7	22.5	100.0
C.Img	100	67	98	99	0	–	–	–	100.0
JabRef	97	79	96	93	27	45.3	26.3	25.0	99.1
Lucene	92	77	97	95	86	52.9	86.9	81.7	100.0
SH-3D	100	70	100	99	0	–	–	–	100.0
T.Mates	100	82	99	99	0	–	–	–	100.0
Bash	92	60	85	86	21	24.6	26.1	27.8	91.8
HDF	93	51	92	93	8	27.3	50.0	40.0	100.0
Chrome	97	74	90	94	574	19.8	20.0	23.3	99.9
HDC	89	68	87	87	19	20.0	12.5	10.8	96.1

The baseline reaches 89 to 100% accuracy on every system. Without folder nodes, FM is 14 to 43 points below it. FFM and FD are within 7 points of it on nine systems and exceed it only on Lucene. Folder information therefore brings both FFM and FD close to the accuracy of the directory hierarchy baseline on most systems.

Directory membership closely matches module membership: on eight systems, 99 to 100% of files belong to the majority module of their directory, and only Bash and HDC have lower purity. Files that depart from the majority module of their directory are therefore rare on most systems, which limits what the analysis can show about them.

On the files the baseline gets wrong, FFM gets most of them wrong on A.UML, JabRef, Bash, Chrome, and HDC, about half on HDF, and most of them right only on Lucene. Compared with FM, FFM is more accurate on these files on A.UML, HDF, and Lucene, similar on Bash and Chrome, and less accurate on JabRef and HDC. JabRef illustrates the distinction between overall accuracy and accuracy on baseline errors: FFM improves over FM by 16 percentage points overall but falls by 19 points on these files. FD is within 10 points of FFM on these files on every system. This analysis does not isolate files whose module differs from the majority module of their directory, so it does not show how well FFM handles such exceptions.

Answer to RQ2.

The effect on mapping performance depends on what is added and on the representation. Moving from the MLP to file-level HGT improves Word2Vec and UXC_{FT} more than Node2Vec. Member nodes give no consistent gain and often lower performance, most strongly for Word2Vec. Folder nodes provide the largest gains, with significant large effects on all ten systems for all three representations. The findings for member and folder nodes remain with two HGT layers.

The directory hierarchy baseline achieves accuracy similar to FFM on most systems, and folder nodes do not consistently outperform folder information supplied directly as file features (FD). On most systems with directory-baseline errors, FFM misclassifies most of these files, and on some it is less accurate than FM. Strong folder-based scores therefore do not establish reliable performance on directory-baseline errors, and the analysis does not isolate files whose directory location is insufficient or misleading.

4.3 Dependency Neighbors, Predictors, and Aggregation Direction (RQ3)

RQ2 shows that the effect of graph context depends on the representation, configuration, and system. However, its comparison of the MLP with file-level HGT changes both the predictor and the availability of dependency messages. RQ3 therefore compares each predictor with dependency neighbors disabled and enabled, using HGT, GAT, and a neighbor MLP (N_{MLP}). It also compares directed and undirected aggregation to examine whether the benefit depends on how dependency neighbors are aggregated. We evaluate all three predictors with all four input representations.

Differences are paired changes in F1 points, enabled minus disabled; positive values favor neighbor information. For direction, differences are undirected minus directed; positive values favor undirected neighbors. We report the median paired difference over the 100 splits. With neighbors disabled, the neighbor MLP reduces to the MLP of RQ1, and GAT and HGT receive no dependency messages.

How does the benefit of dependency neighbors vary across predictors and representations? Table 8 shows the change in F1 when neighbor information is enabled, for each predictor and input representation. With undirected neighbors, the median change is positive in every cell on both metrics. GAT and the N_{MLP} improve on at least eight systems in every cell and are never significantly lower, whereas HGT gains least with every representation and is significantly lower on a few systems with Node2Vec and Word2Vec. With directed neighbors, the median macro-F1 is lower with Node2Vec for all three predictors, by 0.6 to 2.8 points, and significant losses occur on up to six systems. For GAT and the N_{MLP}, the gain from undirected neighbors is smallest with Node2Vec and largest with UXC_{ZS}.

This comparison differs from RQ2, where file-level message passing (F) is compared with the MLP, which changes the predictor together with the available structure. Here the predictor is fixed, and dependency neighbors are enabled and disabled, so a gain of F over the MLP does not imply that HGT benefits from dependency neighbors. With Node2Vec, for example, F raises the median macro-F1 over the MLP, whereas HGT with directed neighbors is lower than HGT without neighbors.

Does the direction of neighbor aggregation matter? Undirected aggregation gives the higher median in almost every cell of Table 8, but the preferred direction depends on the system. Figure 3 shows gains from undirected aggregation on five to eight systems per combination of predictor and input representation, with significant differences in both directions.

On macro-F1, SH-3D, T.Mates, and JabRef score higher with directed aggregation in all 12 combinations, and C.Img in eight. Lucene, Bash, Chrome, and HDC score higher with undirected aggregation in all 12 combinations, and A.UML and HDF in 11. The largest differences are 14.6 points in favor of undirected aggregation (Bash, Word2Vec, N_{MLP}) and 8.7 points in favor of directed aggregation (T.Mates, Word2Vec, N_{MLP}). Micro-F1 shows the same directional pattern for seven systems, while JabRef, C.Img, and A.UML show mixed results. Adding reverse edges also expands the neighborhood, so this comparison does not isolate propagation direction from neighborhood size.

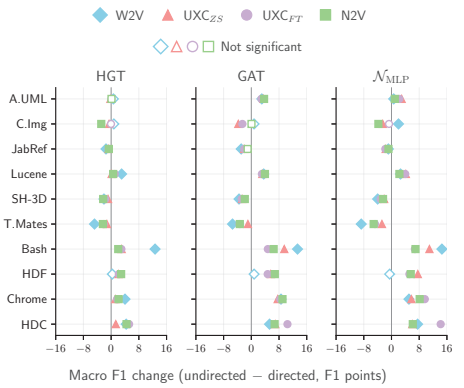


Figure 3: Macro-F1 differences (undirected − directed), in F1 points, by predictor and input representation. Each point shows median paired differences over 100 splits; negative values favor directed aggregation. Hollow markers indicate non-significant differences.

Which predictor scores highest? Figure 4 pools the ten systems with undirected neighbors: each point is the median of one system and the boxes show the spread across systems. GAT has the highest median macro-F1 with every representation and the highest median micro-F1 with every representation except Node2Vec, where HGT is marginally higher. GAT also scores highest most often: in each split we count which of the four predictors scores highest, and GAT has the largest share of wins, averaged over systems, with

Table 8: Changes in F1 from enabling neighbor information (D−off, UD−off) and switching from directed to undirected aggregation (UD−D), by model and input representation. Each cell reports the median across ten systems of the per-system median paired change over runs, in F1 points. Parentheses follow the format (systems with a positive median change; significant gains/significant losses). D: directed (source-to-target); UD: undirected; off: neighbor information disabled. Blue shading indicates gains and purple indicates losses, with darker shades indicating larger magnitudes.

		N2V		W2V		UXC_{ZS}		UXC_{FT}	
Model	Contrast	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1	Macro F1	Micro F1
HGT	D−off	−0.6 (4; 4/6)	+1.6 (6; 6/4)	+0.2 (5; 4/5)	+1.8 (7; 7/3)	+1.6 (9; 7/0)	+2.7 (10; 10/0)	+0.7 (7; 6/3)	+1.8 (8; 8/0)
	UD−off	+0.5 (8; 5/2)	+0.8 (9; 8/0)	+2.9 (7; 7/3)	+4.5 (8; 8/2)	+1.8 (10; 10/0)	+2.5 (10; 10/0)	+0.9 (10; 8/0)	+1.6 (10; 10/0)
	UD−D	+0.3 (6; 5/4)	+0.1 (5; 3/4)	+0.7 (7; 4/3)	+1.5 (6; 5/4)	0.0 (5; 4/3)	+0.1 (5; 5/4)	+0.6 (6; 5/3)	+0.2 (5; 4/4)
GAT	D−off	−2.8 (4; 3/6)	−0.8 (3; 2/5)	+1.5 (8; 6/2)	+2.4 (7; 7/1)	+3.8 (10; 9/0)	+6.2 (10; 10/0)	+0.7 (6; 4/2)	+2.2 (8; 7/1)
	UD−off	+1.4 (9; 8/0)	+1.4 (10; 10/0)	+5.6 (8; 7/0)	+6.7 (8; 8/0)	+8.2 (10; 10/0)	+9.5 (10; 10/0)	+3.8 (10; 8/0)	+4.1 (10; 10/0)
	UD−D	+3.8 (7; 6/2)	+2.2 (8; 8/1)	+1.9 (7; 5/3)	+1.6 (7; 5/3)	+3.3 (6; 6/4)	+2.2 (6; 6/2)	+3.0 (6; 6/4)	+1.3 (7; 6/1)
$\mathcal{N}_{MLP}$	D−off	−0.6 (4; 4/5)	+0.8 (6; 5/4)	+3.4 (9; 9/1)	+2.7 (9; 8/1)	+4.8 (8; 7/1)	+5.1 (9; 9/0)	+2.8 (7; 6/2)	+2.9 (7; 7/2)
	UD−off	+1.2 (10; 8/0)	+1.2 (9; 8/0)	+5.1 (10; 10/0)	+4.9 (10; 10/0)	+7.7 (10; 10/0)	+7.3 (10; 10/0)	+5.8 (10; 10/0)	+4.6 (10; 10/0)
	UD−D	+1.6 (6; 6/4)	+0.4 (6; 5/3)	+1.3 (6; 6/3)	+1.9 (6; 6/2)	+3.5 (6; 6/4)	+2.5 (6; 6/4)	+3.1 (6; 6/3)	+1.3 (7; 7/3)

every representation and on both metrics. All three predictors with neighborhood aggregation exceed the MLP without neighbors in every median comparison. As without neighbors, UXC_{FT} has the highest median macro-F1 across systems for every predictor, with GAT and UXC_{FT} giving the highest scores overall.

Most of the variation in F1 lies between systems and not between runs of one system. The choice of predictor matters most for Word2Vec and UXC_{ZS}, the two representations without dependency information, and less for Node2Vec and UXC_{FT}.

Answer to RQ3.

Dependency neighbors do not benefit all predictors equally. With undirected aggregation, median changes are positive for every predictor and representation, but GAT and the $\mathcal{N}_{MLP}$ improve more consistently across systems than HGT, which shows significant losses in some cases. Directed aggregation is less reliable, particularly with Node2Vec, where median macro-F1 decreases for all three predictors. Although undirected aggregation generally gives larger gains, it is not preferable on every system: several systems consistently favor directed aggregation across predictors and representations. These comparisons also change neighborhood size, so they do not isolate direction alone. With undirected neighbors, GAT has the highest median macro-F1 across systems for every representation, and UXC_{FT} remains the strongest representation by this measure for every predictor.

5 Discussion

The research questions examine where architectural signal resides and how a mapping model can use it. RQ1 compares file representations before message passing, RQ2 examines file, member, and folder context and compares the folder gains with a directory-based rule, and RQ3 examines how predictor choice and aggregation direction affect the contribution of dependency neighbors. This section discusses these findings together.

Dependency Information in File Representations

Dependency information can support mapping before message passing, through the graph position encoded by Node2Vec or through dependency-guided adaptation of a code encoder. The advantage of these representations over Word2Vec, and of fine-tuned UniXcoder over its zero-shot counterpart, suggests that dependency information is useful beyond what the evaluated lexical and pretrained code representations provide. The representations also differ in training, vector size, and source content, so these comparisons are between representation pipelines and not between isolated information sources.

The usefulness of this signal varies with the relation between dependency structure and the reference architecture. Node2Vec's micro-F1 is lower on the three least homophilic systems, SH-3D, HDF, and T.Mates (Table 2), with a median of 58 compared with 72 on the remaining seven. Node2Vec learns dependency proximity from files that co-occur on random walks, without using the GT mapping. When dependencies frequently cross module boundaries, files from different modules may co-occur on walks and receive similar vectors, making the modules harder to distinguish.

The advantage of fine-tuned UniXcoder over Node2Vec on SH-3D and T.Mates suggests that code content may provide complementary evidence in these mixed neighborhoods. HDF does not follow this pattern, so low homophily alone does not explain their relative performance; HDF is also the smallest system, and every representation has comparatively low micro-F1 on it. Homophily is used here to interpret the observed pattern, not to select a representation, since computing it requires the reference mapping.

Encoding dependencies in the initial representation and aggregating dependency neighbors during mapping are two uses of the same information source, and the second does not necessarily replace the first. With undirected aggregation, the median gains are generally larger for Word2Vec and zero-shot UniXcoder than for Node2Vec and fine-tuned UniXcoder, yet fine-tuned UniXcoder still has the highest median macro-F1 for every predictor. Conversely,

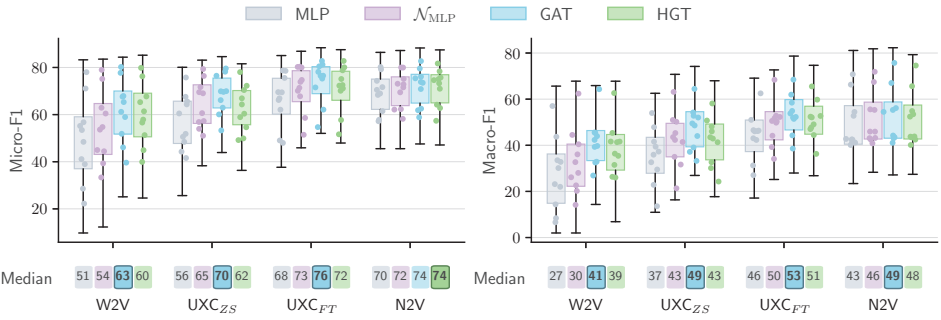


Figure 4: Micro-F1 (left) and macro-F1 (right) of the MLP, $\mathcal{N}_{\text{MLP}}$, GAT, and HGT with undirected neighbors, by input representation. Each point is the median of one system over 100 runs; boxes show the spread across systems, and the values below are the median of medians across systems, with the highest in each group in bold.

useful dependency information in a representation does not guarantee a benefit from further aggregation: RQ3 shows that this benefit depends on both aggregation direction and predictor choice.

Folder Organization and Architectural Exceptions

The gains from adding folder nodes in RQ2 show that directory organization helps reproduce the reference mapping. They do not show that this information must reach a file through message passing over folder nodes, since supplying the same information directly in the file vector matches or exceeds the accuracy of folder nodes on most systems. Extending a directory assignment is also different from identifying files that depart from it. Comparing FM, FFM, and FD on the directory baseline's errors helps assess this distinction (Section 4.2.2), but does not isolate architectural exceptions from errors caused by missing local supervision.

Lucene and HDF have pure directories under the reference mapping, so their baseline errors do not reflect conflicts between directory and module membership. They may instead arise from directories without labeled examples, although the table does not establish this. FFM recovers most baseline errors on Lucene and half on HDF, but only about a fifth on A.UML, whose directories are also pure. Directory purity therefore does not ensure that folder nodes recover baseline errors.

Bash and HDC have mixed directories. On baseline errors, FFM is similar to FM on Bash and less accurate on HDC. JabRef further illustrates why overall gains are insufficient: adding folder context raises overall accuracy while lowering accuracy on baseline errors.

The directory rule serves as a reference point for interpreting the learned models. Where directory organization closely follows the architecture, it already reproduces most of the mapping. On most evaluated systems, the added modeling complexity therefore brings little accuracy over the rule. Evaluating the additional value of a learned mapper requires examining both files in directories without labeled examples and files whose module differs from their directory's majority module.

Dependency Neighbors and Aggregation Direction

The benefit of dependency neighbors depends on how the predictor aggregates them. With undirected aggregation, HGT gains least, while GAT achieves the highest median macro-F1 with every representation, although differences between predictors are small relative to variation across systems. GAT and HGT differ in their attention formulations and parameterizations, so the comparison does not isolate why GAT performs better. These findings concern the evaluated file-level predictors, not the usefulness of dependency types in general. RQ2 uses HGT for the folder- and member-aware configurations; whether the folder gains persist with other predictors remains untested.

The preferred aggregation direction appears to depend more on the system than on the predictor or representation, although the pattern is not uniform across metrics. Undirected aggregation generally gives larger gains, but several systems consistently favor directed aggregation. Agreement across predictors and representations describes the stability of these system-level preferences rather than providing independent evidence, since the comparisons share systems and splits.

Undirected aggregation adds reverse edges, so the comparison does not isolate propagation direction from neighborhood size. A reversed-edges-only setting would help distinguish these effects and remains for future work.

With Node2Vec, directed aggregation lowers median macro-F1 for all three predictors. Since its vectors were learned on an undirected graph, this may reflect a mismatch between the neighborhoods encoded during initialization and those used during prediction, but this explanation remains untested.

Connectivity and Neighbor Information

Connectivity describes how much neighboring context is available, measured here by average node degree, while homophily describes whether linked files share a module. Few neighbors limit the context

available to shallow message passing, whereas Node2Vec can encode more distant context through random walks. Limited connectivity may therefore constrain the benefit of additional aggregation even when the initial structural representation remains useful.

With Node2Vec and undirected aggregation, no predictor changes significantly on Bash and HDC, the two lowest-degree systems. The depth analysis is also consistent with this interpretation: higher average degree is associated with larger gains from a second message-passing layer, particularly in FFM.

Connectivity may also help explain the effect of aggregation direction. T.Mates and SH-3D combine low homophily with high average degree and favor directed aggregation in RQ3. Bash and HDC favor undirected aggregation, as does HDF, which has low homophily but much lower average degree than T.Mates and SH-3D. Reverse edges may supply missing context in sparse neighborhoods but introduce additional cross-module context in dense, low-homophily neighborhoods. This mechanism remains untested: the ten-system comparison cannot separate connectivity from homophily, system size, or label scarcity.

Implications for Evaluation

Evaluation should establish what the input representation already provides, then isolate access to neighborhood information within a fixed predictor. Comparing a GNN model with an MLP changes both the predictor and the use of dependency-neighborhood aggregation. Enabling and disabling neighborhood aggregation within the same predictor family, while keeping the file representations fixed, provides a more targeted comparison. For the neighbor MLP, this comparison includes an increase in input width and parameter count. The same applies to folder information: a comparison of graph variants should include a model that receives that information directly as file features.

Micro-F1 and macro-F1 should be read together, since correctly mapping more files does not necessarily improve performance across modules. For tools intended to identify architectural exceptions, aggregate scores should also be accompanied by performance on files whose module differs from the majority module of their directory.

The intended use also matters. If mapping supports code reorganization, the existing directory structure may be something to reconsider rather than reproduce. Strong performance based on that structure does not establish which information would remain useful when the organization changes. The present study does not test this setting, but motivates evaluating information sources against the intended task rather than treating any representation or predictor as generally preferable.

6 Related Work

Architecture recovery and code-to-architecture mapping both infer a system's modular structure from implementation artifacts, such as static dependencies, code semantics, and directory organization. Recovery produces a decomposition without a reference architecture, while mapping assigns entities to the modules of a given one. We evaluate mapping and draw on recovery research for representations of dependencies, lexical content, and project organization.

Recovery techniques have historically been built around one dominant signal: dependency graphs, partitioned by search-based optimization [18, 32, 33, 36, 48] or clustering [1, 38, 40, 41, 49, 59], identifiers and comments [11, 15, 28, 53], or file names and paths [2]. Each signal is noisy in isolation, since shared utilities introduce cross-module edges and reused terminology mixes entities from many modules, so hybrid methods combine sources [35, 37, 46, 50, 51, 56, 64, 65].

Mapping began with orphan adoption [60] and the attraction functions of HuGMe [7, 8], and later varied the signal and the learner: textual similarity [4], supervised classifiers [3], and semi-supervised Naive Bayes combining lexical and dependency-derived information [43, 45]. Text-based approaches compare classifiers for module assignment [12], recommend mappings from natural-language architecture descriptions [57], or integrate package- and class-level granularity [58]. More recent work uses TF-IDF, GloVe, and LLM representations [27], extracts architectural concepts from documentation and code [13], or prompts LLMs to map implementation units to a reference architecture, which faces context-size limits and repeated processing costs on large codebases [55]. Johansson and Caporuscio map functions and files to textual module descriptions with pre-trained embeddings and find files classified more reliably than functions, with performance associated with vocabulary overlap and class separability [26].

Representation-learning work on recovery encoded one or two sources at a time. NEGAR clusters Node2Vec embeddings of the dependency graph alone [6]. Jabeen et al. [23] study dependencies alone and with folder organization, comparing random walk-based methods and variational graph autoencoders. GAER [22], a graph autoencoder, and ARL [24], a contrastively trained GAT, each fix folder and identifier tokens into one encoding without comparing alternatives. For mapping, GNNs were evaluated on homogeneous and dependency-type-aware graphs without folder or member nodes and without separating graph contribution from feature contribution [21]. Its extension combines Word2Vec identifier embeddings and binary bag-of-words representations of folder path tokens with dependencies in one model [25].

These approaches motivate our distinction between representing an information source and using it during prediction. Text-based mapping reports weaker results for functions than for files [26], but does not examine whether member information helps as context for the file. Our comparisons span lexical, structural, and contextual inputs but retain differences in their training procedures and feature content, so the matched dependency controls and directory-error analyses assess specific design choices and do not turn the broader comparison into an isolated ranking of information sources.

7 Threats to Validity

Generalizability: We evaluate ten open-source systems in Java, C, and C++, each with one reference architecture. The findings may not extend to other systems or alternative architectural decompositions. The study is transductive: representation learning and graph processing can use unlabeled test nodes and their edges. It therefore does not establish performance on newly arriving files outside the observed graph or on maintaining a mapping as the system evolves.

Other information sources, including change history and dynamic dependencies, are outside the evaluated scope.

Dependencies and reference mappings: Dependencies are extracted using static analysis, which may miss or misresolve dependencies. Preprocessing and build configuration can also affect the dependencies available for analysis. Because predictors that use neighbors depend on the extracted graph, such errors can affect their predictions. Models with neighbors enabled and disabled are trained separately. Their comparison therefore measures the effect of training with access to neighbor information, rather than removing neighbors from a fixed trained model.

We made a best-effort attempt to identify and obtain the source versions corresponding to the published mappings, using the available version information and documentation. Despite these efforts, some mismatches between the analyzed file sets and the mappings remain. These may reflect incomplete documentation of the original source versions or build configurations, which limits our ability to establish an exact correspondence between the analyzed source code and the published mappings.

Files can plausibly belong to more than one module, and annotators may assign them differently. Because the initial labeled set is sampled from the same mapping used for evaluation, an incorrect assignment can both mislead the model during training and penalize an architecturally plausible prediction.

Experimental variation: We use 100 runs per configuration. These runs reuse the same systems and can include the same files, so they are not independent samples. With this many runs, small differences can reach statistical significance, so we report results per system and include effect sizes to distinguish the magnitude of a difference from its significance. The comparisons were not fixed in advance, so the analyses are exploratory and may favor patterns specific to this benchmark.

The runs use different initial labeled sets, but the contribution of stochastic model training to the observed variation was not assessed separately. Input representations were generated once per system and reused across runs, so variation arising from their generation was not assessed. The seeds governing model initialization and training were not retained, limiting exact reproducibility. Cliff's δ treats the paired scores as two independent groups, so it describes the separation of the two score distributions and not the paired differences.

Interpretation of results: Performance measures agreement with the reference architecture, not whether a prediction is the only architecturally reasonable assignment. Folder organization may reflect the same decisions used to define that architecture, so strong directory-based predictions do not by themselves demonstrate recovery of architectural intent. The directory-error analysis applies only where the directory baseline makes errors, which excludes C.Img, SH-3D, and T.Mates in most or all splits, and it does not isolate files whose module differs from the majority module of their directory. Message-passing depth was varied only for HGT in RQ2, with one and two layers, so the results do not show how deeper models or other predictors respond. It reports accuracy only, so it does not show how performance differs between small and large modules.

Relationships between performance and structural properties are reported as associations, not causal effects, since the benchmark does not allow these properties to be separated from one another or from system size and label availability. The fine-tuning gain is that of the whole dependency-guided procedure, since no alternative adaptation objective was evaluated, so it is not attributed to dependency topology alone.

8 Conclusion

This study examines how code text, static dependencies, folder organization, and member-level information contribute to code-to-architecture mapping on ten systems. The findings show that mapping performance depends not only on which information is available but also on how it is represented and used.

Representations that encode dependency information generally outperform the evaluated lexical and zero-shot code representations. Node2Vec and dependency-guided fine-tuned UniXcoder outperform Word2Vec on most systems, and fine-tuning improves UniXcoder over its zero-shot counterpart on nearly all systems. However, neither Node2Vec nor fine-tuned UniXcoder is consistently best across systems.

Folder nodes produce the largest and most consistent gains within the HGT variants, but a directory rule using the same labeled files reaches similar accuracy on most systems, and folder paths supplied as file features match or exceed folder nodes on most systems. These gains do not consistently translate into better predictions on files the rule assigns incorrectly. Member nodes provide no consistent improvement with one or two HGT layers, and their effects cannot generally be separated from changes in feature construction and placement. These structural findings are specific to HGT; whether they persist with other predictors remains untested.

Keeping the input representations fixed while enabling and disabling access to dependency-neighborhood information shows that neighbor aggregation can complement representations that already encode dependencies. With undirected aggregation, the median change across systems is positive on both metrics for every predictor and representation. In this setting, GAT achieves the highest median macro-F1, while HGT gains the least. The better aggregation direction varies between systems, although the experiments do not isolate propagation direction from neighborhood size. Evaluation should therefore distinguish information already encoded in the representations from the contribution of neighborhood access. The findings support assessing these contributions for the intended mapping task, rather than treating any representation or predictor as generally preferable.

Future work. Future research could clarify how different aspects of graph context contribute to the observed differences. Comparing incoming, outgoing, and combined neighborhoods at matched sizes would help distinguish the effects of propagation direction and neighborhood size. Adding member nodes while preserving each file's own features would help separate the effects of graph structure and feature placement. Another direction is configuration selection on unfamiliar systems. Homophily, which helped interpret the system differences, requires reference module assignments, so selection rules based on properties observable in the dependency

graph alone could be developed on some systems and evaluated on others. Finally, a model trained on one version could be evaluated on new and changed files in later versions without retraining, to test whether the learned information supports mapping as systems evolve.

References

[1] Periklis Andritsos, Panayiotis Tsaparas, Renée J Miller, and Kenneth C Sevcik. 2004. LIMBO: Scalable clustering of categorical data. In *International Conference on Extending Database Technology*. Springer, 123–146. doi:10.1007/978-3-540-24741-8_9

[2] Nicolas Anquetil and Timothy C Lethbridge. 1999. Recovering software architecture from the names of source files. *Journal of Software Maintenance: Research and Practice* 11, 3 (1999), 201–221. doi:10.1002/(sici)1096-908x(199905/06)11:3<201::aid-smr192>3.0.co;2-1

[3] Maryum Bibi, Onaiza Maqbool, and Jaweria Kanwal. 2016. Supervised learning for orphan adoption problem in software architecture recovery. *Malaysian Journal of Computer Science* 29, 4 (2016), 287–313.

[4] Roberto Almeida Bittencourt, Gustavo Jansen de Souza Santos, Dalton Dario Serey Guerrero, and Gail C. Murphy. 2010. Improving Automated Mapping in Reflexion Models Using Information Retrieval Techniques. In *17th Working Conference on Reverse Engineering*. 163–172.

[5] Max Brunsfeld and contributors. 2018. Tree-sitter: An Incremental Parsing System for Programming Tools. https://tree-sitter.github.io/tree-sitter/. Accessed: 2026-04-22.

[6] Jiayi Chen, Zhixing Wang, Yuchen Jiang, Jun Pang, Tian Zhang, Minxue Pan, and Jianwen Sun. 2022. NEGAR: network embedding guided architecture recovery for software systems. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 367–376. doi:10.1109/apsec57359.2022.00049

[7] Andreas Christl, Rainer Koschke, and M-A Storey. 2005. Equipping the reflexion method with automated clustering. In *12th Working Conference on Reverse Engineering (WCRE'05)*. IEEE, 10–pp.

[8] Andreas Christl, Rainer Koschke, and Margaret-Anne Storey. 2007. Automated clustering to support the reflexion method. *Information and Software Technology* 49, 3 (2007), 255–274.

[9] Paul Clements, David Garlan, Reed Little, Robert Nord, and Judith Stafford. 2003. Documenting software architectures: views and beyond. In *25th International Conference on Software Engineering*. IEEE, 740–741. doi:10.1109/icse.2003.1201264

[10] Norman Cliff. 1993. Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin* 114, 3 (1993), 494. doi:10.1037/0033-2909.114.3.494

[11] Anna Corazza, Sergio Di Martino, Valerio Maggio, and Giuseppe Scanniello. 2011. Investigating the use of lexical information for software system clustering. In *2011 15th European Conference on Software Maintenance and Reengineering*. IEEE, 35–44. doi:10.1109/csmr.2011.8

[12] Alexander Florean, Laoa Jalal, Zipani Tom Sinkala, and Sebastian Herold. 2021. A Comparison of Machine Learning-Based Text Classifiers for Mapping Source Code to Architectural Modules. In *15th European Conference on Software Architecture*.

[13] Dominik Fuchß, Haoyu Liu, Tobias Hey, Jan Keim, and Anne Koziolek. 2025. Enabling architecture traceability by llm-based architecture component name extraction. In *2025 IEEE 22nd International Conference on Software Architecture (ICSA)*. IEEE, 1–12.

[14] Joshua Garcia, Ivo Krka, Chris Mattmann, and Nenad Medvidovic. 2013. Obtaining ground-truth software architectures. In *35th International Conference on Software Engineering (ICSE)*. IEEE, 901–910. doi:10.1109/icse.2013.6606639

[15] Joshua Garcia, Daniel Popescu, Chris Mattmann, Nenad Medvidovic, and Yuanfang Cai. 2011. Enhancing architectural recovery using concerns. In *2011 26th IEEE/ACM International Conference on Automated Software Engineering (ASE 2011)*. IEEE, 552–555. doi:10.1109/ase.2011.6100123

[16] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 855–864. doi:10.1145/2939672.2939754

[17] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 7212–7225.

[18] Mark Harman and Xin Yao. 2010. Software module clustering as a multi-objective search problem. *IEEE Transactions on Software Engineering* 37, 2 (2010), 264–282. doi:10.1109/tse.2010.26

[19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[20] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of the Web Conference 2020*. 2704–2710.

[21] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2025. Graph Convolution Networks for Mapping Source Code Entities to Architectural Modules. In *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 472–481. doi:10.1109/icsa-c65153.2025.00071

[22] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. GAER: Graph Auto-Encoders for Unsupervised Software Architecture Recovery. *Proceedings of the ACM on Software Engineering* 3, FSE, Article FSE180 (July 2026), 22 pages. doi:10.1145/3808187

[23] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. Graph Embeddings for Software Architecture Recovery. *Nordic Machine Intelligence* (2026). doi:10.5617/nmi.13153

[24] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. Graph Representation Learning for Software Architecture Recovery. In *ESANN*.

[25] Rakhshanda Jabeen, Morgan Ericsson, Jonas Nordqvist, and Anna Wingkvist. 2026. When Does the Dependency Graph Help Code-to-Architecture Mapping? arXiv:2610.19769 [cs.SE] https://arxiv.org/abs/2610.19769

[26] Nils Johansson and Mauro Caporuscio. 2026. Mapping source code to software architecture using embedding models: Insights from an analysis of data properties. *Information and Software Technology* 198 (2026), 108200. doi:10.1016/j.infsof.2026.108200

[27] Nils Johansson, Mauro Caporuscio, and Tobias Olsson. 2024. Mapping Source Code to Software Architecture by Leveraging Large Language Models. In *European Conference on Software Architecture*. Springer, 133–149. doi:10.1007/978-3-031-70797-1_8

[28] Masoud Kargar, Ayaz Isazadeh, and Habib Izadkhah. 2017. Semantic-based software clustering using hill climbing. In *2017 International Symposium on Computer Science and Software Engineering Conference (CSSE)*. IEEE, 55–60. doi:10.1109/csicsse.2017.8320117

[29] Maurice G Kendall. 1938. A new measure of rank correlation. *Biometrika* 30, 1-2 (1938), 81–93.

[30] Jörg Lenhard, Martin Blom, and Sebastian Herold. 2019. Exploring the suitability of source code metrics for indicating architectural inconsistencies. *Software Quality Journal* 27, 1 (2019), 241–274.

[31] Thibaud Lutellier, Devin Chollak, Joshua Garcia, Lin Tan, Derek Rayside, Nenad Medvidović, and Robert Kroeger. 2017. Measuring the impact of code dependencies on software architecture recovery techniques. *IEEE Transactions on Software Engineering* 44, 2 (2017), 159–181. doi:10.1109/tse.2017.2671865

[32] Spiros Mancoridis, Brian S Mitchell, Yihfarn Chen, and Emden R Gansner. 1999. Bunch: A clustering tool for the recovery and maintenance of software system structures. In *Proceedings IEEE International Conference on Software Maintenance (ICSM'99)*. IEEE, 50–59. doi:10.1109/icsm.1999.792498

[33] Onaiza Maqbool and Haroon Atique Babri. 2004. The weighted combined algorithm: A linkage algorithm for software clustering. In *Eighth European Conference on Software Maintenance and Reengineering (CSMR 2004)*. IEEE, 15–24. doi:10.1109/csmr.2004.1281402

[34] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems* 26 (2013).

[35] Janardan Misra, KM Annervaz, Vikrant Kaulgud, Shubhashis Sengupta, and Gary Titus. 2012. Software clustering: Unifying syntactic and semantic features. In *2012 19th Working Conference on Reverse Engineering*. IEEE, 113–122. doi:10.1109/wcre.2012.21

[36] Brian S Mitchell and Spiros Mancoridis. 2006. On the automatic modularization of software systems using the bunch tool. *IEEE Transactions on Software Engineering* 32, 3 (2006), 193–208. doi:10.1109/tse.2006.31

[37] Wiem Mkaouer, Marouane Kessentini, Adnan Shaout, Patrice Koligheu, Slim Bechikh, Kalyanmoy Deb, and Ali Ouni. 2015. Many-objective software remodularization using NSGA-III. *ACM Transactions on Software Engineering and Methodology* 24, 3 (2015), 1–45. doi:10.1145/2729974

[38] Sina Mohammadi and Habib Izadkhah. 2019. A new algorithm for software clustering considering the knowledge of dependency between artifacts in the source code. *Information and Software Technology* 105 (2019), 252–256. doi:10.1016/j.infsof.2018.09.001

[39] Vinod Nair and Geoffrey E. Hinton. 2010. Rectified Linear Units Improve Restricted Boltzmann Machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*. Omnipress, 807–814. https://www.cs.toronto.edu/~fritz/absps/reluICML.pdf

[40] Rashid Naseem, Mustafa Mat Deris, Onaiza Maqbool, and Sara Shahzad. 2019. Euclidean space based hierarchical clusterers combinations: an application to software clustering. *Cluster Computing* 22, Suppl 3 (2019), 7287–7311. doi:10.1007/s10586-017-1408-0

[41] Rashid Naseem, Onaiza Maqbool, and Siraj Muhammad. 2013. Cooperative clustering for software modularization. *Journal of Systems and Software* 86, 8 (2013), 2045–2062. doi:10.1016/j.jss.2013.03.080

[42] Hung Viet Nguyen. 2023. Depends: Multilingual Dependency Analysis Tool. https://github.com/multilang-depends/depends. Accessed: 2025-08-17.

[43] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2019. Semi-automatic mapping of source code using Naive Bayes. In *Proceedings of the 13th European Conference on Software Architecture*. 209–216.

[44] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2021. s4rdm3x: A Tool Suite to Explore Code to Architecture Mapping Techniques. *Journal of Open Source Software* 6, 58 (2021).

[45] Tobias Olsson, Morgan Ericsson, and Anna Wingkvist. 2022. To automatically map source code entities to architectural modules with Naive Bayes. *Journal of Systems and Software* 183 (2022), 111095. doi:10.1016/j.jss.2021.111095

[46] Marios Papachristou. 2019. Software clusterings with vector semantics and the call graph. In *Proceedings of the 27th ACM Joint ESEC/FSE*. 1184–1186. doi:10.1145/3338906.3342483

[47] David Lorge Parnas. 1972. On the criteria to be used in decomposing systems into modules. *Commun. ACM* 15, 12 (1972), 1053–1058.

[48] Saeed Parsa and Omid Bushehrian. 2005. A new encoding scheme and a framework to investigate genetic clustering algorithms. *Journal of Research and Practice in Information Technology* 37, 1 (2005), 127–143.

[49] Babak Pourasghar, Habib Izadkhah, Ayaz Isazadeh, and Shahriar Lotfi. 2021. A graph-based clustering algorithm for software systems modularization. *Information and Software Technology* 133 (2021), 106469. doi:10.1016/j.infsof.2020.106469

[50] Shiva Prasad Reddy Puchala, Jitender Kumar Chhabra, and Amit Rathee. 2022. Ensemble clustering based approach for software architecture recovery. *International Journal of Information Technology* 14, 4 (2022), 2013–2019. doi:10.1007/s41870-021-00846-0

[51] Amit Rathee and Jitender Kumar Chhabra. 2017. Software remodularization by estimating structural and conceptual relations among classes and using hierarchical clustering. In *Advanced Informatics for Computing Research: ICAICR 2017*. Springer, 94–106. doi:10.1007/978-981-10-5780-9_9

[52] Mark Richards and Neal Ford. 2025. *Fundamentals of Software Architecture: A Modern Engineering Approach*. O'Reilly Media.

[53] Michele Risi, Giuseppe Scanniello, and Genoveffa Tortora. 2012. Using fold-in and fold-out in the architecture recovery of software systems. *Formal Aspects of Computing* 24, 3 (2012), 307–330. doi:10.1007/s00165-011-0199-y

[54] Jeanine Romano, Jeffrey D Kromrey, Jesse Coraggio, and Jeff Skowronek. 2006. Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen'sd for evaluating group differences on the NSSE and other surveys. In *annual meeting of the Florida Association of Institutional Research*, Vol. 177.

[55] Satrio Adi Rukmono, Lina Ochoa, and Michel Chaudron. 2024. Deductive software architecture recovery via chain-of-thought prompting. In *44th International Conference on Software Engineering: New Ideas and Emerging Results*. 92–96. doi:10.1145/3639476.3639776

[56] Nafiseh Sadat Jalali, Habib Izadkhah, and Shahriar Lotfi. 2019. Multi-objective search-based software modularization: Structural and non-structural features. *Soft Computing* 23, 21 (2019). doi:10.1007/s00500-018-3666-z

[57] Zipani Tom Sinkala and Sebastian Herold. 2021. InMap: Automated Interactive Code-to-Architecture Mapping Recommendations. In *18th International Conference on Software Architecture*. 173–183.

[58] Zipani Tom Sinkala and Sebastian Herold. 2023. An integrated approach to package and class code-to-architecture mapping using InMap. In *2023 IEEE 20th International Conference on Software Architecture (ICSA)*. IEEE, 164–174.

[59] Navid Teymourian, Habib Izadkhah, and Ayaz Isazadeh. 2020. A fast clustering algorithm for modularization of large-scale software systems. *IEEE Transactions on Software Engineering* 48, 4 (2020), 1451–1462. doi:10.1109/tse.2020.3022212

[60] Vassilios Tzerpos and Richard C Holt. 1997. The orphan adoption problem in architecture maintenance. In *Working Conference on Reverse Engineering*. IEEE, 76–82.

[61] Aäron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748* (2018).

[62] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017). doi:10.48550/arXiv.1710.10903

[63] Frank Wilcoxon. 1945. Individual comparisons by ranking methods. *Biometrics bulletin* 1, 6 (1945), 80–83. doi:10.2307/3001968

[64] Kaiyuan Yang, Junfeng Wang, Zhiyang Fang, Peng Wu, and Zihua Song. 2022. Enhancing software modularization via semantic outliers filtration and label propagation. *Information and Software Technology* 145 (2022), 106818. doi:10.1016/j.infsof.2021.106818

[65] Yiran Zhang, Zhengzi Xu, Chengwei Liu, Hongxu Chen, Jianwen Sun, Dong Qiu, and Yang Liu. 2023. Software architecture recovery with information fusion. In *Proceedings of the 31st ACM Joint ESEC/FSE*. 1535–1547. doi:10.1145/3611643.3616285